DPGeometry

目录

12 动态规划 1

12.1 序列、区间、数位与基础状态 DP 1

12.2 背包家族 7

12.3 树、轮廓线、概率与期望 DP 12

12.4 DP 优化与选型 17

13 搜索、随机化与博弈论 28

13.1 启发式搜索与指数级搜索 28

13.2 随机化与局部搜索 31

13.3 公平组合游戏 34

13.4 图上博弈与对抗搜索 41

14 概率与期望 45

14.1 期望递推、方程与生成函数 45

15 二维计算几何 47

15.1 点、向量、直线与线段 47

15.2 多边形、包含与 Minkowski 和 52

15.3 凸包、切点与旋转卡壳 60

15.4 半平面交与平面区域 68

15.5 圆与圆弧 72

15.6 扫描线、最近点与高级平面结构 81

16 三维、球面与特殊几何 93

16.1 三维基础运算与旋转 93

16.2 三维点线面与距离 98

16.3 空间相交与位置判定 102

16.4 三维凸包与多面体 108

16.5 球与最小覆盖球 113

16.6 球面与地理计算 119

16.7 特殊空间几何 123

17 经典模型、专题与结论表 127

17.1 构造、搜索与经典模型 127

17.2 图论、偏序与变换结论 135

17.3 微积分、恒等式与常数表 142

17.4 大模拟、牌型与赛事排名 148

12. 动态规划

12.1 序列、区间、数位与基础状态 DP

通用约定

```
constexpr i64 INF = numeric_limits<i64>::max() / 4;
constexpr i64 NINF = numeric_limits<i64>::lowest() / 4;
```

- 最小化不可达状态用 `INF`，最大化不可达状态用 `NINF`
- 不要直接用 `numeric_limits<i64>::max()` 后继续加法
- 从不可达状态转移前先判断，或保证有限值与增量的绝对值远小于 `INF`
- 计数 DP 的不可达状态通常是 `0`
- 状态压缩前先写清 `dp` 的含义、初值、转移顺序、答案位置

最大子段和

状态： `dp[i]` 表示以 `i` 结尾的非空最大子段和

$$dp_i = \max(a_i, dp_{i-1} + a_i)$$

答案为 $\max_i dp_i$

```
i64 cur = a[0], ans = a[0];
for (int i = 1; i < n; ++ i) {
    cur = max(a[i], cur + a[i]);
    ans = max(ans, cur);
}
```

- 全负数组仍返回最大单点
- 若允许空段，令 `ans = max<i64>(0, ans)`
- 时间 $O(n)$，空间 $O(1)$

LIS 最长上升子序列

基础状态： `dp[i]` 表示以 `i` 结尾的 LIS 长度

$$dp_i = 1 + \max_{j<i, a_j<a_i} dp_j$$

朴素复杂度 $O(n^2)$
经典优化： `d[len-1]` 保存长度为 `len` 的上升子序列最小末尾

```
template<class T>
int lis(const vector<T> &a, bool strict = true) {
    vector<T> d;
    for (const T &x : a) {
        auto it = strict ? rangs::lower_bound(d, x) : ranges::upper_bound(d, x);
        if (it == d.end()) d.push_back(x);
        else *it = x;
    }
    return d.size();
}
```

- 严格上升用 `lower_bound`
- 不下降用 `upper_bound`
- `d` 不是实际的一条 LIS，只记录每个长度的最优末尾
- 若 `a[i]` 很大但可离散化，也可把朴素转移改成 BIT / 线段树前缀最大值
- 时间 $O(n \log n)$，空间 $O(n)$

LCS 最长公共子序列

状态： `dp[i][j]` 表示 `a[0..i)` 与 `b[0..j)` 的 LCS 长度

$$dp_{i,j} = \begin{cases} dp_{i-1,j-1} + 1 & a_{i-1} = b_{j-1} \\ \max(dp_{i-1,j}, dp_{i,j-1}) & a_{i-1} \neq b_{j-1} \end{cases}$$

滚动数组写法

```
template<class S>
int lcs(const S &a, const S &b) {
    int n = a.size(), m = b.size();
    vector<int> dp(m + 1);
    for (int i = 1; i <= n; ++ i) {
        int pre{};
        for (int j = 1; j <= m; ++ j) {
            int old{ dp[j] };
            if (a[i - 1] == b[j - 1]) dp[j] = pre + 1;
            else dp[j] = max(dp[j], dp[j - 1]);
            pre = old;
        }
    }
    return dp[m];
}
```

- 仅求长度时把较短序列放第二维
- 两序列都是同一组互异元素的排列时，可映射位置后转 LIS，复杂度降为 $O(n \log n)$
- 普通时间 $O(nm)$，滚动后空间 $O(m)$

最长公共子串

与 LCS 不同，要求选出的元素在两个序列中都连续
定义 `dp[j]` 为当前处理到 `a[i]` 时，以 `a[i]`、`b[j - 1]` 结尾的最长公共子串长度

$$dp_{i,j} = \begin{cases} dp_{i-1,j-1} + 1 & a_{i-1} = b_{j-1} \\ 0 & a_{i-1} \neq b_{j-1} \end{cases}$$

滚动数组时必须倒序枚举 `j`

```
template<class Seq>
int lcstr(const Seq &a, const Seq &b) {
    int m{ int(b.size()) }, ans{};
    vector<int> dp(m + 1);
    for (const auto &x : a) {
        for (int j = m - 1; j >= 0; -- j) {
            dp[j + 1] = x == b[j] ? dp[j] + 1 : 0;
            ans = max(ans, dp[j + 1]);
        }
    }
    return ans;
}
```

- LCS 失配时继承相邻状态，公共子串失配时必须清零
- 时间复杂度 $O(nm)$
- 空间复杂度 $O(m)$

LCIS 最长公共上升子序列

状态： `dp[j]` 表示以 `b[j]` 结尾且已经匹配当前 `a` 前缀的 LCIS 长度

```
template<class T>
int lcis(const vector<T> &a, const vector<T> &b) {
    vector<int> dp(b.size());
    for (const T &x : a) {
```

```
        int best{};
        for (int j = 0; j < (int)b.size(); ++ j) {
            if (b[j] < x) best = max(best, dp[j]);
            else if (b[j] == x) dp[j] = max(dp[j], best + 1);
        }
    }
    return b.empty() ? 0 : *max_element(dp.begin(), dp.end());
}
```

- `best` 只允许来自 `b[j] < x` 的旧状态
- 两个判断不能随意改成两个独立 `if`
- 时间 $O(nm)$，空间 $O(m)$

编辑距离

状态：`dp[i][j]` 表示把 `a[0..i)` 变成 `b[0..j)` 的最小代价

$$dp_{i,j} = \min \begin{cases} dp_{i-1,j} + 1 & \text{删除} \\ dp_{i,j-1} + 1 & \text{插入} \\ dp_{i-1,j-1} + [a_{i-1} \neq b_{j-1}] & \text{替换} \end{cases}$$

```
template<class S>
int edit_dis(const S &a, const S &b) {
    int n = a.size(), m = b.size();
    vector<int> dp(m + 1);
    iota(dp.begin(), dp.end(), 0);
    for (int i = 1; i <= n; ++ i) {
        int pre{ dp[0] };
        dp[0] = i;
        for (int j = 1; j <= m; ++ j) {
            int ol{ dp[j] };
            dp[j] = min({dp[j] + 1, dp[j - 1] + 1, pre + (a[i - 1] != b[j - 1])});
            pre = old;
        }
    }
    return dp[m];
}
```

- 不同操作代价时直接替换三个转移项
- 时间 $O(nm)$，空间 $O(m)$

最长回文子序列

状态：`dp[l][r]` 表示闭区间 `[l,r]` 的最长回文子序列长度

$$dp_{l,r} = \begin{cases} dp_{l+1,r-1} + 2 & a_l = a_r \\ \max(dp_{l+1,r}, dp_{l,r-1}) & a_l \neq a_r \end{cases}$$

- 区间长度从小到大枚举
- 单点初值 `dp[i][i] = 1`
- 也可转化为原串与反串的 LCS
- 时间 $O(n^2)$，空间 $O(n^2)$

区间 DP 通用形态

若最后一次操作把 `[l,r)` 分为 `[l,k)` 与 `[k,r)`，常见状态为

$$dp_{l,r} = \operatorname{opt}_{l<k<r}\{dp_{l,k} + dp_{k,r} + w(l,r,k)\}$$

枚举顺序

```
for (int len = 2; len <= n; ++ len) {
    for (int l = 0; l + len <= n; ++ l) {
        int r = l + len;
        for (int k = l + 1; k < r; ++ k) {
```

```
            // dp[l][r] ← dp[l][k], dp[k][r]
        }
    }
}
```

- 普通复杂度通常 $O(n^3)$
- 若最优决策单调，检查 12.4 的分治优化或 Knuth 优化
- 环形区间通常复制一倍，只计算长度不超过 `n` 的区间

区间博弈 DP

两人轮流从区间两端取数，双方都最优，求先手相对后手的最大净胜分
定义：

$$dp_{l,r} = \text{面对区间 } [l, r) \text{ 时当前玩家相对另一人的最大净胜分}$$

转移：

$$dp_{l,r} = \max \begin{cases} a_l - dp_{l+1,r} \\ a_{r-1} - dp_{l,r-1} \end{cases}$$

边界：

$$dp_{i,i} = 0$$

```
i64 interval_game(const vector<i64> &a) {
    int n{ int(a.size()) };
    vector dp(n + 1, vector<i64>(n + 1));
    for (int len = 1; len <= n; ++ len) {
        for (int l = 0; l + len <= n; ++ l) {
            int r{ l + len };
            dp[l][r] = max(a[l] - dp[l + 1][r], a[r - 1] - dp[l][r - 1]);
        }
    }
    return dp[0][n];
}
```

若所有取得的数最终都会被分完，记总和为 `sum`、净胜分为 `d`，则：

$$A = \frac{sum + d}{2}, \qquad B = \frac{sum - d}{2}$$

核心技巧是直接维护双方得分差，避免同时记录先手和后手两个状态
时间复杂度 $O(n^2)$，空间复杂度 $O(n^2)$

划分 DP / 分段 DP

将前 `n` 个元素划分为恰好 `K` 个非空连续段，每段 `[j,i)` 的代价为 `w(j,i)`

定义：

$$dp_{k,i} = \text{前 } i \text{ 个元素划成 } k \text{ 段的最优值}$$

转移：

$$dp_{k,i} = \min_{k-1 \le j < i} \{dp_{k-1,j} + w(j,i)\}$$

初始化：

$$dp_{0,0} = 0$$

其余状态为 `INF`

直接转移复杂度 $O(Kn^2)$，只依赖上一层时空间可压到 $O(n)$

```
vector<i64> dp(n + 1, INF), ndp(n + 1);
dp[0] = 0;
for (int k = 1; k ≤ K; ++ k) {
    fill(ndp.begin(), ndp.end(), INF);
    for (int i = k; i ≤ n; ++ i) {
        for (int j = k - 1; j < i; ++ j) {
            if (dp[j] ≠ INF) chmin(ndp[i], dp[j] + w(j, i));
        }
    }
    dp.swap(ndp);
}
```

优化入口

- `opt[k][i]` 随 `i` 单调 → 分治优化
- `w(j,i)` 展开后出现 `k_j x_i+b_j` → 斜率优化
- 转移矩阵满足 Monge 性质 → 检查决策单调
- 要求恰好 `K` 段且段数可通过罚值控制 → 检查 WQS 二分

石子合并

状态：`dp[l][r]` 表示把 `[l,r)` 合成一堆的最小代价

$$dp_{l,r} = \min_{l<k<r}\{dp_{l,k} + dp_{k,r} + sum(l,r)\}$$

- 区间和用前缀和 $O(1)$ 求出
- 普通时间 $O(n^3)$，空间 $O(n^2)$
- 若题目满足四边形不等式，可进一步检查 Knuth 优化

矩阵链乘

矩阵 i 的维度为 $p_i \times p_{i+1}$
状态：`dp[l][r]` 表示乘完 `[l,r)` 的最少标量乘法次数

$$dp_{l,r} = \min_{l<k<r}\{dp_{l,k} + dp_{k,r} + p_l p_k p_r\}$$

这是最典型的“枚举最后断点”的区间 DP

DAG DP

有向无环图上按拓扑序转移
最长路形式

$$dp_v = \max_{u\to v}(dp_u + w_{u,v})$$

计数形式

$$cnt_v = \sum_{u\to v} cnt_u$$

- 关键不是 DFS 还是拓扑排序，而是所有依赖状态必须先计算完成
- 图本身的拓扑排序与路径恢复统一使用图论板子
- 时间通常 $O(n+m)$

TSP 状压 DP

固定从 `0` 出发
状态：`dp[s][u]` 表示访问集合 `s` 且当前停在 `u` 的最小代价

$$dp_{s\cup\{v\},v} = \min_{u\in s}\{dp_{s,u} + d_{u,v}\}$$

初值 `dp[1][0] = 0`
回路答案

$$\min_u dp_{2^n-1,u} + d_{u,0}$$

- Hamilton 路径删掉最后回到 `0` 的一步
- 时间 $O(n^2 2^n)$，空间 $O(n2^n)$

子集 DP 常见状态

- `dp[s]`：恰好选择集合 `s` 的最优值或方案数
- `dp[s][u]`：集合 `s` 内额外记录最后点 `u`
- `dp[s][k]`：集合 `s` 内额外记录数量、阶段或边界状态
- 枚举 `s` 的非空真子集会自然产生 $O(3^n)$，不能误判成 $O(n2^n)$
- SOS DP、Gosper Hack、Subset Sum 位集统一放在位运算与集合枚举板子

子集划分 DP

将集合划分成若干组，已知单独处理子集 `T` 的代价 `w[T]`
定义：

$$dp_S = \text{划分集合 } S \text{ 的最优代价}$$

转移：

$$dp_S = \min_{\varnothing \neq T \subseteq S} \{dp_{S \setminus T} + w_T\}$$

直接枚举：

```
vector<i64> dp(1 << n, INF);
dp[0] = 0;
for (int s = 1; s < 1 << n; ++ s) {
    for (int t = s; t; t = (t - 1) & s) {
        chmin(dp[s], dp[s ^ t] + w[t]);
    }
}
```

所有 `S` 枚举其所有子集的总复杂度为：

$$\sum_{S \subseteq U} 2^{|S|} = 3^n$$

若组之间没有顺序，可强制当前选出的 `T` 包含 `S` 的最低位，减少重复转移：

```
for (int s = 1; s < 1 << n; ++ s) {
    int b{ __builtin_ctz(s) };
    for (int t = s; t; t = (t - 1) & s) {
        if (t >> b & 1) chmin(dp[s], dp[s ^ t] + w[t]);
    }
}
```

若要求恰好划分成 `K` 组，增加组数一维：

$$dp_{k,S} = \min_{T \subseteq S} \{dp_{k-1,S \setminus T} + w_T\}$$

数位 DP

适合统计 `[0,x]` 中十进制表示满足局部约束的整数
常用状态

- `pos`：当前数位
- `st`：与后续有关的压缩状态
- `lim`：前缀是否仍贴着上界
- `beg`：是否已经出现非前导零
  只有 `lim = false` 的状态能直接记忆化

```
template<class Trans, class Accept>
i64 dig_dp(i64 x, int S, int init, Trans trans, Accept accept) {
    if (x < 0) return 0;
```

```
    string s = to_string(x);
    int n = s.size();
    vector memo(n, vector(S, array<i64, 2>{-1, -1}));
    auto dfs = [&](auto &&self, int p, int st, bool lim, bool beg) -> i64 {
        if (p == n) return accept(st, beg);
        if (!lim && memo[p][st][beg] != -1) return memo[p][st][beg];
        int up = lim ? s[p] - '0' : 9;
        i64 ans = 0;
        for (int d = 0; d <= up; ++ d) {
            bool nbeg = beg || d;
            int ns = trans(st, d, beg, nbeg);
            if (ns < 0) continue;
            ans += self(self, p + 1, ns, lim && d == up, nbeg);
        }
        if (!lim) memo[p][st][beg] = ans;
        return ans;
    };
    return dfs(dfs, 0, init, true, false);
}
```

- 区间 `[l,r]` 用 `dig_dp(r) - dig_dp(l - 1)`
- 数字 `0` 是否计入由 `accept(st,false)` 决定
- 若前导零不应进入自动机，必须在 `trans` 中显式区分 `beg`
- 计数可能超过 `i64` 时改用取模类或高精度

自动机 DP

字符串长度为 `n`，自动机状态数为 `S`
状态： `dp[i][s]` 表示处理前 `i` 个字符后停在自动机状态 `s` 的方案数或最优值

$$dp_{i+1,go(s,c)} \mathrel{+}= dp_{i,s}$$

常见组合

- KMP 自动机：禁止出现一个模式串
- AC 自动机：禁止或统计多个模式串
- DFA：局部规则、余数、连续段长度等状态
- 长度极大且转移与位置无关时，把转移写成矩阵快速幂

DP 套 DP

当题目要求统计“另一个 DP 的结果满足某条件”的对象时，不要直接把整张内层 DP 表作为状态
处理顺序

1. 写出内层 DP
2. 找出只会影响未来转移的信息
3. 把等价内层状态编号
4. 外层在这些编号上继续 DP
5. 状态固定且长度很大时再矩阵化
   典型形式是数位 DP + 自动机状态、轮廓线 + 连通性状态、在线算法状态机化

选型速查

特征	首选模型
只依赖前一位置或少量历史	线性 DP
两个前缀互相匹配	LCS / 编辑距离
最后一步把区间分成两段	区间 DP
只需记已选择集合	状压 DP
上界按数位限制	数位 DP
字符约束只依赖有限状态	自动机 DP
状态依赖形成 DAG	DAG DP
转移式出现滑窗、前缀最值、直线、决策单调	转 12.4 检查优化

12.2 背包家族

定位： 记录容量型 DP 的经典状态、循环方向与常见变形
以下默认最大化问题，不可达值使用

```
constexpr i64 NINF = numeric_limits<i64>::lowest() / 4;
```

统一状态

二维基础状态
`dp[i][j]`：只考虑前 `i` 种物品，容量条件为 `j` 时的最优值
压掉物品维后通常写成 `dp[j]`
初始化先区分两种语义

- 容量不超过 `W`：通常 `dp[0..W] = 0`
- 容量恰好为 `W`：`dp[0] = 0`，其余为 `NINF`

0/1 背包

每件物品至多选一次
二维转移

$$dp_{i,j} = \max(dp_{i-1,j}, dp_{i-1,j-w_i} + v_i)$$

压维后容量必须逆序

```
vector<i64> dp(W + 1);
for (int i = 0; i < n; ++ i) {
    for (int j = W; j >= w[i]; -- j) {
        dp[j] = max(dp[j], dp[j - w[i]] + v[i]);
    }
}
```

- 逆序保证 `dp[j-w[i]]` 仍来自上一层
- 时间 $O(nW)$，空间 $O(W)$

完全背包

每件物品可选任意次

$$dp_j = \max(dp_j, dp_{j-w_i} + v_i)$$

容量正序

```
for (int i = 0; i < n; ++ i) {
    for (int j = w[i]; j <= W; ++ j) {
        dp[j] = max(dp[j], dp[j - w[i]] + v[i]);
    }
}
```

- 正序允许本层刚更新的状态继续使用当前物品
- 时间 $O(nW)$，空间 $O(W)$

多重背包

第 `i` 种物品最多选 `c[i]` 件
朴素转移

$$dp_{i,j} = \max_{0 \le t \le c_i, tw_i \le j} \{dp_{i-1,j-tw_i} + tv_i\}$$

朴素复杂度 $O(W \sum c_i)$

二进制拆分

把 `c` 拆成 `1,2,4,...,rest` 个物品，再做 0/1 背包

```
for (int i = 0; i < n; ++ i) {
    int c = min(cnt[i], W / w[i]);
    for (int k = 1; c; k <<= 1) {
        int t = min(k, c);
        int ww = w[i] * t;
        i64 vv = v[i] * t;
        for (int j = W; j >= ww; -- j) {
            dp[j] = max(dp[j], dp[j - ww] + vv);
        }
        c -= t;
    }
}
```

时间 $O(W\sum\log(c_i+1))$

单调队列优化

固定一件物品 `(w,v,c)`，按容量模 `w` 分组
令 `j = r + kw`

$$ndp_{r+kw}=kv+\max_{k-c\le t\le k}\{dp_{r+tw}-tv\}$$

括号内是长度 `c+1` 的滑动窗口最大值，可用单调队列

```
vector<i64> old = dp;
for (int r = 0; r < w && r <= W; ++ r) {
    deque<int> q;
    for (int k = 0; r + k * w <= W; ++ k) {
        while (!q.empty() && q.front() < k - c) q.pop_front();
        if (old[r + k * w] != NINF) {
            i64 x = old[r + k * w] - 1LL * k * v;
            while (!q.empty()) {
                int t = q.back();
                i64 y = old[r + t * w] - 1LL * t * v;
                if (y > x) break;
                q.pop_back();
            }
            q.push_back(k);
        }
        if (!q.empty()) {
            int t = q.front();
            dp[r + k * w] = old[r + t * w] + 1LL * (k - t) * v;
        } else dp[r + k * w] = NINF;
    }
}
```

- 必须从整层 `old` 转移
- 单件物品处理 $O(W)$，总时间 $O(nW)$

混合背包

每种物品分别属于 0/1、完全或多重
只需按类型选对应循环

```
if (c == -1) {
    for (int j = W; j >= w; -- j) dp[j] = max(dp[j], dp[j - w] + v);
} else if (c == 0) {
    for (int j = w; j <= W; ++ j) dp[j] = max(dp[j], dp[j - w] + v);
} else {
    // 二进制拆分或单调队列
}
```

分组背包

物品分成若干组，每组至多选一件
状态： 处理完前若干组后，容量为 `j` 的最优值

一组内所有选择必须从上一层 `old` 转移

```
for (const auto &g : group) {
    auto old = dp;
    for (int j = 0; j ⩽ W; ++ j) {
        dp[j] = old[j];
        for (auto [w, v] : g) {
            if (j ⩾ w) dp[j] = max(dp[j], old[j - w] + v);
        }
    }
}
```

时间 $O(W\sum|G_i|)$

二维费用背包

每件物品同时消耗两种容量 `a[i]`，`b[i]`
0/1 形式

$$dp_{x,y}=\max(dp_{x,y},dp_{x-a_i,y-b_i}+v_i)$$

两个容量都逆序

```
for (int i = 0; i < n; ++ i) {
    for (int x = A; x ⩾ a[i]; -- x) {
        for (int y = B; y ⩾ b[i]; -- y) {
            dp[x][y] = max(dp[x][y], dp[x - a[i]][y - b[i]] + v[i]);
        }
    }
}
```

时间 $O(nAB)$，空间 $O(AB)$

有依赖的背包

典型条件是选择儿子前必须选择父亲

状态：`dp[u][j]` 表示必须选择 `u`，并在 `u` 子树内恰好使用容量 `j` 的最大价值
初始化

```
dp[u][w[u]] = v[u];
```

合并一个儿子 `v`

$$ndp_{j+k}=\max(ndp_{j+k},dp_{u,j}+dp_{v,k})$$

```
for (int j = min(W, su); j ⩾ 0; -- j) {
    if (dp[u][j] == NINF) continue;
    for (int k = 1; k ⩽ min(W - j, sv); ++ k) {
        if (dp[v][k] == NINF) continue;
        ndp[j + k] = max(ndp[j + k], dp[u][j] + dp[v][k]);
    }
}
```

- 森林可接一个体积价值都为 `0` 的虚根
- 一般上界 $O(nW^2)$，实际可用子树有效容量截断
- 特殊先修结构可能有更强优化，不能无证明套用

恰好装满与不可达状态

最大化时

```
vector<i64> dp(W + 1, NINF);
dp[0] = 0;
```

转移前检查来源可达

```
if (dp[j - w] != NINF) {
    dp[j] = max(dp[j], dp[j - w] + v);
}
```

- `NINF` 不要取 `numeric_limits<i64>::lowest()` 本身再参与加法
- 最小化背包同理改用 `INF`

方案数背包

统计恰好装满的方案数

0/1 背包

```
vector<i64> dp(W + 1);
dp[0] = 1;
for (int i = 0; i < n; ++ i) {
    for (int j = W; j >= w[i]; -- j) {
        dp[j] = (dp[j] + dp[j - w[i]]) % mod;
    }
}
```

完全背包只需把容量改为正序

最优值的方案数

同时维护 `{val,cnt}`
若新值更优则覆盖计数，相等则累加计数

```
if (nv > val[j]) {
    val[j] = nv;
    cnt[j] = cnt[j - w];
} else if (nv == val[j]) {
    cnt[j] = (cnt[j] + cnt[j - w]) % mod;
}
```

计数背包的组合与排列

完全背包计数时，循环顺序决定统计对象

组合数

每种物品可选任意次，选择顺序不同视为同一种方案
物品在外、容量在内：

```
vector<i64> dp(W + 1);
dp[0] = 1;
for (int w : a) {
    for (int j = w; j <= W; ++ j) {
        dp[j] = (dp[j] + dp[j - w]) % mod;
    }
}
```

此时每种方案只会按照固定的物品顺序生成一次

排列数

每种物品可选任意次，选择顺序不同视为不同方案
容量在外、物品在内：

```
vector<i64> dp(W + 1);
dp[0] = 1;
for (int j = 1; j <= W; ++ j) {
    for (int w : a) if (w <= j) {
        dp[j] = (dp[j] + dp[j - w]) % mod;
```

```
    }
}
```

例如物品重量为 `{1,2}`、容量为 `3`

- 组合：`1+1+1`、`1+2`，共 `2` 种
- 排列：`1+1+1`、`1+2`、`2+1`，共 `3` 种

> ⚠ Warning
>
> 若 `a` 中相同重量表示同一种物品，计数组合前应先去重，否则相同重量会被当作不同物品类型

方案恢复

若需要恢复一组 0/1 背包方案，最稳妥的是保留二维 `dp[i][j]`
从 `(n,W)` 逆推

- 若 `dp[i][j] == dp[i-1][j]`，可不选第 `i` 件
- 若 `j ≥ w[i-1]` 且 `dp[i][j] == dp[i-1][j-w[i-1]] + v[i-1]`，可选第 `i` 件
- 有平局时恢复规则决定字典序或是否偏向选择
  只存一维数组时若仍要恢复，需要额外父指针或重新计算

前 K 优解

若题目要求前 `K` 个方案值，可让每个容量维护长度 `K` 的降序数组，再把“不选”和“选当前物品”两列归并

- 时间通常 $O(nWK)$
- 必须先确认题目区分“不同方案”还是“不同价值”
- 相同价值若代表不同方案，不能去重

Subset Sum

只判断某个和是否可达时，状态为布尔值
0/1 形式

$$dp_j \mid= dp_{j-w_i}$$

若容量较大且只做可达性，可用 `bitset` 或动态位集把整层转移改成位移与或

背包选型速查

条件	循环或方法
每件至多一次	容量逆序
每件无限次	容量正序
每件最多 `c` 次	二进制拆分 / 单调队列
每组至多一件	整组从 `old` 转移
两种容量	两维都按 0/1 或完全规则枚举
父子依赖	树上分组合并
恰好装满	仅 `dp[0] = 0`，其余不可达
只求可达性	布尔 DP / bitset
需要方案数	值相等时累加计数
需要方案	二维 DP 或父指针

12.3 树、轮廓线、概率与期望 DP

定位： 记录树上局部状态、换根、网格轮廓状态以及概率与期望方程

树形 DP 通用形态

先任选根，把无根树转成父子关系
常见状态写成 `dp[u][s]`，其中 `s` 只描述 `u` 与父亲、儿子交界处需要保留的信息
处理顺序通常是后序 DFS

```
auto dfs = [&](auto &&self, int u, int fa) -> void {
    // init dp[u]
    for (int v : g[u]) if (v != fa) {
        self(self, v, u);
        // merge dp[v] into dp[u]
    }
};
```

树上最大权独立集

相邻点不能同时选择
状态

- `dp[u][0]`：不选 `u` 的最大权值
- `dp[u][1]`：选择 `u` 的最大权值
  转移

$$dp_{u,0} = \sum_{v} \max(dp_{v,0}, dp_{v,1})$$

$$dp_{u,1} = w_u + \sum_{v} dp_{v,0}$$

答案 `max(dp[root][0], dp[root][1])`
时间 $O(n)$，空间 $O(n)$

树上最小点覆盖

要求每条边至少一个端点被选择
状态

- `dp[u][0]`：不选 `u`
- `dp[u][1]`：选择 `u`
  转移

$$dp_{u,0} = \sum_{v} dp_{v,1}$$

$$dp_{u,1} = w_u + \sum_{v} \min(dp_{v,0}, dp_{v,1})$$

若是无权点覆盖令 `w[u] = 1`

树上最小支配集

每个点必须被自己、父亲或儿子至少一个选中
三状态最常用

- `dp[u][0]`：选择 `u`
- `dp[u][1]`：不选 `u`，由至少一个儿子覆盖
- `dp[u][2]`：不选 `u`，等待父亲覆盖
  对每个儿子 `v`

```
dp[u][0] += min(dp[v][0], dp[v][1], dp[v][2])
base     += min(dp[v][0], dp[v][1])
extra     = min(extra, dp[v][0] - min(dp[v][0], dp[v][1]))
dp[u][1]  = base + extra
dp[u][2]  = base
```

- 叶子的 `dp[u][1]` 不可达
- 根不能处于等待父亲覆盖状态
- 答案 `min(dp[root][0], dp[root][1])`

树上最大匹配

每个点最多属于一条匹配边
可设

- `dp[u][0]`：`u` 不与儿子匹配
- `dp[u][1]`：`u` 与恰好一个儿子匹配
  先令

$$base = \sum_v \max(dp_{v,0}, dp_{v,1})$$

若让 `u-v` 成为匹配边，则 `v` 不能再与自己的儿子匹配

$$dp_{u,1} = \max_v \{base - \max(dp_{v,0}, dp_{v,1}) + dp_{v,0} + w_{u,v}\}$$

$$dp_{u,0} = base$$

无权最大匹配令边权为 `1`

树上背包

当每个子树需要再维护容量维时，状态变成 `dp[u][j]`
每合并一个儿子就是一次分组卷积
完整模型见 12.2 有依赖的背包

换根 DP

目标是求每个点作为根时的答案
基本步骤

1. 任选根做一次向下 DP
2. 再从父亲向儿子传递“树外贡献”
3. 得到所有根的答案

所有点距离和

树边 `u-v` 权值为 `w`
先求

- `siz[u]`：子树大小
- `down[u]`：`u` 到自己子树所有点的距离和
  从 `u` 换根到儿子 `v`

$$ans_v = ans_u + (n - 2siz_v)w$$

原因是 `v` 子树内 `siz[v]` 个点距离减少 `w`，其余点距离增加 `w`

一般换根

若儿子贡献可以结合，常见三种写法

- 可逆运算：总贡献减去当前儿子再传下去
- 维护最大值与次大值：排除一个儿子时取另一个
- 前后缀合并：适用性最广，可排除任意一个儿子

行轮廓状压 DP

网格的一维较短时，只保存当前行或列的边界状态
常见状态

- `mask`：当前行哪些位置被选择、占用或从上一行延伸
- `dp[row][mask]`：处理完前若干行后的方案数或最优值
  典型复杂度取决于合法状态数 `S`，常见为 $O(nS^2)$ 或 $O(nmS)$

网格独立集

每行选择若干格，同行相邻和上下相邻不能同时选择
先预处理合法行状态

```
vector<int> st;
for (int s = 0; s < 1 << m; ++ s) {
    if (!(s & (s << 1))) st.push_back(s);
}
```

两行状态 `a,b` 可相邻当且仅当

```
(a & b) == 0
```

有障碍时还要求

```
(a & block[row]) == 0
```

若斜对角也冲突，再检查 `(a << 1) & b` 与 `(a >> 1) & b`

多米诺骨牌铺满

用 `1 x 2` 骨牌铺满 `n x m`
状态： `mask` 表示当前行被上一行竖放骨牌占用的位置
逐格处理

- 当前位已占用：跳到下一格
- 当前位空：可横放，或竖放并在下一行留下一个占用位
  最终答案必须回到 `mask = 0`
  通常让 `m = min(n,m)`，指数维尽量小

插头 DP

适合 Hamilton 回路、连通块、路径覆盖等需要保存边界连通关系的网格问题
常见轮廓有 `m+1` 个插头位置
单回路常用 2 bit 括号表示

- `0`：无插头
- `1`：左括号
- `2`：右括号

```
struct Plug {
    u64 s{};
    int get(int p) const { return s >> (p * 2) & 3; }
    void set(int p, int x) {
        s &= ~(3ULL << (p * 2));
        s |= u64(x) << (p * 2);
    }
    int right(int p, int n) const {
        int b = 0;
        for (int i = p; i < n; ++ i) {
            int x = get(i);
            if (x == 1) ++ b;
            else if (x == 2) -- b;
            if (!b) return i;
        }
        return -1;
    }
    int left(int p) const {
        int b = 0;
        for (int i = p; i >= 0; -- i) {
            int x = get(i);
            if (x == 2) ++ b;
            else if (x == 1) -- b;
            if (!b) return i;
        }
        return -1;
    }
};
```

- 插头 DP 没有一套通用完整转移，闭环时机、障碍与连通要求都会改变规则

- 单回路通常只允许在最后一个合法格闭合最终回路
- 若 `2*(m+1) > 64`，不能继续用 `u64` 保存全部插头

三进制轮廓状态

若每个边界位置只需独立保存 `0/1/2`，可用三进制整数编码

$$state = \sum_i d_i 3^i$$

- 适合每格独立三状态
- 不适合直接表示任意连通块等价关系

Steiner Tree 状压 DP

给定 `k` 个终端点，求连接全部终端的最小子图
状态：`dp[s][v]` 表示连接终端集合 `s` 且当前汇合在 `v` 的最小代价
同一点合并两个终端集合

$$dp_{s,v} = \min_{t \subset s, t \neq 0} \{dp_{t,v} + dp_{s \setminus t,v}\}$$

随后对每个 `s` 做一次多源最短路松弛
典型复杂度 $O(3^k n + 2^k (m+n)\log n)$
具体图论实现统一放 Steiner Tree 板子，此处只保留状态模型

概率 DP

若状态转移无环，常正向维护到达概率
状态：`dp[u]` 表示到达状态 `u` 的总概率
对边 `u → v`，转移概率为 `p`

$$dp_v \mathrel{+}= dp_u p$$

- 多条路径到同一状态时概率相加
- 依赖图是 DAG 时按拓扑序转移
- 概率和是否等于 `1` 由题意决定，失败或停留状态要显式建模

期望的线性性

若总贡献可以写成：

$$X = \sum_i X_i$$

则无论各随机变量是否独立，都有：

$$E[X] = \sum_i E[X_i]$$

特别地，令指示变量：

$$X_i = [A_i]$$

则：

$$E[X_i] = P(A_i)$$

因此：

$$E\left[\sum_i [A_i]\right] = \sum_i P(A_i)$$

常见识别

题目询问下列量的期望时，优先尝试拆成单个元素或事件的贡献：

- 满足条件的元素个数
- 被覆盖的位置数
- 逆序对数量
- 边、点、区间产生的总贡献
  例如随机排列的逆序对数：

$$X = \sum_{i<j} [p_i > p_j]$$

每一对成为逆序对的概率均为 $\frac{1}{2}$，因此：

$$E[X] = \binom{n}{2} \frac{1}{2}$$

> Tip
>
> 期望可加性不要求独立，但 $E[XY] = E[X]E[Y]$ 一般需要额外条件

若目标是最大值、最小值、首次到达时间等不能直接拆成加法贡献的量，再考虑概率 DP、期望 DP 或尾和公式

期望 DP

从状态 `u` 出发到结束的期望代价为 `E[u]`
基本方程

$$E_u = c_u + \sum_v p_{u,v} E_v$$

若除自环外其余依赖无环，设自环概率为 `p`

$$E_u = c_u + pE_u + \sum_{v \neq u} p_{u,v} E_v$$

移项得到

$$E_u = \frac{c_u + \sum_{v \neq u} p_{u,v} E_v}{1 - p}$$

- 终止状态通常 `E = 0`
- `p = 1` 且每步有正代价时期望发散
- 一般有环期望要建立 `(I-P)E=c` 线性方程组，不可强行按拓扑顺序递推
- 浮点无穷可写 `numeric_limits<ld>::infinity()`

概率与期望方向判断

- 问“到达某状态的概率”时优先尝试正向分配概率
- 问“从当前状态到结束还需要多少”时优先尝试逆向期望方程
- 有自环先尝试移项消掉
- 有一般环再考虑高斯消元、吸收马尔可夫链或题目特有结构

选型速查

结构	典型状态
相邻点不能同时选	树上独立集 `dp[u][0/1]`
每条边至少覆盖一个端点	树上点覆盖 `dp[u][0/1]`
每个点必须被邻近选点覆盖	树上支配集三状态
每个点最多匹配一条边	树上匹配两状态
求所有根答案	换根 DP
网格短边很小	行轮廓状压
边界还要记录连通关系	插头 DP
到达概率	正向概率 DP
剩余期望代价	逆向期望方程

12.4 DP 优化与选型

定位： 从转移式的结构判断能否把枚举候选的复杂度降下来

优化选择表

转移特征	方法	常见变化
只依赖上一层或前几层	滚动数组	空间降一维
区间和、区间固定代价反复出现	前缀和 / 差分	单次 $O(n) \to O(1)$
候选只在滑动窗口内	单调队列	$O(nk) \to O(n)$
候选是前缀最值或偏序最值	BIT / 线段树	$O(n^2) \to O(n \log n)$
候选可写成直线在 `x` 处取最值	CHT / Li Chao	$O(n^2) \to O(n)$ 或 $O(n \log n)$
一层最优决策随位置单调	分治优化	常见 $O(Kn^2) \to O(Kn \log n)$
区间决策满足 Knuth 单调	Knuth	$O(n^3) \to O(n^2)$
左半状态批量贡献右半状态	CDQ	常见 $O(n^2) \to O(n \log^2 n)$
恰好选 `K` 个且关于数量凸或凹	WQS 二分	去掉数量维再二分罚金
固定小状态且转移与位置无关	矩阵快速幂	$O(nS^2) \to O(S^3 \log n)$
布尔可达性批量平移	bitset	约降一个机器字因子
转移本质是卷积	FFT / NTT	$O(n^2) \to O(n \log n)$
一元凸分段线性函数反复加折线	Slope Trick	每次 $O(\log n)$

先改状态再改转移，一个多余状态维通常比任何数据结构都贵

数据结构优化

若转移是前缀、后缀或偏序范围内的最值

$$dp_i = w_i + \max_{j<i, a_j<a_i} dp_j$$

把 `a` 离散化后，用 BIT 或线段树维护键值前缀最大值
典型应用

- 带权 LIS
- 二维偏序 DP
- 坐标范围内的最优前驱
  注意同一第一关键字不能互相转移时，要分组查询后再批量更新

斜率优化 DP

先把转移严格化成

$$dp_i = c_i + \operatorname{opt}_{j<i}(k_j x_i + b_j)$$

此时

- 历史状态 `j` 对应直线 `y = k_j x + b_j`
- 当前状态 `i` 对应查询横坐标 `x_i`

单调斜率 + 单调查询

支持最大值或最小值、斜率递增或递减、查询点递增或递减

```
template<bool MAX = true, bool K_INC = true, bool X_INC = true>
struct MonoCHT {
    struct Line {
        i64 k, b;
        i128 get(i64 x) const { return (i128)k * x + b; }
    };
    vector<Line> q;
    int h{};
    MonoCHT() = default;
    explicit MonoCHT(int n) { q.reserve(n); }
    static bool better(const Line &a, const Line &b, i64 x) {
        if constexpr (MAX) return a.get(x) ≥ b.get(x);
        else return a.get(x) ≤ b.get(x);
```

```
    }
    static bool bad(const Line &a, const Line &b, const Line &c) {
        i128 x = ((i128)b.b - a.b) * ((i128)b.k - c.k);
        i128 y = ((i128)c.b - b.b) * ((i128)a.k - b.k);
        if constexpr (MAX == K_INC) return x >= y;
        else return x <= y;
    }
    void add(i64 k, i64 b) {
        Line c{k, b};
        if ((int)q.size() > h && q.back().k == k) {
            if (better(q.back(), c, 0)) return;
            q.pop_back();
        }
        while ((int)q.size() - h >= 2 && bad(q[q.size() - 2], q.back(), c)) q.pop_back();
        q.push_back(c);
    }
    i128 query(i64 x) {
        static constexpr bool FRONT = X_INC == (MAX == K_INC);
        if constexpr (FRONT) {
            while ((int)q.size() - h >= 2 && better(q[h + 1], q[h], x)) ++ h;
            return q[h].get(x);
        } else {
            while ((int)q.size() - h >= 2 && better(q[q.size() - 2], q.back(), x)) q.pop_back();
            return q.back().get(x);
        }
    }
};
```

模板参数

- `MAX = true/false`：最大值 / 最小值
- `K_INC = true/false`：斜率不降 / 不升
- `X_INC = true/false`：查询 `x` 不降 / 不升

两种单调性满足时 `add/query` 均摊 $O(1)$

单调斜率 + 任意查询

查询点不单调时，凸包保留完整并二分最优直线

```
template<bool MAX = true, bool K_INC = true>
struct BinCHT {
    struct Line {
        i64 k, b;
        i128 get(i64 x) const { return (i128)k * x + b; }
    };
    vector<Line> q;
    BinCHT() = default;
    explicit BinCHT(int n) { q.reserve(n); }
    static bool better(i128 a, i128 b) {
        if constexpr (MAX) return a >= b;
        else return a <= b;
    }
    static bool bad(const Line &a, const Line &b, const Line &c) {
        i128 x = ((i128)b.b - a.b) * ((i128)b.k - c.k);
        i128 y = ((i128)c.b - b.b) * ((i128)a.k - b.k);
        if constexpr (MAX == K_INC) return x >= y;
        else return x <= y;
    }
    void add(i64 k, i64 b) {
        Line c{k, b};
        if (!q.empty() && q.back().k == k) {
            if (better(q.back().b, b)) return;
            q.pop_back();
        }
        while (q.size() >= 2 && bad(q[q.size() - 2], q.back(), c)) q.pop_back();
        q.push_back(c);
    }
    i128 query(i64 x) const {
        int l = 0, r = q.size() - 1;
        while (l < r) {
```

```
            int m = (l + r) / 2;
            if (better(q[m + 1].get(x), q[m].get(x))) l = m + 1;
            else r = m;
        }
        return q[l].get(x);
    }
};
```

查询 $O(\log n)$

CHT 使用条件

- 斜率必须按 `K_INC` 指定方向加入
- `MonoCHT` 的查询必须按 `X_INC` 指定方向进行
- 相同斜率只保留截距更优的直线
- `query()` 前必须至少有一条线
- `i64` 乘法与交叉乘积先提升到 `i128`
- 斜率也不单调时优先考虑 Li Chao、CDQ + CHT 或重新排序
- 查询不单调但斜率单调时用 `BinCHT`
- 浮点版不能直接照搬整数交叉乘，需统一相对误差比较

Li Chao Tree

适合

- 斜率任意顺序加入
- 查询点任意顺序
- 横坐标范围已知或可离散
  单次加入和查询通常 $O(\log C)$ 或 $O(\log n)$
  具体实现统一使用数据结构章节的 Li Chao Tree，不在 DP 文件重复维护

Monge / 四边形不等式与决策单调

很多 DP 优化的关键并不是某个数据结构，而是证明最优决策位置单调
考虑：

$$cur_i = \min_j \{pre_j + w(j,i)\}$$

记最小的最优决策为：

$$opt_i = \min \arg\min_j \{pre_j + w(j,i)\}$$

若代价满足 Monge 性质：

$$w(a,c) + w(b,d) \le w(a,d) + w(b,c)$$

其中：

$$a \le b, \quad c \le d$$

则在常见合法区间下通常可推出：

$$opt_i \le opt_{i+1}$$

加入只依赖 `j` 的项 `pre[j]` 不会破坏 Monge 性质

优化入口

若已经证明：

$$opt_i \le opt_{i+1}$$

则形如：

$$cur_i = \min_j \{pre_j + w(j,i)\}$$

的分层 DP 应优先检查分治优化
若区间 DP：

$$dp_{l,r} = \min_{l<k<r}\{dp_{l,k} + dp_{k,r} + w(l,r)\}$$

还能得到更强的：

$$opt_{l,r-1} \le opt_{l,r} \le opt_{l+1,r}$$

则检查 Knuth 优化
常见充分条件是 `w` 同时满足四边形不等式与区间包含单调性，但具体题目应按实际状态证明

> ⚠ Warning
>
> 不要因为代价看起来“凸”或样例中的决策位置单调就直接套优化
> 分治优化真正需要的是决策单调，Knuth 需要更强的决策范围关系

最大化问题可对目标整体取负后转成最小化统一判断

分治优化 DP

典型分层转移

$$dp_{t,i} = \min_{k\le i}\{dp_{t-1,k} + w(k,i)\}$$

若一层的最优决策 `opt[i]` 随 `i` 单调不下降，可递归计算中点，只搜索允许的决策区间

```
template<class Cost>
void dc_dp(int l, int r, int ql, int qr,
           const vector<i64> &pre, vector<i64> &cur, Cost cost) {
    if (l > r) return;
    int m = (l + r) / 2;
    pair<i64, int> best{INF, -1};
    int R = min(m, qr);
    for (int k = ql; k <= R; ++ k) {
        if (pre[k] == INF) continue;
        best = min(best, {pre[k] + cost(k, m), k});
    }
    cur[m] = best.first;
    int p = best.second;
    dc_dp(l, m - 1, ql, p, pre, cur, cost);
    dc_dp(m + 1, r, p, qr, pre, cur, cost);
}
```

- 若要求 `k < i`，把 `R` 改为 `min(m - 1, qr)`
- 每层常见 $O(n \log n)$，总计 $O(Kn \log n)$
- 有些标准结构可以进一步分析到每层 $O(n)$
- 决策单调必须证明或引用已知结论

四边形不等式与 Knuth 优化

区间 DP 形如

$$dp_{l,r} = \min_{l<k<r}\{dp_{l,k} + dp_{k,r} + w(l,r)\}$$

若满足

$$opt[l][r-1] \le opt[l][r] \le opt[l+1][r]$$

则只需在这个小区间枚举 `k`

```
template<class Cost>
i64 knuth(int n, Cost cost) {
    vector dp(n + 1, vector<i64>(n + 1, INF));
    vector opt(n + 1, vector<int>(n + 1));
    for (int i = 0; i <= n; ++ i) {
```

```
        dp[i][i] = 0;
        opt[i][i] = i;
        if (i < n) {
            dp[i][i + 1] = 0;
            opt[i][i + 1] = i + 1;
        }
    }
    for (int len = 2; len ⩽ n; ++ len) {
        for (int l = 0; l + len ⩽ n; ++ l) {
            int r = l + len;
            int L = max(l + 1, opt[l][r - 1]);
            int R = min(r - 1, opt[l + 1][r]);
            for (int k = L; k ⩽ R; ++ k) {
                i64 x = dp[l][k] + dp[k][r] + cost(l, r);
                if (x < dp[l][r]) dp[l][r] = x, opt[l][r] = k;
            }
        }
    }
    return dp[0][n];
}
```

时间 $O(n^2)$，空间 $O(n^2)$
不能因为转移长得像区间 DP 就默认满足 Knuth 条件

CDQ 分治优化 DP

适合时间方向固定为 `j < i`，但候选加入顺序不适合直接在线维护

```
template<class Relax>
void cdq(int l, int r, Relax relax) {
    if (r - l ⩽ 1) return;
    int m = (l + r) / 2;
    cdq(l, m, relax);
    relax(l, m, r);
    cdq(m, r, relax);
}
```

`relax(l,m,r)` 只允许用 `[l,m)` 已确定状态更新 `[m,r)`
内部常配合

- BIT / 线段树
- CHT
- 排序扫描
  若每层 `relax` 为 $O(n\log n)$，总复杂度常见 $O(n\log^2 n)$

WQS 二分

用于“恰好选 `K` 个”的最优化，并且最优值关于选择数量具有离散凸或凹性质
最大化时给每次选择加罚金 `-lambda`
原目标

$$\max\{value \mid cnt = K\}$$

调整后

$$\max(value - \lambda cnt)$$

一次 `calc(lambda)` 同时返回调整后最优值与对应选择数
若 `lambda` 增大时最优 `cnt` 单调不增，就可二分 `lambda`
恢复原答案

$$ans = adjusted + \lambda K$$

关键点

- 同调整值时的 `cnt` tie-break 必须固定，通常最大化问题偏向更大的 `cnt`
- 二分范围必须覆盖最优罚金
- 并非所有带 `K` 限制的 DP 都可 WQS

bitset 优化

布尔可达性转移

$$dp \leftarrow dp \vee (dp \ll w)$$

适合 0/1 Subset Sum 等只关心可达性的状态
复杂度从 $O(nW)$ 降到约 $O(nW/word)$
动态位集实现统一放位运算章节

SOS DP（子集 / 超集 DP）

对所有状态 `S` 同时统计其全部子集或超集的信息，也称快速 Zeta 变换
典型形式：

$$f_S = \sum_{T \subseteq S} a_T$$

或：

$$f_S = \sum_{T \supseteq S} a_T$$

直接对每个 `S` 枚举所有子集总复杂度为：

$$O(3^n)$$

SOS DP 利用每一位依次加入贡献，将复杂度降为：

$$O(n2^n)$$

子集 Zeta 变换

变换后：

$$f_S = \sum_{T \subseteq S} a_T$$

```
template<class T>
void sub_zeta(vector<T> &f, int n) {
    for (int i = 0; i < n; ++ i) {
        for (int s = 0; s < 1 << n; ++ s) {
            if (s >> i & 1) f[s] += f[s ^ (1 << i)];
        }
    }
}
```

例如 `a[S]` 表示状态 `S` 的出现次数，变换后 `a[S]` 表示所有 `T ⊆ S` 的出现次数之和

子集 Möbius 逆变换

将：

$$f_S = \sum_{T \subseteq S} a_T$$

还原为原数组 `a`

```
template<class T>
void sub_mobius(vector<T> &f, int n) {
    for (int i = 0; i < n; ++ i) {
        for (int s = 0; s < 1 << n; ++ s) {
            if (s >> i & 1) f[s] -= f[s ^ (1 << i)];
        }
    }
}
```

即：

$$a_S = \sum_{T \subseteq S} (-1)^{|S|-|T|} f_T$$

超集 Zeta 变换

变换后：

$$f_S = \sum_{T \supseteq S} a_T$$

```
template<class T>
void sup_zeta(vector<T> &f, int n) {
    for (int i = 0; i < n; ++ i) {
        for (int s = 0; s < 1 << n; ++ s) {
            if (!(s >> i & 1)) f[s] += f[s | 1 << i];
        }
    }
}
```

例如 `a[S]` 表示状态 `S` 的出现次数，变换后 `a[S]` 表示所有 `T ⊇ S` 的出现次数之和

超集 Möbius 逆变换

```
template<class T>
void sup_mobius(vector<T> &f, int n) {
    for (int i = 0; i < n; ++ i) {
        for (int s = 0; s < 1 << n; ++ s) {
            if (!(s >> i & 1)) f[s] -= f[s | 1 << i];
        }
    }
}
```

最大值 / 最小值版本

若只需要：

$$f_S = \max_{T \subseteq S} a_T$$

可将加法改为 `max`

```
template<class T>
void sub_max(vector<T> &f, int n) {
    for (int i = 0; i < n; ++ i) {
        for (int s = 0; s < 1 << n; ++ s) {
            if (s >> i & 1) f[s] = max(f[s], f[s ^ (1 << i)]);
        }
    }
}
```

类似地可得到：

- 子集最小值：`max` 改为 `min`
- 超集最大值：使用超集 Zeta 的枚举方向
- 超集最小值：同时改为 `min`
  此类 `max / min` 变换通常不可通过简单逆变换恢复原数组

与补集结合

全集：

```
int all{ (1 << n) - 1 };
```

若已经对子集和做过 `sub_zeta`，则与 `S` 不相交的所有状态之和为：

```
f[all ^ s]
```

因为：

$$T \cap S = \varnothing \iff T \subseteq \overline{S}$$

类似地，包含 `S` 的所有状态直接使用超集 Zeta 后的 `f[S]`

OR 卷积

若：

$$c_S = \sum_{A \cup B = S} a_A b_B$$

对子集 Zeta 变换有：

$$\hat{c}_S = \hat{a}_S \hat{b}_S$$

因此可以：

```
sub_zeta(a, n);
sub_zeta(b, n);
for (int s = 0; s < 1 << n; ++ s) a[s] *= b[s];
sub_mobius(a, n);
```

得到 OR 卷积，复杂度 $O(n2^n)$

AND 卷积

若：

$$c_S = \sum_{A \cap B = S} a_A b_B$$

使用超集 Zeta 变换：

```
sup_zeta(a, n);
sup_zeta(b, n);
for (int s = 0; s < 1 << n; ++ s) a[s] *= b[s];
sup_mobius(a, n);
```

同样为 $O(n2^n)$

识别

出现以下形式时优先考虑 SOS DP：

$$f_S = \sum_{T \subseteq S} a_T \]\[f_S = \sum_{T \supseteq S} a_T$$

或需要对所有 `S` 批量查询：

- `S` 的所有子集贡献
- `S` 的所有超集贡献
- 与 `S` 不相交的状态贡献
- OR / AND 卷积
  若对每个状态枚举：

```
for (int t = s; ; t = (t - 1) & s)
```

且转移只是在对子集信息做可合并的统计，应检查能否改为 SOS DP

与子集划分 DP 的区别

子集划分 DP：

$$dp_S = \min_{T \subseteq S} \{dp_{S \setminus T} + w_T\}$$

转移同时依赖 `T` 与 `S \ T`，普通 SOS DP 通常不能直接将其从 $O(3^n)$ 降为 $O(n2^n)$
SOS DP 适合的是对所有子集或超集做统一的加法、计数、最大值、最小值等聚合
更一般的：

$$c_S = \sum_{A \sqcup B = S} a_A b_B$$

属于子集卷积，不等同于普通 SOS DP

复杂度

四种 Zeta / Möbius 变换均为：

- 时间复杂度 $O(n2^n)$
- 空间复杂度 $O(2^n)$
- 原地变换无需额外数组

矩阵快速幂加速

若状态数 `S` 固定且每一步使用同一个线性转移

$$v_{i+1} = v_i T$$

则

$$v_n = v_0 T^n$$

- 普通 DP $O(nS^2)$
- 矩阵快速幂 $O(S^3 \log n)$
- `S` 大时矩阵未必更快
- 稀疏转移可考虑稀疏矩阵、线性递推或 Bostan-Mori
  矩阵实现统一使用线性代数章节

卷积优化

若转移出现

$$dp_i = \sum_{j=0}^{i} a_j b_{i-j}$$

或分治后能把跨区间贡献写成卷积，可考虑 FFT / NTT
典型方向

- 计数 DP
- 生成函数
- 分治卷积
- 在线卷积
  卷积实现统一使用多项式章节

Slope Trick（凸分段线性函数 DP）

维护关于变量 `x` 的一元凸分段线性函数，适合 DP 状态本身是一整个凸函数的情况
常见操作：

$$f(x) \leftarrow f(x) + (a - x)_+$$

$$f(x) \leftarrow f(x) + (x - a)_+$$

$$f(x) \leftarrow f(x) + |x - a|$$

以及：

$$g(x) = \min_{x-r \le y \le x-l} f(y)$$

通常只需维护函数的最小值以及斜率发生变化的位置，可用两个堆实现

与斜率优化 DP 的区别

斜率优化维护的是一组直线：

$$\text{opt}_j(k_j x + b_j)$$

Slope Trick 维护的是一个完整的凸分段线性函数：

$$f(x)$$

识别时：

- 历史状态 `j` 能整理成 `k_jx_i+b_j` → CHT / Li Chao
- `dp_i(x)` 本身是关于 `x` 的凸折线，并不断加入 `|x-a|` 或 $(x-a)_+$ → Slope Trick

```
template<class T = i64>
struct SlopeTrick {
```

```
    T ans{}, dl{}, dr{};
    priority_queue<T> l;
    priority_queue<T, vector<T>, greater<T>> r;
    T top_l() const { return l.empty() ? numeric_limits<T>::lowest() / 4 : l.top() + dl; }
    T top_r() const { return r.empty() ? numeric_limits<T>::max() / 4 : r.top() + dr; }
    void push_l(T x) { l.push(x - dl); }
    void push_r(T x) { r.push(x - dr); }
    void add_l(T a) {
        if (!r.empty() && a > top_r()) {
            ans += a - top_r();
            push_r(a);
            T x = top_r();
            r.pop();
            push_l(x);
        } else push_l(a);
    }
    void add_r(T a) {
        if (!l.empty() && a < top_l()) {
            ans += top_l() - a;
            push_l(a);
            T x = top_l();
            l.pop();
            push_r(x);
        } else push_r(a);
    }
    void add_abs(T a) { add_l(a), add_r(a); }
    void shift(T a, T b) { dl += a, dr += b; }
    T get() const { return ans; }
};
```

`shift(a,b)` 对应

$$f_{new}(x) = \min_{x-b \le y \le x-a} f_{old}(y)$$

且要求 `a ≤ b`
每次加折线 $O(\log n)$，平移与查询最小值 $O(1)$

赛场检查顺序

---

1. 状态是否有多余维度
2. 能否滚动数组或原地转移
3. 区间量能否前缀和预处理
4. 候选是否形成滑动窗口
5. 候选是否只是前缀 / 后缀最值
6. 转移能否整理成 `k*x+b`
7. 最优决策是否单调
8. 区间 DP 是否满足 Knuth 条件
9. 数量限制是否具有 WQS 所需凸性或凹性
10. 布尔状态能否 bitset
11. 转移是否固定线性，可否矩阵快速幂
12. 是否出现卷积结构

转移特征速查

转移特征	优先检查
固定区间内取最值	单调队列
前缀 / 后缀范围取最值	BIT / 线段树
$\mathrm{opt}(kx+b)$	CHT / Li Chao
最优决策位置单调	分治优化
$opt_{l,r-1} \le opt_{l,r} \le opt_{l+1,r}$	Knuth
恰好选 `K` 个且数量可被罚值控制	WQS
状态是一元凸分段线性函数	Slope Trick
每个状态枚举所有子集	SOS / 子集卷积

转移特征	优先检查
固定小状态、步数极大	矩阵快速幂
转移是卷积形式	FFT / NTT

13. 搜索、随机化与博弈论

13.1 启发式搜索与指数级搜索

A*

适用于边权非负的隐式状态图
`nxt(s)` 返回所有 `{下一状态，代价}`，`h(s)` 返回到目标的代价下界，`goal(s)` 判断目标状态
复杂度最坏指数级；空间与实际访问状态数同阶

```
template<class S, class Next, class Heu, class Goal, class Hash = hash<S>>
optional<i64> astar(const S &st, Next nxt, Heu h, Goal goal, Hash hs = Hash()) {
    struct Node {
        i128 f;
        i64 d;
        S s;
    };
    auto cmp = [](const Node &a, const Node &b) { return a.f != b.f ? a.f > b.f : a.d < b.d; };
    priority_queue<Node, vector<Node>, decltype(cmp)> q(cmp);
    unordered_map<S, i64, Hash> dis(0, hs);
    constexpr i64 INF{ numeric_limits<i64>::max() };
    dis[st] = 0;
    q.push({h(st), 0, st});
    while (!q.empty()) {
        Node cur = q.top();
        q.pop();
        auto it = dis.find(cur.s);
        if (it == dis.end() || it->second != cur.d) continue;
        if (goal(cur.s)) return cur.d;
        for (auto [v, w] : nxt(cur.s)) {
            if (w > INF - cur.d) continue;
            i64 nd = cur.d + w;
            auto jt = dis.find(v);
            if (jt != dis.end() && jt->second <= nd) continue;
            dis[v] = nd;
            q.push({i128(nd) + h(v), nd, move(v)});
        }
    }
    return nullopt;
}
```

- `astar(st,nxt,h,goal)`：返回从 `st` 到任意目标状态的最小代价；不可达返回 `nullopt`
- `nxt(s)`：返回 `{状态，边权}` 序列，要求边权非负
- `h(s)`：从 `s` 到目标真实最短距离的下界；要求所有目标满足 `h(goal)=0`
- `goal(s)`：判断 `s` 是否为目标
- `S` 需要支持相等比较和哈希；特殊状态可传自定义 `Hash`

注意事项

- `h(s)=0` 时退化为 Dijkstra
- `h` 必须可采纳，即不能高估真实剩余代价，否则可能剪掉最优解
- 模板允许状态重新入队，因此可处理可采纳但不一致的启发式
- 启发式越接近真实距离通常越快，但计算 `h` 本身也不能过重
- g 距离与返回值使用 i64，f=g+h 使用 i128；超过 i64 上界的非负路径不再扩展。因此 nullopt 表示不可达或最短距离超出 i64 范围。h 返回 i64 范围内的整数；同 f 时优先较大 g，不改变最优性
- 若需要恢复方案，在松弛时额外记录父状态和操作

常见用法

```
auto ans = astar(st,
    [&](const State &s) {
        vector<pair<State, i64>> res;
        // 生成后继状态
        return res;
    },
    [&](const State &s) {
        return heuristic(s);
    },
    [&](const State &s) {
        return s == target;
    });
if (ans) cout << *ans << '\n';
```

IDA*

适用于单位代价、答案深度较小且状态无法全部存储的搜索
`gen(s,last)` 返回可选操作编号，状态需支持 `apply(op)` / `undo(op)`
复杂度最坏指数级，路径空间为 $O(d)$；还需计入状态及递归各层生成的操作列表，若每层至多 b 个操作则为 $O(S+bd)$，其中 d 为当前搜索深度

```
template<class S, class Heu, class Gen, class Goal>
optional<vector<int>> ida(S st, Heu h, Gen gen, Goal goal, int ub) {
    vector<int> path;
    constexpr i64 INF{ numeric_limits<i64>::max() };
    auto dfs = [&](auto &&self, S &s, i64 dep, i64 bnd) -> i64 {
        i64 f = dep + h(s);
        if (f > bnd) return f;
        if (goal(s)) return -1;
        i64 nb = INF;
        int last = path.empty() ? -1 : path.back();
        for (int op : gen(s, last)) {
            s.apply(op);
            path.push_back(op);
            i64 t = self(self, s, dep + 1, bnd);
            if (t == -1) return -1;
            nb = min(nb, t);
            path.pop_back();
            s.undo(op);
        }
        return nb;
    };
    for (i64 bnd = h(st); bnd <= ub;) {
        S s = st;
        i64 nb = dfs(dfs, s, 0, bnd);
        if (nb == -1) return path;
        if (nb == INF) break;
        bnd = nb;
    }
    return nullopt;
}
```

- `ida(st,h,gen,goal,ub)`：在答案不超过 `ub` 时返回操作序列，否则返回 `nullopt`
- `h(s)`：从当前状态到目标至少还需多少步，返回非负 int，且不能高估；目标处为 0。深度与阈值中间量用 i64，避免 dep+h 溢出
- `gen(s,last)`：返回当前可执行操作；可根据 `last` 排除立即撤销的反向操作
- `S::apply(op)` / `S::undo(op)`：执行、撤销一次操作

注意事项

- 每轮不是简单令深度界加一，而是跳到本轮所有越界状态中最小的 `dep+h`
- `h` 越强，实际搜索量通常越小；若 `h=0`，基本退化为迭代加深 DFS
- `gen` 应优先安排更可能接近答案的操作，并尽量去掉立即反向、重复和对称状态
- 模板默认每步代价为 `1`；带权状态搜索优先使用 A*
- `ub` 必须覆盖可能答案；若题目无自然上界，可根据题意自行扩大

常见用法

```
auto ans = ida(st,
    [&](const State &s) { return heuristic(s); },
```

```
    [&](const State &s, int last) {
        vector<int> ops;
        // 生成操作，并根据 last 剪去反向操作
        return ops;
    },
    [&](const State &s) { return s == target; },
    100);
```

最大团

使用位压缩、贪心染色上界与分支限界，支持至多 `64` 个点。最大独立集等价于补图最大团
复杂度最坏指数级；每个搜索节点的贪心染色为 $O(n^2)$ 的位运算级处理，`n ≤ 64`

```
struct MaxClique {
    int n, best{};
    u64 ans{};
    vector<u64> g;
    explicit MaxClique(int n_) : n(n_), g(n) {}
    void add(int u, int v) {
        if (u == v) return;
        g[u] |= 1ULL << v;
        g[v] |= 1ULL << u;
    }
    void dfs(u64 p, int sz, u64 cur) {
        if (!p) {
            if (sz > best) best = sz, ans = cur;
            return;
        }
        array<int, 64> v, c;
        int m = 0, col = 0;
        u64 rem = p;
        while (rem) {
            ++ col;
            u64 q = rem;
            while (q) {
                int x = __builtin_ctzll(q);
                u64 b = 1ULL << x;
                v[m] = x;
                c[m ++] = col;
                rem ^= b;
                q ^= b;
                q &= ~g[x];
            }
        }
        for (int i = m - 1; i >= 0; -- i) {
            if (sz + c[i] <= best) return;
            int x = v[i];
            u64 b = 1ULL << x;
            if (!(p & b)) continue;
            dfs(p & g[x], sz + 1, cur | b);
            p ^= b;
        }
    }
    pair<int, vector<int>> solve() {
        best = 0;
        ans = 0;
        u64 p = n == 64 ? ~0ULL : (1ULL << n) - 1;
        dfs(p, 0, 0);
        vector<int> res;
        for (int i = 0; i < n; ++ i) if (ans >> i & 1ULL) res.push_back(i);
        return {best, res};
    }
};
```

- `MaxClique(n)`：建立 `n` 个点的无向图，要求 `0 ≤ n ≤ 64`
- `add(u,v)`：加入无向边
- `solve()`：返回 `{最大团大小，最大团点集}`
- 最大独立集：对原图取补图后调用最大团

分支限界

指数级搜索最重要的是尽早获得较优可行解，并使用正确方向的界剪枝：

- 最大化：维护当前最好答案，若 `当前值 + 乐观上界 ⩽ best` 则剪枝
- 最小化：若 `当前值 + 乐观下界 ⩾ best` 则剪枝
- 优先搜索更可能得到优解的分支，可以更早收紧 `best`
- 固定变量顺序、按度数排序、强制选择、对称性消除、支配关系都可能显著减少搜索树
- 上界或下界必须保证正确，界太弱只影响速度，界错误会直接产生错误答案

注意事项

- 使用分支限界搜索
- `sz + c[i] ⩽ best` 时剪枝，其中 `c[i]` 是贪心染色得到的最大团上界
- 模板仅适用于无向简单图
- 贪心染色把候选点分为若干独立集，因此颜色数是候选子图最大团大小的上界
- `n ⩽ 64` 时位运算版本非常方便；更大规模应改动态 `bitset` 或其他最大团实现
- 最大团仍是指数级算法，复杂图上的实际速度高度依赖结构和搜索顺序

常见用法

```
MaxClique mc(n);
for (auto [u, v] : edges) mc.add(u, v);
auto [sz, ver] = mc.solve();
```

最大独立集：

```
MaxClique mc(n);
vector<vector<bool>> e(n, vector<bool>(n));
for (auto [u, v] : edges) e[u][v] = e[v][u] = true;
for (int i = 0; i < n; ++ i) {
    for (int j = i + 1; j < n; ++ j) {
        if (!e[i][j]) mc.add(i, j);
    }
}
auto [sz, ver] = mc.solve();
```

13.2 随机化与局部搜索

随机数

```
mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
template<class T>
T rnd(T l, T r) {
    if constexpr (is_integral_v<T>) return uniform_int_distribution<T>(l, r)(rng);
    else return uniform_real_distribution<T>(l, r)(rng);
}
```

接口说明

- `rng`：64 位 Mersenne Twister 随机数生成器
- `rnd<T>(l,r)`：
  - 整数类型：均匀生成 $[l,r]$
  - 浮点类型：均匀生成 $[l,r)$
- 随机打乱直接使用 `shuffle(a.begin(),a.end(),rng)`

注意事项

- 调试时建议使用固定种子，便于复现：`rng.seed(123456789);`
- `rand()` 随机范围和质量较差，不用于 64 位随机数、随机打乱或随机哈希
- 若随机算法单次失败概率不超过 p，独立运行 t 次仍全部失败的概率不超过 p^t
- Monte Carlo 算法可能返回错误答案；Las Vegas 算法答案始终正确，只随机运行时间
- 若随机得到的候选可以快速验证，优先考虑 Las Vegas：不断生成候选直到验证成功
- 随机打乱只能改善依赖输入顺序的期望性能，不能修复算法本身错误的最坏复杂度

常见用法

随机整数：

```
int x = rnd(0, n - 1);
i64 y = rnd<i64>(1, (i64)1E18);
```

随机浮点数：

```
double x = rnd(0.0, 1.0);
```

随机下标：

```
int p = rnd(0, (int)a.size() - 1);
```

随机打乱：

```
shuffle(a.begin(), a.end(), rng);
```

随机选择两个不同位置：

```
int x = rnd(0, n - 1);
int y = rnd(0, n - 2);
if (y >= x) ++ y;
```

复杂度

- rnd 的整数类型限标准分布支持的 short/int/long/long long 及其 unsigned 版本（不含 bool/char/i128），要求 l<=r；浮点限 float/double/long double，要求有限 l<r 且区间宽度可表示
- `rnd()`：期望 $O(1)$
- `shuffle()`：$O(n)$

模拟退火

统一按最小化目标函数编写。状态原地修改，通过 `apply/undo` 避免每次复制整个状态

```
template<class S, class F, class G, class A, class U>
pair<S, double> sa(S s, F f, G gen, A apply, U undo, double t0, double te, double k, int rep = 1) {
    assert(isfinite(t0) && isfinite(te) && isfinite(k) && t0 > 0 && te >= 0 && 0 < k && k < 1 && rep >= 1);
    uniform_real_distribution<double> prob(0, 1);
    S best = s;
    double cur = f(s), ans = cur;
    for (double t = t0; t > te;) {
        for (int z = 0; z < rep; ++ z) {
            auto op = gen(s, t);
            apply(s, op);
            double nv = f(s);
            double d = nv - cur;
            if (d <= 0 || prob(rng) < exp(-d / t)) {
                cur = nv;
                if (cur < ans) {
                    ans = cur;
                    best = s;
                }
            } else {
                undo(s, op);
            }
        }
        double nt = t * k;
        if (!(nt < t)) break;
        t = nt;
    }
    return {best, ans};
}
```

接口说明

- `sa(s,f,gen,apply,undo,t0,te,k,rep)`：返回 `{搜索到的最优状态，最优目标值}`
- `s`：初始状态

- `f(s)`：计算目标值，模板默认最小化
- `gen(s,t)`：根据当前状态和温度生成一次操作
- `apply(s,op)`：执行操作
- `undo(s,op)`：撤销操作，必须与 `apply` 严格互逆
- `t0`：初始温度
- `te`：终止温度
- `k`：降温系数，要求 $0 < k < 1$
- `rep`：每个温度执行的尝试次数，要求 >=1
- 温度参数必须有限，t0>0、te>=0；t0<=te 时不执行搜索。f(s) 必须返回有限 double；连续变量撤销应保存原值，不能假定浮点加减精确互逆

注意事项

- 当前值 `cur` 和历史最优值 `ans` 必须分开维护；模拟退火允许暂时接受更差状态
- 当新状态更优，即 $d \le 0$ 时直接接受；否则以 $e^{-d/t}$ 的概率接受
- 温度较高时容易接受差解，用于跳出局部最优；温度较低时逐渐接近普通爬山
- 必须返回搜索过程中保存的 `best`，不能直接返回最终状态
- 最大化时可令 `f(s)=-score(s)`
- `t0`、`te`、`k`、`rep` 没有固定最优值，需要结合目标值尺度和时限调整
- `k` 越接近 `1`，降温越慢，搜索通常越充分，但运行时间也越长
- 邻域设计通常比参数微调更重要；应尽量让一次修改既能局部调整，又能跨越局部最优
- `gen` 可利用温度调整扰动范围：高温大范围修改，低温小范围修改
- 若 `f(s)` 计算较慢，应尽量维护一次修改造成的目标值增量，而不是每次完整重算
- 大状态应使用 `apply/undo` 原地修改，避免频繁复制 `vector`、排列或图
- 模拟退火不保证找到全局最优解，但实现仍应保证状态合法、撤销正确和返回历史最优，重要题目应多次随机重启并保存全局最好答案

常见用法

排列中每次随机交换两个位置：

```
struct Op {
    int x, y;
};

auto [best, val] = sa(a,
    [&](const vector<int> &a) {
        return cost(a);
    },
    [&](const vector<int> &a, double t) {
        int x = rnd(0, (int)a.size() - 1);
        int y = rnd(0, (int)a.size() - 1);
        return Op{x, y};
    },
    [&](vector<int> &a, Op op) {
        swap(a[op.x], a[op.y]);
    },
    [&](vector<int> &a, Op op) {
        swap(a[op.x], a[op.y]);
    },
    1000, 1E-6, 0.995, 5);
```

若交换相同位置没有意义：

```
auto gen = [&](const vector<int> &a, double t) {
    int n = a.size();
    int x = rnd(0, n - 1);
    int y = rnd(0, n - 2);
    if (y >= x) ++ y;
    return Op{x, y};
};
```

区间翻转：

```
struct Op {
    int l, r;
};
auto gen = [&](const vector<int> &a, double t) {
    int l = rnd(0, (int)a.size() - 1);
    int r = rnd(0, (int)a.size() - 1);
    if (l > r) swap(l, r);
```

```
    return Op{l, r};
};
auto apply = [&](vector<int> &a, Op op) {
    reverse(a.begin() + op.l, a.begin() + op.r + 1);
};
auto undo = apply;
```

连续变量可让扰动范围随温度变化：

```
auto gen = [&](const vector<double> &a, double t) {
    int p = rnd(0, (int)a.size() - 1);
    double d = rnd(-t, t);
    return tuple{p, d, a[p]};
};
auto apply = [&](vector<double> &a, auto op) {
    a[get<0>(op)] += get<1>(op);
};
auto undo = [&](vector<double> &a, auto op) {
    a[get<0>(op)] = get<2>(op);
};
```

多次随机重启：

```
constexpr double INF{ numeric_limits<double>::max() };
double ans = INF;
for (int z = 0; z < 20; ++ z) {
    auto s = init();
    auto [best, val] = sa(s, f, gen, apply, undo, 1000, 1E-6, 0.995, 5);
    ans = min(ans, val);
}
```

复杂度

当 0<te<t0 且忽略浮点舍入时，温度层数为

$$L = \left\lceil \frac{\log(te/t0)}{\log k} \right\rceil$$

te=0 时该公式不适用；实现检测 t*k 未能严格减小时退出，避免次正规浮点数导致停滞。
每层尝试 rep 次，令 C 包含生成、执行/撤销、目标计算和必要的最优状态复制，则：

- 时间复杂度：$O(L \cdot rep \cdot C)$
- 额外空间主要为当前状态和最优状态各一份，即 $O(S)$

13.3 公平组合游戏

默认讨论双方轮流行动、信息完全、无随机且双方策略最优的公平组合游戏。除特别说明外均采用 normal play：无法行动者输

Nim

```
u64 nim_sum(const vector<u64> &a) {
    u64 s{};
    for (u64 x : a) s ^= x;
    return s;
}

optional<pair<int, u64>> nim_move(const vector<u64> &a) {
    u64 s = nim_sum(a);
    if (!s) return nullopt;
    for (int i = 0; i < (int)a.size(); ++ i) {
        u64 x = a[i] ^ s;
        if (x < a[i]) return pair{i, x};
    }
    return nullopt;
}
```

接口说明

- `nim_sum(a)`：返回所有堆大小的异或和
- `nim_move(a)`：返回一种必胜操作 `{堆编号，操作后的大小}`
- `nim_move(a)==nullopt`：当前为必败态

注意事项

- 普通 Nim 每次只能选择一堆并减少任意正数颗石子
- 先手必胜当且仅当

$$a_1 \oplus a_2 \oplus \cdots \oplus a_n \neq 0$$

- 必胜操作一定可以把总异或和变成 `0`
- 若一次能同时操作多堆，不能直接套普通 Nim
- 若最后一步者输，应使用 Anti-Nim

常见用法

只判断胜负：

```
bool win = nim_sum(a) != 0;
```

构造一步必胜操作：

```
if (auto mv = nim_move(a)) {
    auto [i, x] = *mv;
    a[i] = x;
}
```

复杂度

- `nim_sum()`：$O(n)$
- `nim_move()`：$O(n)$
- 空间：$O(1)$

SG 与 Mex

适用于一定会终止的 impartial game。一个状态的 SG 值为所有后继 SG 值的 `mex`，多个独立子游戏的 SG 值取异或

```
struct Mex {
    vector<int> vis;
    int tim{ 1 };
    explicit Mex(int n) : vis(n + 1) {}
    void clear() {
        if (++ tim == numeric_limits<int>::max()) {
            fill(vis.begin(), vis.end(), 0);
            tim = 1;
        }
    }
    void add(int x) {
        if (0 <= x && x < (int)vis.size()) vis[x] = tim;
    }
    int get() const {
        int x{};
        while (x < (int)vis.size() && vis[x] == tim) ++ x;
        return x;
    }
};

vector<int> sg_dag(const vector<vector<int>> &g, const vector<int> &ord) {
    int d{};
    for (auto &e : g) d = max(d, (int)e.size());
    Mex mex(d + 1);
    vector<int> sg(g.size());
    for (int u : ord) {
        mex.clear();
        for (int v : g[u]) mex.add(sg[v]);
        sg[u] = mex.get();
    }
```

```
    return sg;
}
```

`ord` 中必须保证每个状态的所有后继均出现在它之前。

接口说明

- `Mex(n)`：支持求值域 `0..n` 的 mex
- `clear()`：开始计算新的集合
- `add(x)`：将 `x` 标记为已出现
- `get()`：返回当前集合的最小未出现非负整数
- `sg_dag(g,ord)`：求 DAG 上所有状态的 SG 值

注意事项

- 定义：

$$SG(u) = \operatorname{mex}\{SG(v) \mid u \to v\}$$

- 无后继状态满足 `SG=0`
- `SG=0` 为必败态，`SG≠0` 为必胜态
- 多个独立子游戏组成的和满足

$$SG = SG_1 \oplus SG_2 \oplus \cdots \oplus SG_k$$

- 一个状态若最多有 d 个不同后继，则其 SG 值至多为 d，因此 Mex 不需要按状态总数开得很大
- `sg_dag()` 要求 `ord` 是逆拓扑序，即后继先计算
- 若状态图有环并可能无限进行，普通 SG 不能直接使用，应做胜/负/平局逆推
- 若游戏规则不是“每次只操作一个独立子游戏”，普通 SG 异或定理可能失效

常见用法

多个独立游戏：

```
int s{};
for (int u : st) s ^= sg[u];
bool win = s != 0;
```

固定取石集合 `{1,3,4}`：

```
int n = 1000000;
vector<int> take{1, 3, 4};
vector<int> sg(n + 1);
Mex mex(take.size() + 1);
for (int i = 1; i <= n; ++ i) {
    mex.clear();
    for (int x : take) {
        if (x > i) break;
        mex.add(sg[i - x]);
    }
    sg[i] = mex.get();
}
```

记忆化搜索也可直接按定义求 SG：

```
auto dfs = [&](auto &&self, int u) -> int {
    if (sg[u] != -1) return sg[u];
    vector<int> a;
    for (int v : g[u]) a.push_back(self(self, v));
    sort(a.begin(), a.end());
    a.erase(unique(a.begin(), a.end()), a.end());
    int x{};
    for (int y : a) {
        if (y != x) break;
        ++ x;
    }
    return sg[u] = x;
};
```

复杂度

- `Mex`：一次 `clear()` 为均摊 $O(1)$，`add()` 为 $O(1)$，`get()` 最坏 $O(d)$
- `sg_dag()`：$O(V+E)$
- 空间：$O(V+D)$，其中 D 为最大出度

Bash 博弈

每次从一堆中取走 `1..m` 颗，无法行动者输
必败态：

$$n \equiv 0 \pmod{m+1}$$

即：

```
bool bash(u64 n, u64 m) {
    return m >= n ? n != 0 : n % (m + 1) != 0;
}
```

接口说明

- `bash(n,m)`：返回先手是否必胜
- `n`：当前石子数量
- `m`：单次最多取走的数量

注意事项

- 这是普通 Bash：每次必须取至少 `1` 颗、至多 `m` 颗
- 必胜时可取

$$n \bmod (m+1)$$

  颗，把状态送入下一必败态
- 若最后一颗者输、双方取石上限不同或下一次上限依赖上一步操作，不能套此结论

常见用法

```
if (bash(n, m)) {
    u64 take = m >= n ? n : n % (m + 1);
}
```

复杂度

- 时间：$O(1)$
- 空间：$O(1)$

Anti-Nim

每次任选一堆取走正数颗石子，但**取走最后一颗石子者输**

```
bool anti_nim(const vector<u64> &a) {
    u64 s{};
    int cnt{};
    bool one = true;
    for (u64 x : a) if (x) {
        s ^= x;
        ++ cnt;
        one &= x == 1;
    }
    return one ? cnt % 2 == 0 : s != 0;
}
```

接口说明

- `anti_nim(a)`：返回当前局面先手是否必胜
- `a[i]`：第 `i` 堆石子数量

注意事项

- 若所有非空堆大小均为 `1`，偶数堆为必胜态
- 若存在一堆大小大于 `1`，结论恢复为普通 Nim：异或和非零时先手必胜
- 空堆不参与堆数统计
- 该结论只针对标准 Anti-Nim，不能直接推广到任意 misère impartial game

常见用法

```
cout << (anti_nim(a) ? "First\n" : "Second\n");
```

复杂度

- 时间：$O(n)$
- 空间：$O(1)$

阶梯 Nim

第 `i` 级台阶上的石子可移动任意正数颗到第 `i-1` 级，`0` 级视为地面，进入地面的石子不再参与游戏

```
bool stair_nim(const vector<u64> &a) {
    u64 s{};
    for (int i = 0; i < (int)a.size(); ++ i) {
        if (i % 2 == 0) s ^= a[i];
    }
    return s != 0;
}
```

接口说明

- `a[0]`：第 `1` 级台阶，即最靠近地面的一级
- `stair_nim(a)`：返回先手是否必胜

注意事项

- 只对奇数级台阶上的石子做普通 Nim 异或
- 若题目编号从 `0` 开始或从顶层开始，必须先转换编号含义，不能机械地异或奇数下标
- 偶数层上的操作可以由对手通过对应操作抵消，本质上只有奇数层决定胜负

常见用法

```
bool win = stair_nim(a);
```

复杂度

- 时间：$O(n)$
- 空间：$O(1)$

Wythoff 博弈

有两堆石子，每次可以从一堆取任意正数颗，或从两堆中取走相同的正数颗
令两堆满足 $a \le b$，记

$$k = b - a, \quad \varphi = \frac{1+\sqrt{5}}{2}$$

必败态恰为

$$(a, b) = (\lfloor k\varphi \rfloor, \lfloor k\varphi \rfloor + k)$$

接口说明

- 先令 `a ≤ b`
- `k=b-a`
- 判断 `a == floor(k*phi)` 即可判断是否为必败态

注意事项

- `(0,0)` 对应 `k=0`，也是必败态

- 不要在大整数范围直接用 `double` 计算 `floor(k*phi)`，临界位置可能因精度误判
- 数据较大时应通过高精度或精确整数平方根计算

$$\lfloor k\varphi \rfloor = \left\lfloor \frac{k + \lfloor k\sqrt{5} \rfloor}{2} \right\rfloor$$

- 两堆规则稍有改变，例如只能同时取固定倍数，就不再是 Wythoff

常见用法

完整 u64 范围可直接精确判断，无须浮点或大整数平方根；返回 true 表示先手必胜：

```
bool wythoff(u64 a, u64 b) {
    if (a > b) swap(a, b);
    u128 k = b - a, x = a;
    if (!k) return a != 0;
    if (x < k) return true;
    return x * (x - k) > k * k || (x + 1) * (x + 1 - k) <= k * k;
}
```

用法：`bool win = wythoff(a,b);`。当 k>0 且 x>=k 时，必败条件等价于 x(x-k)<=k²<(x+1)(x+1-k)。由于 a<b<=u64 上界，所有乘积均可放入 u128；k=0 与 x<k 已单独处理。

复杂度

- 上面的 u128 精确比较：$O(1)$
- 精确整数平方根版本：取决于整数平方根实现

Fibonacci Nim

一堆 `n` 颗石子，首步不能全部取完；之后每次最多取走上一次取走数量的两倍
`z(n)` 表示 `n` 的 Zeckendorf 分解中的最小 Fibonacci 项

```
u64 fib_low(u64 n) {
    static const vector<u64> f = [] {
        vector<u64> a{1, 2};
        constexpr u64 INF{ numeric_limits<u64>::max() };
        while (u128(a.end()[-1]) + a.end()[-2] <= INF) a.push_back(a.end()[-1] + a.end()[-2]);
        return a;
    }();
    u64 res{};
    for (int i = int(upper_bound(f.begin(), f.end(), n) - f.begin()) - 1; i >= 0; --i) {
        if (f[i] <= n) {
            n -= f[i];
            res = f[i];
        }
    }
    return res;
}

bool fib_nim(u64 n, u64 lim) {
    return n && lim >= fib_low(n);
}
```

接口说明

- `fib_low(n)`：返回 `n` 的 Zeckendorf 分解中的最小项
- `fib_nim(n,lim)`：还剩 `n` 颗、当前最多可取 `lim` 颗时，返回当前玩家是否必胜
- 标准首步 n>=1 时调用 `fib_nim(n,n-1)`；n=0 为终止必败态，fib_low(0)=0
- 完整 u64 Fibonacci 表仅首次调用构造，之后复用

注意事项

- Zeckendorf 定理：每个正整数都能唯一表示为若干个不相邻 Fibonacci 数之和
- 当前最大可取数量 `lim >= fib_low(n)` 时必胜，并可取 `fib_low(n)` 颗
- 标准首步不能取完，因此 `n` 本身为 Fibonacci 数时先手必败
- Fibonacci 序列按 `1,2,3,5,...` 使用
- 若“下一步最多取上一步的倍数”不是 `2`，结论不再成立

常见用法

标准 Fibonacci Nim：

```
bool win = fib_nim(n, n - 1);
```

构造必胜操作：

```
u64 x = fib_low(n);
if (x && lim ≥ x) {
    // 取 x 颗
}
```

复杂度

- Fibonacci 数数量为 $O(\log n)$
- `fib_low()`：$O(\log n)$
- 空间：$O(\log n)$

Anti-SG：适用范围

---

misère 规则为**无法行动者胜**。普通 SG 值一般不足以判断这种游戏的胜负，不能将任意子游戏先求普通 SG，再套 Anti-Nim 的规则。
反例：状态 0 无后继，1→0，2→{0,1}，3→2。普通 SG 依次为 {0,1,2,0}，misère 胜负依次为 {胜,负,胜,负}。状态 0 与 3 的 SG 都为 0，但胜负不同。
下列函数保留旧名以兼容调用，**只用于真正的 Nim 堆大小，或已经另外证明在 misère 下可作同样替换的游戏**；它不是通用 Anti-SG 算法。

```
bool anti_sg(const vector<int> &a) {
    int s{};
    bool big = false;
    for (int x : a) {
        s ^= x;
        big |= x > 1;
    }
    return big ? s ≠ 0 : s == 0;
}
```

接口说明

- a[i] 是非负 Nim 堆大小，不是任意游戏的普通 SG 值；完整 u64 堆请用 anti_nim
- 任意有限无环状态图应按“终止状态胜；存在必败后继则胜，否则负”逆推
- 多个一般子游戏需在组合状态上计算，或使用已证明适用的 misère quotient 等方法
- 可用 13.4 GameGraph 对完整组合状态图逆推，将所有终止状态 fix 为 WIN；不能把普通 SG 值直接异或代替组合状态
- 理论参考：Aaron N. Siegel, Misère Games and Misère Quotients

常见用法

```
vector<int> a{1, 1, 2}; // 实际 Nim 堆，而非任意状态的 SG 值
bool win = anti_sg(a);
```

复杂度

- 时间：$O(n)$
- 空间：$O(1)$

Green Hackenbush 树上删边

根固定，每次删除一条边，并同时删去因此与根断开的部分；无法行动者输。

```
u64 hack_sg(const vector<vector<int>> &g, int u, int fa = -1) {
    vector<int> p(g.size(), -1), ord{u};
    p[u] = fa;
    for (int i = 0; i < int(ord.size()); ++ i) {
        int x = ord[i];
        for (int v : g[x]) if (v ≠ p[x]) p[v] = x, ord.push_back(v);
    }
    vector<u64> s(g.size());
    for (int i = int(ord.size()) - 1; i > 0; -- i) {
        int x = ord[i];
        s[p[x]] ^= s[x] + 1;
```

```
    }
    return s[u];
}
```

接口说明

- `hack_sg(g,root)`：返回以 `root` 为根的树上删边游戏 SG 值
- 返回值为 `0` 时当前玩家必败

注意事项

- 叶子节点 SG 值为 `0`
- 对每条子树分支：

$$SG_{\text{branch}} = SG_{\text{child}} + 1$$

  各分支再异或
- 此模板只适用于树
- 一般无向图删边游戏需要先使用 Fusion Principle 等化简，不能直接找一棵 DFS 树后套本模板
- 有边权或其他删边规则时还需要相应扩展，不能机械使用 `+1`

常见用法

```
u64 s = hack_sg(g, root);
cout << (s ? "First\n" : "Second\n");
```

复杂度

- 时间：$O(n)$
- 额外空间：$O(n)$；显式遍历并逆序合并，避免长链递归爆栈

13.4 图上博弈与对抗搜索

图上胜负与平局逆推

在有向状态图上双方轮流沿一条出边移动。默认无法行动者输；若双方都能避免失败并无限进行，则为平局

```
struct GameGraph {
    enum { LOSE = -1, DRAW = 0, WIN = 1, UNK = 2 };
    int n;
    vector<vector<int>> g, rg;
    vector<int> ini, st, go;
    explicit GameGraph(int n_) : n(n_), g(n), rg(n), ini(n, UNK), st(n), go(n, -1) {}
    void add(int u, int v) {
        g[u].push_back(v);
        rg[v].push_back(u);
    }
    void fix(int u, int s) { ini[u] = s; }
    void solve() {
        vector<int> rem(n);
        queue<int> q;
        fill(st.begin(), st.end(), DRAW);
        fill(go.begin(), go.end(), -1);
        for (int u = 0; u < n; ++ u) {
            rem[u] = g[u].size();
            if (ini[u] != UNK) {
                st[u] = ini[u];
                q.push(u);
            } else if (!rem[u]) {
                st[u] = LOSE;
                q.push(u);
            }
        }
        while (!q.empty()) {
            int v = q.front();
            q.pop();
            for (int u : rg[v]) if (st[u] == DRAW) {
                if (st[v] == LOSE) {
```

```
                    st[u] = WIN;
                    go[u] = v;
                    q.push(u);
                } else if (-- rem[u] == 0) {
                    st[u] = LOSE;
                    q.push(u);
                }
            }
        }
        for (int u = 0; u < n; ++ u) if (ini[u] == UNK && go[u] == -1) {
            if (st[u] == DRAW) {
                for (int v : g[u]) if (st[v] == DRAW) {
                    go[u] = v;
                    break;
                }
            } else if (!g[u].empty()) {
                go[u] = g[u][0];
            }
        }
    }
};
```

接口说明

- `GameGraph(n)`：建立 `n` 个状态，编号为 `0..n-1`
- `add(u,v)`：当前玩家可从状态 `u` 移动到状态 `v`
- `fix(u,s)`：预先指定状态 `u` 为 `WIN` 或 `LOSE`
- `solve()`：求所有状态的胜负和平局情况
- `st[u]`：
  - `WIN`：当前玩家必胜
  - `LOSE`：当前玩家必败
  - `DRAW`：双方最优策略下均可避免失败，但无法强制获胜
- `go[u]`：一种最优后继状态；终止状态为 `-1`

注意事项

核心递推只有三条：

- 存在一个后继为 `LOSE`，当前状态就是 `WIN`
- 所有后继都是 `WIN`，当前状态就是 `LOSE`
- 最终仍不能确定的状态就是 `DRAW`

模板从已确定状态沿反图逆推。`rem[u]` 表示尚未被证明为 `WIN` 的后继数量；当所有后继都已确定为 `WIN` 时，`u` 才能确定为 `LOSE`

`st[u]` 始终表示**轮到当前玩家行动时**的结果。若两名玩家可执行的操作不同，应把“轮到谁”也放入状态：

```
id = u * 2 + turn;
```

`fix(u,s)` 同样从状态中“当前玩家”的角度定义。例如某玩家走到目标点后立刻获胜，而状态已经切换给另一玩家，则该目标状态对新的当前玩家应标为 `LOSE`

默认所有出度为 `0` 且未 `fix` 的状态都是 `LOSE`

无限循环只有在题目规定其结果为平局时才能直接使用该模板。若重复局面判负、存在步数限制或超时判定，应把对应信息加入状态

这里求的是 `WIN / LOSE / DRAW`，**不是 SG 值**。不能利用结果异或合并多个独立游戏；独立公平组合游戏仍使用 `13.3` 的 SG 定理

常见用法

普通“无路可走者输”的图上博弈：

```
GameGraph game(n);
for (auto [u, v] : edges) game.add(u, v);
game.solve();

if (game.st[s] == GameGraph::WIN) {
    int v = game.go[s];
} else if (game.st[s] == GameGraph::LOSE) {
    // 后手必胜
} else {
    // 平局
}
```

双方移动规则不同：

```
GameGraph game(2 * n);
auto id = [&](int u, int t) {
    return 2 * u + t;
};

for (int u = 0; u < n; ++ u) {
    for (int v : g0[u]) game.add(id(u, 0), id(v, 1));
    for (int v : g1[u]) game.add(id(u, 1), id(v, 0));
}
game.solve();
```

预设特殊终止状态：

```
game.fix(u, GameGraph::WIN);
game.fix(v, GameGraph::LOSE);
game.solve();
```

复杂度

- 建图：$O(V+E)$
- `solve()`：$O(V+E)$
- 空间：$O(V+E)$

Alpha-Beta 对抗搜索

用于双人、零和、完全信息的有限深度对抗搜索。使用 Negamax 统一双方逻辑，并在 Minimax 基础上加入 Alpha-Beta 剪枝

```
template<class S, class F, class G, class A, class U>
i64 ab(S &s, int dep, i64 l, i64 r, F f, G gen, A ap, U un) {
    auto dfs = [&](auto &&self, int d, i64 a, i64 b) -> i64 {
        if (d <= 0) return f(s);
        auto ops = gen(s);
        if (ops.empty()) return f(s);
        for (auto &op : ops) {
            ap(s, op);
            i64 v = -self(self, d - 1, -b, -a);
            un(s, op);
            a = max(a, v);
            if (a >= b) break;
        }
        return a;
    };
    return dfs(dfs, dep, l, r);
}
```

接口说明

- `ab(s,dep,l,r,f,gen,ap,un)`：全窗口时返回当前玩家视角下的最优估值；窄窗口可能只返回上/下界，不能无条件当精确值缓存
- `s`：当前状态，搜索过程中原地修改
- `dep`：剩余搜索深度
- `[l,r)`：当前 Alpha-Beta 搜索窗口
- `f(s)`：返回当前状态对当前玩家的估值
- `gen(s)`：返回当前状态所有可执行操作
- `ap(s,op)`：执行操作，并切换到对手行动的状态
- `un(s,op)`：撤销操作
- 根节点通常使用足够大的窗口 `[-INF,INF]`；l<r，边界和估值必须可安全取负（不能为 i64 最小值），所有真实估值宜严格位于全窗口内部；dep<=0 直接评估

注意事项

Negamax 利用零和博弈的关系

$$V(s) = -V(\text{next}(s))$$

因此不需要分别写 `max()` 和 `min()`
`f(s)` 必须始终从当前行动玩家的角度估值：

- 越大越有利
- 越小越不利
- 双方视角互为相反数

搜索窗口中：

- `a`：当前至少能够取得的分数
- `b`：对手不会允许当前玩家达到的上界
- 当 `a ≥ b` 时，剩余分支已经不可能影响祖先决策，可以直接剪枝

操作顺序不会影响正确性，但会极大影响速度。应优先搜索更可能优秀的操作，例如：

- 上一轮搜索得到的最优操作
- 吃子、得分等强制操作
- 启发式估值较高的操作

若有明确终局，应让 `gen(s)` 返回空操作，并由 `f(s)` 返回准确的终局分数。终局分数通常应远大于普通局面估值
常见写法：

```
WIN - step
```

比固定返回 `WIN` 更好，可以让算法优先选择更快获胜；失败时对应：

```
-WIN + step
```

从而尽量延迟失败
若相同局面会通过多条路径反复到达，可以加入置换表。但 Alpha-Beta 剪枝得到的返回值不一定是精确值，不能无条件直接记忆化。标准置换表需要区分：

- `EXACT`：精确值
- `LOWER`：下界
- `UPPER`：上界

即使本节点遍历了全部操作，也可能因未超过传入 alpha 而只返回界，不能仅凭“未剪枝”当成 EXACT。以进入节点时的窗口 [a0,b0) 分类：返回值 <=a0 为 UPPER，>=b0 为 LOWER，严格落在窗口内部才可标为 EXACT；缓存还需区分搜索深度和局面状态。
迭代加深常与 Alpha-Beta 配合：

```
for (int dep = 1; dep ⩽ lim; ++ dep) {
    ans = ab(s, dep, -INF, INF, f, gen, ap, un);
}
```

上一层得到的最优操作可以用于下一层的操作排序，通常能显著加强剪枝

常见用法

基本调用：

```
constexpr i64 INF{ numeric_limits<i64>::max() };

auto f = [&](const State &s) -> i64 {
    return eval(s);
};

auto gen = [&](const State &s) {
    vector<int> ops;
    // 生成并尽量按优秀程度排序
    return ops;
};

auto ap = [&](State &s, int op) {
    s.apply(op);
};

auto un = [&](State &s, int op) {
    s.undo(op);
};

i64 ans = ab(s, dep, -INF, INF, f, gen, ap, un);
```

需要求根节点的最优操作时（dep>=1；终局 ops 为空时 bop=-1）：

```
i64 best = -INF;
int bop = -1;
auto ops = gen(s);
for (int op : ops) {
```

```
    ap(s, op);
    i64 v = -ab(s, dep - 1, -INF, -best, f, gen, ap, un);
    un(s, op);
    if (bop == -1 || v > best) {
        best = v;
        bop = op;
    }
}
```

若不想利用根节点已有的 `best` 缩小窗口，也可直接写：

```
i64 v = -ab(s, dep - 1, -INF, INF, f, gen, ap, un);
```

代码更直观，但剪枝略弱。

复杂度

设平均分支数为 b，搜索深度为 d：

- Minimax 最坏时间复杂度：$O(b^d)$
- Alpha-Beta 最坏时间复杂度仍为 $O(b^d)$
- 操作顺序理想时约为 $O(b^{d/2})$
- 递归空间：$O(d)$，不计状态本身与每层生成的操作

14. 概率与期望

14.1 期望递推、方程与生成函数

```
// ===== Markov 链方程：依赖高斯消元 =====
template<class T>
vector<vector<T>> markov_system(const vector<vector<T>> &p,
        const vector<char> &term, const vector<T> &cost) {
    int n = p.size();
    vector a(n, vector<T>(n + 1));
    for (int u = 0; u < n; ++ u) {
        a[u][u] = 1;
        if (term[u]) continue;
        for (int v = 0; v < n; ++ v) a[u][v] -= p[u][v];
        a[u][n] = cost[u];
    }
    return a;
}
// ===== 树上随机游走 =====
vector<i64> tree_random_walk_to_root(const vector<vector<int>> &g, int root) {
    int n = g.size();
    vector<int> fa(n, -1), sub(n, 1), order{root};
    fa[root] = root;
    for (int i = 0; i < (int)order.size(); ++ i) {
        int u = order[i];
        for (int v : g[u]) if (fa[v] == -1) {
            fa[v] = u;
            order.push_back(v);
        }
    }
    for (int i = n - 1; i > 0; -- i) {
        int u = order[i];
        sub[fa[u]] += sub[u];
    }
    vector<i64> e(n);
    for (int u : order) if (u != root) {
        e[u] = e[fa[u]] + 2LL * sub[u] - 1;
    }
    return e;
}
```

接口说明

接口	功能	复杂度
`markov_system(p, term, cost)`	构造吸收 Markov 链的增广矩阵	(O(n^2))
`tree_random_walk_to_root(g, root)`	求树上每个点首次到达吸收根的期望步数	(O(n))

`markov_system()`

- `p[u][v]`：状态 `u` 转移到 `v` 的概率。
- `term[u]`：是否为终止状态；终止状态的未知量固定为 0。
- `cost[u]`：状态 `u` 的期望即时成本；计算首次吸收步数时，非终止状态填 1。
- 返回 `n × (n + 1)` 增广矩阵，交给 11.1 矩阵与线性方程组 中对应类型的高斯消元求解。
- `T` 可使用 `ld` 或静态模整数 `MInt`，但求解器必须支持对应数域。

`tree_random_walk_to_root()`

- `g`：`0-based` 无权无向树邻接表。
- `root`：吸收点，首次抵达即停止。
- 返回 `e[u]`：从 `u` 首次到达 `root` 的期望步数。
- 输入必须是一棵合法树；不执行合法性检查。

注意事项

- 有限期望：若从某状态有正概率永远无法吸收，则首次吸收时间的期望为无穷。构造线性方程不等于保证存在唯一的有限解。
- 转移概率：每个非终止状态的完整转移概率之和必须为 1，包括自环和进入终止状态的概率。
- 即时成本：`cost[u]` 是进行下一次转移时产生的期望成本，包含最后一次进入终止状态的转移成本。
- 自环处理：若除自环外的后继状态均已求出，可直接使用 $E = (c + \sum p_i E_i)/(1 - p_{\text{self}})$，不需要高斯消元。对应 12.3 树、轮廓线、概率与期望 DP。
- 有环依赖：一般需要解线性方程；若去掉自环后形成 DAG，可移项后按逆拓扑序递推。
- 期望与最短路：当状态可以选择动作时，Bellman 方程可能包含取最小值操作。不能直接套用 Dijkstra，除非证明定值顺序的正确性。
- 模意义期望：概率须能表示为模数下的元素，求解所需的除数必须可逆；模意义方程有解不代表原随机过程的期望有限。
- 树上随机游走：只适用于无权无向树、每一步等概率选择相邻点的模型。结果可能达到 $O(n^2)$，使用 `i64` 保存。
- PGF 截断：只保留低次项会丢失尾部概率，不能直接把截断结果的矩当作原分布的矩。

常见用法

1. DAG 期望 DP

终止状态期望为 0，按照后继先于前驱的顺序计算。

```
// g[u] 存储 (to, probability, cost)
vector<ld> e(n);
for (int u : reverse_topological) {
    if (term[u]) continue;
    for (auto [v, p, cost] : g[u]) {
        e[u] += p * (cost + e[v]);
    }
}
```

2. 吸收 Markov 链期望

求每个状态首次到达终止状态的期望步数。

```
vector<ld> cost(n, 1);
auto a = markov_system(transition, term, cost);
auto res = gauss_real(a);
// 确认唯一解后：
// E[u] = res.particular[u];
```

若只关心到达指定终止集合的概率，可将终止状态的方程右端设为相应边界值（目标为 1，其他终止状态为 0），再求解。

3. 首达时间的方差

设 $E_u = \mathbb{E}[T_u]$，$M_u = \mathbb{E}[T_u^2]$。每走一步计 1，且最终吸收时：

$$(I - P)E = \mathbf{1}, \qquad (I - P)M = 2E - \mathbf{1}$$

先求 `E`，再以 `2 * E[u] - 1` 作为非终止状态的右端求 `M`：

```
vector<ld> rhs(n);
for (int u = 0; u < n; ++ u) {
    if (!term[u]) rhs[u] = 2 * E[u] - 1;
}
auto a = markov_system(transition, term, rhs);
```

```
auto res = gauss_real(a);
// 确认唯一解后：
// variance[u] = res.particular[u] - E[u] * E[u];
```

这里假定 `E` 已经正确求出，且所有相关状态的首达时间二阶矩有限。

4. 树上随机游走

```
auto e = tree_random_walk_to_root(g, root);
cout << e[source] << '\n';
```

核心公式：以 `root` 为根，`sub[u]` 表示子树大小，则

$$E_u = E_{\mathrm{fa}(u)} + 2\,\mathrm{sub}(u) - 1, \qquad E_{\mathrm{root}} = 0$$

5. 概率生成函数

对于非负整数随机变量 X，令

$$G_X(z) = \sum_{k\ge 0} \Pr(X = k) z^k$$

独立随机变量之和对应 PGF 相乘，直接调用 10.1 FFT、NTT 与 FWT 的 `convolution`，不再重复实现。
求期望与方差：

$$\begin{aligned} \mathbb{E}[X] &= G'_X(1), \\ \mathrm{Var}(X) &= G''_X(1) + G'_X(1) - G'_X(1)^2. \end{aligned}$$

若只给出完整概率数组 `p[k]`，可直接累加：

```
ld e = 0, e2 = 0;
for (int k = 0; k < (int)p.size(); ++ k) {
    e += (ld)k * p[k];
    e2 += (ld)k * k * p[k];
}
ld variance = e2 - e * e;
```

复杂度

操作	时间复杂度	空间复杂度
DAG 期望 DP	$O(V+E)$	$O(V)$，不计图
Markov 方程构造	$O(n^2)$	$O(n^2)$
稠密高斯消元	$O(n^3)$	$O(n^2)$
树上随机游走期望	$O(n)$	$O(n)$
朴素 PGF 卷积	$O(ab)$	$O(a+b)$
FFT / NTT 卷积	$O(n \log n)$	$O(n)$
PGF 一阶、二阶矩	$O(n)$	$O(1)$

15. 二维计算几何

15.1 点、向量、直线与线段

数值与版本约定

整数版的 `Point` 加减、数乘结果必须可由 `i64` 表示，叉积、距离平方及其累加结果必须可由有符号 `i128` 表示，转换为 `i128` 后计算不等于支持任意满范围 `i64` 输入的所有几何运算
浮点版使用 `EPS = 1E-12L` 进行几何分支判定，`EPS` 不是题目输出误差标准，最终答案通常按

$$\frac{|a-b|}{\max(1,|b|)} \le 10^{-6}$$

判断，其中 a 为输出值，b 为标准值。本仓库验证使用更严格的 `1e-9` 阈值；建议至少输出 `12` 位小数，输出位数本身不能保证数值算法精度
叉积、长度、长度平方具有不同量纲，不要通过 `sgn(norm2(v)) == 0` 判断方向向量是否退化

整数结构体

```
int sgn(i128 x) { return (x > 0) - (x < 0); }

struct Point {
    i64 x{ 0 }, y{ 0 };
    Point() = default;
    Point(i64 x_, i64 y_) : x(x_), y(y_) {}
    Point operator+() const { return *this; }
    Point operator-() const { return Point(-x, -y); }
    Point &operator+=(Point p) { x += p.x; y += p.y; return *this; }
    Point &operator-=(Point p) { x -= p.x; y -= p.y; return *this; }
    friend Point operator+(Point a, Point b) { return a += b; }
    friend Point operator-(Point a, Point b) { return a -= b; }
    Point &operator*=(i128 k) { x *= k; y *= k; return *this; }
    friend Point operator*(Point p, i128 k) { return p *= k; }
    friend Point operator*(i128 k, Point p) { return p *= k; }
    friend bool operator==(Point a, Point b) { return a.x == b.x && a.y == b.y; }
    friend bool operator!=(Point a, Point b) { return !(a == b); }
    // 原始坐标字典序
    friend auto operator<=>(Point a, Point b) = default;
};
bool same_pt(Point a, Point b) { return a == b; }
struct Line {
    Point p, dir;
    Line() = default;
    Line(Point p_, Point dir_) : p(p_), dir(dir_) {}
    // 构造经过 a、b 的直线，a == b 时退化为单点 a
    static Line through(Point a, Point b) { return Line(a, b - a); }
};
struct Segment {
    Point a, b;
    Segment() = default;
    Segment(Point a_, Point b_) : a(a_), b(b_) {}
};
```

通用函数

点与向量

```
i128 dot(Point a, Point b) { return i128(a.x) * b.x + i128(a.y) * b.y; }
i128 cross(Point a, Point b) { return i128(a.x) * b.y - i128(a.y) * b.x; }
i128 dot(Point o, Point a, Point b) {
    return (i128(a.x) - o.x) * (i128(b.x) - o.x) + (i128(a.y) - o.y) * (i128(b.y) - o.y);
}
i128 cross(Point o, Point a, Point b) {
    return (i128(a.x) - o.x) * (i128(b.y) - o.y) - (i128(a.y) - o.y) * (i128(b.x) - o.x);
}
i128 norm2(Point p) { return dot(p, p); }
i128 dist2(Point a, Point b) {
    i128 dx{ i128(a.x) - b.x }, dy{ i128(a.y) - b.y };
    return dx * dx + dy * dy;
}
Point rotate90(Point p) { return Point(-p.y, p.x); }
// ===== 极角、方向分类 =====
// 返回极角所在半平面，上半平面为 0，下半平面为 1
// 浮点版同样使用原始比较，不能使用 sgn()，否则可能破坏严格弱序
int arg_half(Point p) { return p.y < 0 || (p.y == 0 && p.x < 0); }
// 从正 x 轴开始逆时针比较极角，同方向按长度升序
bool arg_less(Point a, Point b) {
    int ha{ arg_half(a) }, hb{ arg_half(b) };
    if (ha != hb) return ha < hb;
    auto turn{ cross(a, b) };
    if (turn != 0) return turn > 0;
    return norm2(a) < norm2(b);
}
enum class CCW {
    CLOCKWISE = -1,        // p 在有向直线 a → b 右侧
    ON_SEGMENT = 0,        // p 在线段 [a,b] 上
    COUNTER_CLOCKWISE = 1, // p 在有向直线 a → b 左侧
    ONLINE_BACK = 2,       // p 与 a、b 共线，但在 a 后方
    ONLINE_FRONT = -2      // p 与 a、b 共线，但越过 b
```

```
};
CCW ccw(Point a, Point b, Point p) {
    int turn{ sgn(cross(a, b, p)) };
    if (turn > 0) return CCW::COUNTER_CLOCKWISE;
    if (turn < 0) return CCW::CLOCKWISE;
    if (sgn(dot(a, b, p)) < 0) return CCW::ONLINE_BACK;
    if (sgn(dist2(a, p) - dist2(a, b)) > 0) return CCW::ONLINE_FRONT;
    return CCW::ON_SEGMENT;
}
```

直线

```
auto dir_cross(Line a, Line b) { return cross(a.dir, b.dir); }
auto dir_dot(Line a, Line b) { return dot(a.dir, b.dir); }
bool parallel(Line a, Line b) { return sgn(dir_cross(a, b)) == 0; }
bool perp(Line a, Line b) { return sgn(dir_dot(a, b)) == 0; }
int pt_side(Point p, Line ln) { return sgn(cross(ln.dir, p - ln.p)); }
bool on_line(Point p, Line ln) {
    if (ln.dir == Point{}) return same_pt(p, ln.p);
    return pt_side(p, ln) == 0;
}
enum class LineRel {
    INTERSECT, // 有唯一交点
    PARALLEL,  // 平行但不重合
    COINCIDENT // 重合
};
// 要求两条直线均非退化
LineRel line_rel(Line a, Line b) {
    if (!parallel(a, b)) return LineRel::INTERSECT;
    return on_line(b.p, a) ? LineRel::COINCIDENT : LineRel::PARALLEL;
}
```

线段

```
// 判断点是否位于闭线段 [a,b] 上，允许线段退化为单点
bool on_seg(Point p, Segment s) {
    return sgn(cross(s.a, s.b, p)) == 0 && sgn(dot(p, s.a, s.b)) <= 0;
}
// 判断两点是否位于直线同侧，stct = true 时不允许点在线上
bool same_sd(Point a, Point b, Line ln, bool stct = true) {
    int sa{ pt_side(a, ln) }, sb{ pt_side(b, ln) };
    if (stct) return sa != 0 && sa == sb;
    return sa == 0 || sb == 0 || sa == sb;
}
// 判断两条线段是否相交，stct = true 时只判断内部严格交叉
bool seg_int(Segment a, Segment b, bool stct = false) {
    int c1{ sgn(cross(a.a, a.b, b.a)) }, c2{ sgn(cross(a.a, a.b, b.b)) };
    int c3{ sgn(cross(b.a, b.b, a.a)) }, c4{ sgn(cross(b.a, b.b, a.b)) };
    bool prop{ c1 != 0 && c2 != 0 && c1 != c2 && c3 != 0 && c4 != 0 && c3 != c4 };
    if (stct) return prop;
    if (prop) return true;
    return (c1 == 0 && on_seg(b.a, a)) || (c2 == 0 && on_seg(b.b, a))
        || (c3 == 0 && on_seg(a.a, b)) || (c4 == 0 && on_seg(a.b, b));
}
```

浮点 Point 与新增函数

浮点版使用本节结构体整体替换整数结构体，然后复制上面的通用函数，其中 `dot()`、`cross()`、`norm2()`、`dist2()` 的返回类型及函数体内的显式转换均由 `i128` 改为 `ld`

浮点结构体

```
using ld = long double;
constexpr ld EPS{ 1E-12L };
const ld PI = acosl(-1.L);
int sgn(ld x, ld eps = EPS) { return (x > eps) - (x < -eps); }
struct Point {
    ld x{ 0 }, y{ 0 };
    Point() = default;
    Point(ld x_, ld y_) : x(x_), y(y_) {}
```

```
    Point operator+() const { return *this; }
    Point operator-() const { return Point(-x, -y); }
    Point &operator+=(Point p) { x += p.x; y += p.y; return *this; }
    Point &operator-=(Point p) { x -= p.x; y -= p.y; return *this; }
    friend Point operator+(Point a, Point b) { return a += b; }
    friend Point operator-(Point a, Point b) { return a -= b; }
    Point &operator*=(ld k) { x *= k; y *= k; return *this; }
    Point &operator/=(ld k) { x /= k; y /= k; return *this; }
    friend Point operator*(Point p, ld k) { return p *= k; }
    friend Point operator*(ld k, Point p) { return p *= k; }
    friend Point operator/(Point p, ld k) { return p /= k; }
    // 原始值精确判等，几何判等使用 same_pt()
    friend bool operator==(Point a, Point b) { return a.x == b.x && a.y == b.y; }
    friend bool operator!=(Point a, Point b) { return !(a == b); }
    // 原始坐标字典序，不能引入 EPS
    friend auto operator<=>(Point a, Point b) = default;
};
bool same_pt(Point a, Point b) { return sgn(a.x - b.x) == 0 && sgn(a.y - b.y) == 0; }
struct Line {
    Point p, dir;
    Line() = default;
    Line(Point p_, Point dir_) : p(p_), dir(dir_) {}
    static Line through(Point a, Point b) { return Line(a, b - a); }
    optional<ld> y_at(ld x) const {
        if (sgn(dir.x) == 0) return nullopt;
        return p.y + dir.y * (x - p.x) / dir.x;
    }
    optional<ld> x_at(ld y) const {
        if (sgn(dir.y) == 0) return nullopt;
        return p.x + dir.x * (y - p.y) / dir.y;
    }
    // 构造 ax + by = c，a、b 均接近 0 时返回 nullopt
    static optional<Line> equation(ld a, ld b, ld c) {
        if (sgn(a) == 0 && sgn(b) == 0) return nullopt;
        Point q = fabsl(a) > fabsl(b) ? Point(c / a, 0) : Point(0, c / b);
        return Line(q, Point(-b, a));
    }
};
struct Segment {
    Point a, b;
    Segment() = default;
    Segment(Point a_, Point b_) : a(a_), b(b_) {}
};
```

浮点新增函数

```
// ===== 长度、角度与旋转 =====
ld length(Point p) { return sqrtl(norm2(p)); }
ld distance(Point a, Point b) { return sqrtl(dist2(a, b)); }
Point rotate(Point p, ld ang) {
    ld c{ cosl(ang) }, s{ sinl(ang) };
    return Point(p.x * c - p.y * s, p.x * s + p.y * c);
}
// 两个非零向量的无向夹角，范围为 [0, PI]
ld vec_ang(Point a, Point b) { return atan2l(fabsl(cross(a, b)), dot(a, b)); }
// ===== 直线交点、投影与距离 =====
optional<Point> line_int(Line a, Line b) {
    ld den{ cross(a.dir, b.dir) };
    if (sgn(den) == 0) return nullopt;
    ld t{ cross(b.p - a.p, b.dir) / den };
    return a.p + a.dir * t;
}
Point proj(Point p, Line ln) {
    if (ln.dir == Point{}) return ln.p;
    ld t{ dot(p - ln.p, ln.dir) / norm2(ln.dir) };
    return ln.p + ln.dir * t;
}
Point refl(Point p, Line ln) { return 2.L * proj(p, ln) - p; }
ld dis_line(Point p, Line ln) {
    if (ln.dir == Point{}) return distance(p, ln.p);
```

```
    return fabsl(cross(ln.dir, p - ln.p)) / length(ln.dir);
}
// 两条非退化直线的无向夹角，范围为 [0, PI]
ld line_ang(Line a, Line b) { return vec_ang(a.dir, b.dir); }
// ===== 线段距离 =====
ld dis_seg(Point p, Segment s) {
    Point dir{ s.b - s.a };
    if (dir == Point{}) return distance(p, s.a);
    ld t{ clamp(dot(p - s.a, dir) / norm2(dir), 0.L, 1.L) };
    return distance(p, s.a + dir * t);
}
ld seg_dis(Segment a, Segment b) {
    if (seg_int(a, b)) return 0.L;
    return min({ dis_seg(a.a, b), dis_seg(a.b, b), dis_seg(b.a, a), dis_seg(b.b, a) });
}
```

接口说明

基础结构

- `Point(x,y)`：构造点或向量
- `Line(p,dir)`：由一点和方向向量构造直线
- `Line::through(a,b)`：构造经过 `a,b` 的直线，$O(1)$
- `Segment(a,b)`：构造闭线段 $[a,b]$

通用接口

- `dot(a,b)`：向量点积，$O(1)$
- `cross(a,b)`：向量叉积，$O(1)$
- `dot(o,a,b)`：$\overrightarrow{oa}\cdot\overrightarrow{ob}$，$O(1)$
- `cross(o,a,b)`：$\overrightarrow{oa}\times\overrightarrow{ob}$，$O(1)$
- `norm2(p)`：向量长度平方，$O(1)$
- `dist2(a,b)`：两点距离平方，$O(1)$
- `rotate90(p)`：向量逆时针旋转 90°，$O(1)$
- `arg_less(a,b)`：从正 x 轴开始逆时针极角比较，同向按长度升序
- `ccw(a,b,p)`：判断 `p` 相对有向线段 `a → b` 的位置，$O(1)$
- `parallel(a,b)`：判断两直线方向是否平行，$O(1)$
- `perp(a,b)`：判断两直线方向是否垂直，$O(1)$
- `pt_side(p,ln)`：返回点在线的左侧 / 线上 / 右侧对应的 `1 / 0 / -1`，$O(1)$
- `on_line(p,ln)`：判断点是否在线上，支持退化直线，$O(1)$
- `line_rel(a,b)`：判断非退化直线唯一相交 / 平行 / 重合，$O(1)$
- `on_seg(p,s)`：判断点是否在闭线段上，$O(1)$
- `same_sd(a,b,ln,stct)`：判断两点是否位于直线同侧，$O(1)$
- `seg_int(a,b,stct)`：判断两线段是否相交，`stct = true` 时只接受内部严格交叉，$O(1)$

浮点新增接口

- `same_pt(a,b)`：按 `EPS` 判断两点重合，$O(1)$
- `length(p)`：向量长度，$O(1)$
- `distance(a,b)`：两点欧氏距离，$O(1)$
- `rotate(p,ang)`：逆时针旋转 `ang` 弧度，$O(1)$
- `vec_ang(a,b)`：两个非零向量的无向夹角，$O(1)$
- `Line::equation(a,b,c)`：构造 $ax+by=c$，非法时返回 `nullopt`，$O(1)$
- `x_at(y)` / `y_at(x)`：根据一个坐标求直线上另一坐标，不存在唯一值时返回 `nullopt`，$O(1)$
- `line_int(a,b)`：两条非平行直线交点，平行或重合返回 `nullopt`，$O(1)$
- `proj(p,ln)`：点在线上的正交投影，$O(1)$
- `refl(p,ln)`：点关于直线的对称点，$O(1)$
- `dis_line(p,ln)`：点到直线距离，$O(1)$
- `line_ang(a,b)`：两条非退化直线的无向夹角，$O(1)$
- `dis_seg(p,s)`：点到线段距离，$O(1)$
- `seg_dis(a,b)`：两线段最短距离，$O(1)$

注意事项

- 整数版的加减和数乘结果必须能由 `i64` 表示，叉积、点积、距离平方及其相关中间结果必须能由 `i128` 表示
- 浮点版的 `operator==` 和 $\Longleftrightarrow$ 比较原始坐标，几何意义下的点重合使用 `same_pt()`
- 浮点排序不能在比较器中使用 `EPS`，`arg_half()`、`arg_less()` 和字典序比较均保持原始值比较，否则可能破坏严格弱序
- `EPS` 只服务于几何分支，不代表最终答案允许误差

- 最终数值答案通常按 $\frac{|a-b|}{\max(1,|b|)} \le 10^{-6}$ 判断，建议输出至少 `12` 位小数
- `parallel()` 对零方向也会返回真，退化直线参与方向夹角、平行、垂直等问题时应根据题意单独处理
- `line_rel()` 要求两条直线均非退化
- `line_int()` 的 `nullopt` 同时表示平行与重合，需要区分时先调用 `line_rel()`
- `vec_ang()` 要求两个向量均非零，`line_ang()` 要求两条直线均非退化
- 判断浮点方向是否退化应使用 `v == Point{}`，只把精确零向量视为退化，不要使用 `sgn(norm2(v)) == 0`

常见用法

排序

```
ranges::sort(p);                  // 原始坐标字典序
ranges::sort(v, arg_less);        // 从正 x 轴开始逆时针极角排序
```

点积判断夹角

```
int rel = sgn(dot(u, v));
if (rel > 0) {}       // 锐角
else if (rel < 0) {}  // 钝角
else {}               // 垂直
```

叉积判断方向

```
int turn = sgn(cross(a, b, p));
if (turn > 0) {}       // p 在 a → b 左侧
else if (turn < 0) {}  // p 在 a → b 右侧
else {}                // 共线
```

构造直线并判断关系

```
Line a = Line::through(p1, p2);
Line b = Line::through(q1, q2);
if (parallel(a, b)) {}
if (perp(a, b)) {}
LineRel rel = line_rel(a, b);
```

判断点与线段

```
if (on_line(p, ln)) {}
if (on_seg(p, seg)) {}
if (seg_int(a, b)) {}
if (seg_int(a, b, true)) {} // 仅内部严格交叉
```

浮点旋转、交点与距离

```
Point q = rotate(p, PI / 3);
ld ang = vec_ang(u, v);
auto isec = line_int(a, b);
if (isec) {}
Point foot = proj(p, ln);
ld d1 = dis_line(p, ln);
ld d2 = dis_seg(p, seg);
```

浮点输出

```
auto fix0 = [](ld x) { return sgn(x) == 0 ? 0.L : x; };
cout << fixed << setprecision(12) << fix0(p.x) << ' ' << fix0(p.y) << '\n';
```

15.2 多边形、包含与 Minkowski 和

有向面积

解决问题： 判断多边形顶点方向，计算多边形面积，并作为重心、凸多边形规范化、Pick 定理等算法的基础

设多边形顶点依次为 $p_0, p_1, \dots, p_{n-1}$，并令 $p_n = p_0$，则有向面积的两倍为

$$S_2 = \sum_{i=0}^{n-1} p_i \times p_{i+1}$$

- $S_2 > 0$：顶点按逆时针排列
- $S_2 < 0$：顶点按顺时针排列
- 实际面积为 $\frac{|S_2|}{2}$

```
using Area2 = decltype(cross(Point{}, Point{}));
// 返回多边形有向面积的两倍；整数版为 i128，浮点版为 ld
Area2 poly_area2(const vector<Point> &poly) {
    Area2 area2{};
    for (int i = 1, n = poly.size(); i + 1 < n; ++ i) {
        area2 += cross(poly[0], poly[i], poly[i + 1]);
    }
    return area2;
}
```

凸多边形规范化与凸性判断

解决问题： 统一凸多边形方向和起点，删除重复点与共线冗余点，使凸包含、Minkowski 和等接口满足前提；也可判断一个按边界排列的多边形是否为凸多边形

`norm_conv()` 进行以下处理：

1. 删除重复末点和连续重复点
2. 非退化多边形统一为逆时针
3. 删除位于相邻两点之间的共线冗余点
4. 将起点旋转为 (y, x) 字典序最小的点
5. 全部共线时只保留线段的两个端点

```
// 规范化按边界顺序给出的凸多边形；输入不是凸集边界时结果没有意义
vector<Point> norm_conv(vector<Point> poly) {
    if (poly.size() > 1 && poly.front() == poly.back()) poly.pop_back();
    vector<Point> pts;
    for (Point p : poly) {
        if (pts.empty() || p != pts.back()) pts.push_back(p);
    }
    if (pts.size() > 1 && pts.front() == pts.back()) pts.pop_back();
    if (pts.size() <= 1) return pts;
    int dir{ sgn(poly_area2(pts)) };
    if (dir == 0) {
        ranges::sort(pts, less{});
        pts.erase(ranges::unique(pts).begin(), pts.end());
        if (pts.size() <= 1) return pts;
        Point a{ pts.front() }, b{ pts.back() };
        if (tie(b.y, b.x) < tie(a.y, a.x)) swap(a, b);
        return {a, b};
    }
    if (dir < 0) ranges::reverse(pts);
    vector<Point> res;
    int n = pts.size();
    for (int i = 0; i < n; ++ i) {
        Point pre{ pts[(i + n - 1) % n] }, cur{ pts[i] }, nxt{ pts[(i + 1) % n] };
        // cur 位于线段 [pre,nxt] 上时，它是凸多边形边界上的冗余点
        if (sgn(cross(pre, cur, nxt)) == 0 && sgn(dot(cur, pre, nxt)) <= 0) continue;
        res.push_back(cur);
    }
    int st{ int(ranges::min_element(res, [](Point a, Point b) {
        return a.y != b.y ? a.y < b.y : a.x < b.x;
    }) - res.begin()) };
    ranges::rotate(res, res.begin() + st);
    return res;
}
// 判断按边界顺序给出的多边形是否为凸多边形；全部共线不视为凸多边形
bool is_conv(const vector<Point> &poly, bool allo_col = true) {
    int n = poly.size();
    if (n < 3) return false;
    int dir{ 0 };
    for (int i = 0; i < n; ++ i) {
```

```
        int turn{ sgn(cross(poly[i], poly[(i + 1) % n], poly[(i + 2) % n])) };
        if (turn == 0) {
            if (!allo_col) return false;
        } else if (dir == 0) dir = turn;
        else if (dir != turn) return false;
    }
    return dir != 0;
}
```

点与多边形的包含关系

解决问题： 判断查询点位于简单多边形外部、边界还是内部；凸多边形经一次规范化后可将单次查询优化到 $O(\log n)$

普通版本使用射线法：从查询点向右作射线，若与边界的有效穿越次数为奇数，则点在内部。代码使用半开纵坐标条件处理顶点，避免同一个顶点被相邻两条边重复计算

凸多边形版本把多边形划分为以 `poly[0]` 为公共顶点的三角形扇，先判断查询点是否位于首尾两条射线之间，再二分所在扇区

```
enum class Contain {
    OUTSIDE,  // 多边形外部
    BOUNDARY, // 多边形边界
    INSIDE    // 多边形内部
};
// 整数版精确比较；浮点版按 EPS 判断两个点是否重合
bool same_geo(Point a, Point b) {
    if constexpr (is_integral_v<decltype(Point{}.x)>) return a == b;
    else return sgn(a.x - b.x) == 0 && sgn(a.y - b.y) == 0;
}
// 判断点与简单多边形的包含关系；多边形方向可以为顺时针或逆时针
Contain in_poly(const vector<Point> &poly, Point p) {
    int wind{ 0 };
    for (int i = 0, n = poly.size(); i < n; ++ i) {
        Point a{ poly[i] }, b{ poly[(i + 1) % n] };
        if (on_seg(p, Segment(a, b))) return Contain::BOUNDARY;
        int turn{ sgn(cross(a, b, p)) };
        if (a.y <= p.y) {
            if (b.y > p.y && turn > 0) ++ wind;
        } else if (b.y <= p.y && turn < 0) -- wind;
    }
    return wind == 0 ? Contain::OUTSIDE : Contain::INSIDE;
}
// poly 必须是 norm_conv() 的结果；支持空集、点、线段和严格凸多边形
Contain in_conv(const vector<Point> &poly, Point p) {
    int n = poly.size();
    if (n == 0) return Contain::OUTSIDE;
    if (n == 1) return same_geo(p, poly[0]) ? Contain::BOUNDARY : Contain::OUTSIDE;
    if (n == 2) return on_seg(p, Segment(poly[0], poly[1])) ? Contain::BOUNDARY : Contain::OUTSIDE;
    int l{ sgn(cross(poly[0], poly[1], p)) }, r{ sgn(cross(poly[0], poly.back(), p)) };
    if (l < 0 || r > 0) return Contain::OUTSIDE;
    if (l == 0) return on_seg(p, Segment(poly[0], poly[1])) ? Contain::BOUNDARY : Contain::OUTSIDE;
    if (r == 0) return on_seg(p, Segment(poly[0], poly.back())) ? Contain::BOUNDARY : Contain::OUTSIDE;
    int low{ 1 }, high{ n - 1 };
    while (high - low > 1) {
        int mid{ (low + high) >> 1 };
        if (sgn(cross(poly[0], poly[mid], p)) >= 0) low = mid;
        else high = mid;
    }
    int side{ sgn(cross(poly[low], poly[high], p)) };
    if (side < 0) return Contain::OUTSIDE;
    if (side == 0) return Contain::BOUNDARY;
    return Contain::INSIDE;
}
```

⚠ 浮点版中，`in_poly()` 的边界判定使用 `EPS`，射线的半开纵坐标比较仍使用原始 `y` 值。不要把半开比较随意改成多个互不一致的 `EPS` 判断，否则顶点附近可能重复计数

周长与重心

解决问题： 计算多边形边界总长度，以及均匀薄片多边形的几何重心

周长为

$$L = \sum_{i=0}^{n-1} |p_{i+1} - p_i|$$

非零面积均匀多边形的重心为

$$C = \frac{\sum_{i=0}^{n-1} (p_i + p_{i+1})(p_i \times p_{i+1})}{3S_2}$$

```
// 返回闭合多边形的周长；空集、单点的周长为 0
ld poly_peri(const vector<Point> &poly) {
    ld peri{ 0 };
    for (int i = 0, n = poly.size(); i < n; ++ i) {
        peri += length(poly[(i + 1) % n] - poly[i]);
    }
    return peri;
}
// 返回均匀薄片多边形的重心；有向面积接近 0 时返回 nullopt
optional<Point> poly_ctr(const vector<Point> &poly) {
    ld area2{ poly_area2(poly) };
    if (sgn(area2) == 0) return nullopt;
    Point num;
    for (int i = 0, n = poly.size(); i < n; ++ i) {
        Point a{ poly[i] - poly[0] }, b{ poly[(i + 1) % n] - poly[0] };
        num += (a + b) * cross(a, b);
    }
    return poly[0] + num / (3 * area2);
}
```

线段是否完全位于简单多边形内

解决问题： 判断闭线段的每一个点是否都位于简单多边形内部或边界，适用于“可见线段是否越出区域”“运动路径是否合法”等问题。
将线段参数化为

$$q(t) = a + t(b - a), \qquad 0 \le t \le 1$$

收集线段与多边形边界相交处的全部参数 t，排序后，任意相邻参数之间的开区间不会再次穿越边界，因此只需检查每段中点

```
// ===== 浮点版专属：依赖 line_int()，整数版若需要应改用有理数 =====
// 边界视为多边形的一部分；poly 必须是无重复末点的简单多边形
bool seg_in(const vector<Point> &poly, Segment seg) {
    if (in_poly(poly, seg.a) == Contain::OUTSIDE || in_poly(poly, seg.b) == Contain::OUTSIDE) {
        return false;
    }
    Point dir{ seg.b - seg.a };
    ld len2{ norm2(dir) };
    if (len2 == 0) return true;
    vector<ld> pars{0.L, 1.L};
    auto add_par = [&](Point p) {
        ld t{ dot(p - seg.a, dir) / len2 };
        if (t >= -EPS && t <= 1 + EPS) pars.push_back(clamp(t, 0.L, 1.L));
    };
    for (int i = 0, n = int(poly.size()); i < n; ++ i) {
        Segment edge(poly[i], poly[(i + 1) % n]);
        if (on_seg(edge.a, seg)) add_par(edge.a);
        if (on_seg(edge.b, seg)) add_par(edge.b);
        if (seg_int(seg, edge, true)) {
            auto isec = line_int(Line::through(seg.a, seg.b), Line::through(edge.a, edge.b));
            if (isec) add_par(*isec);
        }
    }
    ranges::sort(pars);
    pars.erase(unique(pars.begin(), pars.end(), [](ld a, ld b) {
        return sgn(a - b) == 0;
    }), pars.end());
    for (int i = 0; i + 1 < pars.size(); ++ i) {
        ld mid{ (pars[i] + pars[i + 1]) / 2 };
        Point p{ seg.a + dir * mid };
        if (in_poly(poly, p) == Contain::OUTSIDE) return false;
```

```
    }
    return true;
}
```

有向直线切割多边形

解决问题： 用一个半平面裁剪多边形；连续对凸多边形的每条边裁剪，可求两个凸多边形的交。
对于有向直线 `ln.p + t * ln.dir`，保留其左侧闭半平面：

$$H = \{p \mid ln.dir \times (p - ln.p) \geq 0\}.$$

```
// ===== 浮点版专属：交点一般不是整数点，依赖 line_int() =====
// 保留有向直线左侧和直线上的部分；输入顶点按边界顺序排列
vector<Point> cut_poly(const vector<Point> &poly, Line ln) {
    vector<Point> res;
    for (int i = 0, n = poly.size(); i < n; ++ i) {
        Point a{ poly[i] }, b{ poly[(i + 1) % n] };
        int sd_a{ pt_side(a, ln) }, sd_b{ pt_side(b, ln) };
        if (sd_a >= 0) res.push_back(a);
        if (sd_a * sd_b < 0) {
            auto isec = line_int(Line::through(a, b), ln);
            if (isec) res.push_back(*isec);
        }
    }
    return res;
}
// 求两个非退化凸多边形的交；返回结果可能为空、点、线段或凸多边形
vector<Point> conv_int(vector<Point> poly, vector<Point> clip) {
    poly = norm_conv(move(poly));
    clip = norm_conv(move(clip));
    if (poly.size() < 3 || clip.size() < 3) return {};
    for (int i = 0, n = clip.size(); i < n && !poly.empty(); ++ i) {
        poly = cut_poly(poly, Line::through(clip[i], clip[(i + 1) % n]));
    }
    return norm_conv(move(poly));
}
```

Ear Clipping 三角剖分

解决问题： 将无孔简单多边形划分为 $n-2$ 个内部互不相交的三角形，便于面积积分、渲染或后续动态规划。
一个凸顶点 b 与相邻顶点 a, c 构成"耳"，当且仅当三角形 $\triangle abc$ 内部和边界上不含当前多边形的其他顶点。剪去一只耳后继续处理剩余多边形。

```
// 返回三角形的原始顶点下标；失败或输入退化时返回空数组
vector<array<int, 3>> ear_clip(vector<Point> poly) {
    int n = poly.size();
    vector<array<int, 3>> tris;
    if (n < 3 || sgn(poly_area2(poly)) == 0) return tris;
    vector<int> org(n);
    ranges::iota(org, 0);
    if (sgn(poly_area2(poly)) < 0) {
        ranges::reverse(poly);
        ranges::reverse(org);
    }
    vector<int> idx(n);
    ranges::iota(idx, 0);
    auto in_tri = [&](Point p, Point a, Point b, Point c) {
        return sgn(cross(a, b, p)) >= 0 && sgn(cross(b, c, p)) >= 0 && sgn(cross(c, a, p)) >= 0;
    };
    while (idx.size() > 3) {
        bool found{ false };
        for (int at = 0; at < idx.size(); ++ at) {
            int pre{ idx[(at + idx.size() - 1) % idx.size()] };
            int cur{ idx[at] }, nxt{ idx[(at + 1) % idx.size()] };
            if (sgn(cross(poly[pre], poly[cur], poly[nxt])) <= 0) continue;
            bool has{ false };
            for (int v : idx) {
                if (v == pre || v == cur || v == nxt) continue;
                if (in_tri(poly[v], poly[pre], poly[cur], poly[nxt])) {
                    has = true;
```

```
                    break;
                }
            }
            if (has) continue;
            tris.push_back({org[pre], org[cur], org[nxt]});
            idx.erase(idx.begin() + at);
            found = true;
            break;
        }
        if (!found) return {};
    }
    tris.push_back({org[idx[0]], org[idx[1]], org[idx[2]]});
    return tris;
}
```

Pick 定理与格点计数

解决问题： 对顶点均为整点的简单多边形，计算边界格点数和内部格点数
对边 $a \to b$，令 $\Delta x = b_x - a_x$、$\Delta y = b_y - a_y$，该边对边界格点总数的贡献为

$$\gcd(|\Delta x|, |\Delta y|)$$

设内部格点数为 I、边界格点数为 B、实际面积为 S，Pick 定理为

$$S = I + \frac{B}{2} - 1, \qquad I = \frac{|S_2| - B + 2}{2}$$

```
// ===== 整数版专属：浮点坐标不适用 Pick 定理 =====
i128 gcd128(i128 a, i128 b) {
    if (a < 0) a = -a;
    if (b < 0) b = -b;
    while (b) {
        a %= b;
        swap(a, b);
    }
    return a;
}
// 返回 { 边界格点数 B, 内部格点数 I }
pair<i128, i128> lat_cnt(const vector<Point> &poly) {
    i128 bnd{ 0 }, area2{ 0 };
    for (int i = 0, n = poly.size(); i < n; ++ i) {
        Point a{ poly[i] }, b{ poly[(i + 1) % n] };
        bnd += gcd128(i128(b.x) - a.x, i128(b.y) - a.y);
        area2 += cross(a, b);
    }
    if (area2 < 0) area2 = -area2;
    return { bnd, (area2 - bnd + 2) / 2 };
}
```

Minkowski 和

解决问题： 合成两个凸集合的所有向量和，常用于凸多边形平移、碰撞判定、距离问题以及把“两点分别取自两个集合”的问题转化为点包含
两个集合的 Minkowski 和定义为

$$A \oplus B = \{a + b \mid a \in A,\ b \in B\}$$

凸多边形的边向量按极角有序，$A \oplus B$ 的边界向量正是两组边向量的归并；同方向边应合并为向量和，反方向边不能合并

```
// 求两个凸多边形、线段或点集凸边界的 Minkowski 和
vector<Point> mink_sum(vector<Point> a, vector<Point> b) {
    a = norm_conv(move(a));
    b = norm_conv(move(b));
    if (a.empty() || b.empty()) return {};
    if (a.size() == 1) {
        for (Point &p : b) p += a[0];
        return b;
    }
    if (b.size() == 1) {
        for (Point &p : a) p += b[0];
```

```
        return a;
    }
    vector<Point> edge_a(a.size()), edge_b(b.size());
    for (int i = 0; i < a.size(); ++ i) edge_a[i] = a[(i + 1) % a.size()] - a[i];
    for (int i = 0; i < b.size(); ++ i) edge_b[i] = b[(i + 1) % b.size()] - b[i];
    vector<Point> res{ a[0] + b[0] };
    int i{ 0 }, j{ 0 };
    while (i < edge_a.size() || j < edge_b.size()) {
        Point edge;
        if (i == edge_a.size()) edge = edge_b[j ++];
        else if (j == edge_b.size()) edge = edge_a[i ++];
        else {
            bool same_dir{ sgn(cross(edge_a[i], edge_b[j])) == 0 && sgn(dot(edge_a[i], edge_b[j])) > 0 };
            if (same_dir) edge = edge_a[i ++] + edge_b[j ++];
            else if (arg_less(edge_a[i], edge_b[j])) edge = edge_a[i ++];
            else edge = edge_b[j ++];
        }
        res.push_back(res.back() + edge);
    }
    res.pop_back(); // 最后一个点与起点重合
    return norm_conv(move(res));
}
```

多边形核

解决问题： 判断简单多边形是否存在一个点，使该点能够看见多边形内的所有点，并求出所有这类点组成的区域。
若简单多边形按逆时针排列，则其核为每条有向边左侧闭半平面的交：

$$\mathrm{Kernel}(P)=\bigcap_{i=0}^{n-1}\{x\mid (p_{i+1}-p_i)\times(x-p_i)\geq 0\}$$

将每条边构造成 `Line::through(poly[i], poly[(i + 1) % n])`，调用半平面交板子即可；交集为空表示多边形没有核。

接口说明

公共约定

- 多边形顶点按边界顺序给出，不重复存储末尾的首点；空集、点和线段仅在接口明确说明时允许
- `poly_area2()`、`in_poly()`、`is_conv()`、`ear_clip()`、`norm_conv()` 和 `mink_sum()` 可用于整数版与浮点版
- 浮点版的几何判定统一通过基础板子的 `sgn()` 使用 `EPS`；排序仍使用原始坐标比较
- `norm_conv()` 只负责规范化已经按凸集边界排列的输入，不会把任意点集求成凸包

面积、方向与规范化

- `poly_area2(poly)`：返回有向面积的两倍；正数表示逆时针，负数表示顺时针，$O(n)$
- `norm_conv(poly)`：规范化凸多边形、线段或点；删除共线冗余点并统一方向、起点，$O(n\log n)$，其中只有退化为共线点集时需要排序，非退化输入为 $O(n)$
- `is_conv(poly, allo_col)`：判断按边界排列的多边形是否凸；`allo_col=false` 时任意连续三点共线都会返回 `false`，$O(n)$

包含关系

- `Contain::OUTSIDE / BOUNDARY / INSIDE`：分别表示外部、边界和内部
- `same_geo(a,b)`：整数版精确判等，浮点版按 `EPS` 判断点重合，$O(1)$
- `in_poly(poly, p)`：判断点与简单多边形的关系；支持顺、逆时针，$O(n)$
- `in_conv(poly, p)`：判断点与规范化凸集的关系；调用前应执行一次 `poly = norm_conv(poly)`，单次查询 $O(\log n)$
- `seg_in(poly, seg)`：判断闭线段是否完全位于简单多边形的内部或边界；仅浮点版，$O(n\log n)$

构造与分解

- `poly_peri(poly)`：返回周长；仅浮点版，$O(n)$
- `poly_ctr(poly)`：返回均匀薄片多边形的重心；面积接近零时返回 `nullopt`，仅浮点版，$O(n)$
- `cut_poly(poly, ln)`：保留有向直线左侧闭半平面内的部分；仅浮点版，$O(n)$
- `conv_int(a, b)`：求两个非退化凸多边形的交；返回数组可能表示空集、点、线段或凸多边形，仅浮点版，逐边裁剪最坏 $O(nm)$
- `ear_clip(poly)`：返回 `n-2` 个三角形的原始顶点下标；输入不合法或无法继续剪耳时返回空数组，$O(n^3)$

格点与 Minkowski 和

- `lat_cnt(poly)`：返回 `{B,I}`，即边界与内部格点数；仅整数版，要求整点简单多边形，$O(n)$
- `mink_sum(a,b)`：返回两个凸边界的 Minkowski 和并自动规范化；允许输入为空集、点、线段或非退化凸多边形，$O(n+m)$，退化输入规范化排序除外

注意事项

- `in_poly()` 只保证简单多边形语义；自交多边形会按非零环绕数规则分类，不应把它当作普通内部判定
- `in_conv()` 的 $O(\log n)$ 前提是输入已由 `norm_conv()` 处理；直接传入带首尾共线冗余点的普通凸多边形可能破坏扇形二分
- `poly_ctr()` 计算的是均匀面积薄片的重心，不是顶点坐标平均值；面积为零时没有该意义下的重心
- `seg_in()` 把边界视为合法区域；只要求严格位于内部时，还需要排除端点或中点落在 `BOUNDARY` 的情况。
- `ear_clip()` 要求无孔简单多边形。连续重复点、共线冗余点会阻止剪耳，应在保留原始下标映射的前提下预处理；带孔或约束三角剖分应使用 CDT，而不是本函数
- `cut_poly()` 与 `conv_int()` 会构造交点，因此默认只放在浮点版；整数输入的交点也可能是分数
- Pick 定理仅适用于整点简单多边形；`|S_2|=B+2` 理论上必为非负偶数，若不是，通常说明输入或面积、边界计数发生错误
- Minkowski 和要求输入按凸边界顺序给出。非凸多边形必须先做凸包或凸分解，不能直接归并原边向量
- `norm_conv()` 使用 `Point::operator==` 删除完全重复点；浮点输入若存在数值误差造成的近重复点，应在调用前按统一 `EPS` 清理
- 多边形核只对按逆时针排列的边使用左半平面；顺时针输入应先反转

常见用法

计算面积并统一为逆时针

```
Area2 area2 = poly_area2(poly);
if (sgn(area2) < 0) ranges::reverse(poly);
// 实际面积为 abs(area2) / 2; 整数版可能出现半整数面积
```

判断点是否位于简单多边形内或边界

```
Contain where = in_poly(poly, q);
if (where != Contain::OUTSIDE) {} // 内部或边界
```

多次查询点是否位于凸多边形

```
poly = norm_conv(move(poly)); // 只预处理一次
for (Point q : qs) {
    Contain where = in_conv(poly, q);
}
```

判断线段是否完全留在区域中

```
// [浮点版专属]
if (seg_in(poly, Segment(a, b))) {} // 允许贴着边界移动
```

用直线裁掉凸多边形的一部分

```
// [浮点版专属]
Line ln = Line::through(a, b);
poly = cut_poly(poly, ln); // 只保留有向直线 a → b 左侧
```

求两个凸多边形交集的面积

```
// [浮点版专属]
vector<Point> isec = conv_int(a, b);
ld area = fabsl(poly_area2(isec)) / 2;
```

对简单多边形三角剖分

```
auto tris = ear_clip(poly);
if (poly.size() >= 3 && tris.empty()) {} // 输入退化、非简单或存在未清理问题
for (auto [a, b, c] : tris) {
    // 三角形顶点为 poly[a]、poly[b]、poly[c]
}
```

使用 Pick 定理

```
// [整数版专属]
auto [bnd, is_in] = lat_cnt(poly);
```

计算 Minkowski 和

```
vector<Point> sum = mink_sum(a, b);
```

用 Minkowski 和判断两个凸多边形是否相交

A 与 B 相交，当且仅当原点属于 $A \oplus (-B)$：

```
vector<Point> neg_b = b;
for (Point &p : neg_b) p = -p;
vector<Point> dif = mink_sum(a, move(neg_b));
bool hit = in_conv(dif, Point{}) != Contain::OUTSIDE;
```

求多边形核

```
if (sgn(poly_area2(poly)) < 0) ranges::reverse(poly);
vector<Line> hps;
for (int i = 0, n = poly.size(); i < n; ++ i) {
    hps.push_back(Line::through(poly[i], poly[(i + 1) % n]));
}
// 将 hps 交给“半平面交”板子；结果为空表示无核
```

复杂度

操作	时间复杂度
`poly_area2()` / `poly_peri()` / `poly_ctr()`	$O(n)$
`norm_conv()`	非退化 $O(n)$；全部共线 $O(n \log n)$
`is_conv()` / `in_poly()`	$O(n)$
`in_conv()`	$O(\log n)$
`seg_in()`	$O(n \log n)$
`cut_poly()`	$O(n)$
`conv_int()`	最坏 $O(nm)$
`ear_clip()`	$O(n^3)$
`lat_cnt()`	$O(n)$
`mink_sum()`	$O(n+m)$，不计退化输入排序

除返回结果外，各判定函数额外空间为 $O(1)$；需要构造新多边形的接口空间复杂度与输出点数同阶

15.3 凸包、切点与旋转卡壳

Andrew 凸包

解决问题： 求包含给定点集的最小凸多边形，作为凸包含、直径、旋转卡壳、Minkowski 和等算法的预处理

按 (x, y) 字典序扫描点集。若当前点使凸链发生顺时针转向，就不断删除链尾

$$(p_{k-1} - p_{k-2}) \times (p - p_{k-1}) < 0$$

`keep_col=false` 时还会删除边上的中间点，得到严格凸包；`true` 时保留所有边界点

```
vector<Point> conv_hull(vector<Point> pts, bool keep_col = false) {
    ranges::sort(pts);
    pts.erase(ranges::unique(pts).begin(), pts.end());
    if (pts.size() <= 1) return pts;
    bool all_col{ true };
    for (int i = 2; i < pts.size(); ++ i) {
        if (sgn(cross(pts[0], pts[1], pts[i])) != 0) {
            all_col = false;
            break;
        }
    }
    if (all_col) return keep_col ? pts : vector<Point>{ pts.front(), pts.back() };
    vector<Point> lo, up;
    auto push = [&](vector<Point> &ch, Point p) {
        while (ch.size() >= 2) {
            int turn{ sgn(cross(ch[ch.size() - 2], ch.back(), p)) };
            if (turn > 0 || (keep_col && turn == 0)) break;
            ch.pop_back();
```

```
        }
        ch.push_back(p);
    };
    for (Point p : pts) push(lo, p);
    for (auto it = pts.rbegin(); it != pts.rend(); ++ it) push(up, *it);
    lo.pop_back();
    up.pop_back();
    lo.insert(lo.end(), up.begin(), up.end());
    return lo;
}
```

输出按逆时针排列，不重复存储首点。除全部共线且 `keep_col=true` 外，起点是 (x, y) 字典序最小点

Graham 扫描（可选）

解决问题： 当题目本身还需要围绕一个基点进行极角排序时，可复用排序结果构造凸包。普通求凸包优先使用更短、更稳定的 Andrew

```
// ===== Graham 扫描：若只使用 Andrew，以下代码可直接删除 =====
vector<Point> gra_hull(vector<Point> pts, bool keep_col = false) {
    ranges::sort(pts, less{});
    pts.erase(ranges::unique(pts).begin(), pts.end());
    if (pts.size() <= 1) return pts;
    bool all_col{ true };
    for (int i = 2; i < pts.size(); ++ i) {
        if (sgn(cross(pts[0], pts[1], pts[i])) != 0) {
            all_col = false;
            break;
        }
    }
    if (all_col) return keep_col ? pts : vector<Point>{ pts.front(), pts.back() };
    int piv{ int(ranges::min_element(pts, [](Point a, Point b) {
            return a.y != b.y ? a.y < b.y : a.x < b.x;
        }) - pts.begin()) };
    swap(pts[0], pts[piv]);
    Point o{ pts[0] };
    sort(pts.begin() + 1, pts.end(), [&](Point a, Point b) {
        return arg_less(a - o, b - o);
    });
    // 保留共线点时，最后一条同角射线必须反转，才能沿凸包边界依次经过这些点。
    if (keep_col) {
        int first{ int(pts.size()) - 1 };
        while (first > 1 && sgn(cross(pts[first - 1] - o, pts.back() - o)) == 0) -- first;
        ranges::reverse(pts.begin() + first, pts.end());
    }
    vector<Point> hull;
    for (Point p : pts) {
        while (hull.size() >= 2) {
            int turn{ sgn(cross(hull[hull.size() - 2], hull.back(), p)) };
            if (turn > 0 || (keep_col && turn == 0)) break;
            hull.pop_back();
        }
        hull.push_back(p);
    }
    return hull;
}
```

直线与凸包的交点（线性兜底）

解决问题： 求一条直线与凸多边形边界的全部交点。一般位置下返回 0、1 或 2 个点；直线与某条边重合时返回该重合边的端点

```
/* ===== 浮点版专属：若不需要直线与凸包求交，以下代码可直接删除 ===== */
vector<Point> line_cv_int(const vector<Point> &hull, Line ln) {
    vector<Point> res;
    auto add = [&](Point p) {
        for (Point old : res) if (same_pt(old, p)) return;
        res.push_back(p);
    };
    for (int i = 0, n = hull.size(); i < n; ++ i) {
        Point a{ hull[i] }, b{ hull[(i + 1) % n] };
        int sd_a{ pt_side(a, ln) }, sd_b{ pt_side(b, ln) };
        if (sd_a == 0) add(a);
```

```
        if (sd_a * sd_b < 0) {
            auto p = line_int(ln, Line::through(a, b));
            if (p) add(*p);
        }
    }
    return res;
}
```

点是否位于凸包内直接使用上一篇的 `in_conv()`；两个凸多边形求交直接使用 `conv_int()`

凸包切点（可靠线性版）

解决问题： 给定凸多边形外一点 `q`，求经过该点的两条支撑线与凸包接触的顶点
对严格凸多边形的顶点 p_i，若相邻点 p_{i-1}, p_{i+1} 位于有向直线 `q → p[i]` 的同侧，则 p_i 是切点

```
// hull 应为无共线冗余点的逆时针凸包；q 应严格位于凸包外
vector<int> tan_ver(const vector<Point> &hull, Point q) {
    int n = hull.size();
    if (n ⩽ 2) {
        vector<int> res(n);
        ranges::iota(res, 0);
        return res;
    }
    vector<int> res;
    for (int i = 0; i < n; ++ i) {
        Point dir{ hull[i] - q };
        int pre{ sgn(cross(dir, hull[(i + n - 1) % n] - q)) };
        int nxt{ sgn(cross(dir, hull[(i + 1) % n] - q)) };
        if (pre == 0 || nxt == 0 || pre == nxt) res.push_back(i);
    }
    return res;
}
```

查询点严格在外部时，通常返回两个下标。查询点落在某条边的延长线上时，这条支撑边的两个端点都会返回，因此严格凸包也可能返回三个下标；需要两个代表切点时使用 `tan_fast()`。若保留了共线冗余点，先调用 `norm_conv()`

凸包快速查询

解决问题： 对严格凸多边形进行方向极值点、直线求交边和凸包外点切点的 $O(\log n)$ 查询
以下接口要求 `poly` 是无共线冗余点的逆时针严格凸多边形。若只进行少量查询，前面的线性版本更短且更容易处理退化情况

```
// 返回使 dot(poly[i],dir) 最大的顶点下标；dir 必须非零
int ext_ver(const vector<Point> &poly, Point dir) {
    int n = poly.size();
    if (n ⩽ 1) return n - 1;
    if (n == 2) return sgn(dot(poly[1] - poly[0], dir)) > 0 ? 1 : 0;
    auto cmp = [&](int i, int j) { return sgn(cross(rotate90(dir), poly[i % n] - poly[j % n])); };
    auto ext = [&](int i) { return cmp(i + 1, i) ⩾ 0 && cmp(i, i - 1 + n) < 0; };
    if (ext(0)) return 0;
    int lo{ 0 }, hi{ n };
    while (lo + 1 < hi) {
        int mid{ (lo + hi) >> 1 };
        if (ext(mid)) return mid;
        int ls{ cmp(lo + 1, lo) }, ms{ cmp(mid + 1, mid) };
        if (ls < ms || (ls == ms && ls == cmp(lo, mid))) hi = mid;
        else lo = mid;
    }
    return lo;
}
// 返回直线与凸包相交位置对应的边下标；ln.dir 必须非零
array<int, 2> line_hull(const vector<Point> &poly, Line ln) {
    int n = poly.size();
    Point a{ ln.p }, b{ ln.p + ln.dir };
    auto cmp = [&](int i) { return sgn(cross(a, poly[i], b)); };
    int ea{ ext_ver(poly, rotate90(a - b)) }, eb{ ext_ver(poly, rotate90(b - a)) };
    if (cmp(ea) < 0 || cmp(eb) > 0) return {-1, -1};
    array<int, 2> res;
    for (int z = 0; z < 2; ++ z) {
        int lo{ eb }, hi{ ea };
        while ((lo + 1) % n ≠ hi) {
            int mid{ ((lo + hi + (lo < hi ? 0 : n)) >> 1) % n };
```

```
            if (cmp(mid) == cmp(eb)) lo = mid;
            else hi = mid;
        }
        res[z] = (lo + (cmp(hi) == 0)) % n;
        swap(ea, eb);
    }
    if (res[0] == res[1]) return {res[0], -1};
    if (cmp(res[0]) == 0 && cmp(res[1]) == 0) {
        int d{ (res[0] - res[1] + n + 1) % n };
        if (d == 0) return {res[0], res[0]};
        if (d == 2) return {res[1], res[1]};
    }
    return res;
}
// q 必须严格位于凸包外；返回两条切线的代表切点下标
array<int, 2> tan_fast(const vector<Point> &poly, Point q) {
    int n{ int(poly.size()) };
    auto vis_e = [&](int i) {
        return sgn(cross(poly[i] - q, poly[(i + 1) % n] - poly[i])) < 0;
    };
    auto vis = [&](int i) { return vis_e(i) || vis_e((i - 1 + n) % n); };
    auto rgt = [&](int i) { return sgn(cross(poly[0] - q, poly[i] - q)) < 0; };
    int lo, hi, mid, ta{ -1 }, tb{ -1 };
    if (vis(0)) {
        lo = 0, hi = n;
        while (hi - lo > 1) {
            mid = (lo + hi) >> 1;
            if (vis(mid) && rgt(mid)) lo = mid;
            else hi = mid;
        }
        tb = lo, lo = 0, hi = n;
        while (hi - lo > 1) {
            mid = (lo + hi) >> 1;
            if (vis((tb + n - mid) % n)) lo = mid;
            else hi = mid;
        }
        ta = (tb + n - lo) % n;
    } else {
        lo = 0, hi = n;
        while (hi - lo > 1) {
            mid = (lo + hi) >> 1;
            if (!vis(mid) && !rgt(mid)) lo = mid;
            else hi = mid;
        }
        ta = (lo + 1) % n;
        lo = 0, hi = n;
        while (hi - lo > 1) {
            mid = (lo + hi) >> 1;
            if (vis((ta + mid) % n)) lo = mid;
            else hi = mid;
        }
        tb = (ta + lo) % n;
    }
    return {ta, tb};
}
```

对踵点与凸包直径（旋转卡壳）

解决问题： 求凸多边形的最远点对，以及枚举互相存在平行支撑线的一组对踵顶点

固定边 p_ip_{i+1}，与其距离最远的顶点使

$$|(p_{i+1}-p_i)\times(p_j-p_i)|$$

最大。边按逆时针旋转时，最远顶点也只会沿凸包单调前进，因此总扫描次数为线性级别

```
using HullArea2 = decltype(cross(Point{}, Point{}));
using HullDist2 = decltype(norm2(Point{}));
// 返回去重后的对踵顶点对；排序去重使本接口总复杂度为 O(n log n)
vector<pair<int, int>> anti_pair(const vector<Point> &hull) {
    int n = hull.size();
```

```
    if (n <= 1) return {};
    if (n == 2) return {{0, 1}};
    vector<pair<int, int>> res;
    auto area = [&](int i, int j, int k) -> HullArea2 {
        HullArea2 value{ cross(hull[j] - hull[i], hull[k] - hull[i]) };
        return value < 0 ? -value : value;
    };
    auto add = [&](int a, int b) {
        if (a == b) return;
        if (a > b) swap(a, b);
        res.push_back({ a, b });
    };
    int j{ 1 };
    for (int i = 0; i < n; ++ i) {
        int nxt_i{ (i + 1) % n };
        while (sgn(area(i, nxt_i, (j + 1) % n) - area(i, nxt_i, j)) > 0) j = (j + 1) % n;
        add(i, j);
        add(nxt_i, j);
        int nxt_j{ (j + 1) % n };
        if (sgn(area(i, nxt_i, nxt_j) - area(i, nxt_i, j)) == 0) {
            add(i, nxt_j);
            add(nxt_i, nxt_j);
        }
    }
    ranges::sort(res);
    res.erase(ranges::unique(res).begin(), res.end());
    return res;
}
// 返回 {最大距离平方, 端点下标 u, 端点下标 v}
tuple<HullDist2, int, int> conv_diam(const vector<Point> &hull) {
    int n = hull.size();
    if (n == 0) return { HullDist2{}, -1, -1 };
    if (n == 1) return { HullDist2{}, 0, 0 };
    if (n == 2) return { dist2(hull[0], hull[1]), 0, 1 };
    HullDist2 best{};
    int ans_u{ 0 }, ans_v{ 0 };
    auto area = [&](int i, int j, int k) -> HullArea2 {
        HullArea2 value{ cross(hull[j] - hull[i], hull[k] - hull[i]) };
        return value < 0 ? -value : value;
    };
    auto upd = [&](int u, int v) {
        HullDist2 cur{ dist2(hull[u], hull[v]) };
        if (cur > best) {
            best = cur;
            ans_u = u;
            ans_v = v;
        }
    };
    int j{ 1 };
    for (int i = 0; i < n; ++ i) {
        int nxt_i{ (i + 1) % n };
        while (sgn(area(i, nxt_i, (j + 1) % n) - area(i, nxt_i, j)) > 0) j = (j + 1) % n;
        upd(i, j);
        upd(nxt_i, j);
        int nxt_j{ (j + 1) % n };
        if (sgn(area(i, nxt_i, nxt_j) - area(i, nxt_i, j)) == 0) {
            upd(i, nxt_j);
            upd(nxt_i, nxt_j);
        }
    }
    return { best, ans_u, ans_v };
}
```

`conv_diam()` 返回距离平方，可直接用于整数版；只有最终输出欧氏距离时才开方

凸包最大面积三角形

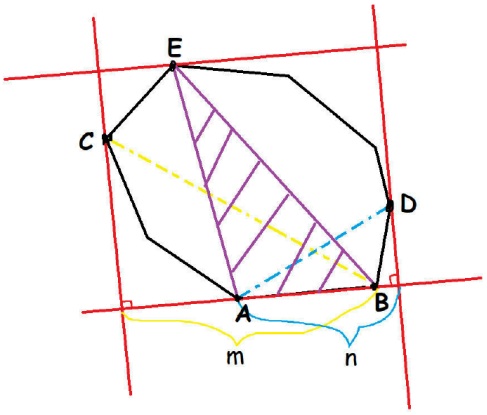
解决问题： 从凸多边形顶点中选择三个点，使三角形面积最大
固定 i, j 后，$|\triangle p_i p_j p_k|$ 随 k 先增后减；对固定 i 依次移动 j 时，最优 k 单调前进

```cpp
// 返回 {最大面积的两倍, 顶点下标 i, j, k}
tuple<HullArea2, int, int, int> max_tri_a2(const vector<Point> &hull) {
    int n = hull.size();
    if (n < 3) return { HullArea2{}, -1, -1, -1 };
    auto area = [&](int i, int j, int k) -> HullArea2 {
        HullArea2 value{ cross(hull[i], hull[j], hull[k]) };
        return value < 0 ? -value : value;
    };
    HullArea2 best{};
    int ans_i{ -1 }, ans_j{ -1 }, ans_k{ -1 };
    for (int i = 0; i + 2 < n; ++ i) {
        int k{ i + 2 };
        for (int j = i + 1; j + 1 < n; ++ j) {
            if (k <= j) k = j + 1;
            while (k + 1 < n && sgn(area(i, j, k + 1) - area(i, j, k)) >= 0) ++ k;
            HullArea2 cur{ area(i, j, k) };
            if (cur > best) {
                best = cur;
                ans_i = i;
                ans_j = j;
                ans_k = k;
            }
        }
    }
    return { best, ans_i, ans_j, ans_k };
}
```

整数版保留"两倍面积"可以避免除法；实际面积为 `area2 / 2`

最小宽度与最小覆盖矩形

解决问题： 求凸多边形的最小宽度，以及边允许任意旋转时的最小面积、最小周长覆盖矩形



凸多边形的最小宽度一定对应某条凸包边的方向。对边向量 $e_i = p_{i+1} - p_i$，该方向对应的宽度为

$$w_i = \max_j \frac{e_i \times (p_j - p_i)}{|e_i|}$$

最优覆盖矩形至少有一条边与凸包边共线。固定方向单位向量 u，令 v 为其逆时针法向量，只需维护点在 u, v 上的最大、最小投影

```cpp
/* ===== 浮点版专属：若不需要宽度与覆盖矩形，以下代码可直接删除 ===== */
ld conv_width(const vector<Point> &hull) {
    int n = hull.size();
    if (n < 3) return 0.L;
    ld ans{ numeric_limits<ld>::infinity() };
    int j{ 1 };
    for (int i = 0; i < n; ++ i) {
        int nxt_i{ (i + 1) % n };
        Point edge{ hull[nxt_i] - hull[i] };
        auto h2 = [&](int k) { return cross(edge, hull[k] - hull[i]); };
        while (sgn(h2((j + 1) % n) - h2(j)) > 0) j = (j + 1) % n;
        ans = min(ans, max(0.L, h2(j)) / length(edge));
    }
```

```
    return ans;
}
struct CoverRect {
    ld area{ 0 }, peri{ 0 };
    array<Point, 4> cor{}; // 逆时针四个顶点
};
// 返回 {最小面积覆盖矩形, 最小周长覆盖矩形}
pair<CoverRect, CoverRect> min_cov_rec(const vector<Point> &hull) {
    int n = hull.size();
    CoverRect min_a, min_p;
    if (n == 0) return { min_a, min_p };
    if (n == 1) {
        min_a.cor.fill(hull[0]);
        min_p = min_a;
        return { min_a, min_p };
    }
    if (n == 2) {
        ld len{ distance(hull[0], hull[1]) };
        min_a.peri = 2 * len;
        min_a.cor = { hull[0], hull[1], hull[1], hull[0] };
        min_p = min_a;
        return {min_a, min_p};
    }
    min_a.area = numeric_limits<ld>::infinity();
    min_p.peri = numeric_limits<ld>::infinity();
    Point fst_e{ hull[1] - hull[0] };
    int top{ 0 }, r{ 0 }, l{ 0 };
    for (int i = 1; i < n; ++ i) {
        if (cross(fst_e, hull[i] - hull[0]) > cross(fst_e, hull[top] - hull[0])) top = i;
        if (dot(fst_e, hull[i] - hull[0]) > dot(fst_e, hull[r] - hull[0])) r = i;
        if (dot(fst_e, hull[i] - hull[0]) < dot(fst_e, hull[l] - hull[0])) l = i;
    }
    for (int i = 0; i < n; ++ i) {
        int nxt_i{ (i + 1) % n };
        Point edge{ hull[nxt_i] - hull[i] };
        auto norm_pr = [&](int k) { return cross(edge, hull[k] - hull[i]); };
        auto edge_pr = [&](int k) { return dot(edge, hull[k] - hull[i]); };
        while (sgn(norm_pr((top + 1) % n) - norm_pr(top)) > 0) top = (top + 1) % n;
        while (sgn(edge_pr((r + 1) % n) - edge_pr(r)) > 0) r = (r + 1) % n;
        while (sgn(edge_pr(l) - edge_pr((l + 1) % n)) > 0) l = (l + 1) % n;
        Point ax{ edge / length(edge) }, ay{ rotate90(ax) };
        Point origin{ hull[i] };
        ld mn_x{ dot(hull[l] - origin, ax) }, mx_x{ dot(hull[r] - origin, ax) };
        ld mn_y{ 0 }, mx_y{ dot(hull[top] - origin, ay) };
        if (mn_x > mx_x) swap(mn_x, mx_x);
        if (mn_y > mx_y) swap(mn_y, mx_y);
        ld w{ mx_x - mn_x }, h{ mx_y - mn_y };
        CoverRect cur;
        cur.area = w * h;
        cur.peri = 2 * (w + h);
        cur.cor = {
            origin + ax * mn_x + ay * mn_y, origin + ax * mx_x + ay * mn_y,
            origin + ax * mx_x + ay * mx_y, origin + ax * mn_x + ay * mx_y
        };
        if (sgn(cur.area - min_a.area) < 0) min_a = cur;
        if (sgn(cur.peri - min_p.peri) < 0) min_p = cur;
    }
    return {min_a, min_p};
}
```

最远 Manhattan 距离

解决问题： 求点集中两点的最大 Manhattan 距离。该问题不需要先求凸包

二维情况下

$$|x_1 - x_2| + |y_1 - y_2| = \max\{|(x_1 + y_1) - (x_2 + y_2)|, |(x_1 - y_1) - (x_2 - y_2)|\}$$

```
// ===== 整数版专属 =====
i128 far_manh(const vector<Point> &pts) {
    if (pts.empty()) return 0;
```

```
    i128 min_s{ i128(pts[0].x) + pts[0].y }, max_s{ min_s };
    i128 min_dif{ i128(pts[0].x) - pts[0].y }, max_dif{ min_dif };
    for (Point p : pts) {
        i128 sum{ i128(p.x) + p.y }, dif{ i128(p.x) - p.y };
        min_s = min(min_s, sum);
        max_s = max(max_s, sum);
        min_dif = min(min_dif, dif);
        max_dif = max(max_dif, dif);
    }
    return max(max_s - min_s, max_dif - min_dif);
}
```

接口说明

凸包构造

- `conv_hull(pts, keep_col)`：Andrew 凸包。默认删除边上中间点，返回逆时针凸包且不重复首点，$O(n \log n)$
- `gra_hull(pts, keep_col)`：Graham 扫描。仅在还需要极角序时建议保留，$O(n \log n)$
- 两个接口都只删除坐标完全相同的点。浮点输入存在近重复点时，应在调用前按统一 `EPS` 清理

查询与切点

- `line_cv_int(hull, ln)`：求直线与凸包边界交点；仅浮点版，$O(n)$
- `tan_ver(hull, q)`：求凸包外一点的切点下标；返回所有候选切点，边延长线上可能多于两个，$O(n)$
- `ext_ver(poly,dir)`：返回指定方向的支撑顶点，$O(\log n)$
- `line_hull(poly,ln)`：返回直线经过的凸包边下标，$O(\log n)$
- `tan_fast(poly,q)`：返回凸包外点的两个代表切点，$O(\log n)$
- 点包含直接调用 `in_conv()`，单次 $O(\log n)$
- 点到凸包的最近距离不需要新板子：遍历每条边并调用 `dis_seg()`，总复杂度 $O(n)$

旋转卡壳

- `anti_pair(hull)`：返回去重后的对踵顶点对，$O(n \log n)$；卡壳扫描本身为 $O(n)$，排序去重为 $O(n \log n)$
- `conv_diam(hull)`：返回 `{距离平方,u,v}`，$O(n)$
- `max_tri_a2(hull)`：返回 `{最大面积的两倍,i,j,k}`，$O(n^2)$
- `conv_width(hull)`：返回最小宽度；仅浮点版，$O(n)$
- `min_cov_rec(hull)`：同时返回最小面积和最小周长覆盖矩形；仅浮点版，$O(n)$
- `far_manh(pts)`：返回最大 Manhattan 距离；仅整数版，$O(n)$，不依赖凸包

注意事项

- 卡壳、切点和凸包含均应使用逆时针凸包。除特别需要边界点外，构造时使用默认 `keep_col=false`；已有凸多边形可先调用 `norm_conv()`
- `conv_hull(..., true)` 在全部共线时按字典序返回所有点，此时结果不是一个环；旋转卡壳接口不能直接使用该退化结果
- 浮点排序不能把 `EPS` 写入 `operator<` 或比较器。近重复点应先量化、聚类或按题意单独清理
- `line_cv_int()` 要求 `ln.dir` 非零。直线与凸包边重合时返回该边端点，不表示无穷多个交点
- `tan_ver()` 和 `tan_fast()` 的查询点应严格位于凸包外。点位于内部或边界时，“两条外切线”不满足接口语义
- 三个快速接口要求至少三个顶点的严格凸多边形；含共线冗余点时应先调用 `norm_conv()`
- `conv_diam()` 和 `max_tri_a2()` 返回平方距离或两倍面积，整数版不要提前转成浮点数
- 最小宽度与覆盖矩形需要构造单位向量和非整数顶点，因此只放在浮点版
- 动态插入、删除凸包属于独立数据结构问题。本页只处理静态点集；不要把静态 Andrew 反复重建误认为动态凸包

常见用法

求点集直径

```
vector<Point> hull = conv_hull(pts);
auto [distance2, u, v] = conv_diam(hull);
```

多次查询点是否位于凸包

直接使用上一页的凸包含接口：

```
vector<Point> hull = norm_conv(conv_hull(pts));
for (Point q : qs) Contain where = in_conv(hull, q);
```

求点到凸包的最近距离

无需新增高度相似的函数，直接扫描基础板子的点到线段距离：

```
// [浮点版专属]
ld ans = numeric_limits<ld>::infinity();
for (int i = 0, n = hull.size(); i < n; ++ i) {
    ans = min(ans, dis_seg(q, Segment(hull[i], hull[(i + 1) % n])));
}
```

若 `q` 位于凸包内部或边界，区域距离应为 0，可先调用 `in_conv()` 判断

求凸包外一点的两条切线

```
vector<Point> hull = norm_conv(conv_hull(pts));
vector<int> tangent = tan_ver(hull, q);
if (tangent.size() == 2) {
    Line first = Line::through(q, hull[tangent[0]]);
    Line second = Line::through(q, hull[tangent[1]]);
}
```

求最小宽度与覆盖矩形

```
// [浮点版专属]
vector<Point> hull = norm_conv(conv_hull(pts));
ld w = conv_width(hull);
auto [min_a, min_p] = min_cov_rec(hull);
```

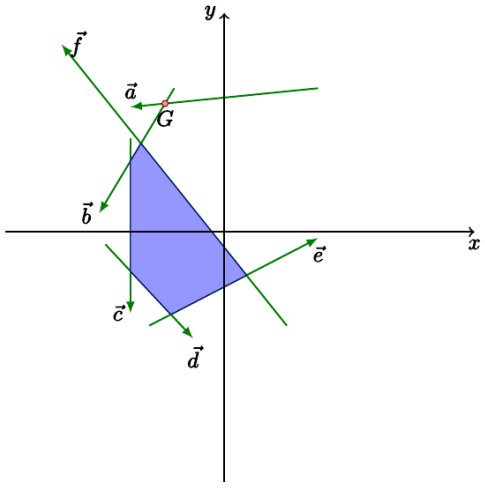
复杂度

操作	时间复杂度	额外空间
`conv_hull()` / `gra_hull()`	$O(n \log n)$	$O(n)$
`line_cv_int()`	$O(n)$	$O(1)$（不计返回值）
`tan_ver()`	$O(n)$	$O(1)$（不计返回值）
`ext_ver()` / `line_hull()` / `tan_fast()`	$O(\log n)$	$O(1)$
`anti_pair()`	$O(n \log n)$	$O(n)$
`conv_diam()`	$O(n)$	$O(1)$
`max_tri_a2()`	$O(n^2)$	$O(1)$
`conv_width()`	$O(n)$	$O(1)$
`min_cov_rec()`	$O(n)$	$O(1)$
`far_manh()`	$O(n)$	$O(1)$

15.4 半平面交与平面区域

浮点半平面交

解决问题： 求若干有向直线左侧闭半平面的公共区域，用于线性不等式可行域、多边形核、二分答案中的几何可行性判定等问题



有向直线由直上一点 p 和非零方向 d 表示，其左侧闭半平面为

$$H(p,d)=\{x \mid d\times(x-p)\geq 0\}.$$

算法先按方向极角排序并删除同方向的较宽松约束，再维护一个双端队列。若相邻两条有效边界的交点落在新半平面外，就从相应一端删除旧边界

```
struct HalfPlane {
    Point p, dir;
    HalfPlane() = default;
    // 有向直线 a → b 的左侧为可行区域；要求 a、b 不重合
    HalfPlane(Point a, Point b) : p(a), dir(b - a) {}
    // q 严格位于右侧时返回 true；边界属于可行区域
    bool out(Point q) const { return sgn(cross(dir, q - p)) < 0; }
    Line ln() const { return Line(p, dir); }
};
optional<Point> hp_cross(const HalfPlane &a, const HalfPlane &b) {
    return line_int(a.ln(), b.ln());
}
// 原始值比较用于 sort，不能把 EPS 写进比较器
bool hp_less(const HalfPlane &a, const HalfPlane &b) {
    int ha{ arg_half(a.dir) }, hb{ arg_half(b.dir) };
    if (ha != hb) return ha < hb;
    ld turn{ cross(a.dir, b.dir) };
    if (turn != 0) return turn > 0;
    // 同方向时让更严格的半平面排在前面
    return cross(a.dir, b.p - a.p) < 0;
}
bool same_hp_dir(const HalfPlane &a, const HalfPlane &b) {
    return sgn(cross(a.dir, b.dir)) == 0 && sgn(dot(a.dir, b.dir)) > 0;
}
// 只返回有界且面积非零的交多边形；空、无界、线段或单点均返回空数组
vector<Point> hp_int(vector<HalfPlane> hps) {
    ranges::sort(hps, hp_less);
    vector<HalfPlane> uniq;
    for (const HalfPlane &hp : hps) {
        if (!uniq.empty() && same_hp_dir(uniq.back(), hp)) {
            if (hp.out(uniq.back().p)) uniq.back() = hp;
        } else uniq.push_back(hp);
    }
    deque<HalfPlane> act;
    deque<Point> isecs; // act 中相邻半平面的交点
    for (const HalfPlane &hp : uniq) {
        while (!isecs.empty() && hp.out(isecs.back())) {
            isecs.pop_back();
            act.pop_back();
        }
        while (!isecs.empty() && hp.out(isecs.front())) {
            isecs.pop_front();
            act.pop_front();
        }
        if (!act.empty()) {
            auto p = hp_cross(act.back(), hp);
            // 在“最终交集有界”的前提下，有效相邻边界不应平行。
            if (!p) return {};
            isecs.push_back(*p);
        }
        act.push_back(hp);
    }
    while (!isecs.empty() && act.front().out(isecs.back())) {
        isecs.pop_back();
        act.pop_back();
    }
    while (!isecs.empty() && act.back().out(isecs.front())) {
        isecs.pop_front();
        act.pop_front();
    }
    if (act.size() < 3) return {};
    auto cls = hp_cross(act.back(), act.front());
    if (!cls) return {};
    isecs.push_back(*cls);
```

```
    vector<Point> res(isecs.begin(), isecs.end());
    res = norm_conv(move(res));
    if (res.size() < 3 || sgn(poly_area2(res)) == 0) return {};
    return res;
}
// 返回逆时针正方形的四个左半平面，即 [-bound,bound]^2
vector<HalfPlane> box(ld bound) {
    Point a{-bound, -bound}, b{ bound, -bound};
    Point c{ bound,  bound}, d{-bound,  bound};
    return {{a, b}, {b, c}, {c, d}, {d, a}};
}
```

线性不等式转换

解决问题： 将二维线性约束转成“有向直线左侧”形式
对非退化约束

$$ax + by \le c$$

先在边界 $ax + by = c$ 上取一点 p，方向应取

$$d = (-b, a)$$

因为

$$d \times (x - p) = c - (ax + by),$$

所以左侧条件 $d \times (x - p) \ge 0$ 恰好等价于 $ax + by \le c$。相反，$ax + by \ge c$ 应使用方向 $(b, -a)$

```
// [浮点版] a、b 不同时为 0
ld a, b, c;
cin >> a >> b >> c;
Point p = fabsl(a) > fabsl(b) ? Point(c / a, 0) : Point(0, c / b);
HalfPlane con(p, p + Point(-b, a)); // a*x + b*y ≤ c
```

当 `a == 0 && b == 0` 时，应在外部单独判断：`0 ≤ c` 是恒真约束，`0 > c` 表示整体无解，不能构造零方向半平面

整数约束与精确性

整数版基础板子的 `pt_side(q, ln)` 已经使用 `i128` 完成精确叉积，因此“某个整数点是否满足半平面”可直接判断：

```
bool ok = pt_side(q, bnd) ≥ 0; // 有向边界左侧或线上
```

不再额外定义与 `Line` 高度重复的 `IHalfPlane`。但完整半平面交需要构造边界交点，整数直线的交点通常为有理数；若题目要求精确输出或严格比较，应使用统一符号的 `i128` 分子、分母表示有理点，不能做整数除法。

多边形核

解决问题： 求简单多边形中能够看见整个多边形的所有点
若简单多边形 $P = (p_0, p_1, \ldots, p_{n-1})$ 按逆时针排列，并令 $p_n = p_0$，则其核为所有边左侧闭半平面的交：

$$\mathrm{Kernel}(P) = \bigcap_{i=0}^{n-1} \{x \mid (p_{i+1} - p_i) \times (x - p_i) \ge 0\}$$

多边形方向可直接使用上一篇的 `poly_area2()` 判断，不重复实现面积函数

```
if (sgn(poly_area2(poly)) < 0) ranges::reverse(poly);
vector<HalfPlane> hps;
for (int i = 0, n = poly.size(); i < n; ++ i) {
    hps.emplace_back(poly[i], poly[(i + 1) % n]);
}
vector<Point> ker = hp_int(move(hps));
// ker.empty() 表示没有面积非零的核；本实现不区分空核与退化为线段/单点的核。
```

核本身一定包含在有界多边形内，通常不需要额外加入大框

平面区域计数

半平面交计算的是凸可行域；一般线段或曲线把平面切成多少区域属于平面图问题，不应把两种算法混成同一份代码
把所有交点加入顶点集合，并将每条原始边在交点处切成不含内部顶点的边后，设平面图有

- V 个不同顶点
- E 条切分后的边
- C 个连通分量
- F 个面，包含无界外部面
  则 Euler 公式为

$$F = E - V + C + 1$$

连通时 $C = 1$，即 $F = E - V + 2$；有界区域数为

$$F - 1 = E - V + C$$

共线重叠边、多个对象共享同一交点、孤立点都会影响 V, E, C，必须先完成平面化后再套公式。
双极定向属于给定平面嵌入上的图论结构，不由半平面交产生，在独立的平面图板子。

接口说明

成员变量

- `HalfPlane::p`：边界直线上的一点
- `HalfPlane::dir`：非零方向向量；从 `p` 沿 `dir` 前进时，左侧为可行区域

接口

- `HalfPlane(a,b)`：用有向直线 `a → b` 构造左半平面，$O(1)$；要求 `a`、`b` 不重合。
- `out(q)`：判断点是否严格位于半平面外，$O(1)$；边界返回 `false`。
- `hp_cross(a,b)`：求两条边界直线的唯一交点；平行或重合返回 `nullopt`，$O(1)$。
- `hp_int(hps)`：返回有界、面积非零的公共交多边形，按逆时针排列且不重复首点，$O(n \log n)$。
- `box(bound)`：构造正方形 $[-bound, bound]^2$ 的四个左半平面，$O(1)$。

返回值语义

`hp_int()` 返回空数组可能表示：

- 约束无解
- 公共交无界
- 公共交只剩一条线段或一个点
- 数值误差使有效边界被误判为平行或交点落在错误一侧
  该接口适合“求有界非退化多边形”或“加入大框后的面积非零可行性”问题；若题目允许退化闭集，需要额外设计可行性判定

注意事项

- 最容易写反的是可行方向：构造 `HalfPlane(a,b)` 后保留的是有向边 `a → b` 左侧。先用一个已知可行点调用 `out()` 自检
- 对 $ax + by \le c$，方向是 $(-b, a)$，不是 $(b, -a)$；后者对应 $ax + by \ge c$
- `hp_less()` 必须形成严格弱序，因此极角排序使用原始叉积，不能使用 `sgn()` 或 `EPS`；几何平行、出界判定再使用 `sgn()`
- 同方向平行约束只保留更严格的一条。反向平行约束可能形成条带；本实现只承诺处理最终交集有界的情况，无界问题应先加入足够大的 `box()`
- 大框只是在题目已知坐标范围内把无界区域截成有限区域。`bound` 必须严格覆盖所有可能答案，同时不能大到使叉积和直线求交严重损失精度
- 二分答案时可以每次重建约束并判断返回值是否非空，但这只判定“存在面积非零的框内区域”。答案允许恰好退化为线或点时不能直接使用
- 坐标或系数尺度很大时，应按误差传播调整 `EPS`，并避免同时混用 `double` 和 `long double`
- 多边形核要求简单多边形按边界顺序给出；顺时针输入必须先反转

常见用法

求无界约束在指定范围内的可行区域

```
vector<HalfPlane> hps = box(1E9L);
for (auto [a, b] : dir_bnds) {
    hps.emplace_back(a, b);
}
vector<Point> reg = hp_int(move(hps));
```

二分答案并判定面积非零可行域

```
bool check(ld ans) {
    vector<HalfPlane> hps = box(BOUND);
    // 按 ans 构造约束，全部转成“左侧可行”
    return !hp_int(move(hps)).empty();
}
```

求线性不等式组的面积

```
vector<HalfPlane> hps = box(BOUND);
for (auto [a, b, c] : cons) {
    Point p = fabsl(a) > fabsl(b) ? Point(c / a, 0) : Point(0, c / b);
    hps.emplace_back(p, p + Point(-b, a)); // a*x+b*y≤c
}
vector<Point> reg = hp_int(move(hps));
ld area = fabsl(poly_area2(reg)) / 2;
```

求多边形核

```
if (sgn(poly_area2(poly)) < 0) ranges::reverse(poly);
vector<HalfPlane> hps;
for (int i = 0, n = poly.size(); i < n; ++ i) {
    hps.emplace_back(poly[i], poly[(i + 1) % n]);
}
vector<Point> ker = hp_int(move(hps));
```

用已有多边形裁剪接口代替小规模半平面交

若约束数很少，且已经有一个初始凸多边形，可以直接反复调用上一页的 `cut_poly()`：

```
// [浮点版专属]
vector<Point> reg = init_poly;
for (Line bnd : bnds) {
    reg = cut_poly(reg, bnd); // 保留有向边界左侧
}
```

这种写法接口更短、容易检查，但若初始顶点数和约束数均为 n，最坏复杂度可达 $O(n^2)$；一般半平面交为 $O(n \log n)$

复杂度

设输入半平面数量为 n

操作	时间复杂度	额外空间
排序与同向去重	$O(n \log n)$	$O(n)$
双端队列扫描	$O(n)$	$O(n)$
`hp_int()` 总计	$O(n \log n)$	$O(n)$
`box()`	$O(1)$	$O(1)$

平面图区域计数在完成平面化并已知 V, E, C 后仅为 $O(1)$；求全部交点、切分边和连通分量的复杂度取决于具体平面图构造方法

15.5 圆与圆弧

圆关系与交点

解决问题： 判断点、直线、线段、两个圆之间的位置关系，并求有限个交点

```
struct Circle {
    Point ctr;
    ld rad{ 0 };
};
enum class CirRel {
    SEPARATE,         // 外离
    EXTERNAL_TANGENT, // 外切
    INTERSECT,        // 相交于两点
    INTERNAL_TANGENT, // 内切
    CONTAIN,          // 内含但不内切
    COINCIDE          // 重合
};
// 返回点相对闭圆盘的位置：圆内 / 圆上 / 圆外为 -1 / 0 / 1
int pt_cir_rel(Point p, Circle cir) { return sgn(distance(p, cir.ctr) - cir.rad); }
CirRel cir_rel(Circle a, Circle b) {
    ld dis{ distance(a.ctr, b.ctr) };
    if (sgn(dis) == 0 && sgn(a.rad - b.rad) == 0) return CirRel::COINCIDE;
```

```
    int outer{ sgn(dis - a.rad - b.rad) }, inner{ sgn(dis - fabsl(a.rad - b.rad)) };
    if (outer > 0) return CirRel::SEPARATE;
    if (outer == 0) return CirRel::EXTERNAL_TANGENT;
    if (inner > 0) return CirRel::INTERSECT;
    if (inner == 0) return CirRel::INTERNAL_TANGENT;
    return CirRel::CONTAIN;
}
// 返回 0 / 1 / 2 个交点；顺序沿 ln.dir 方向递增
vector<Point> line_cir_int(Line ln, Circle cir) {
    Point foot{ proj(cir.ctr, ln) };
    ld dis{ distance(foot, cir.ctr) };
    int rel{ (dis > cir.rad) - (dis < cir.rad) };
    if (rel > 0) return {};
    if (rel == 0) return {foot};
    ld rem{ (cir.rad - dis) * (cir.rad + dis) };
    Point unit{ ln.dir / length(ln.dir) }, off{ unit * sqrtl(max(0.L, rem)) };
    return {foot - off, foot + off};
}
// 返回 0 / 1 / 2 个交点；重合圆因交点无穷多而返回空数组
vector<Point> cir_int(Circle a, Circle b) {
    Point dir{ b.ctr - a.ctr };
    ld dis{ length(dir) };
    if (dis == 0) return {};
    ld sum{ a.rad + b.rad }, dif{ fabsl(a.rad - b.rad) };
    if (dis > sum || dis < dif) return {};
    ld x{ ((a.rad - b.rad) * (a.rad + b.rad) + dis * dis) / (2 * dis) };
    ld h2{ (sum + dis) * (sum - dis) * (dis + dif) * (dis - dif) / (4 * dis * dis) };
    Point mid{ a.ctr + dir * (x / dis) };
    if (h2 == 0) return { mid };
    Point off{ rotate90(dir) * (sqrtl(max(0.L, h2)) / dis) };
    return { mid - off, mid + off };
}
// 判断闭线段是否与闭圆盘有公共点
bool seg_disk(Segment seg, Circle cir) { return sgn(dis_seg(cir.ctr, seg) - cir.rad) <= 0; }
// 判断闭线段是否与圆周有公共点；整段严格位于圆内时返回 false
bool seg_cir(Segment seg, Circle cir) {
    ld near{ dis_seg(cir.ctr, seg) };
    ld far{ max(distance(seg.a, cir.ctr), distance(seg.b, cir.ctr)) };
    return sgn(near - cir.rad) <= 0 && sgn(far - cir.rad) >= 0;
}
// 返回圆上离 p 最近、最远的点；p 等于圆心时两者均不唯一
optional<pair<Point, Point>> near_far(Point p, Circle cir) {
    Point dir{ p - cir.ctr };
    if (norm2(dir) == 0) return nullopt;
    dir = dir / length(dir);
    return pair(cir.ctr + dir * cir.rad, cir.ctr - dir * cir.rad);
}
```

接口说明

- `pt_cir_rel(p,cir)`：返回 `-1 / 0 / 1`，分别表示点在圆内、圆上、圆外，$O(1)$
- `cir_rel(a,b)`：返回两圆六种关系之一，$O(1)$
- `line_cir_int(ln,cir)`：返回直线与圆周的有限交点；要求 `ln.dir` 非零，$O(1)$
- `cir_int(a,b)`：返回两圆周的有限交点；重合圆返回空数组，调用前可先用 `cir_rel()` 区分，$O(1)$
- `seg_disk()`：判断线段是否接触闭圆盘，$O(1)$
- `seg_cir()`：判断线段是否接触圆周，$O(1)$
- `near_far()`：返回最近点和最远点；查询点为圆心时返回 `nullopt`，$O(1)$

注意事项

- `Circle::rad` 必须非负
- 直线与圆交点依赖投影，退化直线没有“沿方向的两个交点”语义，调用前保证 `ln.dir` 非零
- `cir_int()` 的空数组可能表示无交点，也可能表示重合圆；需要区分时先调用 `cir_rel()`。关系判定使用 `EPS`，交点构造按实际计算值判断相切，以免把两个距离超过输出容差的近切交点合并
- `seg_disk()` 与 `seg_cir()` 语义不同：整条线段位于圆内时，前者为 `true`，后者为 `false`
- 查询点等于圆心时，圆上最近点和最远点都不唯一，可按题意任取 `ctr + Point(rad,0)`

常见用法

```
CirRel rel = cir_rel(a, b);
vector<Point> isecs = cir_int(a, b);
if (seg_cir(seg, cir)) {} // 线段碰到圆周
```

复杂度

所有接口时间复杂度与额外空间均为 $O(1)$，不计返回数组

圆的切线与公切线

解决问题： 求圆外一点向圆引出的切线，以及两个圆的全部公切线

```
// 返回 p 到 cir 的切点；圆内无切点，圆上一个，圆外两个
vector<Point> tan_pts(Point p, Circle cir) {
    Point dir{ p - cir.ctr };
    ld dis2{ norm2(dir) }, r2{ cir.rad * cir.rad };
    ld dis{ length(dir) };
    int rel{ (dis > cir.rad) - (dis < cir.rad) };
    if (rel < 0) return {};
    if (rel == 0) return {p};
    Point base{ cir.ctr + dir * (r2 / dis2) };
    Point off{ rotate90(dir) * (cir.rad * sqrtl(max(0.L, (dis - cir.rad) * (dis + cir.rad))) / dis2) };
    return { base + off, base - off };
}
// 返回 p 到 cir 的切线；圆内无切线，圆上一条，圆外两条
vector<Line> tan_line(Point p, Circle cir) {
    vector<Point> tans{ tan_pts(p, cir) };
    vector<Line> res;
    for (Point tan : tans) {
        if (same_pt(p, tan)) res.emplace_back(p, rotate90(p - cir.ctr));
        else res.push_back(Line::through(p, tan));
    }
    return res;
}
// 返回每条公切线在 a、b 上的切点；切点对可能重合
vector<pair<Point, Point>> com_tan_pts(Circle a, Circle b) {
    vector<pair<Point, Point>> res;
    Point dir{ b.ctr - a.ctr };
    ld dis2{ norm2(dir) };
    if (dis2 == 0) return res;
    // sign = 1: 外公切线; sign = -1: 内公切线
    for (int sign : {-1, 1}) {
        ld dr{ a.rad - sign * b.rad };
        ld dis{ sqrtl(dis2) };
        if (dis < fabsl(dr)) continue;
        ld h2{ (dis - fabsl(dr)) * (dis + fabsl(dr)) };
        for (int side : {-1, 1}) {
            if (h2 == 0 && side == 1) continue;
            Point nor{ (dir * dr + rotate90(dir) * (sqrtl(max(0.L, h2)) * side)) / dis2 };
            res.push_back({ a.ctr + nor * a.rad, b.ctr + nor * (sign * b.rad) });
        }
    }
    return res;
}
```

接口说明

- `tan_pts(p,cir)`：返回切点，数量为 $0/1/2$，$O(1)$
- `tan_line(p,cir)`：返回对应切线，数量为 $0/1/2$，$O(1)$
- `com_tan_pts(a,b)`：返回全部公切线在两圆上的切点对，数量为 0 到 4，$O(1)$
- `com_tan_pts()` 中外公切线和内公切线均可能出现；相切时两种方向中某一组会退化为一条

注意事项

- 两圆同心时没有唯一可枚举的公切线，本接口返回空数组；等圆重合时实际有无穷多条公切线
- 两圆相切时，某条公切线的两个切点可能重合。此时不能用 `Line::through(p1,p2)` 构造直线，应使用半径的垂线方向：`rotate90(p1-a.ctr)`
- 半径为 0 的“圆”会退化为点，切线语义通常需要按题意单独处理

常见用法

```
for (Line ln : tan_line(q, cir)) {
    // 圆上点的切线也具有非零方向
}
for (auto [pa, pb] : com_tan_pts(a, b)) {
    Line tan = same_pt(pa, pb) ? Line(pa, rotate90(pa - a.ctr)) : Line::through(pa, pb);
}
```

复杂度

所有接口时间复杂度与额外空间均为 $O(1)$，不计返回数组

三角形相关圆与最小覆盖圆

解决问题： 求三角形外接圆、内切圆，以及覆盖点集的最小圆

```
optional<Circle> out_cir(Point a, Point b, Point c) {
    ld den{ 2 * cross(b - a, c - a) };
    if (den == 0) return nullopt;
    ld b2{ norm2(b - a) }, c2{ norm2(c - a) };
    Point ctr{ a + Point((b2 * (c.y - a.y) - c2 * (b.y - a.y)) / den, (c2 * (b.x - a.x) - b2 * (c.x - a.x)) / den) };
    return Circle{ctr, distance(ctr, a)};
}
optional<Circle> in_cir(Point a, Point b, Point c) {
    ld sd_a{ distance(b, c) }, sd_b{ distance(c, a) };
    ld side_c{ distance(a, b) }, peri{ sd_a + sd_b + side_c };
    if (peri == 0 || cross(a, b, c) == 0) return nullopt;
    Point ctr{ (a * sd_a + b * sd_b + c * side_c) / peri };
    return Circle{ctr, dis_line(ctr, Line::through(a, b))};
}
Circle dia_cir(Point a, Point b) {
    return Circle{ (a + b) / 2, distance(a, b) / 2};
}
bool cir_has(Circle cir, Point p) { return sgn(distance(cir.ctr, p) - cir.rad) <= 0; }
// 随机增量最小覆盖圆；空点集返回圆心 (0,0)、半径 0
Circle min_cir(vector<Point> pts, uint64_t seed = 0xC1AC1EULL) {
    mt19937_64 gen(seed);
    ranges::shuffle(pts, gen);
    Circle res{{0, 0}, -1};
    auto out = [&](Point p) { return res.rad < 0 || !cir_has(res, p); };
    for (int i = 0; i < pts.size(); ++ i) if (out(pts[i])) {
        res = Circle{pts[i], 0};
        for (int j = 0; j < i; ++ j) if (out(pts[j])) {
            res = dia_cir(pts[i], pts[j]);
            for (int k = 0; k < j; ++ k) if (out(pts[k])) {
                auto cand = out_cir(pts[i], pts[j], pts[k]);
                if (cand) {
                    res = *cand;
                } else {
                    // 三点共线时取最远点对的直径圆
                    array<pair<Point, Point>, 3> pairs{{
                        {pts[i], pts[j]},
                        {pts[i], pts[k]},
                        {pts[j], pts[k]}
                    }};
                    res = dia_cir(pairs[0].first, pairs[0].second);
                    for (auto [a, b] : pairs) {
                        Circle cir{ dia_cir(a, b) };
                        if (cir.rad > res.rad) res = cir;
                    }
                }
            }
        }
    }
    if (pts.empty()) return Circle{{0, 0}, 0};
    return res;
}
```

接口说明

- `out_cir(a,b,c)`：求非共线三点的外接圆；共线时返回 `nullopt`，$O(1)$

- `in_cir(a,b,c)`：求非退化三角形的内切圆；退化时返回 `nullopt`，$O(1)$
- `dia_cir(a,b)`：返回以 ab 为直径的圆，$O(1)$
- `cir_has(cir,p)`：判断点是否位于闭圆盘内，$O(1)$
- `min_cir(pts, seed)`：返回覆盖全部点的最小圆；期望 $O(n)$

注意事项

- 外接圆和内切圆均要求三点不共线
- 最小覆盖圆的 `seed` 参数有固定默认值，便于复现；可传入其他种子检查不同插入顺序。
- 最小覆盖圆使用随机增量；期望复杂度为 $O(n)$，最坏复杂度可达 $O(n^3)$
- 最小覆盖圆的边界最多由三个输入点确定：一个点、两个直径端点，或三个外接圆点
- 输入中允许重复点；若浮点近重点会影响稳定性，可在调用前按统一 `EPS` 清理
- 三点共线时，覆盖它们的最小圆由最远点对作为直径

常见用法

```
Circle cover = min_cir(pts);
auto outer = out_cir(a, b, c);
auto inner = in_cir(a, b, c);
```

复杂度

操作	时间复杂度	额外空间
外接圆 / 内切圆	$O(1)$	$O(1)$
最小覆盖圆	期望 $O(n)$，最坏 $O(n^3)$	$O(n)$

固定半径圆覆盖最多点

解决问题： 给定平面点集与半径 r，平移一个半径固定的圆，使闭圆盘覆盖的输入点数量最大
固定一个输入点 p_i 位于圆周上。对另一个点 p_j，圆心绕 p_i 的极角必须落在一个闭区间中；扫描这些角区间的最大重叠数即可

```
int max_pts_cir(const vector<Point> &pts, ld rad) {
    assert(sgn(rad) >= 0);
    if (pts.empty()) return 0;
    if (sgn(rad) == 0) {
        int ans{ 1 };
        for (Point anc : pts) {
            int count{ 0 };
            for (Point p : pts) count += same_pt(anc, p);
            ans = max(ans, count);
        }
        return ans;
    }
    int ans{ 1 };
    for (int i = 0; i < pts.size(); ++ i) {
        vector<pair<ld, int>> evs;
        int act0{ 1 }; // 锚点 pts[i]
        for (int j = 0; j < pts.size(); ++ j) if (i != j) {
            ld dis{ distance(pts[i], pts[j]) };
            if (dis == 0) { ++ act0; continue; } // 重复锚点在所有角度下都被覆盖
            if (sgn(dis - 2 * rad) > 0) continue;
            ld mid{ atan2l(pts[j].y - pts[i].y, pts[j].x - pts[i].x) };
            if (mid < 0) mid += 2 * PI;
            ld delta{ acosl(clamp(dis / (2 * rad), -1.L, 1.L)) }, l{ mid - delta }, r{ mid + delta };
            while (l < 0) l += 2 * PI;
            while (r >= 2 * PI) r -= 2 * PI;
            if (l <= r) {
                evs.push_back({l, 1});
                evs.push_back({r, -1});
            } else {
                ++ act0;
                evs.push_back({r, -1});
                evs.push_back({l, 1});
            }
        }
        // 闭区间：同角度先加入，再删除
        ranges::sort(evs, [](auto a, auto b) {
            return a.first != b.first ? a.first < b.first : a.second > b.second;
```

```
        });
        int act{ act0 };
        ans = max(ans, act);
        for (auto [ang, delta] : evs) {
            (void)ang;
            act += delta;
            ans = max(ans, act);
        }
    }
    return ans;
}
```

接口说明

- `max_pts_cir(pts,rad)`：返回一个半径为 `rad` 的闭圆盘最多能覆盖多少个输入点，$O(n^2 \log n)$
- 圆心位置不固定，接口只返回最大数量，不恢复圆心
- `rad=0` 时返回某个位置上近重点的最大数量

注意事项

- 最优圆可以平移到至少有一个输入点位于边界，因此枚举边界锚点不会漏解
- 角区间是闭区间；事件同角时必须先处理 `+1` 再处理 `-1`，否则会漏掉相切情形
- 距离大于 $2r$ 的点不可能与锚点同时被覆盖
- 若需要恢复圆心，在更新答案时保存锚点与当前事件角，圆心为 `anc + Point(cos(ang),sin(ang))*rad`

常见用法

```
int ans = max_pts_cir(pts, rad);
```

复杂度

时间复杂度：$O(n^2 \log n)$
额外空间：$O(n)$

圆与多边形面积交

解决问题： 求两个圆的公共面积，以及一个圆与任意简单多边形的公共面积
对多边形每条有向边 (a,b)，把三角形 Oab 按与圆的交点分段；圆内部分使用叉积面积，圆外部分使用扇形面积

```
ld sec_area(Point a, Point b, ld rad) { return rad * rad * atan2l(cross(a, b), dot(a, b)) / 2; }
// 圆心固定为原点，返回有向三角形 Oab 与圆盘的有向面积交
ld cir_tri_int(Point a, Point b, ld rad) {
    Point dir{ b - a };
    vector<ld> pars{0, 1};
    ld qa{ norm2(dir) }, qb{ 2 * dot(a, dir) };
    ld qc{ norm2(a) - rad * rad }, disc{ qb * qb - 4 * qa * qc };
    if (sgn(qa) != 0 && sgn(disc) >= 0) {
        ld root{ sqrtl(max(0.L, disc)) };
        for (ld t : {(-qb - root) / (2 * qa), (-qb + root) / (2 * qa)}) {
            if (t > EPS && t < 1 - EPS) pars.push_back(t);
        }
    }
    ranges::sort(pars);
    ld area{ 0 };
    for (int i = 0; i + 1 < pars.size(); ++ i) {
        Point l{ a + dir * pars[i] }, r{ a + dir * pars[i + 1] };
        Point mid{ (l + r) / 2 };
        if (sgn(length(mid) - rad) <= 0) area += cross(l, r) / 2;
        else area += sec_area(l, r, rad);
    }
    return area;
}
ld cir_poly_area(Circle cir, const vector<Point> &poly) {
    ld area{ 0 };
    for (int i = 0, n = poly.size(); i < n; ++ i) {
        area += cir_tri_int(poly[i] - cir.ctr, poly[(i + 1) % n] - cir.ctr, cir.rad);
    }
    return fabsl(area);
}
ld cir_int_area(Circle a, Circle b) {
    ld dis{ distance(a.ctr, b.ctr) };
```

```
    if (sgn(dis - a.rad - b.rad) ≥ 0) return 0;
    if (sgn(dis - fabsl(a.rad - b.rad)) ≤ 0) {
        ld rad{ min(a.rad, b.rad) };
        return PI * rad * rad;
    }
    ld ang_a{ 2 * acosl(clamp((a.rad * a.rad + dis * dis - b.rad * b.rad) / (2 * a.rad * dis), -1.L, 1.L)) };
    ld ang_b{ 2 * acosl(clamp((b.rad * b.rad + dis * dis - a.rad * a.rad) / (2 * b.rad * dis), -1.L, 1.L)) };
    return (a.rad * a.rad * (ang_a - sinl(ang_a)) + b.rad * b.rad * (ang_b - sinl(ang_b))) / 2;
}
```

接口说明

- `sec_area(a,b,rad)`：返回从向量 `a` 逆时针/顺时针转到 `b` 的有向扇形面积，角度范围由 `atan2` 决定，$O(1)$
- `cir_tri_int(a,b,rad)`：圆心为原点时，返回有向三角形 Oab 与圆盘的有向面积交，$O(1)$
- `cir_poly_area(cir,poly)`：返回圆盘与简单多边形的公共面积，$O(n)$
- `cir_int_area(a,b)`：返回两个闭圆盘的公共面积，$O(1)$

注意事项

- 多边形顶点可以顺时针或逆时针排列，最终接口会取绝对值；底层三角贡献必须保留方向
- `cir_poly_area()` 适用于简单多边形，不要求凸
- 半径必须非负；两圆面积交公式中的反三角函数参数必须 `clamp` 到 $[-1,1]$
- 相离和相切时公共面积为 0；包含时公共面积为较小圆面积

常见用法

```
ld com_cir = cir_int_area(a, b);
ld com_poly = cir_poly_area(cir, poly);
```

复杂度

操作	时间复杂度	额外空间
两圆面积交	$O(1)$	$O(1)$
圆与 n 边形面积交	$O(n)$	$O(1)$

圆面积并与可见圆弧

解决问题： 求多个圆盘的面积并、并集边界长度，以及每个圆未被其他圆覆盖的可见圆弧
每个圆只对其未被其他圆覆盖的弧做 Green 公式积分。重复圆通过下标打破平局，只保留一个圆贡献边界

```
// 返回第 idx 个圆未被其他圆覆盖的角区间，区间位于 [0,2PI]
vector<pair<ld, ld>> vis_arcs(const vector<Circle> &cirs, int idx) {
    Circle cir{ cirs[idx] };
    if (sgn(cir.rad) == 0) return {};
    vector<pair<ld, ld>> cov;
    for (int j = 0; j < cirs.size(); ++ j) if (j != idx) {
        Circle oth{ cirs[j] };
        ld dis{ distance(cir.ctr, oth.ctr) };
        // cir 被严格更大的圆或更早出现的重复圆完全覆盖
        if (sgn(dis + cir.rad - oth.rad) ≤ 0 && (sgn(cir.rad - oth.rad) < 0 || idx > j)) return {};
        // 外离/外切，或 oth 完全位于 cir 内部，不覆盖 cir 的边界弧
        if (sgn(dis - cir.rad - oth.rad) ≥ 0 || sgn(dis + oth.rad - cir.rad) ≤ 0) continue;
        ld mid{ atan2l(oth.ctr.y - cir.ctr.y, oth.ctr.x - cir.ctr.x) };
        if (mid < 0) mid += 2 * PI;
        ld delta{ acosl(clamp((cir.rad * cir.rad + dis * dis
            - oth.rad * oth.rad) / (2 * cir.rad * dis), -1.L, 1.L)) };
        ld l{ mid - delta }, r{ mid + delta };
        while (l < 0) l += 2 * PI, r += 2 * PI;
        while (l ≥ 2 * PI) l -= 2 * PI, r -= 2 * PI;
        if (r ≤ 2 * PI) {
            cov.push_back({l, r});
        } else {
            cov.push_back({l, 2 * PI});
            cov.push_back({0, r - 2 * PI});
        }
    }
    ranges::sort(cov);
    vector<pair<ld, ld>> mgd;
    for (auto itv : cov) {
```

```
        if (mgd.empty() || itv.first > mgd.back().second + EPS) mgd.push_back(itv);
        else mgd.back().second = max(mgd.back().second, itv.second);
    }
    vector<pair<ld, ld>> vis;
    ld st{ 0 };
    for (auto [l, r] : mgd) {
        if (l > st + EPS) vis.push_back({st, l});
        st = max(st, r);
    }
    if (st < 2 * PI - EPS) vis.push_back({st, 2 * PI});
    return vis;
}
ld cir_uni_area(const vector<Circle> &cirs) {
    ld area{ 0 };
    Point origin{ cirs.empty() ? Point{} : cirs[0].ctr };
    for (int i = 0; i < cirs.size(); ++ i) {
        Circle cir{ cirs[i] };
        cir.ctr -= origin; // 平移后积分，避免大坐标相消
        for (auto [l, r] : vis_arcs(cirs, i)) {
            Point p1{ Point(cosl(l), sinl(l)) * cir.rad };
            Point p2{ Point(cosl(r), sinl(r)) * cir.rad };
            area += (cross(cir.ctr, p2 - p1) + cir.rad * cir.rad * (r - l)) / 2;
        }
    }
    return max(0.L, area);
}
ld cir_uni_peri(const vector<Circle> &cirs) {
    ld peri{ 0 };
    for (int i = 0; i < cirs.size(); ++ i) {
        for (auto [l, r] : vis_arcs(cirs, i)) {
            peri += cirs[i].rad * (r - l);
        }
    }
    return peri;
}
```

接口说明

- `vis_arcs(cirs,idx)`：返回第 `idx` 个圆未被其他圆盘覆盖的角区间，$O(n \log n)$
- `cir_uni_area(cirs)`：返回所有闭圆盘的面积并，$O(n^2 \log n)$
- `cir_uni_peri(cirs)`：返回圆盘并集边界总长度，$O(n^2 \log n)$
- 若只需要某个圆的可见弧长度，累加 `rad*(r-l)` 即可

注意事项

- 半径必须非负；半径为 0 的圆不贡献面积和边界长度
- 重复圆通过输入下标去重：只保留下标较小者的完整边界
- 相切只产生单点，不改变面积并与边界长度，因此按“不覆盖弧”处理
- `vis_arcs()` 每次都会重新扫描全部圆；若同时求面积和周长，建议先缓存每个圆的可见区间，避免重复计算
- Green 公式使用逆时针圆弧；角区间统一存为 `l < r` 且位于 $[0, 2\pi]$

常见用法

```
ld area = cir_uni_area(cirs);
ld peri = cir_uni_peri(cirs);
```

复杂度

时间复杂度：$O(n^2 \log n)$
除返回区间外，单次扫描额外空间为 $O(n)$

圆反演与常见结论

解决问题： 快速调用圆反演、Apollonius 轨迹、最近圆对与圆弧/扇形重心公式

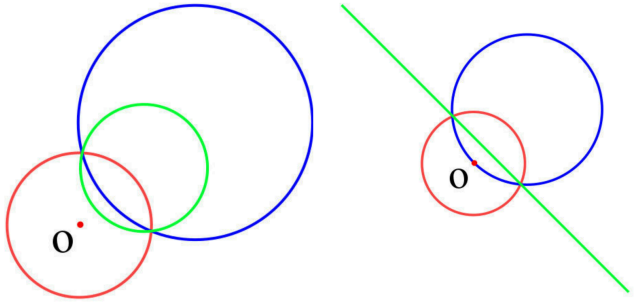
圆反演

以 O 为中心、幂为 R^2 的反演满足

$$OP \cdot OP' = R^2, \qquad P' = O + (P - O)\frac{R^2}{|P - O|^2}$$

```
// power = R^2; 反演中心本身没有有限像
optional<Point> inv_pt(Point p, Point o, ld power) {
    Point dir{ p - o };
    ld dis2{ norm2(dir) };
    if (dis2 == 0) return nullopt;
    return o + dir * (power / dis2);
}
```

不过反演中心的圆仍变为不过反演中心的圆；过反演中心的圆变为不过反演中心的直线，反向关系也成立。切点不为反演中心时，相切关系保持



Apollonius 圆

满足

$$\frac{|PA|}{|PB|} = k$$

的点集在 $k = 1$ 时为 AB 的垂直平分线，在 $k \neq 1$ 时为圆

```
// A、B 必须不同，k 必须非负；返回垂直平分线或 Apollonius 圆
optional<variant<Line, Circle>> apol_loc(Point a, Point b, ld k) {
    if (same_pt(a, b) || sgn(k) < 0) return nullopt;
    if (sgn(k - 1) == 0) {
        Point mid{ (a + b) / 2 };
        return Line(mid, rotate90(b - a));
    }
    ld k2{ k * k }, den{ 1 - k2 }, rad{ k * distance(a, b) / fabsl(den) };
    Point ctr{ (a - b * k2) / den };
    return Circle{ ctr, rad };
}
```

最近圆对

两个闭圆盘之间的距离为

$$\max(0, |c_i - c_j| - r_i - r_j)$$

```
// 返回 {最短距离,i,j}; 圆少于两个时返回 {INF,-1,-1}
tuple<ld, int, int> close_cir(const vector<Circle> &cirs) {
    ld ans{ numeric_limits<ld>::infinity() };
    int a{ -1 }, b{ -1 };
    for (int i = 0; i < cirs.size(); ++ i) {
        for (int j = i + 1; j < cirs.size(); ++ j) {
            ld cur{ max(0.L, distance(cirs[i].ctr, cirs[j].ctr) - cirs[i].rad - cirs[j].rad) };
            if (cur < ans) {
                ans = cur;
                a = i;
                b = j;
            }
        }
    }
    return {ans, a, b};
}
```

扇形与圆弧重心

圆心角为 θ（弧度，$0 < \theta \leq 2\pi$）时：

- 均匀扇形薄片的重心到圆心距离：

$$\frac{4r\sin(\theta/2)}{3\theta}$$

- 均匀圆弧线的重心到圆心距离：

$$\frac{2r\sin(\theta/2)}{\theta}$$

```
ld sec_ctr_dis(ld rad, ld ang) {
    if (sgn(ang) == 0) return 2 * rad / 3;
    return 4 * rad * sinl(ang / 2) / (3 * ang);
}
ld arc_ctr_dis(ld rad, ld ang) {
    if (sgn(ang) == 0) return rad;
    return 2 * rad * sinl(ang / 2) / ang;
}
```

接口说明

- `inv_pt(p,o,power)`：反演点；反演中心返回 `nullopt`，$O(1)$
- `apol_loc(a,b,k)`：返回 `Line` 或 `Circle`；非法输入返回 `nullopt`，$O(1)$
- `close_cir(cirs)`：暴力求最近圆对，$O(n^2)$
- `sec_ctr_dis()`、`arc_ctr_dis()`：返回重心沿角平分线到圆心的距离，$O(1)$

注意事项

- 反演的 `power` 通常取正数 R^2；反演中心没有有限像
- `apol_loc()` 返回 `variant<Line,Circle>`，使用 `holds_alternative<Line>()` 区分
- `A=B` 时距离比轨迹退化，接口返回 `nullopt`
- 扇形薄片与圆弧线的重心公式不同，不要混用
- 圆上整点属于数论问题。只有化为整数圆心下的 $x^2+y^2=r^2$ 后，才能使用平方和表示数等结论
- 大量圆的最近圆对需要空间分治、网格或包围盒结构；本页只提供可靠的 $O(n^2)$ 兜底

常见用法

```
if (auto image = inv_pt(p, o, rad * rad)) {
    Point inv = *image;
}
auto locus = apol_loc(a, b, ratio);
```

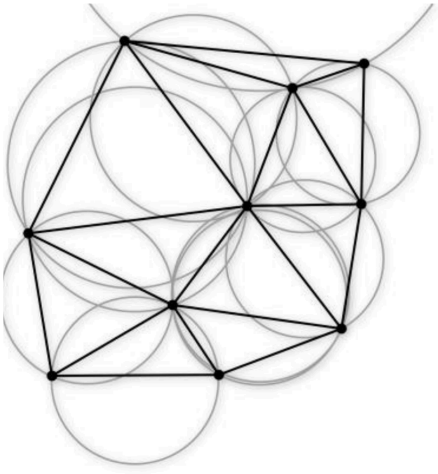
复杂度

除最近圆对为 $O(n^2)$ 外，其余接口均为 $O(1)$

15.6 扫描线、最近点与高级平面结构

Delaunay 三角剖分

解决问题： 将平面点集连接成满足空圆性质的平面三角网，并导出原输入下标表示的边与三角形
Delaunay 三角形满足：其外接圆内部不含其他输入点。四点共圆时剖分不唯一



```
struct Delaunay {
    struct Vertex {
        Point p;
        int id{ -1 };
    };
private:
    struct Edge {
        int to{ 0 };
        list<Edge>::iterator rev;
        explicit Edge(int to_ = 0) : to(to_) {}
    };
    vector<list<Edge>> wk;
    // 返回 -1 / 0 / 1：p 在 abc 外接圆内 / 圆上 / 圆外。
    // 先以 p 为原点展开二维行列式，不依赖三维 Point3，也避免绝对坐标平方相消。
    static int in_cir(Point a, Point b, Point c, Point p) {
        if (sgn(cross(a, b, c)) < 0) swap(b, c);
        a -= p; b -= p; c -= p;
        ld det{ norm2(a) * cross(b, c) + norm2(b) * cross(c, a) + norm2(c) * cross(a, b) };
        return -sgn(det);
    }
    void add_edge(int u, int v) {
        wk[u].push_front(Edge(v));
        wk[v].push_front(Edge(u));
        wk[u].begin()->rev = wk[v].begin();
        wk[v].begin()->rev = wk[u].begin();
    }
    bool prop_int(int a, int b, int c, int d) const {
        return seg_int(Segment(vs[a].p, vs[b].p), Segment(vs[c].p, vs[d].p), true);
    }
    void divide(int l, int r) {
        if (r - l <= 2) {
            for (int i = l; i <= r; ++i) {
                for (int j = i + 1; j <= r; ++j) {
                    if (j == i + 2 && sgn(cross(vs[i].p, vs[i + 1].p, vs[j].p)) == 0) continue;
                    add_edge(i, j);
                }
            }
            return;
        }
        int mid{ (l + r) >> 1 };
        divide(l, mid);
        divide(mid + 1, r);
        int lo_l{ l }, lo_r{ r };
        while (true) {
            bool mov{ false };
            Point p_l{ vs[lo_l].p };
            Point p_r{ vs[lo_r].p };
            for (auto it = wk[lo_l].begin(); it != wk[lo_l].end(); ++it) {
                Point cand{ vs[it->to].p };
                int turn{ sgn(cross(p_r, p_l, cand)) };
                if (turn > 0 || (turn == 0 && norm2(cand - p_r) < norm2(p_l - p_r))) {
                    lo_l = it->to;
                    mov = true;
                    break;
                }
            }
            if (mov) continue;
            for (auto it = wk[lo_r].begin(); it != wk[lo_r].end(); ++it) {
                Point cand{ vs[it->to].p };
                int turn{ sgn(cross(p_l, p_r, cand)) };
                if (turn < 0 || (turn == 0 && norm2(cand - p_l) < norm2(p_r - p_l))) {
                    lo_r = it->to;
                    mov = true;
                    break;
                }
            }
            if (!mov) break;
        }
        add_edge(lo_l, lo_r);
        while (true) {
```

```
            int cho{ -1 }, side{ 0 };
            for (auto it = wk[lo_l].begin(); it != wk[lo_l].end(); ++ it) {
                int v{ it->to };
                if (sgn(cross(vs[lo_l].p, vs[lo_r].p, vs[v].p)) > 0
                        && (cho == -1 || in_cir(vs[lo_l].p, vs[lo_r].p, vs[cho].p, vs[v].p) < 0)) {
                    cho = v;
                    side = -1;
                }
            }
            for (auto it = wk[lo_r].begin(); it != wk[lo_r].end(); ++ it) {
                int v{ it->to };
                if (sgn(cross(vs[lo_r].p, vs[v].p, vs[lo_l].p)) > 0
                        && (cho == -1 || in_cir(vs[lo_l].p, vs[lo_r].p, vs[cho].p, vs[v].p) < 0)) {
                    cho = v;
                    side = 1;
                }
            }
            if (cho == -1) break;
            if (side == -1) {
                for (auto it = wk[lo_l].begin(); it != wk[lo_l].end();) {
                    int v{ it->to };
                    if (prop_int(lo_l, v, lo_r, cho)) {
                        wk[v].erase(it->rev);
                        wk[lo_l].erase(it ++);
                    } else ++ it;
                }
                lo_l = cho;
                add_edge(lo_l, lo_r);
            } else {
                for (auto it = wk[lo_r].begin(); it != wk[lo_r].end();) {
                    int v{ it->to };
                    if (prop_int(lo_r, v, lo_l, cho)) {
                        wk[v].erase(it->rev);
                        wk[lo_r].erase(it++);
                    } else ++ it;
                }
                lo_r = cho;
                add_edge(lo_l, lo_r);
            }
        }
    }
    void fin_g() {
        g.assign(vs.size(), {});
        for (int u = 0; u < vs.size(); ++ u) {
            for (const Edge &edge : wk[u]) g[u].push_back(edge.to);
            ranges::sort(g[u], [&](int a, int b) {
                return arg_less(vs[a].p - vs[u].p, vs[b].p - vs[u].p);
            });
        }
        wk.clear();
    }
public:
    vector<Vertex> vs;      // 去重、排序后的内部点
    vector<vector<int>> g;    // 内部下标邻接表，邻居按逆时针极角排列
    void build(vector<Vertex> pts) {
        ranges::sort(pts, [](const Vertex &a, const Vertex &b) { return a.p < b.p; });
        vs.clear();
        for (Vertex v : pts) {
            if (vs.empty() || !same_pt(vs.back().p, v.p)) vs.push_back(v);
            else vs.back().id = min(vs.back().id, v.id);
        }
        wk.assign(vs.size(), {});
        if (!vs.empty()) divide(0, int(vs.size()) - 1);
        fin_g();
    }
    void build(const vector<Point> &pts) {
        vector<Vertex> idxs(pts.size());
        for (int i = 0; i < pts.size(); ++ i) idxs[i] = Vertex{pts[i], i};
        build(move(idxs));
    }
    vector<pair<int, int>> edges() const {
```

```
        vector<pair<int, int>> res;
        for (int u = 0; u < vs.size(); ++ u) {
            for (int v : g[u]) if (u < v) {
                res.push_back({ vs[u].id, vs[v].id });
            }
        }
        return res;
    }
    // 利用平面嵌入遍历所有逆时针有界面；Delaunay 有界面应为三角形
    vector<array<int, 3>> tris() const {
        int n = vs.size();
        vector<unordered_map<int, int>> pos(n);
        vector<vector<char>> used(n);
        for (int u = 0; u < n; ++ u) {
            used[u].assign(g[u].size(), false);
            pos[u].reserve(g[u].size() * 2 + 1);
            for (int i = 0; i < g[u].size(); ++ i) {
                pos[u][g[u][i]] = i;
            }
        }
        vector<array<int, 3>> res;
        for (int st_u = 0; st_u < n; ++ st_u) {
            for (int st_idx = 0; st_idx < g[st_u].size(); ++ st_idx) {
                if (used[st_u][st_idx]) continue;
                vector<int> face;
                int u{ st_u }, idx{ st_idx };
                while (!used[u][idx]) {
                    used[u][idx] = true;
                    face.push_back(u);
                    int v{ g[u][idx] }, rev_idx{ pos[v].at(u) };
                    u = v;
                    idx = (rev_idx - 1 + g[v].size()) % int(g[v].size());
                }
                if (face.size() != 3) continue;
                vector<Point> poly;
                for (int v : face) poly.push_back(vs[v].p);
                if (sgn(poly_area2(poly)) <= 0) continue; // 排除外部面与退化面
                array<int, 3> tri{ vs[face[0]].id, vs[face[1]].id, vs[face[2]].id };
                ranges::sort(tri);
                res.push_back(tri);
            }
        }
        ranges::sort(res);
        res.erase(ranges::unique(res).begin(), res.end());
        return res;
    }
};
```

接口说明

- `Delaunay::Vertex{p,id}`：显式保存原编号；`id` 应唯一且非负
- `build(vector<Point>)`：原输入下标自动作为 `id`
- `build(vector<Vertex>)`：保留调用者给出的 `id`；按坐标排序并用 `same_pt()` 去重
- `vs`：内部点，已排序去重；不要把内部下标当作原输入编号
- `g[u]`：内部下标邻接表，邻居按绕 `vs[u]` 的逆时针极角排列
- `edges()`：返回原 `id` 表示的无向 Delaunay 边，每条边一次
- `tris()`：返回原 `id` 表示的有界三角面，三元组内部排序

注意事项

- 近共线、近共圆或恶意构造数据需要精确/自适应 `orient2d`、`in_cir` 谓词
- 共线或四点共圆时 Delaunay 剖分不唯一，不要依赖某一条对角线必然出现
- 全部共线时没有二维三角形，但仍可能返回连接相邻点的图边
- 近重点会被合并，只保留该组最小 `id`；必须保留重复点时先自行分组并保存映射
- `tris()` 依赖 15.2 的 `poly_area2()` 判断面方向；若只需要边，可删除整个 `tris()`
- Voronoi 是 Delaunay 的对偶：每个三角形外心是 Voronoi 顶点，凸包边对应无界射线；裁剪到边界框时可调用 15.2 的 `cut_poly()` 或 15.4 的半平面交

常见用法

```
Delaunay del;
del.build(pts);
vector<pair<int, int>> edges = del.edges();
vector<array<int, 3>> tris = del.tris();
```

复杂度

- 构造：常规/非退化数据下 $O(n \log n)$
- 平面 Delaunay 图边数：$O(n)$
- `edges()`：$O(n)$
- `tris()`：$O(n)$ 期望/平面图规模，哈希表额外空间 $O(n)$

矩形面积并与周长并

解决问题： 求大量轴对齐矩形的面积并与并集周长
矩形统一视为半开区域 $[x_1, x_2) \times [y_1, y_2)$；退化矩形忽略。扫描线沿 x 轴移动，线段树维护当前纵截面覆盖长度、连续覆盖段数和左右端覆盖状态

```
struct Rectangle {
    i64 x1, y1, x2, y2;
};
struct RectUnion {
    i128 area{ 0 }, peri{ 0 };
};
RectUnion rec_union(const vector<Rectangle> &recs) {
    struct Event {
        i64 x, y1, y2;
        int delta;
    };
    vector<Event> evs;
    vector<i64> ys;
    for (Rectangle rec : recs) {
        if (rec.x1 >= rec.x2 || rec.y1 >= rec.y2) continue;
        evs.push_back({ rec.x1, rec.y1, rec.y2, 1 });
        evs.push_back({ rec.x2, rec.y1, rec.y2, -1 });
        ys.push_back(rec.y1);
        ys.push_back(rec.y2);
    }
    if (evs.empty()) return {};
    ranges::sort(evs, [](const Event &a, const Event &b) {
        if (a.x != b.x) return a.x < b.x;
        return a.delta > b.delta; // 同 x 先加入，再删除
    });
    ranges::sort(ys);
    ys.erase(ranges::unique(ys).begin(), ys.end());
    int m{ int(ys.size()) - 1 };
    vector<int> cover(4 * max(1, m)), segs(4 * max(1, m));
    vector<i128> len(4 * max(1, m));
    vector<char> l_cov(4 * max(1, m)), r_cov(4 * max(1, m));
    auto pull = [&](auto self, int p, int l, int r) -> void {
        if (cover[p] > 0) {
            len[p] = i128(ys[r]) - ys[l];
            segs[p] = 1;
            l_cov[p] = r_cov[p] = true;
            return;
        }
        if (r - l == 1) {
            len[p] = segs[p] = 0;
            l_cov[p] = r_cov[p] = false;
            return;
        }
        int lc{ p << 1 }, rc{ p << 1 | 1 };
        len[p] = len[lc] + len[rc];
        segs[p] = segs[lc] + segs[rc] - (r_cov[lc] && l_cov[rc]);
        l_cov[p] = l_cov[lc];
        r_cov[p] = r_cov[rc];
    };
    auto upd = [&](auto self, int p, int l, int r, int ql, int qr, int delta) -> void {
        if (ql <= l && r <= qr) {
            cover[p] += delta;
            pull(pull, p, l, r);
```

```
            return;
        }
        int mid{ (l + r) >> 1 };
        if (ql < mid) self(self, p << 1, l, mid, ql, qr, delta);
        if (mid < qr) self(self, p << 1 | 1, mid, r, ql, qr, delta);
        pull(pull, p, l, r);
    };
    RectUnion res;
    i64 pre_x{ evs[0].x };
    for (int i = 0; i < evs.size(); ) {
        i64 x{ evs[i].x };
        i128 dx{ i128(x) - pre_x };
        res.area += len[1] * dx;
        res.peri += i128(2) * segs[1] * dx;
        int j{ i };
        while (j < evs.size() && evs[j].x == x) {
            i128 bef{ len[1] };
            int l{ int(ranges::lower_bound(ys, evs[j].y1) - ys.begin()) };
            int r{ int(ranges::lower_bound(ys, evs[j].y2) - ys.begin()) };
            upd(upd, 1, 0, m, l, r, evs[j].delta);
            res.peri += len[1] > bef ? len[1] - bef : bef - len[1];
            ++ j;
        }
        pre_x = x;
        i = j;
    }
    return res;
}
```

接口说明

- `Rectangle{x1,y1,x2,y2}`：表示半开矩形 $[x_1,x_2)\times[y_1,y_2)$
- `rec_union(recs)`：返回 `{area,peri}`，两者均为 `i128`，$O(n\log n)$
- 面积与周长按几何长度计算；重叠边、内部边均不会重复计入

注意事项

- 同一横坐标的事件必须先处理进入事件，再处理离开事件；否则首尾相接的矩形会产生虚假的公共竖边
- 水平边贡献为 `2 * 当前连续覆盖段数 * dx`
- 每次区间修改前后的覆盖长度差，恰好是该事件产生的竖边贡献；同坐标先加后删可正确抵消公共边
- 若只求面积，可删除 `segs / l_cov / r_cov` 与所有周长累加

常见用法

```
RectUnion res = rec_union(recs);
auto print128 = [](i128 x) {
    if (x == 0) { cout << '0'; return; }
    // 此处面积、周长均非负
    string s;
    while (x) { s += char('0' + x % 10); x /= 10; }
    ranges::reverse(s);
    cout << s;
};
print128(res.area); cout << ' '; print128(res.peri); cout << '\n';
```

复杂度

时间复杂度：$O(n\log n)$
空间复杂度：$O(n)$

矩形覆盖层数

解决问题： 求被恰好 k 个矩形覆盖的面积，或被至少 k 个矩形覆盖的面积

```
struct RectCover {
    vector<i128> exact;    // exact[k]: 恰好覆盖 k 次的面积
    vector<i128> ge; // ge[k]: 至少覆盖 k 次的面积
};
RectCover rec_cov(const vector<Rectangle> &recs) {
    struct Event {
        i64 x, y1, y2;
```

```
        int delta;
    };
    vector<Event> evs;
    vector<i64> ys;
    for (Rectangle rec : recs) {
        if (rec.x1 >= rec.x2 || rec.y1 >= rec.y2) continue;
        evs.push_back({rec.x1, rec.y1, rec.y2, 1});
        evs.push_back({rec.x2, rec.y1, rec.y2, -1});
        ys.push_back(rec.y1);
        ys.push_back(rec.y2);
    }
    int n = recs.size();
    RectCover res;
    res.exact.assign(n + 1, 0);
    res.ge.assign(n + 1, 0);
    if (evs.empty()) return res;
    ranges::sort(evs, [](const Event &a, const Event &b) { return a.x < b.x; });
    ranges::sort(ys);
    ys.erase(ranges::unique(ys).begin(), ys.end());
    vector<int> cover(ys.size() - 1);
    i64 pre_x{ evs[0].x };
    for (int i = 0; i < evs.size();) {
        i64 x{ evs[i].x };
        i128 dx{ i128(x) - pre_x };
        for (int j = 0; j + 1 < ys.size(); ++ j) {
            if (cover[j] == 0) continue;
            i128 h{ i128(ys[j + 1]) - ys[j] };
            res.exact[cover[j]] += h * dx;
        }
        int nxt{ i };
        while (nxt < evs.size() && evs[nxt].x == x) {
            int l{ int(ranges::lower_bound(ys, evs[nxt].y1) - ys.begin()) };
            int r{ int(ranges::lower_bound(ys, evs[nxt].y2) - ys.begin()) };
            for (int j = l; j < r; ++ j) cover[j] += evs[nxt].delta;
            ++ nxt;
        }
        pre_x = x;
        i = nxt;
    }
    for (int k = n; k >= 1; -- k) {
        res.ge[k] = res.exact[k] + (k < n ? res.ge[k + 1] : 0);
    }
    return res;
}
```

接口说明

- `exact[k]`：被恰好 k 个有效矩形覆盖的面积
- `ge[k]`：被至少 k 个有效矩形覆盖的面积
- `rec_cov()`：返回全部层数，时间复杂度 $O(n^2)$
- 面积并等于 `ge[1]`；恰好覆盖两次等于 `exact[2]`

注意事项

- 本版逐个维护离散纵段，适合中等 n；只求面积并/周长并时优先使用 15.6.2 的 $O(n\log n)$ 线段树版
- `exact` 和 `ge` 的长度均为 `recs.size()+1`；退化矩形不会贡献面积
- 先统计当前横条面积，再处理该横坐标的全部事件
- 原始"大于等于层数"逐层累加会退化到 $O(n^3)$；本版先统计 `exact[k]`，最后做一次后缀和，因此为 $O(n^2)$

常见用法

```
RectCover res = rec_cov(recs);
i128 uni_a = res.ge.size() > 1 ? res.ge[1] : 0;
i128 ex2 = res.exact.size() > 2 ? res.exact[2] : 0;
```

复杂度

时间复杂度：$O(n^2)$
空间复杂度：$O(n)$

三角形面积并

解决问题： 求任意若干三角形的面积并；三角形之间可以相交、包含或退化

把所有顶点横坐标与不同三角形边的**真交点**横坐标作为分段点。相邻分段内部边界次序不变，竖截面区间并长度是一次函数，可用梯形公式精确积分

```
optional<ld> tri_edge_x(Point a, Point b, Point c, Point d) {
    Segment s1(a, b), s2(c, d);
    if (!seg_int(s1, s2, true)) return nullopt;
    auto p = line_int(Line::through(a, b), Line::through(c, d));
    if (!p) return nullopt;
    return p->x;
}
ld tri_slice(const vector<array<Point, 3>> &tris, ld x) {
    vector<pair<ld, ld>> itvs;
    for (const auto &tri : tris) {
        vector<ld> ys;
        for (int edge = 0; edge < 3; ++ edge) {
            Point a{ tri[edge] }, b{ tri[(edge + 1) % 3] };
            if (sgn(a.x - b.x) == 0) continue;
            if (!(min(a.x, b.x) < x && x < max(a.x, b.x))) continue;
            ld ratio{ (x - a.x) / (b.x - a.x) };
            ys.push_back(a.y + (b.y - a.y) * ratio);
        }
        if (ys.size() >= 2) {
            auto [lo, hi] = ranges::minmax_element(ys);
            itvs.push_back({*lo, *hi});
        }
    }
    if (itvs.empty()) return 0;
    ranges::sort(itvs);
    ld ans{ 0 }, l{ itvs[0].first }, r{ itvs[0].second };
    for (int i = 1; i < itvs.size(); ++ i) {
        if (itvs[i].first <= r + EPS) r = max(r, itvs[i].second);
        else {
            ans += r - l;
            l = itvs[i].first;
            r = itvs[i].second;
        }
    }
    return ans + r - l;
}
ld tri_area(vector<array<Point, 3>> tris) {
    tris.erase(remove_if(tris.begin(), tris.end(), [](const auto &tri) {
        return sgn(cross(tri[0], tri[1], tri[2])) == 0;
    }), tris.end());
    vector<ld> xs;
    for (const auto &tri : tris) {
        for (Point p : tri) xs.push_back(p.x);
    }
    for (int i = 0; i < tris.size(); ++ i) {
        for (int j = i + 1; j < tris.size(); ++ j) {
            for (int a = 0; a < 3; ++ a) {
                for (int b = 0; b < 3; ++ b) {
                    auto x = tri_edge_x(
                        tris[i][a], tris[i][(a + 1) % 3],
                        tris[j][b], tris[j][(b + 1) % 3]);
                    if (x) xs.push_back(*x);
                }
            }
        }
    }
    ranges::sort(xs);
    xs.erase(ranges::unique(xs, [](ld a, ld b) {
        return sgn(a - b) == 0;
    }).begin(), xs.end());
    ld area{ 0 };
    for (int i = 0; i + 1 < xs.size(); ++ i) {
        ld w{ xs[i + 1] - xs[i] };
        if (sgn(w) == 0) continue;
        ld x1{ xs[i] + w / 3 }, x2{ xs[i] + 2 * w / 3 };
        area += w * (tri_slice(tris, x1) + tri_slice(tris, x2)) / 2;
    }
```

```
    return area;
}
```

接口说明

- `tri_area(tris)`：返回任意三角形集合的面积并；自动忽略退化三角形
- `tri_slice()`：返回竖线 `x=常数` 与所有三角形相交区间的并长度，主要供主算法调用
- 边交点直接复用基础板子的严格线段相交与直线交点，不重复维护另一套几何谓词

注意事项

- 必须加入不同三角形边的交点横坐标；只按顶点横坐标分段会漏掉截面端点次序变化
- 端点相交的横坐标已经作为某个三角形顶点加入，无需额外作为“真交点”处理
- 共线重叠边不产生唯一交点，其端点横坐标已经包含在分段集合中
- 本实现依赖统一 `EPS` 合并近重合横坐标，适合普通竞赛浮点数据，不适合极端病态尺度
- 复杂度较高，只适合三角形数量较小的题目

常见用法

```
ld area = tri_area(tris);
```

复杂度

最多产生 $O(n^2)$ 个分段横坐标，每段合并 $O(n)$ 个区间：

- 时间复杂度：$O(n^3 \log n)$
- 空间复杂度：$O(n^2)$

平面最近点对

解决问题： 在平面点集中找出欧氏距离最近的一对点，并返回距离平方与两个端点

```
using CloseDis2 = decltype(norm2(Point{}));
struct ClosePair {
    CloseDis2 dis2;
    Point a, b;
};
void upd(optional<ClosePair> &ans, Point a, Point b) {
    CloseDis2 cur{ dist2(a, b) };
    if (!ans || cur < ans->dis2) ans = ClosePair{cur, a, b};
}
optional<ClosePair> close_impl(vector<Point> &p, vector<Point> &b, int l, int r) {
    if (r - l <= 3) {
        optional<ClosePair> ans;
        for (int i = l; i < r; ++ i) {
            for (int j = i + 1; j < r; ++ j) {
                upd(ans, p[i], p[j]);
            }
        }
        sort(p.begin() + l, p.begin() + r, [](Point a, Point b) {
            return a.y != b.y ? a.y < b.y : a.x < b.x;
        });
        return ans;
    }
    int mid{ (l + r) >> 1 };
    auto mid_x{ p[mid].x };
    auto ans_l{ close_impl(p, b, l, mid) };
    auto ans_r{ close_impl(p, b, mid, r) };
    optional<ClosePair> ans{ ans_l };
    if (ans_r && (!ans || ans_r->dis2 < ans->dis2)) ans = ans_r;
    merge(p.begin() + l, p.begin() + mid, p.begin() + mid, p.begin() + r, b.begin(), [](Point a, Point b) {
        return a.y != b.y ? a.y < b.y : a.x < b.x;
    });
    copy(b.begin(), b.begin() + r - l, p.begin() + l);
    int cnt{}; // 归并结果已复制回 p，复用 b 保存条带
    for (int i = l; i < r; ++ i) {
        CloseDis2 dx{ CloseDis2(p[i].x) - mid_x };
        CloseDis2 dx2{ CloseDis2(dx) * dx };
        if (ans && dx2 >= ans->dis2) continue;
        for (int j = cnt - 1; j >= 0; -- j) {
            CloseDis2 dy{ CloseDis2(p[i].y) - b[j].y };
```

```
            CloseDis2 dy2{ CloseDis2(dy) * dy };
            if (ans && dy2 >= ans->dis2) break;
            upd(ans, p[i], b[j]);
        }
        b[cnt ++] = p[i];
    }
    return ans;
}
optional<ClosePair> close_pair(vector<Point> p) {
    if (p.size() < 2) return nullopt;
    ranges::sort(p);
    vector<Point> b(p.size());
    return close_impl(p, b, 0, p.size());
}
```

接口说明

- `close_pair(pts)`：返回最近点对；点数少于 2 时返回 `nullopt`
- `dis2` 是距离平方
- `a`、`b` 是最近点本身；若需要原下标，可把输入改为 `{Point,id}` 并同步比较坐标

注意事项

- 顶层先按 (x, y) 排序；递归过程中各区间会改为按 y 排序，这是算法的一部分
- 输入存在重复点时会正确返回距离平方 0
- 整数版保留平方距离即可避免开方误差；输出欧氏距离时最后再转换为 `ld` 并 `sqrtl()`
- 不要在按 y 归并后再次使用当前位置推断原始按 x 下标

常见用法

```
auto res = close_pair(pts);
if (res) {
    auto dis2 = res->dis2;
    Point a = res->a;
    Point b = res->b;
}
```

复杂度

时间复杂度：$O(n \log n)$
额外空间：$O(n)$

欧氏最小生成树

解决问题： 以平面点为完全图顶点、欧氏距离为边权，求最小生成树总长度与所选边

```
struct EuclidMST {
    ld w{ 0 };
    vector<pair<int, int>> edges;
};
EuclidMST euc_mst(const vector<Point> &p) {
    int n = p.size();
    EuclidMST res;
    if (n == 0) return res;
    vector<ld> best(n, numeric_limits<ld>::infinity());
    vector<int> pa(n, -1);
    vector<char> used(n);
    best[0] = 0;
    for (int it = 0; it < n; ++ it) {
        int u{ -1 };
        for (int i = 0; i < n; ++ i) if (!used[i] && (u == -1 || best[i] < best[u])) {
            u = i;
        }
        used[u] = true;
        res.w += sqrtl(best[u]);
        if (pa[u] != -1) res.edges.push_back({pa[u], u});
        for (int v = 0; v < n; ++ v) if (!used[v]) {
            ld dis2{ ld(norm2(p[u] - p[v])) };
            if (dis2 < best[v]) {
                best[v] = dis2;
                pa[v] = u;
```

```
            }
        }
    }
    return res;
}
```

接口说明

- `euc_mst(pts)`：返回最小生成树总长度与 $n-1$ 条原输入下标边
- 空点集返回权值 0、空边集；单点同样返回空边集
- 内部比较使用距离平方，只在边被加入答案时开方

大规模做法

欧氏 MST 一定是 Delaunay 图的子图。点数较大时：

1. 调用 15.6.1 的 `Delaunay::edges()` 得到 $O(n)$ 条候选边
2. 用已有 `DSU` 和 Kruskal 按欧氏距离排序

注意事项

- 本版是完全图 Prim，适合 n 为几千量级的正确兜底
- 若坐标为整数，`norm2()` 可先精确计算，再转换为 `ld`；坐标过大时确认转换精度是否满足题意
- Delaunay 大规模方案要处理重复点映射与浮点退化

常见用法

```
EuclidMST res = euc_mst(pts);
cout << fixed << setprecision(10) << res.w << '\n';
```

复杂度

时间复杂度：$O(n^2)$
空间复杂度：$O(n)$，不计返回边

六边形轴坐标

解决问题： 在规则六边形网格上表示格子，计算距离并枚举六个相邻格
使用 axial 坐标 (q,r)，隐含第三个 cube 坐

$$s=-q-r$$

两格距离为

$$\frac{|\Delta q|+|\Delta r|+|\Delta s|}{2}$$

```
struct Hex {
    i64 q{ 0 }, r{ 0 };
    friend bool operator==(Hex a, Hex b) { return a.q == b.q && a.r == b.r; }
    friend bool operator<(Hex a, Hex b) { return tie(a.q, a.r) < tie(b.q, b.r); }
    Hex &operator+=(Hex oth) { q += oth.q; r += oth.r; return *this; }
    friend Hex operator+(Hex a, Hex b) { return a += b; }
    friend Hex operator-(Hex a, Hex b) { return Hex{a.q - b.q, a.r - b.r}; }
};
i64 hex_dis(Hex a, Hex b) {
    i64 dq{ a.q - b.q }, dr{ a.r - b.r }, ds{ -dq - dr };
    return (llabs(dq) + llabs(dr) + llabs(ds)) / 2;
}
array<Hex, 6> hex_nb(Hex cell) {
    constexpr array<Hex, 6> dir {{
        {1, 0}, {1, -1}, {0, -1},
        {-1, 0}, {-1, 1}, {0, 1}
    }};
    array<Hex, 6> res;
    for (int i = 0; i < 6; ++ i) res[i] = cell + dir[i];
    return res;
}
```

接口说明

- `Hex{q,r}`：一个 axial 坐标格子
- `hex_dis(a,b)`：返回最少相邻移动次数，$O(1)$
- `hex_nb(cell)`：按固定环绕顺序返回六个相邻格，$O(1)$

注意事项

- axial 坐标只有两个显式分量，但距离计算必须补出 $s = -q - r$
- 不同题目的屏幕坐标、行列坐标与 axial 坐标方向可能不同；先确定输入到 (q, r) 的映射
- 若需要哈希，可对 `(q,r)` 使用现有 pair 哈希，不必另写几何哈希

常见用法

```
i64 dis = hex_dis(st, tar);
for (Hex v : hex_nb(cur)) {
    // BFS / DFS
}
```

复杂度

所有接口时间复杂度与额外空间均为 $O(1)$

自适应 Simpson 积分

```
template<class F, class T = long double>
struct AdaptiveSimpson {
    F f;
    T eps;
    int limit;
    AdaptiveSimpson(F f_, T eps_ = 1e-10L, int limit_ = 30)
        : f(f_), eps(eps_), limit(limit_) {}
    T calc(T l, T r, T fl, T fm, T fr) const {
        return (r - l) * (fl + 4 * fm + fr) / 6;
    }
    T dfs(T l, T r, T fl, T fm, T fr, T s, T e, int dep) const {
        T m = (l + r) / 2, lm = (l + m) / 2, rm = (m + r) / 2;
        if (dep == 0 || lm == l || rm == r) return s;
        T f1 = f(lm), f2 = f(rm);
        T sl = calc(l, m, fl, f1, fm);
        T sr = calc(m, r, fm, f2, fr);
        T delta = sl + sr - s;
        if (abs(delta) <= 15 * e) return sl + sr + delta / 15;
        return dfs(l, m, fl, f1, fm, sl, e / 2, dep - 1)
             + dfs(m, r, fm, f2, fr, sr, e / 2, dep - 1);
    }
    T integrate(T l, T r) const {
        if (l == r) return 0;
        if (l > r) return -integrate(r, l);
        T m = (l + r) / 2;
        T fl = f(l), fm = f(m), fr = f(r);
        return dfs(l, r, fl, fm, fr, calc(l, r, fl, fm, fr), eps, limit);
    }
};
```

接口说明

- `AdaptiveSimpson(f, eps, limit)`：构造积分器。`f` 为一元函数，`eps` 为目标绝对误差量级，`limit` 为最多递归层数；构造 $O(1)$
- `integrate(l, r)`：计算定积分的数值近似；耗时取决于被积函数与自适应细分次数

适用场景

用于近似计算有限区间上的一元定积分 $\int_l^r f(x)\,dx$，要求能够快速计算任意采样点的 `f(x)`

- **推荐**：连续、足够光滑且难以直接求原函数的积分；曲线面积、体积、概率积分等
- **分段处理**：存在已知的不连续点、不光滑点时，将区间拆开后分别积分
- **不宜直接使用**：区间内有奇点、不可积函数、隐藏的窄尖峰、高频振荡、无穷积分区间，或要求严格精确答案的情况

使用说明

```
auto f = [](long double x) { return sinl(x); };
auto simpson = AdaptiveSimpson(f, 1e-12L, 30);
long double ans = simpson.integrate(0.L, acosl(-1.L)); // 约 2
```

- `f`：被积函数，必须在采样点有定义并返回有限值
- `eps`：目标绝对误差量级，默认 `1e-10L`
- `limit`：最大递归深度，默认 `30`
- `integrate(l, r)`：返回积分近似值，允许 `l > r`

注意事项

- `eps` 控制的是自适应 Simpson 的误差估计，**不提供严格误差保证**
- `eps` 是局部误差估计的控制参数，不是严格的全局误差证明。达到 `limit` 或浮点无法进一步二分时返回现有近似结果
- 已知分段点时，优先分别积分；如果要求总绝对误差量级为 `eps`，可将误差预算分配给各段
- 达到最大递归深度或浮点数无法继续二分时，当前实现会返回已有近似值，不报告未收敛
- 含端点奇点的积分需要先做变量代换或使用其他适当方法
- 复杂度取决于实际采样次数，通常无法仅由积分区间长度给出固定上界
- 适合有限区间上的平滑或分段充分光滑函数。间断、尖峰、奇点、振荡、高度抵消情形不能保证达到 `eps`；已知分段点时应拆分积分区间
- `f` 必须在所有采样点有定义并给出有限值；不应有不可重复的副作用

复杂度

操作	时间复杂度
`integrate`	$O(K)$，K 为实际采样节点数；最坏随递归上限指数增长

额外空间：$O(\text{limit})$（栈空间）

16. 三维、球面与特殊几何

16.1 三维基础运算与旋转

数值与版本约定

与二维 `Point` 一致：**整数版负责精确的向量运算与方向判定；浮点版负责长度、投影、角度及任意轴旋转**。两版同名代码是替换关系，不应同时粘贴

- 整数版：输入坐标、加减、数乘与取反结果必须能由 `i64` 表示；叉积三个分量、点积、混合积与距离平方的**每个中间结果及总和**均须在有符号 `i128` 范围内。使用 `i128` 计算不能保证任意满范围 `i64` 坐标安全
- 浮点版：使用 `ld = long double`、`EPS = 1E-12L`；`EPS` 用于几何分支判断，不是题目输出误差
- 三维叉积是**向量**，混合积是**标量**；三维不存在与二维完全等价的全局逆时针极角顺序
- 与二维板子相同，浮点结构的 `==` 和 `⟺` 比较原始值，几何意义上的重合使用 `same_pt()`

整数结构体

```
int sgn(i128 x) { return (x > 0) - (x < 0); }
struct Point3 {
    i64 x{ 0 }, y{ 0 }, z{ 0 };
    Point3() = default;
    Point3(i64 x_, i64 y_, i64 z_) : x(x_), y(y_), z(z_) {}
    Point3 operator+() const { return *this; }
    Point3 operator-() const { return {-x, -y, -z}; }
    Point3 &operator+=(Point3 p) {
        x += p.x; y += p.y; z += p.z;
        return *this;
    }
    Point3 &operator-=(Point3 p) {
        x -= p.x; y -= p.y; z -= p.z;
        return *this;
    }
    friend Point3 operator+(Point3 a, Point3 b) { return a += b; }
    friend Point3 operator-(Point3 a, Point3 b) { return a -= b; }
    Point3 &operator*=(i128 k) {
        x *= k; y *= k; z *= k;
        return *this;
```

```
    }
    friend Point3 operator*(Point3 p, i128 k) { return p *= k; }
    friend Point3 operator*(i128 k, Point3 p) { return p *= k; }
    friend bool operator==(Point3 a, Point3 b) {
        return a.x == b.x && a.y == b.y && a.z == b.z;
    }
    friend auto operator<=>(Point3 a, Point3 b) = default;
};
bool same_pt(Point3 a, Point3 b) { return a == b; }
```

整数基础函数

```
// ===== 点积、叉积、混合积：其他空间判定依赖 =====
i128 dot(Point3 a, Point3 b) {
    return i128(a.x) * b.x + i128(a.y) * b.y + i128(a.z) * b.z;
}
// 返回 i128 三元组，避免把叉积误收窄到 Point3(i64)
array<i128, 3> cross(Point3 a, Point3 b) {
    return {
        i128(a.y) * b.z - i128(a.z) * b.y,
        i128(a.z) * b.x - i128(a.x) * b.z,
        i128(a.x) * b.y - i128(a.y) * b.x
    };
}
i128 triple(Point3 a, Point3 b, Point3 c) {
    auto t = cross(b, c);
    return i128(a.x) * t[0] + i128(a.y) * t[1] + i128(a.z) * t[2];
}
// 用宽整数差分，避免计算两个位置差时提前发生 i64 溢出
i128 dot(Point3 o, Point3 a, Point3 b) {
    return (i128(a.x) - o.x) * (i128(b.x) - o.x)
         + (i128(a.y) - o.y) * (i128(b.y) - o.y)
         + (i128(a.z) - o.z) * (i128(b.z) - o.z);
}
array<i128, 3> cross(Point3 o, Point3 a, Point3 b) {
    i128 x1 = i128(a.x) - o.x, y1 = i128(a.y) - o.y, z1 = i128(a.z) - o.z;
    i128 x2 = i128(b.x) - o.x, y2 = i128(b.y) - o.y, z2 = i128(b.z) - o.z;
    return {y1 * z2 - z1 * y2, z1 * x2 - x1 * z2, x1 * y2 - y1 * x2};
}
// det(b-a,c-a,d-a)，返回有向六倍四面体体积
i128 orient(Point3 a, Point3 b, Point3 c, Point3 d) {
    auto n = cross(a, b, c);
    return n[0] * (i128(d.x) - a.x)
         + n[1] * (i128(d.y) - a.y)
         + n[2] * (i128(d.z) - a.z);
}

i128 norm2(Point3 p) { return dot(p, p); }
i128 dist2(Point3 a, Point3 b) {
    i128 x = i128(a.x) - b.x, y = i128(a.y) - b.y, z = i128(a.z) - b.z;
    return x * x + y * y + z * z;
}

// ===== 共线、共面、垂直 =====
bool collinear(Point3 a, Point3 b, Point3 c) {
    auto t = cross(a, b, c);
    return t[0] == 0 && t[1] == 0 && t[2] == 0;
}
bool coplanar(Point3 a, Point3 b, Point3 c, Point3 d) {
    return orient(a, b, c, d) == 0;
}
bool parallel(Point3 a, Point3 b) {
    auto t = cross(a, b);
    return t[0] == 0 && t[1] == 0 && t[2] == 0;
}
bool perp(Point3 a, Point3 b) { return dot(a, b) == 0; }

// ===== 90 度坐标轴旋转 =====
```

```
// 均为沿坐标轴正向观察、按右手法则旋转 +90°
Point3 rotate90x(Point3 p) { return {p.x, -p.z, p.y}; }
Point3 rotate90y(Point3 p) { return {p.z, p.y, -p.x}; }
Point3 rotate90z(Point3 p) { return {-p.y, p.x, p.z}; }
```

浮点结构体

以下代码整体替换整数结构体及整数基础函数；不必再复制其 `sgn(i128)`、`dot()`、`cross()` 等函数

```
using ld = long double;
constexpr ld EPS{ 1E-12L };
const ld PI = acosl(-1.L);
int sgn(ld x, ld eps = EPS) { return (x > eps) - (x < -eps); }

struct Point3 {
    ld x{ 0 }, y{ 0 }, z{ 0 };
    Point3() = default;
    Point3(ld x_, ld y_, ld z_) : x(x_), y(y_), z(z_) {}
    Point3 operator+() const { return *this; }
    Point3 operator-() const { return {-x, -y, -z}; }
    Point3 &operator+=(Point3 p) {
        x += p.x; y += p.y; z += p.z;
        return *this;
    }
    Point3 &operator-=(Point3 p) {
        x -= p.x; y -= p.y; z -= p.z;
        return *this;
    }
    friend Point3 operator+(Point3 a, Point3 b) { return a += b; }
    friend Point3 operator-(Point3 a, Point3 b) { return a -= b; }
    Point3 &operator*=(ld k) {
        x *= k; y *= k; z *= k;
        return *this;
    }
    Point3 &operator/=(ld k) {
        x /= k; y /= k; z /= k;
        return *this;
    }
    friend Point3 operator*(Point3 p, ld k) { return p *= k; }
    friend Point3 operator*(ld k, Point3 p) { return p *= k; }
    friend Point3 operator/(Point3 p, ld k) { return p /= k; }
    friend bool operator==(Point3 a, Point3 b) {
        return a.x == b.x && a.y == b.y && a.z == b.z;
    }
    friend auto operator<=>(Point3 a, Point3 b) = default;
};

bool same_pt(Point3 a, Point3 b) {
    return sgn(a.x - b.x) == 0
        && sgn(a.y - b.y) == 0 && sgn(a.z - b.z) == 0;
}
```

浮点基础函数

```
// ===== 点积、叉积、混合积 =====
ld dot(Point3 a, Point3 b) { return a.x*b.x + a.y*b.y + a.z*b.z; }
Point3 cross(Point3 a, Point3 b) {
    return {a.y*b.z - a.z*b.y, a.z*b.x - a.x*b.z, a.x*b.y - a.y*b.x};
}
ld triple(Point3 a, Point3 b, Point3 c) { return dot(a, cross(b, c)); }
ld dot(Point3 o, Point3 a, Point3 b) { return dot(a - o, b - o); }
Point3 cross(Point3 o, Point3 a, Point3 b) { return cross(a - o, b - o); }
ld orient(Point3 a, Point3 b, Point3 c, Point3 d) {
    return triple(b - a, c - a, d - a);
}
ld norm2(Point3 p) { return dot(p, p); }
```

```
ld dist2(Point3 a, Point3 b) { return norm2(a - b); }

// ===== 长度、角度与单位化 =====
ld length(Point3 p) { return sqrtl(norm2(p)); }
ld distance(Point3 a, Point3 b) { return length(a - b); }
optional<Point3> unit(Point3 p) {
    ld len = length(p);
    if (len == 0) return nullopt;
    return p / len;
}
// 两个非零向量的无向夹角 [0, PI]
ld vec_ang(Point3 a, Point3 b) {
    return atan2l(length(cross(a, b)), dot(a, b));
}

// ===== 向量投影与对称 =====
// 以下函数的 dir / normal 必须非零
Point3 proj_vec(Point3 p, Point3 dir) {
    return dir * (dot(p, dir) / norm2(dir));
}
Point3 refl_vec(Point3 p, Point3 normal) {
    return p - 2.L * proj_vec(p, normal);
}

// ===== 旋转：只需基础运算与长度，可独立删除 =====
// 右手法则绕坐标轴旋转 ang 弧度
Point3 rotate_x(Point3 p, ld ang) {
    ld c = cosl(ang), s = sinl(ang);
    return {p.x, p.y*c - p.z*s, p.y*s + p.z*c};
}
Point3 rotate_y(Point3 p, ld ang) {
    ld c = cosl(ang), s = sinl(ang);
    return {p.x*c + p.z*s, p.y, -p.x*s + p.z*c};
}
Point3 rotate_z(Point3 p, ld ang) {
    ld c = cosl(ang), s = sinl(ang);
    return {p.x*c - p.y*s, p.x*s + p.y*c, p.z};
}
// Rodrigues：绕经过原点、方向为 axis 的轴旋转
Point3 rotate_axis(Point3 p, Point3 axis, ld ang) {
    ld len = length(axis);
    // 前置条件：axis ≠ 0
    Point3 u = axis / len;
    ld c = cosl(ang), s = sinl(ang);
    return p*c + cross(u, p)*s + u*(dot(u, p)*(1.L - c));
}
// 绕经过 o、方向为 axis 的空间直线旋转
Point3 rotate_about(Point3 p, Point3 o, Point3 axis, ld ang) {
    return o + rotate_axis(p - o, axis, ang);
}

// ===== 空间姿态判定 =====
// 此处仅判定数学上的零向量；大规模数值下应自定相对容差
bool parallel(Point3 a, Point3 b) {
    ld la = length(a), lb = length(b);
    if (la == 0 || lb == 0) return false;
    return length(cross(a / la, b / lb)) ≤ EPS;
}
bool perp(Point3 a, Point3 b) {
    ld la = length(a), lb = length(b);
    if (la == 0 || lb == 0) return false;
    return fabsl(dot(a / la, b / lb)) ≤ EPS;
}
bool collinear(Point3 a, Point3 b, Point3 c) {
    Point3 u = b - a, v = c - a;
    if (u == Point3{} || v == Point3{}) return true;
    return parallel(u, v);
}
bool coplanar(Point3 a, Point3 b, Point3 c, Point3 d) {
    Point3 u = b-a, v = c-a, w = d-a;
    ld lu = length(u), lv = length(v), lw = length(w);
```

```
    if (lu == 0 || lv == 0 || lw == 0) return true;
    return fabsl(triple(u / lu, v / lv, w / lw)) <= EPS;
}
```

成员与类型

- `Point3(x,y,z)`：构造空间点或向量；原始字典序比较
- 整数版 `cross()` 返回 `array<i128,3>`；浮点版返回 `Point3`
- 浮点版 `unit()` 返回 `optional<Point3>`，零向量返回 `nullopt`

基础接口

接口	返回值或用途	时间复杂度
`dot(a,b)`	点积	$O(1)$
`cross(a,b)`	向量叉积，方向满足右手法则	$O(1)$
`triple(a,b,c)`	标量三重积 $\mathbf{a}\cdot(\mathbf{b}\times\mathbf{c})$	$O(1)$
`dot(o,a,b)`	$\overrightarrow{oa}\cdot\overrightarrow{ob}$	$O(1)$
`cross(o,a,b)`	$\overrightarrow{oa}\times\overrightarrow{ob}$	$O(1)$
`orient(a,b,c,d)`	$\det(b-a,c-a,d-a)$，有向六倍四面体体积	$O(1)$
`norm2(p)` / `dist2(a,b)`	长度平方 / 距离平方	$O(1)$
`collinear(a,b,c)`	三点共线（允许重合）	$O(1)$
`coplanar(a,b,c,d)`	四点共面（允许退化）	$O(1)$
`parallel(a,b)` / `perp(a,b)`	非零向量的平行 / 垂直判断	$O(1)$

浮点扩展接口

接口	用途	时间复杂度
`length(p)` / `distance(a,b)`	长度 / 欧氏距离	$O(1)$
`unit(p)`	单位化；零向量返回 `nullopt`	$O(1)$
`vec_ang(a,b)`	两非零向量无向夹角，范围 $[0,\pi]$	$O(1)$
`proj_vec(p,dir)`	向量 `p` 在非零 `dir` 上的投影	$O(1)$
`refl_vec(p,normal)`	向量 `p` 关于过原点、法向量为 `normal` 的平面的对称向量	$O(1)$
`rotate_x/y/z(p,ang)`	绕原点处坐标轴旋转	$O(1)$
`rotate_axis(p,axis,ang)`	绕过原点、方向 `axis` 的轴旋转	$O(1)$
`rotate_about(p,o,axis,ang)`	绕过 `o`、方向 `axis` 的空间直线旋转	$O(1)$

注意事项

- `orient(a,b,c,d)` 的符号由向量顺序决定；交换 `b,c,d` 中任意两个点，符号反转。`orient / 6` 才是有向四面体体积。
- 三维叉积不应强行用 `Point3<i64>` 储存，结果即使输入坐标适合 `i64`，仍可能超出 `i64`。
- 整数 `Point3` 的 `operator-` 是便捷接口，若坐标差可能超过 `i64`，必须使用直接提升到 `i128` 的 `dot(o,a,b)`、`cross(o,a,b)`、`orient()`、`dist2()` 等接口；即使使用这些接口，`i128` 中间结果仍不可溢出。
- `parallel()` 和 `perp()` 整数版将零向量分别判作与所有向量平行及垂直（代数约定），在几何问题中需自行排除零向量；浮点版遇到零向量返回 `false`。
- 浮点排序必须使用原始坐标比较，不可在比较器中引入 `EPS`。
- `vec_ang()`、`proj_vec()`、`refl_vec()`、`rotate_axis()`、`rotate_about()` 的相应方向向量必须非零；`unit()` 则会明确报告零向量。
- `parallel()` / `coplanar()` 的浮点判定基于归一化方向，适合有限、正常范围的数值；接近退化或坐标量级很大时仍可能受到消去误差影响，不是精确谓词。
- 绕轴旋转角度均为**弧度**；沿旋转轴正向观察时，正角满足右手法则。`rotate_about` 中 `axis` 是方向向量，不是轴上第二个点。
- 旋转通常改变浮点数的末位，不应使用原始 `operator==` 检查旋转后的几何相等。

常见用法

整数叉积与四点共面

```
Point3 a{0, 0, 0}, b{1, 0, 0}, c{0, 1, 0}, d{0, 0, 1};
auto n = cross(a, b, c);    // {0, 0, 1}, array<i128, 3>
i128 vol6 = orient(a, b, c, d); // 1
bool same_plane = coplanar(a, b, c, d); // false
```

浮点角度、单位向量与投影

```
Point3 a{1, 0, 0}, b{0, 1, 0};
ld ang = vec_ang(a, b);          // PI / 2
auto u = unit(Point3{2, 0, 0}); // (1, 0, 0)
Point3 foot = proj_vec(Point3{1, 2, 3}, Point3{0, 0, 1}); // (0,0,3)
```

绕任意空间直线旋转

```
Point3 o{1, 2, 3};
Point3 axis{0, 0, 1};
Point3 p{2, 2, 3};
Point3 q = rotate_about(p, o, axis, PI / 2); // (1,3,3)
cout << fixed << setprecision(12)
     << q.x << ' ' << q.y << ' ' << q.z << '\n';
```

复杂度

所有公开操作均为 $O(1)$ 时间、$O(1)$ 额外空间

16.2 三维点线面与距离

依赖 3.1： 仅复制对应数值版本的 `Point3`、`dot`、`cross`、`norm2`、`length`、`distance` 等基础函数。整数版与浮点版分别使用，不能混合定义。本节只处理直线/线段/平面的基础关系、正交投影及最短距离；严格的交点分类和三角形相交放在 3.3

整数版：精确位置关系

适用于整数坐标的共线、共面、点在线上/线段上、两非退化直线平行/垂直、直线与平面平行/垂直以及平面同侧判定。避免除法及开平方。

```
struct Line3 {
    Point3 p, dir;
    Line3() = default;
    Line3(Point3 p_, Point3 dir_) : p(p_), dir(dir_) {}
    // a ≠ b, 且 b-a 的每个分量必须可由 i64 表示
    static Line3 through(Point3 a, Point3 b) { return {a, b - a}; }
};
struct Segment3 {
    Point3 a, b;
    Segment3() = default;
    Segment3(Point3 a_, Point3 b_) : a(a_), b(b_) {}
};
struct Plane3 {
    Point3 a, b, c;
    Plane3() = default;
    Plane3(Point3 a_, Point3 b_, Point3 c_) : a(a_), b(b_), c(c_) {}
};

// ===== 精确位置判定 =====
bool on_line(Point3 p, Line3 l) {
    if (l.dir == Point3{}) return p == l.p;
    i128 x = i128(p.x) - l.p.x, y = i128(p.y) - l.p.y;
    i128 z = i128(p.z) - l.p.z;
    return i128(l.dir.y) * z - i128(l.dir.z) * y == 0
        && i128(l.dir.z) * x - i128(l.dir.x) * z == 0
        && i128(l.dir.x) * y - i128(l.dir.y) * x == 0;
}
bool on_seg(Point3 p, Segment3 s) {
    return collinear(s.a, s.b, p) && dot(p, s.a, s.b) <= 0;
}
// 前提：平面 a,b,c 不共线
int pt_side(Point3 p, Plane3 pl) { return sgn(orient(pl.a, pl.b, pl.c, p)); }
bool on_plane(Point3 p, Plane3 pl) { return coplanar(pl.a, pl.b, pl.c, p); }
// stct=true：严格同侧；false：把处于平面的点视为两侧皆可
bool same_sd(Point3 p, Point3 q, Plane3 pl, bool stct = true) {
    int a = pt_side(p, pl), b = pt_side(q, pl);
    return stct ? (a != 0 && a == b) : (a == 0 || b == 0 || a == b);
}
// 以下方向关系都要求输入方向非零；平面须由三非共线点定义
bool line_parallel(Line3 a, Line3 b) { return parallel(a.dir, b.dir); }
```

```
bool line_perp(Line3 a, Line3 b) { return perp(a.dir, b.dir); }
// 法向量来自宽整数叉积，避免存回 i64
bool line_plane_parallel(Line3 l, Plane3 pl) {
    auto n = cross(pl.a, pl.b, pl.c);
    return n[0] * l.dir.x + n[1] * l.dir.y + n[2] * l.dir.z == 0;
}
bool line_plane_perp(Line3 l, Plane3 pl) {
    auto n = cross(pl.a, pl.b, pl.c);
    // n × dir == 0，乘积须处于 i128 范围
    return n[1] * l.dir.z == n[2] * l.dir.y
        && n[2] * l.dir.x == n[0] * l.dir.z
        && n[0] * l.dir.y == n[1] * l.dir.x;
}
```

整数版额外限制： `on_line` 直接采用宽整数差分，不要求 `p-l.p` 仍落在 `i64` 范围内。`Line3::through` 构造仍需保证坐标差可由 `i64` 表示。`orient` 与宽整数叉积、点积仍要求所有中间乘积及累加值在 `i128` 范围内。`Plane3` 不负责三点共线检查，调用方应排除退化平面。

浮点版：结构体与判定

复用 3.1 的浮点 `Point3`、`EPS`、`sgn`、`dot`、`cross`、`norm2`、`length`、`distance`、`unit`。本节浮点版 `Plane3` 采用**一点 + 法向量**表示，避免反复构造三个顶点

```
struct Line3 {
    Point3 p, dir;
    Line3() = default;
    Line3(Point3 p_, Point3 dir_) : p(p_), dir(dir_) {}
    static Line3 through(Point3 a, Point3 b) { return {a, b - a}; }
};
struct Segment3 {
    Point3 a, b;
    Segment3() = default;
    Segment3(Point3 a_, Point3 b_) : a(a_), b(b_) {}
};
struct Plane3 {
    Point3 p, n;
    Plane3() = default;
    Plane3(Point3 p_, Point3 n_) : p(p_), n(n_) {}
    // 三点不共线时有效
    static Plane3 through(Point3 a, Point3 b, Point3 c) {
        return {a, cross(b - a, c - a)};
    }
    // ax+by+cz=d，要求法向量 (a,b,c) 非零
    static optional<Plane3> equation(ld a, ld b, ld c, ld d) {
        Point3 n{a, b, c};
        ld nn = norm2(n);
        if (nn == 0) return nullopt;
        return Plane3(n * (d / nn), n);
    }
};

// ===== 位置判定 =====
// 方向类函数均要求直线、平面非退化
bool on_line(Point3 p, Line3 l) {
    ld len = length(l.dir);
    if (len == 0) return same_pt(p, l.p);
    Point3 v = p - l.p;
    return length(cross(l.dir / len, v)) ⩽ EPS;
}
bool on_seg(Point3 p, Segment3 s) {
    Point3 v = s.b - s.a;
    ld vv = norm2(v);
    if (vv == 0) return same_pt(p, s.a);
    ld t = dot(p - s.a, v) / vv;
    return t ⩾ -EPS && t ⩽ 1 + EPS && on_line(p, Line3{s.a, v});
}
// 正负侧遵循法向量 n，前提：n ≠ 0
ld signed_dis_plane(Point3 p, Plane3 pl) {
    return dot(p - pl.p, pl.n) / length(pl.n);
}
int pt_side(Point3 p, Plane3 pl) { return sgn(signed_dis_plane(p, pl)); }
bool on_plane(Point3 p, Plane3 pl) { return pt_side(p, pl) == 0; }
```

```
bool same_sd(Point3 p, Point3 q, Plane3 pl, bool stct = true) {
    int a = pt_side(p, pl), b = pt_side(q, pl);
    return stct ? (a != 0 && a == b) : (a == 0 || b == 0 || a == b);
}
bool line_parallel(Line3 a, Line3 b) { return parallel(a.dir, b.dir); }
bool line_perp(Line3 a, Line3 b) { return perp(a.dir, b.dir); }
bool line_plane_parallel(Line3 l, Plane3 pl) { return perp(l.dir, pl.n); }
bool line_plane_perp(Line3 l, Plane3 pl) { return parallel(l.dir, pl.n); }
bool plane_parallel(Plane3 a, Plane3 b) { return parallel(a.n, b.n); }
bool plane_perp(Plane3 a, Plane3 b) { return perp(a.n, b.n); }

// ===== 投影、对称与基本距离 =====
// 退化直线按单点处理；退化线段按端点处理
Point3 proj(Point3 p, Line3 l) {
    ld dd = norm2(l.dir);
    return dd == 0 ? l.p : l.p + l.dir * (dot(p - l.p, l.dir) / dd);
}
Point3 refl(Point3 p, Line3 l) { return 2.L * proj(p, l) - p; }
ld dis_line(Point3 p, Line3 l) { return distance(p, proj(p, l)); }
Point3 proj(Point3 p, Plane3 pl) {
    return p - pl.n * (dot(p - pl.p, pl.n) / norm2(pl.n));
}
Point3 refl(Point3 p, Plane3 pl) { return 2.L * proj(p, pl) - p; }
ld dis_plane(Point3 p, Plane3 pl) { return fabsl(signed_dis_plane(p, pl)); }
Point3 closest(Point3 p, Segment3 s) {
    Point3 v = s.b - s.a;
    ld vv = norm2(v);
    if (vv == 0) return s.a;
    ld t = clamp(dot(p - s.a, v) / vv, 0.L, 1.L);
    return s.a + v * t;
}
ld dis_seg(Point3 p, Segment3 s) { return distance(p, closest(p, s)); }
```

浮点版：空间直线与线段最近点

```
struct Closest3 {
    Point3 a, b; // 两对象上的最近点
    ld dist;     // 两点距离
};

// ===== 两条无限直线：独立扩展；退化直线作为点处理 =====
// 平行时选第一条直线的基点 a.p 作为一端，另一端取其投影
Closest3 closest(Line3 a, Line3 b) {
    Point3 u = a.dir, v = b.dir, w = a.p - b.p;
    ld A = norm2(u), C = norm2(v);
    ld D = dot(u, w), E = dot(v, w);
    ld s = 0, t = 0;
    if (A == 0 && C == 0) return {a.p, b.p, distance(a.p, b.p)};
    if (A == 0) t = E / C;
    else if (C == 0) s = -D / A;
    else {
        Point3 n = cross(u, v);
        ld det = norm2(n);
        // 叉积形式避免近乎平行时 A*C-B*B 相减丢失有效位
        if (det > 0) {
            s = dot(cross(v, w), n) / det;
            t = dot(cross(u, w), n) / det;
        } else {
            t = E / C;
        }
    }
    Point3 p = a.p + u * s, q = b.p + v * t;
    return {p, q, distance(p, q)};
}
ld line_dis(Line3 a, Line3 b) { return closest(a, b).dist; }

// ===== 两条闭线段：独立扩展；允许一条或两条线段退化为点 =====
// 返回全局最近点对，不可只取四个端点到对方线段的距离
Closest3 closest(Segment3 x, Segment3 y) {
    Point3 u = x.b - x.a, v = y.b - y.a, w = x.a - y.a;
```

```
    ld A = norm2(u), B = dot(u, v), C = norm2(v);
    ld D = dot(u, w), E = dot(v, w);
    ld s = 0, t = 0;
    if (A == 0 && C == 0) {
        return {x.a, y.a, distance(x.a, y.a)};
    } else if (A == 0) {
        t = clamp(E / C, 0.L, 1.L);
    } else if (C == 0) {
        s = clamp(-D / A, 0.L, 1.L);
    } else {
        // 叉积形式避免近乎平行时 A*C-B*B 相减丢失有效位
        Point3 n = cross(u, v);
        ld det = norm2(n);
        s = det > 0 ? clamp(dot(cross(v, w), n) / det, 0.L, 1.L) : 0.L;
        t = (B * s + E) / C;
        if (t < 0) {
            t = 0;
            s = clamp(-D / A, 0.L, 1.L);
        } else if (t > 1) {
            t = 1;
            s = clamp((B - D) / A, 0.L, 1.L);
        }
    }
    Point3 p = x.a + u * s, q = y.a + v * t;
    return {p, q, distance(p, q)};
}
ld seg_dis(Segment3 a, Segment3 b) { return closest(a, b).dist; }
```

接口说明

基础结构与位置关系

接口	作用	时间
`Line3(p,dir)` / `Line3::through(a,b)`	直线的一点与方向 / 过两点的直线	$O(1)$
`Segment3(a,b)`	闭线段	$O(1)$
整数 `Plane3(a,b,c)`	三点确定平面	$O(1)$
浮点 `Plane3(p,n)` / `through(a,b,c)`	点+法向量 / 三点构造平面	$O(1)$
浮点 `Plane3::equation(a,b,c,d)`	方程 $ax+by+cz=d$；零法向量返回 `nullopt`	$O(1)$
`on_line(p,l)` / `on_seg(p,s)`	点在直线 / 闭线段上（支持退化）	$O(1)$
`pt_side(p,pl)` / `on_plane(p,pl)`	点相对有向平面的正负侧 / 在平面上	$O(1)$
`same_sd(p,q,pl,stct)`	同侧判定；非严格时平面上的点两侧皆可	$O(1)$
`line_parallel` / `line_perp`	两直线方向平行 / 垂直	$O(1)$
`line_plane_parallel` / `line_plane_perp`	直线方向与平面平行 / 垂直	$O(1)$
`plane_parallel` / `plane_perp`	两平面平行 / 垂直（仅浮点版）	$O(1)$

浮点距离与投影

接口	作用	时间
`proj(p, Line3)` / `proj(p, Plane3)`	点到无限直线 / 平面的正交投影	$O(1)$
`refl(p, Line3)` / `refl(p, Plane3)`	点关于直线 / 平面的镜像点	$O(1)$
`dis_line(p,l)` / `dis_plane(p,pl)`	点到直线 / 平面的距离	$O(1)$
`signed_dis_plane(p,pl)`	点到有向平面的带符号距离	$O(1)$
`closest(p, Segment3)` / `dis_seg(p,s)`	点在线段上的最近点 / 距离	$O(1)$
`closest(Line3,Line3)` / `line_dis(a,b)`	两无限直线最近点对 / 距离	$O(1)$
`closest(Segment3,Segment3)` / `seg_dis(a,b)`	两闭线段最近点对 / 距离	$O(1)$

注意事项

- 3.1 与 3.2 的依赖：本文件代码必须置于对应版本的 3.1 之后；整数与浮点两套 `Line3` / `Plane3` 不可同时复制。`Line3`、`Plane3` 不做自动合法性校验。
- 平面必须非退化，直线/平面方向判断也要求方向非零；投影和距离函数允许退化直线 / 线段，视为单点。
- `pt_side` 的符号：整数版由 `orient(a,b,c,p)` 决定，浮点版由 `dot(p-pl.p,pl.n)` 决定；`Plane3::through(a,b,c)` 的法向量为 `(b-a)×(c-a)`。
- `line_plane_parallel` 仅指方向平行，直线也可能恰好在平面内；`line_parallel` 不区分两直线重合或平行不重合。

- 两条空间直线可能异面，其方向既不平行也不相交；`line_dis` 才能反映异面最短距离。求最近点对时，平行情况下最近点不唯一，返回任意有效一对。
- 空间两线段距离不能简单推广二维板子的“四端点到另一线段距离取最小”公式；异面线段的两个内部点可能是最近点。
- `EPS` 为绝对坐标距离阈值，浮点谓词不能保证极大坐标或极小夹角下的鲁棒性；高精度要求需使用自适应/精确谓词，避免绝对误差与相对误差混用。
- 对于近乎平行的直线/线段，点对参数求解可能数值不稳定；距离通常更稳定，极端条件建议提高精度或独立复核。
- 浮点 `on_seg` 对参数 `t` 采用容差，容差是无量纲参数，而 `on_line` 使用坐标距离的 `EPS`；数据尺度悬殊时应自行调整。
- 整数版 `line_plane_parallel` 和 `line_plane_perp` 内部继续产生额外 `i128` 乘积；必须确认这些表达式不会溢出。

常见用法

空间点到直线、线段、平面（浮点版）

```
Point3 p{2, 3, 4};
Line3 ln = Line3::through({0, 0, 0}, {1, 0, 0});
Segment3 seg{{0, 0, 0}, {1, 0, 0}};
Plane3 pl = Plane3::through({0, 0, 0}, {1, 0, 0}, {0, 1, 0});
Point3 foot1 = proj(p, ln);    // (2,0,0)
Point3 foot2 = proj(p, pl);    // (2,3,0)
ld d1 = dis_line(p, ln);       // 5
ld d2 = dis_seg(p, seg);       // sqrt(26)
ld d3 = dis_plane(p, pl);      // 4
```

空间异面直线、线段最近点对（浮点版）

```
Line3 l1 = Line3::through({0, 0, 0}, {1, 0, 0});
Line3 l2 = Line3::through({0, 0, 1}, {0, 1, 1});
auto res = closest(l1, l2);    // res.dist == 1

Segment3 s1{{-1, 0, 0}, {1, 0, 0}};
Segment3 s2{{0, -1, 1}, {0, 1, 1}};
auto ans = closest(s1, s2);   // 最近点是两段内部点，距离 1
```

精确平面同侧（整数版）

```
Plane3 pl{{0, 0, 0}, {1, 0, 0}, {0, 1, 0}};
int side = pt_side({0, 0, 5}, pl); // +1
bool ok = same_sd({0, 0, 2}, {1, 1, 3}, pl); // true
```

复杂度

所有函数：时间复杂度 $O(1)$，额外空间 $O(1)$

16.3 空间相交与位置判定

依赖： 按需要选择 3.1、3.2 的同一种数值版本。整数版使用 `Point3`、`Line3`、`Segment3`、`Plane3`、`cross()`、`orient()`、`sgn()`、`on_line()`、`on_seg()` 等；浮点版额外使用 `length()`、`distance()`、`signed_dis_plane()`、`dis_line()`、`seg_dis()` 等。
本节区分几何对象的位置关系与交点坐标：整数版提供精确的相交/包含判定，不做可能产生有理数的坐标计算；浮点版提供直线与直线、直线与平面、两平面的唯一交点/交线。线段与三角形、两个三角形默认包含边界接触和共面重叠

整数版：空间线、面的位置关系

```
// ===== 两条空间直线的关系 =====
enum class LineLineRel {
    INVALID,     // 无效：至少一条直线退化
    INTERSECT,   // 相交：唯一交点
    PARALLEL,    // 平行：不重合，无交点
    COINCIDENT,  // 重合：无穷多个交点
    SKEW         // 异面：不平行、不相交
};
// ===== 空间直线与平面的关系 =====
enum class LinePlaneRel {
    INVALID,     // 无效：直线或平面退化
    INTERSECT,   // 相交：唯一交点
    PARALLEL,    // 平行：不包含，无交点
    CONTAINED    // 包含：整条直线位于平面内
};
// ===== 两个空间平面的关系 =====
```

```
enum class PlanePlaneRel {
    INTERSECT,   // 相交：交集是一条直线
    PARALLEL,    // 平行：不重合，无交线
    COINCIDENT   // 重合：两个平面完全相同
};
// ===== 直线、平面分类 =====
LineLineRel line_rel(Line3 a, Line3 b) {
    if (a.dir == Point3{} || b.dir == Point3{}) return LineLineRel::INVALID;
    if (parallel(a.dir, b.dir)) {
        return on_line(b.p, a) ? LineLineRel::COINCIDENT : LineLineRel::PARALLEL;
    }
    auto n = cross(a.dir, b.dir);
    i128 v = n[0] * (i128(b.p.x) - a.p.x)
           + n[1] * (i128(b.p.y) - a.p.y)
           + n[2] * (i128(b.p.z) - a.p.z);
    return v == 0 ? LineLineRel::INTERSECT : LineLineRel::SKEW;
}
LinePlaneRel line_plane_rel(Line3 l, Plane3 pl) {
    if (l.dir == Point3{}) return LinePlaneRel::INVALID;
    auto n = cross(pl.a, pl.b, pl.c);
    if (n[0] == 0 && n[1] == 0 && n[2] == 0) return LinePlaneRel::INVALID;
    i128 den = n[0] * l.dir.x + n[1] * l.dir.y + n[2] * l.dir.z;
    if (den != 0) return LinePlaneRel::INTERSECT;
    return on_plane(l.p, pl) ? LinePlaneRel::CONTAINED : LinePlaneRel::PARALLEL;
}
PlanePlaneRel plane_rel(Plane3 a, Plane3 b) {
    // 两个平面均要求三个定义点不共线
    i128 u = orient(a.a, a.b, a.c, b.a);
    i128 v = orient(a.a, a.b, a.c, b.b);
    i128 w = orient(a.a, a.b, a.c, b.c);
    if (u != v || u != w) return PlanePlaneRel::INTERSECT;
    return u == 0 ? PlanePlaneRel::COINCIDENT : PlanePlaneRel::PARALLEL;
}
```

整数版：线段和三角形相交

```
struct Triangle3 {
    Point3 a, b, c;
    Triangle3() = default;
    Triangle3(Point3 a_, Point3 b_, Point3 c_) : a(a_), b(b_), c(c_) {}
};
// ===== 二维投影辅助：供空间线段/三角形相交使用 =====
// k=0/1/2 分别丢弃 x/y/z；均保持循环坐标次序
using P2 = pair<i64, i64>;
P2 proj2(Point3 p, int k) {
    if (k == 0) return {p.y, p.z};
    if (k == 1) return {p.z, p.x};
    return {p.x, p.y};
}
i128 cross2(P2 a, P2 b, P2 c) {
    return (i128(b.first) - a.first) * (i128(c.second) - a.second)
         - (i128(b.second) - a.second) * (i128(c.first) - a.first);
}
bool on_seg2(P2 p, P2 a, P2 b) {
    return cross2(a, b, p) == 0
        && min(a.first, b.first) <= p.first && p.first <= max(a.first, b.first)
        && min(a.second, b.second) <= p.second && p.second <= max(a.second, b.second);
}
bool seg_int2(P2 a, P2 b, P2 c, P2 d) {
    int x = sgn(cross2(a, b, c)), y = sgn(cross2(a, b, d));
    int z = sgn(cross2(c, d, a)), w = sgn(cross2(c, d, b));
    if (x == 0 && on_seg2(c, a, b)) return true;
    if (y == 0 && on_seg2(d, a, b)) return true;
    if (z == 0 && on_seg2(a, c, d)) return true;
    if (w == 0 && on_seg2(b, c, d)) return true;
    return x * y < 0 && z * w < 0;
}
int drop_axis(array<i128, 3> n) {
```

```
    auto ab = [](i128 x) -> __uint128_t {
        return x < 0 ? __uint128_t(-(x + 1)) + 1 : __uint128_t(x);
    };
    int k = 0;
    for (int i = 1; i < 3; ++ i) if (ab(n[i]) > ab(n[k])) k = i;
    return k;
}
// ===== 两空间闭线段相交：允许退化为点 =====
// 不能仅做二维投影相交：必须先排除异面直线
bool seg_int3(Segment3 a, Segment3 b) {
    if (a.a == a.b) return on_seg(a.a, b);
    if (b.a == b.b) return on_seg(b.a, a);
    if (orient(a.a, a.b, b.a, b.b) != 0) return false;
    auto n = cross(a.b - a.a, b.b - b.a);
    if (n[0] == 0 && n[1] == 0 && n[2] == 0) {
        if (!collinear(a.a, a.b, b.a)) return false;
        return on_seg(a.a, b) || on_seg(a.b, b)
            || on_seg(b.a, a) || on_seg(b.b, a);
    }
    int k = drop_axis(n);
    return seg_int2(proj2(a.a, k), proj2(a.b, k), proj2(b.a, k), proj2(b.b, k));
}
// ===== 点与空间三角形：允许三角形退化为三条边的并 =====
bool on_tri(Point3 p, Triangle3 t, bool stct = false) {
    auto n = cross(t.a, t.b, t.c);
    if (n[0] == 0 && n[1] == 0 && n[2] == 0) {
        if (stct) return false;
        return on_seg(p, {t.a, t.b}) || on_seg(p, {t.b, t.c})
            || on_seg(p, {t.c, t.a});
    }
    if (orient(t.a, t.b, t.c, p) != 0) return false;
    int k = drop_axis(n);
    P2 a = proj2(t.a, k), b = proj2(t.b, k);
    P2 c = proj2(t.c, k), q = proj2(p, k);
    int s = sgn(cross2(a, b, c));
    int x = s * sgn(cross2(a, b, q));
    int y = s * sgn(cross2(b, c, q));
    int z = s * sgn(cross2(c, a, q));
    return stct ? (x > 0 && y > 0 && z > 0)
                : (x >= 0 && y >= 0 && z >= 0);
}
// ===== 线段与空间三角形：包含共面、边界与退化情形 =====
bool seg_tri_int(Segment3 s, Triangle3 t) {
    auto n = cross(t.a, t.b, t.c);
    if (n[0] == 0 && n[1] == 0 && n[2] == 0) {
        return seg_int3(s, {t.a, t.b}) || seg_int3(s, {t.b, t.c})
            || seg_int3(s, {t.c, t.a});
    }
    int x = sgn(orient(t.a, t.b, t.c, s.a));
    int y = sgn(orient(t.a, t.b, t.c, s.b));
    if (x == 0 && on_tri(s.a, t)) return true;
    if (y == 0 && on_tri(s.b, t)) return true;
    if (x == 0 && y == 0) {
        return seg_int3(s, {t.a, t.b}) || seg_int3(s, {t.b, t.c})
            || seg_int3(s, {t.c, t.a});
    }
    if (x == 0 || y == 0 || x == y) return false;
    int u = sgn(orient(s.a, s.b, t.a, t.b));
    int v = sgn(orient(s.a, s.b, t.b, t.c));
    int w = sgn(orient(s.a, s.b, t.c, t.a));
    return (u >= 0 && v >= 0 && w >= 0)
        || (u <= 0 && v <= 0 && w <= 0);
}
// ===== 两三角形相交 =====
bool tri_tri_int(Triangle3 a, Triangle3 b) {
    for (auto s : {Segment3{a.a, a.b}, {a.b, a.c}, {a.c, a.a}}) {
        if (seg_tri_int(s, b)) return true;
    }
    for (auto s : {Segment3{b.a, b.b}, {b.b, b.c}, {b.c, b.a}}) {
        if (seg_tri_int(s, a)) return true;
    }
```

```
    return false;
}
```

浮点版：直线与平面交点、两平面交线

依赖 3.1、3.2 浮点版的 `Line3`、`Plane3`、`Point3` 与基础函数；无须复制整数版的二维投影辅助代码

```
// ===== 空间位置关系 =====
enum class LineLineRel {
    INVALID,     // 无效：至少一条直线退化
    INTERSECT,   // 相交：唯一交点
    PARALLEL,    // 平行：不重合，无交点
    COINCIDENT,  // 重合：无穷多个公共点
    SKEW         // 异面：不平行、不相交
};
enum class LinePlaneRel {
    INVALID,     // 无效：直线或平面退化
    INTERSECT,   // 相交：唯一交点
    PARALLEL,    // 平行：不包含，无交点
    CONTAINED    // 包含：整条直线位于平面内
};
enum class PlanePlaneRel {
    INVALID,     // 无效：至少一个平面退化
    INTERSECT,   // 相交：交集是一条直线
    PARALLEL,    // 平行：不重合，无交线
    COINCIDENT   // 重合：两个平面完全相同
};
// ===== 无限直线、平面位置分类：可独立删除 =====
LineLineRel line_rel(Line3 a, Line3 b) {
    ld la = length(a.dir), lb = length(b.dir);
    if (la == 0 || lb == 0) return LineLineRel::INVALID;
    Point3 u = a.dir / la, v = b.dir / lb;
    Point3 n = cross(u, v);
    if (length(n) <= EPS) {
        return dis_line(b.p, a) <= EPS
            ? LineLineRel::COINCIDENT : LineLineRel::PARALLEL;
    }
    ld d = fabsl(dot(b.p - a.p, n)) / length(n);
    return d <= EPS ? LineLineRel::INTERSECT : LineLineRel::SKEW;
}
optional<Point3> line_int(Line3 a, Line3 b) {
    if (line_rel(a, b) != LineLineRel::INTERSECT) return nullopt;
    Point3 w = b.p - a.p, n = cross(a.dir, b.dir);
    ld t = dot(cross(w, b.dir), n) / norm2(n);
    return a.p + a.dir * t;
}

struct LinePlaneHit {
    LinePlaneRel rel;
    optional<Point3> point; // 仅 INTERSECT 时存在
};
LinePlaneHit line_plane_int(Line3 l, Plane3 pl) {
    ld dl = length(l.dir), dn = length(pl.n);
    if (dl == 0 || dn == 0) return {LinePlaneRel::INVALID, nullopt};
    ld c = dot(l.dir / dl, pl.n / dn);
    if (fabsl(c) <= EPS) {
        bool inside = fabsl(signed_dis_plane(l.p, pl)) <= EPS;
        return {inside ? LinePlaneRel::CONTAINED : LinePlaneRel::PARALLEL, nullopt};
    }
    ld t = dot(pl.p - l.p, pl.n) / dot(l.dir, pl.n);
    return {LinePlaneRel::INTERSECT, l.p + l.dir * t};
}
struct PlanePlaneHit {
    PlanePlaneRel rel;
    optional<Line3> line; // 仅 INTERSECT 时存在
};
PlanePlaneHit plane_plane_int(Plane3 a, Plane3 b) {
    ld la = length(a.n), lb = length(b.n);
    if (la == 0 || lb == 0) return {PlanePlaneRel::INVALID, nullopt};
```

```
    Point3 n1 = a.n / la, n2 = b.n / lb;
    Point3 dir = cross(n1, n2);
    ld den = norm2(dir);
    if (length(dir) ⩽ EPS) {
        bool inside = fabsl(signed_dis_plane(b.p, a)) ⩽ EPS;
        return {inside ? PlanePlaneRel::COINCIDENT : PlanePlaneRel::PARALLEL, nullopt};
    }
    // 以 a.p 为局部原点，避免在世界坐标下直接构造大常数
    ld d = dot(b.p - a.p, n2);
    Point3 p = a.p + cross(dir, n1) * (d / den);
    return {PlanePlaneRel::INTERSECT, Line3{p, dir}};
}
```

浮点版：三角形和线段相交

```
struct Triangle3 {
    Point3 a, b, c;
    Triangle3() = default;
    Triangle3(Point3 a_, Point3 b_, Point3 c_) : a(a_), b(b_), c(c_) {}
};
// ===== 浮点空间线段相交：依赖 3.2 的 seg_dis() =====
bool seg_int3(Segment3 a, Segment3 b) { return seg_dis(a, b) ⩽ EPS; }
// ===== 点与空间三角形：允许退化成线段或点 =====
bool on_tri(Point3 p, Triangle3 t, bool stct = false) {
    Point3 n = cross(t.b - t.a, t.c - t.a);
    ld ln = length(n);
    ld scale = max({distance(t.a, t.b), distance(t.b, t.c), distance(t.c, t.a)});
    if (scale == 0 || ln ⩽ EPS * scale) {
        if (stct) return false;
        return on_seg(p, {t.a, t.b}) || on_seg(p, {t.b, t.c})
            || on_seg(p, {t.c, t.a});
    }
    Point3 u = n / ln;
    if (fabsl(dot(p - t.a, u)) > EPS) return false;
    auto edge_side = [&](Point3 a, Point3 b) -> ld {
        return dot(u, cross(b - a, p - a)) / distance(a, b);
    };
    ld x = edge_side(t.a, t.b), y = edge_side(t.b, t.c), z = edge_side(t.c, t.a);
    return stct ? (x > EPS && y > EPS && z > EPS)
                : (x ⩾ -EPS && y ⩾ -EPS && z ⩾ -EPS);
}
// ===== 线段与空间三角形：兼容共面、边界及退化 =====
bool seg_tri_int(Segment3 s, Triangle3 t) {
    Point3 n = cross(t.b - t.a, t.c - t.a);
    ld ln = length(n);
    ld scale = max({distance(t.a, t.b), distance(t.b, t.c), distance(t.c, t.a)});
    if (scale == 0 || ln ⩽ EPS * scale) {
        return seg_int3(s, {t.a, t.b}) || seg_int3(s, {t.b, t.c})
            || seg_int3(s, {t.c, t.a});
    }
    Point3 u = n / ln;
    ld d0 = dot(s.a - t.a, u), d1 = dot(s.b - t.a, u);
    int x = sgn(d0), y = sgn(d1);
    if (x == 0 && on_tri(s.a, t)) return true;
    if (y == 0 && on_tri(s.b, t)) return true;
    if (x == 0 && y == 0) {
        return seg_int3(s, {t.a, t.b}) || seg_int3(s, {t.b, t.c})
            || seg_int3(s, {t.c, t.a});
    }
    if (x == 0 || y == 0 || x == y) return false;
    Point3 p = s.a + (s.b - s.a) * (d0 / (d0 - d1));
    return on_tri(p, t);
}
// ===== 两三角形：内部交叉、共面包含、边界相切均为 true =====
bool tri_tri_int(Triangle3 a, Triangle3 b) {
    for (auto s : {Segment3{a.a, a.b}, {a.b, a.c}, {a.c, a.a}}) {
        if (seg_tri_int(s, b)) return true;
    }
```

```
    for (auto s : {Segment3{b.a, b.b}, {b.b, b.c}, {b.c, b.a}}) {
        if (seg_tri_int(s, a)) return true;
    }
    return false;
}
```

公共接口

接口	含义	时间复杂度
`Triangle3(a,b,c)`	三角形三个顶点，允许退化为线段或点	$O(1)$
`line_rel(a,b)`	无限直线：唯一交点 / 平行 / 重合 / 异面 / 非法	$O(1)$
`seg_int3(a,b)`	两空间闭线段是否相交（含边界和退化）	$O(1)$
`on_tri(p,t,stct)`	点是否在三角形上；`stct=true` 仅非退化三角形相对内部	$O(1)$
`seg_tri_int(s,t)`	闭线段与三角形相交（共面、包含、相切均计入）	$O(1)$
`tri_tri_int(a,b)`	两空间三角形相交（允许退化）	$O(1)$

整数版额外接口

接口	含义	时间复杂度
`line_plane_rel(l,pl)`	唯一交点 / 平行 / 直线包含于平面 / 非法	$O(1)$
`plane_rel(a,b)`	两非退化平面相交 / 平行 / 重合	$O(1)$

浮点版额外接口

接口	含义	时间复杂度
`line_int(a,b)`	两空间直线唯一交点，其他情况 `nullopt`	$O(1)$
`line_plane_int(l,pl)`	返回 `LinePlaneHit{rel,point}`；仅唯一交点时 `point` 有值	$O(1)$
`plane_plane_int(a,b)`	返回 `PlanePlaneHit{rel,line}`；仅相交成直线时 `line` 有值	$O(1)$

结果枚举： `LineLineRel::{INVALID,INTERSECT,PARALLEL,COINCIDENT,SKEW}`；`LinePlaneRel::{INVALID,INTERSECT,PARALLEL,CONTAINED}`；浮点 `PlanePlaneRel` 另含 `INVALID`。`INTERSECT` 对直线/平面表示唯一点、对两平面表示唯一交线；不要仅通过 `optional` 空值区分平行与重合。

注意事项

- **输入约定：** `Line3(p,dir)` 中 `dir` 是方向，不是第二个端点；整数 `Plane3(a,b,c)` 使用三点，浮点 `Plane3(p,n)` 使用一点与法向量，接口不可混用。
- 整数版 `line_rel()`、`line_plane_rel()` 对零方向返回 `INVALID`；`plane_rel()` 及平面分类要求平面非退化。整数版不返回除法得到的有理数交点。
- **共面分支不可省略：** `seg_tri_int()` 若线段两端都在三角形平面上，需要检查三角形内端点与三条边相交；只判断两端异侧会漏掉共面重叠。
- **空间线段必须先验共面：** 二维投影交叉不保证三维线段相交，异面时投影可能重合。
- **包含边界：** `seg_int3`、`seg_tri_int`、`tri_tri_int` 默认允许共享顶点、共享边、切触及部分/完全重合；如题目要求真正穿过内部，应另外构造严格谓词，不能简单地把 `≤` 全部改成 `<`。
- **退化三角形：** 本板子按三条闭边的并处理，允许退化为线段或点；`on_tri(...,true)` 对退化三角形返回 `false`。这不代表退化对象具有二维内部。
- **整数溢出：** `orient`、空间叉积、`line_rel` 的扩展混合积等每一步和累加都必须落在 `i128` 内；`seg_int3` 计算两线段方向 `b-a` 时，差值还需落在 `i64` 中。若输入到达 `i64` 极限，不能只靠 `i128` 保证所有公式安全。
- 浮点版以 `EPS` 控制**距离误差**与**无量纲角度误差**，不等价于精确几何谓词。近共面、近相切、极大坐标或极细长三角形可能需要更高精度/自适应谓词；分类容差可能将接近相交的对象视为相交。
- 浮点 `line_int`、`line_plane_int`、`plane_plane_int` 对近乎平行的输入可能数值敏感。特别是两平面夹角很小，交线坐标绝对误差可能显著放大。
- 浮点 `seg_int3` 直接复用 3.2 的 `seg_dis()`；如只复制三角形部分，必须保留 3.2 相应的最近点及距离函数；它和后续相交谓词共享同一 `EPS`。
- `on_tri()` 浮点版使用投影到所在平面后的边侧距离做包容判定，`stct=true` 要求三个边侧距离都严格大于 `EPS`；点至三角形所在平面的距离仍允许 `EPS` 误差。

常见用法

判断空间直线是否异面

```
Line3 a = Line3::through(p1, p2);
Line3 b = Line3::through(q1, q2);
if (line_rel(a, b) == LineLineRel::SKEW) {
    // 两条无限直线异面，没有交点
}
```

线段与空间三角形

```
Triangle3 tri{a, b, c};
Segment3 seg{p, q};
if (seg_tri_int(seg, tri)) {
    // 包括穿过内部、触边、端点接触和共面重叠
}
```

两三角形相交

```
Triangle3 a{p0, p1, p2}, b{q0, q1, q2};
if (tri_tri_int(a, b)) {
    // 包括共面包含与共享边界
}
```

求空间直线与平面的唯一交点（浮点版）

```
Line3 l = Line3::through(a, b);
Plane3 pl = Plane3::through(p, q, r);
auto hit = line_plane_int(l, pl);
if (hit.rel == LinePlaneRel::INTERSECT) {
    Point3 x = *hit.point;
}
```

求两平面的交线（浮点版）

```
Plane3 a = Plane3::through(a0, a1, a2);
Plane3 b = Plane3::through(b0, b1, b2);
auto hit = plane_plane_int(a, b);
if (hit.rel == PlanePlaneRel::INTERSECT) {
    Line3 l = *hit.line; // l.p + t*l.dir
}
```

16.4 三维凸包与多面体

依赖 3.1： 选择同一种数值版本的 `Point3`、`cross()`、`orient()`、`collinear()` 与 `sgn()`；浮点版还需 `length()`、`EPS` 和 `ld`。
本板子是 **增量法三维凸包**：输出统一朝外的三角面，并提供三维体积、表面积、体积重心、点包含与重心到表面最短距离；全点共面的情形退化为**单侧平面凸多边形**，分别提供平面面积及面积重心。输入允许重复点、共线和共面，但浮点近退化输入受 `EPS` 与数值精度限制

```
// ===== 三维凸包增量法：依赖 3.1 同一种数值版本的 Point3 / orient / cross / collinear =====
// 整数版还需 i128、sgn(i128)；浮点版还需 ld、EPS、sgn(ld)、length()。
// f 的三角面法向朝外；顶点下标对应 h.p（已去重、重排），不是原始输入下标。
template<class P = Point3>
struct Hull3 {
    using T = decltype(P{}.x);
    using Wide = conditional_t<is_integral_v<T>, i128, ld>;
    vector<P> p;
    vector<array<int, 3>> f;
    vector<int> poly; // dim == 1 时为线段端点，dim == 2 时为共面凸多边形环
    int dim{ -1 };    // -1 空集；0 点；1 线段；2 共面多边形；3 三维实体
    Hull3() = default;
    explicit Hull3(vector<P> ps) { build(move(ps)); }
    // ===== 几何谓词：整数精确；浮点比较点到平面的有符号距离 =====
    static int side(P a, P b, P c, P d) {
        if constexpr (is_integral_v<T>) {
            return sgn(orient(a, b, c, d));
        } else {
            P n = cross(a, b, c);
            ld s = length(n);
            return s == 0 ? 0 : sgn(orient(a, b, c, d) / s);
        }
    }
    static ld coord(P a, int k) {
        return k == 0 ? ld(a.x) : k == 1 ? ld(a.y) : ld(a.z);
    }
    static pair<T, T> project(P a, int k) {
        if (k == 0) return {a.y, a.z};
        if (k == 1) return {a.z, a.x};
```

```
        return {a.x, a.y};
    }
    static int turn2(P a, P b, P c, int k) {
        auto [ax, ay] = project(a, k);
        auto [bx, by] = project(b, k);
        auto [cx, cy] = project(c, k);
        if constexpr (is_integral_v<T>) {
            i128 x = i128(bx) - ax, y = i128(by) - ay;
            return sgn(x * (i128(cy) - ay) - y * (i128(cx) - ax));
        } else {
            ld x = bx - ax, y = by - ay;
            ld len = hypotl(x, y);
            return len == 0 ? 0 : sgn((x * (cy - ay) - y * (cx - ax)) / len);
        }
    }
    static int drop_axis(P a, P b, P c) {
        if constexpr (is_integral_v<T>) {
            auto n = cross(a, b, c);
            auto ab = [](i128 x) -> __uint128_t {
                return x < 0 ? __uint128_t(-(x + 1)) + 1 : __uint128_t(x);
            };
            int k = 0;
            for (int i = 1; i < 3; ++ i) if (ab(n[i]) > ab(n[k])) k = i;
            return k;
        } else {
            P n = cross(a, b, c);
            ld v[] = {fabsl(n.x), fabsl(n.y), fabsl(n.z)};
            return int(max_element(v, v + 3) - v);
        }
    }
    // ===== 共面凸包：只在 dim == 2 时使用；可与三维算法独立裁剪 =====
    void build_planar(int k) {
        vector<int> id(p.size());
        iota(id.begin(), id.end(), 0);
        sort(id.begin(), id.end(), [&](int a, int b) {
            return project(p[a], k) < project(p[b], k);
        });
        for (int i : id) {
            while (poly.size() >= 2 &&
                   turn2(p[poly[poly.size()-2]], p[poly.back()], p[i], k) <= 0)
                poly.pop_back();
            poly.push_back(i);
        }
        int base = int(poly.size()) + 1;
        for (int j = int(id.size()) - 2; j >= 0; -- j) {
            int i = id[j];
            while (int(poly.size()) >= base &&
                   turn2(p[poly[poly.size()-2]], p[poly.back()], p[i], k) <= 0)
                poly.pop_back();
            poly.push_back(i);
        }
        poly.pop_back();
    }
    // ===== 核心：可独立构造三维凸包，O(n^2) 期望哈希复杂度 =====
    void build(vector<P> ps) {
        p = move(ps);
        sort(p.begin(), p.end());
        p.erase(unique(p.begin(), p.end()), p.end());
        f.clear(); poly.clear(); dim = -1;
        int n = int(p.size());
        if (n == 0) return;
        dim = 0;
        if (n == 1) return;
        dim = 1;
        int j = 2;
        while (j < n && collinear(p[0], p[1], p[j])) ++ j;
        if (j == n) { poly = {0, n - 1}; return; }
        swap(p[2], p[j]);
        dim = 2;
        int k = 3;
        while (k < n && side(p[0], p[1], p[2], p[k]) == 0) ++ k;
```

```
        if (k == n) { build_planar(drop_axis(p[0], p[1], p[2])); return; }
        swap(p[3], p[k]);
        dim = 3;

        f = {{{0, 1, 2}, {0, 3, 1}, {0, 2, 3}, {1, 3, 2}}};
        for (int i = 0; i < 4; ++ i) {
            auto &t = f[i];
            int other = 0;
            while (other == t[0] || other == t[1] || other == t[2]) ++ other;
            if (side(p[t[0]], p[t[1]], p[t[2]], p[other]) > 0) swap(t[1], t[2]);
        }
        for (int i = 4; i < n; ++ i) {
            vector<char> vis(f.size());
            int visible = 0;
            for (int j = 0; j < int(f.size()); ++ j) {
                auto [a, b, c] = f[j];
                vis[j] = side(p[a], p[b], p[c], p[i]) > 0;
                visible += vis[j];
            }
            if (!visible) continue;
            unordered_map<uint64_t, pair<int, int>> border;
            border.reserve(size_t(visible) * 6 + 1);
            auto add = [&](int a, int b) {
                uint64_t rev = (uint64_t(uint32_t(b)) << 32) | uint32_t(a);
                if (auto it = border.find(rev); it != border.end()) border.erase(it);
                else border.emplace((uint64_t(uint32_t(a)) << 32) | uint32_t(b), pair{a, b});
            };
            vector<array<int, 3>> nf;
            nf.reserve(f.size() + visible * 2);
            for (int j = 0; j < int(f.size()); ++ j) {
                auto [a, b, c] = f[j];
                if (vis[j]) { add(a, b); add(b, c); add(c, a); }
                else nf.push_back(f[j]);
            }
            for (auto [key, edge] : border) {
                auto [a, b] = edge;
                nf.push_back({a, b, i});
            }
            f = move(nf);
        }
    }
    // ===== 凸包判定：只适用 dim == 3 =====
    // 包含边界；strict=true 表示严格内部。dim<3 时均返回 false。
    bool contains(P q, bool strict = false) const {
        if (dim != 3) return false;
        for (auto [a, b, c] : f) {
            int t = side(p[a], p[b], p[c], q);
            if (strict ? t >= 0 : t > 0) return false;
        }
        return true;
    }
    // ===== 三维体积、表面积与体积重心：若只需面片，可整段删除 =====
    // dim≠3 返回 nullopt，不能把共面多边形当三维实体。
    optional<Wide> volume6() const {
        if (dim != 3) return nullopt;
        Wide ans = 0;
        for (auto [a, b, c] : f) ans += orient(p[0], p[a], p[b], p[c]);
        return ans;
    }
    optional<ld> volume() const {
        auto v6 = volume6();
        if (!v6) return nullopt;
        return ld(*v6) / 6;
    }
    static array<ld, 3> delta(P a, P b) {
        return {ld(a.x) - ld(b.x), ld(a.y) - ld(b.y), ld(a.z) - ld(b.z)};
    }
    static array<ld, 3> cross_ld(array<ld, 3> a, array<ld, 3> b) {
        return {a[1]*b[2]-a[2]*b[1], a[2]*b[0]-a[0]*b[2], a[0]*b[1]-a[1]*b[0]};
    }
    static ld len(array<ld, 3> a) {
```

```
        return sqrtl(a[0]*a[0] + a[1]*a[1] + a[2]*a[2]);
    }
    optional<ld> surface_area() const {
        if (dim != 3) return nullopt;
        ld ans = 0;
        for (auto [a, b, c] : f)
            ans += len(cross_ld(delta(p[b], p[a]), delta(p[c], p[a]))) / 2;
        return ans;
    }
    optional<array<ld, 3>> centroid() const {
        if (dim != 3) return nullopt;
        ld sum = 0;
        array<ld, 3> acc{};
        for (auto [a, b, c] : f) {
            ld w = ld(orient(p[0], p[a], p[b], p[c]));
            sum += w;
            auto u = delta(p[a], p[0]), v = delta(p[b], p[0]), t = delta(p[c], p[0]);
            for (int k = 0; k < 3; ++ k) acc[k] += w * (u[k] + v[k] + t[k]);
        }
        if (sum <= 0) return nullopt;
        return array<ld, 3>{ld(p[0].x) + acc[0]/(4*sum), ld(p[0].y) + acc[1]/(4*sum), ld(p[0].z) + acc[2]/(4*sum)};
    }
    // ===== 到表面距离：仅对体积重心计算 =====
    optional<ld> centroid_surface_distance() const {
        auto c = centroid();
        if (!c) return nullopt;
        ld ans = numeric_limits<ld>::infinity();
        for (auto [a, b, d] : f) {
            auto u = delta(p[b], p[a]), v = delta(p[d], p[a]);
            auto n = cross_ld(u, v);
            ld nl = len(n);
            if (nl == 0) continue;
            ld s = 0;
            for (int k = 0; k < 3; ++ k) s += n[k] * ((*c)[k] - coord(p[a], k));
            ans = min(ans, fabsl(s) / nl);
        }
        return isfinite(ans) ? optional<ld>(ans) : nullopt;
    }
    // ===== 共面情形的单侧面积与面积重心：只在 dim == 2 使用 =====
    optional<ld> planar_area() const {
        if (dim != 2) return nullopt;
        ld ans = 0;
        for (int i = 1; i + 1 < int(poly.size()); ++ i)
            ans += len(cross_ld(delta(p[poly[i]], p[poly[0]]), delta(p[poly[i+1]], p[poly[0]]))) / 2;
        return ans;
    }
    optional<array<ld, 3>> planar_centroid() const {
        if (dim != 2) return nullopt;
        ld sum = 0;
        array<ld, 3> acc{};
        for (int i = 1; i + 1 < int(poly.size()); ++ i) {
            auto u = delta(p[poly[i]], p[poly[0]]);
            auto v = delta(p[poly[i+1]], p[poly[0]]);
            ld w = len(cross_ld(u, v)) / 2;
            sum += w;
            for (int k = 0; k < 3; ++ k) acc[k] += w * (u[k] + v[k]);
        }
        if (sum == 0) return nullopt;
        return array<ld, 3>{ld(p[poly[0]].x) + acc[0]/(3*sum), ld(p[poly[0]].y) + acc[1]/(3*sum), ld(p[poly[0]].z) +
acc[2]/(3*sum)};
    }
};
```

成员变量

- `p`：构造后内部的点数组；**去重且可能重排**，下标不同于输入点原始下标。
- `f`：仅在 `dim == 3` 时有效，存储 `array<int,3>`，表示三维凸包的**朝外有向三角面**。相邻三角面的公共边方向相反，多个相邻三角面可能共面。
- `poly`：`dim == 1` 时为线段两端点的下标；`dim == 2` 时为平面凸多边形的循环顶点序列；其他维数下为空。
- `dim`：`-1` 空集、`0` 单点、`1` 线段、`2` 共面凸多边形、`3` 有正体积的三维凸多面体。

接口

接口	作用	时间复杂度
`Hull3<>(points)` / `build(points)`	构造凸包、判定退化维数	预期 $O(n^2)$
`contains(q, strict)`	三维凸多面体内/边界判定；`strict=true` 排除边界	$O(F)$
`volume6()`	有向三角面求和得到**六倍体积**；整数版返回 `optional<i128>`，浮点版为 `optional<ld>`	$O(F)$
`volume()`	三维凸多面体体积 `optional<ld>`	$O(F)$
`surface_area()`	三维实体**外表面**面积 `optional<ld>`	$O(F)$
`centroid()`	三维实体**均匀体积密度重心** `optional<array<ld,3>>`	$O(F)$
`centroid_surface_distance()`	三维体积重心到多面体表面的最短距离 `optional<ld>`	$O(F)$
`planar_area()`	共面凸多边形的**单侧面积** `optional<ld>`	$O(h)$
`planar_centroid()`	共面凸多边形的面积重心 `optional<array<ld,3>>`	$O(h)$

`F = f.size()` 是输出三角面数，`h = poly.size()` 是共面凸多边形顶点数；三维凸包中 `F = O(n)`。

关键数学约定

三维有向四面体六倍体积为

$$6V_{abcd} = \det(b-a, c-a, d-a) = (b-a)\cdot((c-a)\times(d-a)).$$

实际计算三维凸包时选择一个凸包顶点 o，对每个向外三角面 (a,b,c) 计算非负的有向六倍体积

$$W_{abc} = \det(a-o, b-o, c-o), \qquad V = \frac{1}{6}\sum_{(a,b,c)\in f} W_{abc}.$$

相同的四面体剖分用于体积重心，

$$G = \frac{\sum_{(a,b,c)\in f} W_{abc}(o+a+b+c)/4}{\sum_{(a,b,c)\in f} W_{abc}}.$$

`centroid()` 在实现中先对 o 做相对位移，避免大坐标参与反复加权时的部分消去误差。

注意事项

- `f` **的元素是三角形，不等于多面体的几何面数。** 例如立方体六个平面，每个平面可能被分为多个三角形；不能直接使用 `f.size()` 作为原始面数。不能保证共面三角划分唯一。
- **面片法向量朝外。** 对于凸包内点 `q`，每个面满足 `orient(a,b,c,q) <= 0`；若严格大于零则处于该面的外侧。
- **退化情形不假装有三维体积。** 当 `dim < 3`，`volume()`、`surface_area()`、`centroid()`、`contains()` 的含义与真三维实体不同：前三者返回 `nullopt`，`contains()` 返回 `false`。`dim == 2` 时使用 `planar_area()` / `planar_centroid()`，它们对应**单面面积**，不是实体"外表面积"。
- **浮点容差带长度量纲。** 通过三角面法向量长度归一化的点到平面有符号距离与 `EPS` 比较；因此同一组数据平移通常不改变尺度，但缩放坐标仍会改变判定结果。极近共面、大坐标小间距、极扁四面体可能触发数值不稳定，无法等同精确谓词。
- **整数运算要防溢出。** `orient()` 的每个中间乘积、三项累加及 `volume6()` 的所有面体积累加，均须处于 `i128` 范围。整数坐标很大时，不能仅凭输入能表示为 `i64` 就认定安全。
- **体积重心不是顶点坐标的简单平均。** 必须采用有向四面体的体积加权；面片数改变不会改变理论结果。
- **到表面距离只对体积重心定义。** 该重心严格位于非退化凸多面体内部，最近距离可由对各支持平面的垂直距离取最小得到；此函数不是"任意外部点到三角网格最短距离"。
- `contains(q, true)` 仅接受真三维内部点，处于任一面上的点都会被排除。浮点版边界由 `EPS` 决定。
- 若后续需要构建面邻接、法向量、棱边集合，直接从三角面 `f` 生成；对使用共面大面的题目，先合并同一支持平面上的三角面。

常见用法

构造三维凸包、求体积与表面积

```
vector<Point3> p = {
    {0, 0, 0}, {1, 0, 0}, {0, 1, 0}, {0, 0, 1}
};
Hull3<> h(p);
if (h.dim == 3) {
    ld v = *h.volume();              // 1/6
    ld s = *h.surface_area();        // (3 + sqrt(3))/2
    auto g = *h.centroid();          // (1/4, 1/4, 1/4)
    ld d = *h.centroid_surface_distance();
}
```

点在凸多面体内及严格内部

```
if (h.dim == 3) {
    bool inside = h.contains(q);        // 边界也算在内
    bool strictly_inside = h.contains(q, true);
}
```

遍历朝外三角面

```
for (auto [a, b, c] : h.f) {
    Point3 A = h.p[a], B = h.p[b], C = h.p[c];
    // 三角面 A → B → C 的叉积法向量朝向外部
}
```

全点共面的多边形

```
Hull3<> h(p);
if (h.dim == 2) {
    ld s = *h.planar_area();
    auto c = *h.planar_centroid();
    for (int id : h.poly) {
        Point3 vertex = h.p[id]; // 按多边形循环顺序
    }
}
```

精确六倍体积（整数版）

```
Hull3<> h(p);                    // 3.1 使用整数 Point3 时
if (auto v6 = h.volume6()) {
    i128 ans6 = *v6;              // 必要时用自定义 i128 输出函数
}
```

复杂度

操作	时间复杂度	额外空间
`build()`	预期 $O(n^2)$，包括初始排序 $O(n \log n)$	$O(n)$
三维 `contains()`	$O(F)$	$O(1)$
体积、表面积、重心、重心到表面距离	$O(F)$	$O(1)$
`planar_area()` / `planar_centroid()`	$O(h)$	$O(1)$

增量算法每次扫描全部当前三角面，并通过哈希容器取消可见面内部的反向边以提取轮廓边。三维凸包始终只有 $O(n)$ 个三角面，故**哈希操作摊还期望**总时间为 $O(n^2)$；对抗性哈希行为下无严格的 $O(n^2)$ 最坏时间保证

16.5 球与最小覆盖球

依赖： 复用 **3.1 浮点版**的 `Point3`、`ld`、`EPS`、`PI`、`sgn(ld)`、`dot()`、`cross()`、`triple()`、`norm2()`、`length()`、`distance()`。球与直线相交额外依赖 **3.2 浮点版** `Line3`，球与平面相交额外依赖 **3.2 浮点版** `Plane3`（一点 `p` + 法向量 `n`）。**不依赖** 3.3、3.4；`min_ball()` 也不依赖 3.2。整数坐标可先转为浮点 `Point3`，不能将整数版/浮点版的 `Point3` 混用
本节所有球均使用**球心 + 半径**表示。球面是边界，实心球包含内部；球面交点/交圆和实心球交体积是不同概念。合法半径为 `r ≥ 0`（`r == 0` 视为单点球），`r < 0` 表示无效球。

基础结构与点的包含判定

```
// ===== 球与圆：依赖 3.1 浮点 Point3 / ld / EPS / dot / cross / norm2 / length / distance =====
// 球面允许 r == 0，此时退化为一个点；r < 0 表示无效球。
struct Sphere3 {
    Point3 c{};
    ld r{ -1 };
    Sphere3() = default;
    Sphere3(Point3 c_, ld r_) : c(c_), r(r_) {}
    bool valid() const { return r ≥ 0; }
    ld area() const { return 4.L * PI * r * r; }
    ld volume() const { return 4.L * PI * r * r * r / 3.L; }
};
```

```
struct Circle3 {
    Point3 c, n; // 圆心、单位法向量
    ld r;        // 圆半径；切点视为 r == 0 的圆
};
// ===== 球面与实体球的点判定 =====
// -1 球内，0 球面上，1 球外；无效球不应调用
int sphere_side(Point3 p, Sphere3 s) {
    return sgn(distance(p, s.c) - s.r);
}
bool in_ball(Point3 p, Sphere3 s) { return sphere_side(p, s) <= 0; }
bool on_sphere(Point3 p, Sphere3 s) { return sphere_side(p, s) == 0; }
```

直线与球面交点

```
// ===== 直线与球面：依赖 3.2 浮点 Line3 =====
enum class SphereLineRel {
    INVALID,  // 无效：球 r < 0 或直线零方向
    DISJOINT, // 相离：没有交点
    TANGENT,  // 相切：唯一公共点
    SECANT    // 相交：两个不同公共点
};
struct SphereLineHit {
    SphereLineRel rel;
    vector<Point3> p; // 0、1 或 2 个球面交点
};
SphereLineHit sphere_line_int(Sphere3 s, Line3 l) {
    if (!s.valid() || norm2(l.dir) == 0) return {SphereLineRel::INVALID, {}};
    ld t = dot(s.c - l.p, l.dir) / norm2(l.dir);
    Point3 foot = l.p + l.dir * t;
    ld d = distance(s.c, foot);
    int rel = sgn(d - s.r);
    if (rel > 0) return {SphereLineRel::DISJOINT, {}};
    if (rel == 0) return {SphereLineRel::TANGENT, {foot}};
    ld h = sqrtl(max(0.L, (s.r - d) * (s.r + d)));
    Point3 v = l.dir * (h / length(l.dir));
    return {SphereLineRel::SECANT, {foot - v, foot + v}};
}
```

平面与球面交圆

```
// ===== 平面与球面：依赖 3.2 浮点 Plane3(p,n)；可整体删除 =====
enum class SpherePlaneRel {
    INVALID,  // 无效：球 r < 0 或平面零法向量
    DISJOINT, // 相离：无公共点
    TANGENT,  // 相切：唯一公共点
    INTERSECT // 相交：公共部分是一个圆
};
struct SpherePlaneHit {
    SpherePlaneRel rel;
    optional<Circle3> circle; // 相切时为半径 0 的退化圆
};
SpherePlaneHit sphere_plane_int(Sphere3 s, Plane3 pl) {
    ld len = length(pl.n);
    if (!s.valid() || len == 0) return {SpherePlaneRel::INVALID, nullopt};
    Point3 n = pl.n / len;
    ld h = dot(s.c - pl.p, n);
    int rel = sgn(fabsl(h) - s.r);
    if (rel > 0) return {SpherePlaneRel::DISJOINT, nullopt};
    Point3 foot = s.c - n * h;
    if (rel == 0) return {SpherePlaneRel::TANGENT, Circle3{foot, n, 0}};
    ld rr = sqrtl(max(0.L, (s.r - fabsl(h)) * (s.r + fabsl(h))));
    return {SpherePlaneRel::INTERSECT, Circle3{foot, n, rr}};
}
```

两个球面的交圆与位置关系

```cpp
// ===== 两球面交集：独立扩展，可整体删除 =====
enum class SphereSphereRel {
    INVALID,          // 无效：存在负半径
    DISJOINT,         // 外离：球面不相交，两个实心球也不相交
    EXTERNAL_TANGENT, // 外切：球面交于一点
    INTERSECT,        // 相交：球面交于一个圆
    INTERNAL_TANGENT, // 内切：球面交于一点
    CONTAINED,        // 内含：较小球严格处于另一实心球内，球面不接触
    COINCIDENT        // 重合：球心、半径均相同
};
struct SphereSphereHit {
    SphereSphereRel rel;
    optional<Circle3> circle; // 相交圆；相切时半径 0；重合时不存在唯一交圆
};
SphereSphereHit sphere_sphere_int(Sphere3 a, Sphere3 b) {
    if (!a.valid() || !b.valid()) return {SphereSphereRel::INVALID, nullopt};
    Point3 v = b.c - a.c;
    ld d = length(v);
    if (sgn(d) == 0) {
        if (sgn(a.r - b.r) == 0) return {SphereSphereRel::COINCIDENT, nullopt};
        return {SphereSphereRel::CONTAINED, nullopt};
    }
    int outer = sgn(d - (a.r + b.r));
    if (outer > 0) return {SphereSphereRel::DISJOINT, nullopt};
    int inner = sgn(d - fabsl(a.r - b.r));
    if (inner < 0) return {SphereSphereRel::CONTAINED, nullopt};
    Point3 n = v / d;
    if (outer == 0) {
        return {SphereSphereRel::EXTERNAL_TANGENT, Circle3{a.c + n * a.r, n, 0}};
    }
    if (inner == 0) {
        // 内切时，切点位于大球朝向小球的一侧
        Point3 p = a.r >= b.r ? a.c + n * a.r : a.c - n * a.r;
        return {SphereSphereRel::INTERNAL_TANGENT, Circle3{p, n, 0}};
    }
    ld x = (d + (a.r - b.r) * ((a.r + b.r) / d)) / 2.L;
    ld rr = sqrtl(max(0.L, (a.r - x) * (a.r + x)));
    return {SphereSphereRel::INTERSECT, Circle3{a.c + n * x, n, rr}};
}
```

两个实心球的交体积

不依赖上述相交关系接口，可以独立保留 `Sphere3` 与本代码块。

```cpp
// ===== 两实心球交体积：只需 Sphere3 和距离 =====
ld sphere_cap_volume(ld r, ld h) {
    h = clamp(h, 0.L, 2.L * r);
    return PI * h * h * (r - h / 3.L);
}
ld ball_overlap_volume(Sphere3 a, Sphere3 b) {
    // 前提：球有效
    ld d = distance(a.c, b.c);
    if (d >= a.r + b.r) return 0;
    if (d <= fabsl(a.r - b.r)) return min(a.volume(), b.volume());
    ld x = (d + (a.r - b.r) * ((a.r + b.r) / d)) / 2.L;
    return sphere_cap_volume(a.r, a.r - x) + sphere_cap_volume(b.r, b.r - (d - x));
}
```

最小覆盖球

只依赖本节 `Sphere3` 和 3.1 浮点 `Point3` 基础函数
实现为随机打乱后的四层增量维护；支持空集、单点、重复点、共线、共面点集。求三点外接圆或四点外接球时采用局部缩放以减轻量级影响，退化时枚举至多四点的低维支撑球。 `min_ball()` 内部先把点集平移到局部原点，并在返回前修正半径以覆盖全部输入点的计算距离。

```cpp
// ===== 最小覆盖球：只依赖 3.1 浮点 Point3、基础函数；可独立复制 =====
// 当 3 或 4 个边界点退化时，从低维候选球中寻找覆盖它们的最小球。
Sphere3 ball2(Point3 a, Point3 b) {
    Point3 c = a + (b - a) / 2.L;
    return {c, distance(c, a)};
}
optional<Sphere3> ball3(Point3 a, Point3 b, Point3 c) {
    Point3 u = b - a, v = c - a;
    ld scale = max(length(u), length(v));
    if (scale == 0) return nullopt;
    u /= scale; v /= scale;
    Point3 n = cross(u, v);
    ld den = 2.L * norm2(n);
    if (den <= 2.L * EPS * EPS * norm2(u) * norm2(v)) return nullopt;
    Point3 o = a + (cross(v, n) * norm2(u) + cross(n, u) * norm2(v)) * (scale / den);
    return Sphere3{o, distance(o, a)};
}
optional<Sphere3> ball4(Point3 a, Point3 b, Point3 c, Point3 d) {
    Point3 u = b - a, v = c - a, w = d - a;
    ld scale = max({length(u), length(v), length(w)});
    if (scale == 0) return nullopt;
    u /= scale; v /= scale; w /= scale;
    ld den = 2.L * triple(u, v, w);
    if (fabsl(den) <= 2.L * EPS * length(u) * length(v) * length(w)) return nullopt;
    Point3 o = a + (cross(v, w) * norm2(u) + cross(w, u) * norm2(v)
                    + cross(u, v) * norm2(w)) * (scale / den);
    return Sphere3{o, distance(o, a)};
}
// 处理边界点共线、共面，亦可作为 <= 4 点的暴力正确性参考
Sphere3 min_ball_small(const vector<Point3> &p) {
    Sphere3 ans;
    auto check = [&](Sphere3 s) {
        // 这里只做极小相对浮点容差，避免明显错误的候选球被接受
        for (auto q : p) {
            ld d = distance(q, s.c);
            if (d > s.r + EPS * max(s.r, d)) return;
        }
        if (!ans.valid() || s.r < ans.r) ans = s;
    };
    int n = p.size();
    for (int i = 0; i < n; ++i) check({p[i], 0});
    for (int i = 0; i < n; ++i) for (int j = 0; j < i; ++j)
        check(ball2(p[i], p[j]));
    for (int i = 0; i < n; ++i) for (int j = 0; j < i; ++j)
        for (int k = 0; k < j; ++k)
            if (auto s = ball3(p[i], p[j], p[k])) check(*s);
    if (n == 4) if (auto s = ball4(p[0], p[1], p[2], p[3])) check(*s);
    return ans;
}
// 随机增量，固定维数期望 O(n)；强退化/病态数据可能影响数值稳定性
optional<Sphere3> min_ball(vector<Point3> p,
                           uint64_t seed = chrono::steady_clock::now().time_since_epoch().count()) {
    if (p.empty()) return nullopt;
    // 平移到局部坐标系，减少大坐标导致的球心精度损失
    Point3 origin = p[0];
    for (auto &q : p) q -= origin;
    mt19937_64 rng(seed);
    shuffle(p.begin(), p.end(), rng);
    auto inside = [](Point3 q, Sphere3 s) {
        ld d = distance(q, s.c);
        return d <= s.r + EPS * max(s.r, d);
    };
    Sphere3 s{p[0], 0};
    for (int i = 1; i < int(p.size()); ++i) {
        if (inside(p[i], s)) continue;
        s = {p[i], 0};
        for (int j = 0; j < i; ++j) {
            if (inside(p[j], s)) continue;
            s = ball2(p[i], p[j]);
            for (int k = 0; k < j; ++k) {
```

```
                if (inside(p[k], s)) continue;
                if (auto t = ball3(p[i], p[j], p[k])) s = *t;
                else s = min_ball_small({p[i], p[j], p[k]});
                for (int l = 0; l < k; ++l) {
                    if (inside(p[l], s)) continue;
                    if (auto t = ball4(p[i], p[j], p[k], p[l])) s = *t;
                    else s = min_ball_small({p[i], p[j], p[k], p[l]});
                }
            }
        }
    }
    // 修补最后的微小舍入：返回球保证覆盖所有输入点的计算距离
    for (auto q : p) s.r = max(s.r, distance(q, s.c));
    s.c += origin;
    return s;
}
```

接口说明

基础接口

接口	含义	复杂度
`Sphere3(c,r)`	球心 `c`、半径 `r`，默认构造是无效球	$O(1)$
`s.valid()`	`r ≥ 0`	$O(1)$
`s.area()`	球面面积 $4\pi r^2$，要求有效球	$O(1)$
`s.volume()`	实心球体积 $\frac{4}{3}\pi r^3$，要求有效球	$O(1)$
`sphere_side(p,s)`	`-1/0/1`：球内 / 球面上 / 球外	$O(1)$
`in_ball(p,s)`	点在闭实心球内（包括球面）	$O(1)$
`on_sphere(p,s)`	点在球面上	$O(1)$
`Circle3{c,n,r}`	空间圆：圆心、单位法向量、圆半径	$O(1)$

相交接口

接口	返回类型	含义	复杂度
`sphere_line_int(s,l)`	`SphereLineHit`	无限直线与球面交点，返回 `0～2` 个点	$O(1)$
`sphere_plane_int(s,pl)`	`SpherePlaneHit`	平面与球面交圆，切点用半径 `0` 的圆表示	$O(1)$
`sphere_sphere_int(a,b)`	`SphereSphereHit`	两球面位置关系及唯一交圆；重合时不返回唯一交圆	$O(1)$
`sphere_cap_volume(r,h)`	`ld`	球冠体积，`h` 限制在 $[0,2r]$	$O(1)$
`ball_overlap_volume(a,b)`	`ld`	两实心球的公共部分体积	$O(1)$

枚举值与中文含义

枚举	值	中文含义	交集
`SphereLineRel`	`INVALID`	无效：球半径为负或直线退化	无法分类
	`DISJOINT`	相离	空集
	`TANGENT`	相切	一个点
	`SECANT`	割线相交	两个不同的点
`SpherePlaneRel`	`INVALID`	无效：球半径为负或平面法向量为零	无法分类
	`DISJOINT`	相离	空集
	`TANGENT`	相切	一个点
	`INTERSECT`	相交	一个非退化空间圆
`SphereSphereRel`	`INVALID`	无效：至少一个球半径为负	无法分类
	`DISJOINT`	外离	球面与实体球均无交集
	`EXTERNAL_TANGENT`	外切	球面交于一个点
	`INTERSECT`	相交	球面交于一个空间圆
	`INTERNAL_TANGENT`	内切	球面交于一个点
	`CONTAINED`	内含、不相切	两球面无交集，小实心球包含于大实心球
	`COINCIDENT`	两球面重合	球面完全相同，没有唯一的交圆

`SphereLineHit::p` 保存交点，空集/切线/割线分别有 `0/1/2` 个元素；`SpherePlaneHit::circle`、`SphereSphereHit::circle` 为 `optional<Circle3>`，只有存在**唯一交圆（含退化切点）**时才有值。**

最小覆盖球接口

接口	含义	复杂度
`ball2(a,b)`	两点作为边界得到的直径球	$O(1)$
`ball3(a,b,c)`	非共线三点的外接圆所在球（球心处于三点平面），近共线返回 `nullopt`	$O(1)$
`ball4(a,b,c,d)`	非共面四点的外接球，近共面返回 `nullopt`	$O(1)$
`min_ball_small(p)`	`p.size()` ≤ 4 时枚举低维候选球，选择可包含所有点的最小者	$O(1)$
`min_ball(p,seed)`	求点集最小覆盖球；空集返回 `nullopt`，`seed` 控制随机打乱	**期望** $O(n)$，最坏 $O(n^4)$

最小覆盖球的 `ball3` / `ball4` 是**外接球构造**，不是任意三点/四点的最小覆盖球。三点形成钝角三角形时，其最小覆盖球可能由最长边的两端决定；单独求 ≤ 4 点最小覆盖球请调用 `min_ball_small()` 或 `min_ball()`。

注意事项

- **数值版本**：本节统一使用浮点 `Point3`；`Line3`、`Plane3` 也必须是 3.2 的浮点版。整数版三维几何中的 `Plane3(a,b,c)` 与这里的 `Plane3(p,n)` 构造约定不同，不能混用。
- **球面 vs 实心球**：`sphere_sphere_int()` 判断的是**两个球面的交集**。`CONTAINED` 时两球面无交集，但实心球交集体积通常是较小球的体积。
- **退化**：半径 `0` 的球视为单点。此时外切与内切在几何语义上可能不唯一，代码优先按外切分支分类；两点球同心同半径视为 `COINCIDENT`，`circle == nullopt`。
- **球面相切**：浮点 `EPS` 分支可把非常接近相切的状态按相切处理；返回的切点只在相应容差内满足球面方程。
- **零方向**：零方向直线或零法向量平面返回 `INVALID`；不尝试做点代替直线或任意平面代替非法平面。
- **交圆法向量**：返回的 `Circle3::n` 已单位化；两球交圆的 `n` 朝向第一个球心到第二个球心，改变两球顺序会反向。
- **最小覆盖球精度**：增量判定使用相对球尺度的容差，不直接使用 `sgn(distance-r)` 的固定绝对阈值，以免微小尺度数据被误判。近共线/近共面时的分支会引入近似，极端病态数据并无严格鲁棒性保证。
- **坐标范围**：`min_ball()` 内部已做平移，但返回绝对球心时仍受 `long double` 表示精度限制；特别大的坐标平移可能损失末位。建议预先按题目容差合理缩放输入。
- **面积与体积**：球冠体积公式适用于球半径 `r` $\geqslant$ `0`；`ball_overlap_volume()` 要求两球有效。输出精度由题目要求决定，通常使用 `fixed << setprecision(12)`。
- **最坏复杂度**：随机增量法的期望线性复杂度建立在随机打乱的前提下，固定极端顺序或浮点近退化可能影响运行时间。

常见用法

直线与球面、平面与球面相交

```
Sphere3 s{{0, 0, 0}, 2};
Line3 l = Line3::through({-3, 0, 0}, {3, 0, 0});
auto a = sphere_line_int(s, l);
if (a.rel == SphereLineRel::SECANT) {
    // a.p[0]、a.p[1] 分别为两个球面交点
}
Plane3 pl{{0, 0, 1}, {0, 0, 1}}; // z = 1
if (auto b = sphere_plane_int(s, pl); b.circle) {
    // b.circle->c == (0,0,1), b.circle->r == sqrt(3)
}
```

两球相交与体积

```
Sphere3 a{{0, 0, 0}, 2}, b{{2, 0, 0}, 2};
auto hit = sphere_sphere_int(a, b);
if (hit.rel == SphereSphereRel::INTERSECT) {
    Point3 c = hit.circle->c; // (1,0,0)
    ld r = hit.circle->r;     // sqrt(3)
}
ld vol = ball_overlap_volume(a, b);
```

最小覆盖球

```
vector<Point3> p{{0,0,0}, {2,0,0}, {0,2,0}, {0,0,2}};
auto ans = min_ball(p); // 可传第二参数 seed 复现实验
if (ans) {
    cout << fixed << setprecision(12)
         << ans->c.x << ' ' << ans->c.y << ' ' << ans->c.z << ' '
         << ans->r << '\n';
}
```

复杂度

除 `min_ball()` 外所有操作均为 $O(1)$ 时间、$O(1)$ 额外空间（相交点数最多 `2`）；`min_ball()` 因复制、随机打乱输入而使用 $O(n)$ 额外空间，**期望** $O(n)$ 时间，最坏 $O(n^4)$

16.6 球面与地理计算

依赖： 复用 **3.1 浮点版**的 `Point3`、`ld`、`PI`、`dot()`、`cross()`、`triple()`、`length()`、`unit()` 和向量四则运算；不依赖 3.2～3.5。`ld` 等类型别名由全局板子提供，**这里不再重复** `using`。整数经纬度输入可隐式转为 `ld`，但本节的空间向量必须使用浮点版 `Point3`。
统一约定： `GeoPoint{lon, lat}` 按 **经度、纬度** 顺序，单位为**角度 (°)**；经度向东为正，纬度向北为正，纬度范围 `[-90°,90°]`。球心在坐标原点，`z` 轴指向北极，`x` 轴指向 `(0°,0°)`，`y` 轴指向 `(90°E,0°)`。输入半径 `r` 应为非负，距离计算要求 `r > 0`；同一次计算必须统一长度单位。

经纬度与三维坐标

```
struct GeoPoint {
    ld lon{ 0 }, lat{ 0 }; // 经度、纬度，单位 °
};
ld geo_rad(ld deg) { return deg * PI / 180.L; }
ld geo_deg(ld rad) { return rad * 180.L / PI; }
ld geo_norm_lon(ld lon) { // [-180, 180)
    lon = remainderl(lon, 360.L);
    if (lon >= 180.L) lon -= 360.L;
    return lon;
}
// ===== 经纬度 → 单位球面坐标：核心基础 =====
Point3 geo_to_unit(GeoPoint g) {
    if (g.lat == 90.L) return {0, 0, 1};
    if (g.lat == -90.L) return {0, 0, -1};
    ld lon = geo_rad(geo_norm_lon(g.lon)), lat = geo_rad(g.lat);
    ld c = cosl(lat);
    return {c * cosl(lon), c * sinl(lon), sinl(lat)};
}
Point3 geo_to_xyz(GeoPoint g, ld r) { return geo_to_unit(g) * r; }
// ===== 空间坐标 → 经纬度 =====
// 任意非零向量可还原方向；零向量返回 nullopt
optional<GeoPoint> geo_from_xyz(Point3 p) {
    if (p == Point3{}) return nullopt;
    ld lon = geo_deg(atan2l(p.y, p.x));
    ld lat = geo_deg(atan2l(p.z, hypotl(p.x, p.y)));
    return GeoPoint{geo_norm_lon(lon), lat};
}
```

圆心角、弦长与球面最短距离

```
// ===== 球面距离：只依赖 3.1 与 geo_to_unit；可独立保留 =====
// 返回 [0, PI]；内部用 atan2，避免 acos 在近重合点处损失精度
ld geo_angle(GeoPoint a, GeoPoint b) {
    Point3 u = geo_to_unit(a), v = geo_to_unit(b);
    return atan2l(length(cross(u, v)), dot(u, v));
}
// 沿大圆较短弧的球面距离；对跖点有无数条等长最短路径
ld geo_arc_dist(GeoPoint a, GeoPoint b, ld r) {
    return r * geo_angle(a, b);
}
// 球内两点的空间直线距离，即弦长
ld geo_chord_dist(GeoPoint a, GeoPoint b, ld r) {
    return 2.L * r * sinl(geo_angle(a, b) / 2.L);
}
```

航向角、沿大圆前进与路径插值

```
// ===== 航向角与目的地：可整体删除 =====
// 初始航向角：从正北顺时针量起，[0, 360) 度
// 两点重合或对跖时无唯一航向，返回 nullopt
optional<ld> geo_bearing(GeoPoint a, GeoPoint b) {
    ld ang = geo_angle(a, b);
    if (ang <= 1E-14L || PI - ang <= 1E-14L) return nullopt;
    ld p = geo_rad(a.lat), q = geo_rad(b.lat);
    ld d = geo_rad(geo_norm_lon(b.lon - a.lon));
    ld x = sinl(d) * cosl(q);
    ld y = cosl(p) * sinl(q) - sinl(p) * cosl(q) * cosl(d);
    ld ans = geo_deg(atan2l(x, y));
    if (ans < 0) ans += 360.L;
    if (ans >= 360.L) ans -= 360.L;
    return ans;
}
// 从 a 沿初始航向 bearing（角度）行进 dist；r > 0
// 可令 dist > PI*r（对应继续沿原大圆行进）
GeoPoint geo_destination(GeoPoint a, ld bearing, ld dist, ld r) {
    ld lon = geo_rad(geo_norm_lon(a.lon));
    ld lat = geo_rad(a.lat), az = geo_rad(bearing);
    Point3 u = geo_to_unit(a);
    Point3 east{-sinl(lon), cosl(lon), 0};
    Point3 north{-sinl(lat)*cosl(lon), -sinl(lat)*sinl(lon), cosl(lat)};
    Point3 dir = north * cosl(az) + east * sinl(az);
    ld t = remainderl(dist / r, 2.L * PI);
    Point3 p = u * cosl(t) + dir * sinl(t);
    return *geo_from_xyz(p);
}
// ===== 大圆短弧插值：只依赖 3.1，可整体删除 =====
// 输入非零向量，0 <= t <= 1；输出单位球面向量
// 对跖点没有唯一短弧，返回 nullopt
optional<Point3> geo_slerp(Point3 a, Point3 b, ld t) {
    auto x = unit(a), y = unit(b);
    if (!x || !y) return nullopt;
    ld ang = atan2l(length(cross(*x, *y)), dot(*x, *y));
    if (PI - ang <= 1E-14L) return nullopt;
    if (ang <= 1E-8L) {
        Point3 p = *x * (1.L - t) + *y * t;
        return p / length(p);
    }
    ld s = sinl(ang);
    Point3 p = *x * (sinl((1.L - t) * ang) / s)
             + *y * (sinl(t * ang) / s);
    return p / length(p);
}
optional<GeoPoint> geo_interpolate(GeoPoint a, GeoPoint b, ld t) {
    auto p = geo_slerp(geo_to_unit(a), geo_to_unit(b), t);
    if (!p) return nullopt;
    return geo_from_xyz(*p);
}
```

球面三角形与球面多边形面积

球面三角形的边均采用较短大圆弧。下面使用单位球向量计算有向球面角盈（立体角）：

$$\Omega(a,b,c) = 2\,\mathrm{atan2}(\det(a,b,c),\, 1 + a\cdot b + b\cdot c + c\cdot a), \quad S = R^2\Omega.$$

Ω 的符号随顶点顺序反转。多边形采用以首点为顶点的有向球面三角形扇形求和，适用于由短大圆弧连接、无自交且所有顶点位于同一个开半球内的简单球面多边形（可以非凸）。多边形顶点按边界依次给出，不重复首点。

```
// ===== 球面三角形：依赖 3.1 浮点 dot/cross/triple/length/unit =====
// 返回有向角盈（球面单位球上的有向面积），范围约为 (-2PI, 2PI)
// 零向量、对跖边或无法唯一确定的退化边界返回 nullopt
optional<ld> sphere_tri_excess(Point3 a, Point3 b, Point3 c) {
    auto pa = unit(a), pb = unit(b), pc = unit(c);
    if (!pa || !pb || !pc) return nullopt;
    a = *pa; b = *pb; c = *pc;
```

```
    auto bad = [&](Point3 u, Point3 v) {
        return dot(u, v) < 0 && length(cross(u, v)) <= 1E-14L;
    };
    if (bad(a, b) || bad(b, c) || bad(c, a)) return nullopt;
    ld y = triple(a, b, c);
    ld x = 1.L + dot(a, b) + dot(b, c) + dot(c, a);
    if (y == 0 && x <= 0) return nullopt;
    return 2.L * atan2l(y, x);
}
// 有向球面三角形面积，r > 0；通常取绝对值获得普通面积
optional<ld> sphere_tri_area(Point3 a, Point3 b, Point3 c, ld r) {
    auto e = sphere_tri_excess(a, b, c);
    if (!e) return nullopt;
    return *e * r * r;
}
// ===== 球面顶角：只依赖 3.1，可独立删除 =====
// 球面三角形 abc 在 b 点的内角（0 到 PI），单位为弧度
// 要求 ba、bc 均确定唯一的短大圆弧
optional<ld> sphere_vertex_angle(Point3 a, Point3 b, Point3 c) {
    auto pa = unit(a), pb = unit(b), pc = unit(c);
    if (!pa || !pb || !pc) return nullopt;
    a = *pa; b = *pb; c = *pc;
    Point3 u = cross(b, cross(a, b));
    Point3 v = cross(b, cross(c, b));
    ld lu = length(u), lv = length(v);
    if (lu <= 1E-14L || lv <= 1E-14L) return nullopt;
    return atan2l(length(cross(u, v)), dot(u, v));
}
// ===== 球面多边形面积：依赖 sphere_tri_excess，可独立删除 =====
// 简单、短大圆弧边界、位于某个开半球内；允许凹多边形
// 返回有向面积：反转顶点顺序会反号；无效输入返回 nullopt
optional<ld> sphere_poly_signed_area(const vector<Point3> &p, ld r) {
    int n = p.size();
    if (n < 3 || r <= 0) return nullopt;
    vector<Point3> q(n);
    for (int i = 0; i < n; ++ i) {
        auto t = unit(p[i]);
        if (!t) return nullopt;
        q[i] = *t;
    }
    for (int i = 0; i < n; ++ i) {
        Point3 a = q[i], b = q[(i + 1) % n];
        ld ang = atan2l(length(cross(a, b)), dot(a, b));
        if (ang <= 1E-14L || PI - ang <= 1E-14L) return nullopt;
    }
    ld sum = 0;
    for (int i = 1; i + 1 < n; ++ i) {
        auto t = sphere_tri_excess(q[0], q[i], q[i + 1]);
        if (!t) return nullopt;
        sum += *t;
    }
    return sum * r * r;
}
optional<ld> sphere_poly_area(const vector<Point3> &p, ld r) {
    auto s = sphere_poly_signed_area(p, r);
    if (!s) return nullopt;
    return fabsl(*s);
}
```

两大圆的交点

```
// ===== 两条大圆的交点：可整体删除 =====
// (a,b)、(c,d) 各确定一条大圆；非重合时返回一对对跖交点
// 输入为球心在原点的非零位置向量；不判断交点是否落在给定短弧内
optional<array<Point3, 2>> geo_great_circle_int(Point3 a, Point3 b,
                                                Point3 c, Point3 d) {
    auto ua = unit(a), ub = unit(b), uc = unit(c), ud = unit(d);
    if (!ua || !ub || !uc || !ud) return nullopt;
```

```
    Point3 n = cross(*ua, *ub), m = cross(*uc, *ud);
    ld ln = length(n), lm = length(m);
    if (ln ⩽ 1E-14L || lm ⩽ 1E-14L) return nullopt;
    n /= ln; m /= lm;
    Point3 v = cross(n, m);
    ld len = length(v);
    if (len ⩽ 1E-14L) return nullopt; // 相同的大圆，无唯一交点对
    v /= len;
    return array<Point3, 2>{v, -v};
}
```

球冠、球带面积

```
// ===== 球冠 / 球带的曲面面积：可整体删除 =====
// h 为沿球轴方向的高度，要求 r ⩾ 0、0 ⩽ h ⩽ 2r
// 球冠面积不包含切割产生的平面底面
ld sphere_cap_area(ld r, ld h) { return 2.L * PI * r * h; }
// 球带：上下两个平行截面之间球面的面积，h 为两平面间的轴向高度
ld sphere_zone_area(ld r, ld h) { return 2.L * PI * r * h; }
```

接口说明

接口	返回 / 几何意义	复杂度
`GeoPoint{lon,lat}`	经度、纬度（角度，经度在前）	—
`geo_rad(deg)` / `geo_deg(rad)`	角度与弧度互换	$O(1)$
`geo_norm_lon(lon)`	经度归一化至 `[-180,180)`	$O(1)$
`geo_to_unit(g)`	经纬度转单位球上的 `Point3`	$O(1)$
`geo_to_xyz(g,r)`	经纬度转半径 `r` 球面的空间坐标	$O(1)$
`geo_from_xyz(p)`	非零空间向量转经纬度；零向量返回 `nullopt`	$O(1)$
`geo_angle(a,b)`	圆心角，弧度，范围 `[0,PI]`	$O(1)$
`geo_arc_dist(a,b,r)`	球面最短距离 `r*圆心角`	$O(1)$
`geo_chord_dist(a,b,r)`	球内直线弦长	$O(1)$
`geo_bearing(a,b)`	从正北顺时针的初始航向角，单位 °；非唯一返回 `nullopt`	$O(1)$
`geo_destination(a,bearing,dist,r)`	从位置出发沿大圆走 `dist` 后的经纬度	$O(1)$
`geo_slerp(a,b,t)` / `geo_interpolate(a,b,t)`	两球面点之间短大圆弧的球面线性插值	$O(1)$
`sphere_tri_excess(a,b,c)`	三点构成的有向球面角盈，单位 `sr`（数值等于单位球面积）	$O(1)$
`sphere_tri_area(a,b,c,r)`	有向球面三角形面积	$O(1)$
`sphere_vertex_angle(a,b,c)`	球面三角形在 `b` 处的内角，单位弧度	$O(1)$
`sphere_poly_signed_area(p,r)`	按顶点顺序计算有向球面多边形面积	$O(n)$
`sphere_poly_area(p,r)`	球面多边形面积非负值	$O(n)$
`geo_great_circle_int(a,b,c,d)`	两不同大圆的一对对跖交点；无唯一交点对返回 `nullopt`	$O(1)$
`sphere_cap_area(r,h)` / `sphere_zone_area(r,h)`	球冠曲面面积 / 球带曲面面积	$O(1)$

注意事项

- **球面模型**：全部公式基于完美球体，而不是 WGS84 等椭球地球模型。纬度按地心球面纬度理解，不能直接保证真实地理测量精度。实际地球半径须按题目给定（如公里与米不可混用）。
- **角度单位**：`GeoPoint`、`geo_bearing`、`geo_destination` 的输入/输出地理角采用 **度**；`geo_angle`、`sphere_vertex_angle`、`sphere_tri_excess` 采用 **弧度 / 球面角盈**，勿混用。
- **经度跨界**：`179°E` 与 `179°W` 实际仅隔经差 `2°`，不要直接使用经度数值差计算路程。经度 `180°` 的返回规范化为 `-180°`。
- **极点**：在北极、南极，经度不是唯一的。`geo_from_xyz` 在恰好 `x=y=0` 时约定经度为 `0°`；`geo_destination` 从极点出发时的航向依据输入经度确定的局部方位系。
- **重合与对跖**：大圆最短距离对所有两点都存在；但重合点没有唯一初始航向，对跖点有无穷条最短大圆弧，`geo_bearing()`、`geo_interpolate()` 遇到这些情形返回 `nullopt`。`geo_destination()` 不需要终点，因此可以任意指定初始航向。
- **三角形与多边形**：采用短大圆弧作为边；对跖边、零向量无效。球面多边形实现只保证**简单、顶点都落在某个开半球、各边为短大圆弧**的情况，允许凹多边形；不能直接推广到绕球超过半球、边界自交、有洞、跨越对跖点、含不唯一大圆边的多边形。代码不额外检查“简单且同半球”这一全局前置条件。
- **有向面积**：三角形和多边形反转顶点顺序应反号，`sphere_poly_area()` 取绝对值；不按“经度增大就一定为正”来判断朝向。面积单位是 `r` 的长度单位的平方。
- **球面角盈**：凸球面多边形的面积满足 $S=R^2(\Sigma\text{内角}-(n-2)\pi)$；代码采用有向三角形 `atan2` 求和，避免显式相减造成小面积时的消去误差。不要用普通平面鞋带公式计算球面面积。

- **浮点尺度**：`atan2(cross,dot)` 通常比直接 `acos(dot)` 更稳定，但浮点精度并非无限；近重合、近对跖以及接近退化三角形时仍可能丢失相对精度。本节 `1E-14L` 用于**角度退化**，与 3.1 的坐标差 `EPS` 判等概念不同。
- **球冠高度**：面积公式要求 $0 \leqslant h \leqslant 2r$；其中 `h=2r` 表示整个球面。

常见用法

经纬度距离：北京到另一点（球体近似）

```
GeoPoint a{116.4L, 39.9L};
GeoPoint b{121.5L, 31.2L};
ld r = 6371.L;                      // 单位：km，示例球半径
ld ang = geo_angle(a, b);           // 弧度
ld arc = geo_arc_dist(a, b, r);     // 大圆最短弧长，km
ld chord = geo_chord_dist(a, b, r);// 穿过球内部的直线距离，km
```

坐标转换与航向

```
Point3 p = geo_to_unit(GeoPoint{90, 0}); // 约等于 (0,1,0)
auto g = geo_from_xyz(p);                // (90°,0°)
auto bearing = geo_bearing({0,0}, {90,0}); // 约为 90°（正东）
GeoPoint q = geo_destination({0,0}, 90, 1000, 6371); // 从赤道出发向东 1000 km
```

八分之一球面（三条赤道 / 经线大圆弧）

```
vector<Point3> tri{{1,0,0}, {0,1,0}, {0,0,1}};
auto e = sphere_tri_excess(tri[0], tri[1], tri[2]); // PI / 2
// 球面三角形面积 = PI * r * r / 2
// 反转三角形顺序，e 的符号反转
```

求球面多边形面积

```
vector<GeoPoint> g{{0,45}, {90,45}, {180,45}, {270,45}};
vector<Point3> p;
for (auto x : g) p.push_back(geo_to_unit(x));
// 这些点位于北半球；顶点间按短大圆弧依次连接
ld r = 1;
auto area = sphere_poly_area(p, r);
if (area) cout << fixed << setprecision(12) << *area << '\n';
```

复杂度

除 `sphere_poly_signed_area()` / `sphere_poly_area()` 需要遍历 n 个顶点、使用 $O(n)$ 额外空间外，其余所有操作均为 $O(1)$ 时间、$O(1)$ 额外空间

16.7 特殊空间几何

依赖： 长方体表面最短路仅使用 3.1 浮点版的 `Point3`、`distance()`。对角顶点特判仅依赖 `ld`；圆柱和圆锥侧面最短路另外使用全局 `PI`。
这里的**表面最短路**指沿物体外表面运动的最短长度，**不是穿过内部的三维直线距离**。优先使用长方体一般版本；若题目只询问两个空间对角顶点，用后面的短公式即可。

长方体表面任意两点最短路

设长方体为 $[0, L] \times [0, W] \times [0, H]$，要求 $L, W, H > 0$。输入的两点必须位于**闭合长方体外表面**（包括棱和顶点）
核心做法：枚举 **6 个面的不重复相邻面序列**，沿公共棱依次展开为平面矩形。对展开后的两端点连线，使用凸多边形裁剪检查它是否按顺序经过相应面片；只有整个线段在合法面片上时才计入答案
在凸多面体上，若一条最短路径重复进入同一个凸面，则可以用该面内的直线段替换中间路径，不会更长。因此，只需枚举**不重复面的路径**；本题只有 6 个面，属于常数规模
面编号： `0: x=0`，`1: x=L`，`2: y=0`，`3: y=W`，`4: z=0`，`5: z=H`

```
// ===== 长方体表面最短路：仅依赖 3.1 浮点 Point3 =====
struct BoxSurface3 {
    struct P2 {
        ld x{}, y{};
        P2 operator+(P2 b) const { return {x + b.x, y + b.y}; }
        P2 operator-(P2 b) const { return {x - b.x, y - b.y}; }
        P2 operator*(ld k) const { return {x * k, y * k}; }
    };
```

```
struct NetFace { // 一个面的展开图; v[i] 对应该面的第 i 个顶点
    array<P2, 4> v{};
};
struct Result {
    ld dist{};
    vector<int> faces; // 2*axis+side: 0 x=0, 1 x=L, ..., 5 z=H
};
array<ld, 3> d;
BoxSurface3(ld L, ld W, ld H) : d{L, W, H} {}
static ld cross2(P2 a, P2 b) { return a.x*b.y - a.y*b.x; }
static ld dot2(P2 a, P2 b) { return a.x*b.x + a.y*b.y; }
static ld len2(P2 a) { return sqrtl(dot2(a, a)); }
static ld coord(Point3 p, int a) {
    return a == 0 ? p.x : a == 1 ? p.y : p.z;
}
// 面 2*a+s 固定第 a 坐标为 s*d[a]; 另外两轴按循环顺序变化
static array<int, 2> free_axes(int f) {
    int a = f / 2;
    return { (a + 1) % 3, (a + 2) % 3 };
}
array<int, 4> vertices(int f) const {
    int a = f/2, b = (a+1)%3, c = (a+2)%3;
    int o = (f&1) << a;
    return {o, o | (1<<b), o | (1<<b) | (1<<c), o | (1<<c)};
}
NetFace initial(int f) const {
    auto ax = free_axes(f);
    return {{{{0, 0}, {d[ax[0]], 0}, {d[ax[0]], d[ax[1]]}, {0, d[ax[1]]}}}};
}
bool contains(Point3 p) const {
    for (int a = 0; a < 3; ++ a) {
        ld tol = 1E-12L * d[a];
        if (coord(p,a) < -tol || coord(p,a) > d[a]+tol) return false;
    }
    for (int a = 0; a < 3; ++ a) {
        ld tol = 1E-12L * d[a];
        if (fabsl(coord(p,a)) <= tol || fabsl(coord(p,a)-d[a]) <= tol) return true;
    }
    return false;
}
int faces_of(Point3 p) const {
    int mask = 0;
    for (int a = 0; a < 3; ++ a) {
        ld tol = 1E-12L * d[a];
        if (fabsl(coord(p,a)) <= tol) mask |= 1 << (2*a);
        if (fabsl(coord(p,a)-d[a]) <= tol) mask |= 1 << (2*a+1);
    }
    return mask;
}
P2 locate(Point3 p, int f, const NetFace &net) const {
    auto [b,c] = free_axes(f);
    return net.v[0] + (net.v[1] - net.v[0]) * (coord(p,b)/d[b])
                    + (net.v[3] - net.v[0]) * (coord(p,c)/d[c]);
}
// 从当前面 cur 沿公共棱展开到相邻面 nxt; 要求 cur/2 != nxt/2
NetFace unfold(int cur, int nxt, const NetFace &nf) const {
    auto u = vertices(cur), v = vertices(nxt);
    int j = -1;
    for (int i = 0; i < 4; ++ i) {
        int a = u[i], b = u[(i+1)%4];
        if (find(v.begin(), v.end(), a) != v.end() &&
            find(v.begin(), v.end(), b) != v.end()) { j = i; break; }
    }
    assert(j != -1);
    int id1 = u[j], id2 = u[(j+1)%4];
    P2 A = nf.v[j], B = nf.v[(j+1)%4];
    P2 e = B - A;
    P2 out{ -e.y / len2(e), e.x / len2(e) };
    P2 center{};
    for (auto p : nf.v) center = center + p * 0.25L;
    if (cross2(e, center - A) > 0) out = out * -1.L;
```

```
        P2 C = A + out * d[cur/2], D = B + out * d[cur/2];
        NetFace ans;
        for (int i = 0; i < 4; ++ i) {
            if (v[i] == id1) ans.v[i] = A;
            if (v[i] == id2) ans.v[i] = B;
            if (v[i] == (id1 ^ (1 << (cur/2)))) ans.v[i] = C;
            if (v[i] == (id2 ^ (1 << (cur/2)))) ans.v[i] = D;
        }
        return ans;
    }
    // 裁剪直线段 P → Q 落在展开面上的参数区间 [l, r]
    optional<pair<ld,ld>> clip(P2 P, P2 Q, const NetFace &nf) const {
        ld l = 0, r = 1;
        ld shortest = numeric_limits<ld>::infinity();
        for (int i = 0; i < 4; ++ i)
            shortest = min(shortest, len2(nf.v[(i+1)%4] - nf.v[i]));
        ld tol = 1E-13L * shortest;
        ld ori = cross2(nf.v[1]-nf.v[0], nf.v[3]-nf.v[0]) > 0 ? 1.L : -1.L;
        for (int i = 0; i < 4; ++ i) {
            P2 A = nf.v[i], e = nf.v[(i+1)%4] - A;
            ld el = len2(e);
            ld a = ori * cross2(e, P-A) / el;
            ld b = ori * cross2(e, Q-P) / el;
            if (b == 0) {
                if (a < -tol) return nullopt;
            } else if (b > 0) {
                l = max(l, (-tol - a) / b);
            } else {
                r = min(r, (-tol - a) / b);
            }
        }
        if (l > r + 1E-12L) return nullopt;
        return pair{clamp(l, 0.L, 1.L), clamp(r, 0.L, 1.L)};
    }
    bool valid(Point3 p, Point3 q, const vector<int> &seq,
               const array<NetFace,6> &net) const {
        P2 P = locate(p, seq.front(), net[seq.front()]);
        P2 Q = locate(q, seq.back(), net[seq.back()]);
        ld reach = 0;
        for (int f : seq) {
            auto seg = clip(P, Q, net[f]);
            if (!seg || seg->first > reach + 1E-12L) return false;
            reach = max(reach, seg->second);
        }
        return reach >= 1.L - 1E-12L;
    }
    // 输入 p、q 均在闭合长方体表面上；尺寸 L,W,H 均严格正
    optional<Result> shortest(Point3 p, Point3 q) const {
        if (min({d[0],d[1],d[2]}) <= 0 || !contains(p) || !contains(q))
            return nullopt;
        int sm = faces_of(p), tm = faces_of(q);
        if (sm & tm) {
            int f = countr_zero((unsigned)(sm & tm));
            return Result{distance(p, q), {f}};
        }
        Result best{numeric_limits<ld>::infinity(), {}};
        array<NetFace,6> net;
        vector<int> seq;
        function<void(int,int)> dfs = [&](int cur,int used) {
            if (tm & (1 << cur)) {
                if (valid(p, q, seq, net)) {
                    P2 P = locate(p, seq.front(), net[seq.front()]);
                    P2 Q = locate(q, cur, net[cur]);
                    ld len = len2(Q - P);
                    if (len < best.dist) best = {len, seq};
                }
                // 末点也可能在其他面，但不可在最短简单面序列中再次经过当前面
            }
            for (int nxt = 0; nxt < 6; ++ nxt) {
                if (cur/2 == nxt/2 || (used & (1 << nxt))) continue;
                net[nxt] = unfold(cur, nxt, net[cur]);
```

```
                seq.push_back(nxt);
                dfs(nxt, used | (1 << nxt));
                seq.pop_back();
            }
        };
        for (int f = 0; f < 6; ++ f) if (sm & (1 << f)) {
            net[f] = initial(f);
            seq = {f};
            dfs(f, 1 << f);
        }
        if (!isfinite(best.dist)) return nullopt;
        return best;
    }
};
```

对角顶点、圆柱侧面和圆锥侧面

这些是**互不依赖**的短板子，可以只复制需要的函数。圆柱和圆锥函数使用**弧度**表示方位角；它们只允许在**侧面**行走，**不允许穿过底面或长方体内部**。

```
// ===== 对角顶点特判：可独立删除，不依赖 BoxSurface3 =====
// 长方体两个空间对角顶点在表面上的最短距离
ld box_opposite_vertices_dist(ld L, ld W, ld H) {
    return min({hypotl(L + W, H), hypotl(L + H, W), hypotl(W + H, L)});
}

// ===== 圆柱侧面最短路：可独立删除，不允许经过上下底面 =====
// θ 单位为弧度；输入 r > 0，侧面两点的 z 可为任意高度
ld cylinder_side_dist(ld r, ld z1, ld t1, ld z2, ld t2) {
    ld dy = r * remainderl(t2 - t1, 2.L * PI);
    return hypotl(z2 - z1, dy);
}

// ===== 正圆锥侧面最短路：可独立删除，不允许经过底面 =====
// 圆锥底面 z=0，顶点 z=h；0 ≤ z1,z2 ≤ h，r,h > 0
ld cone_side_dist(ld r, ld h, ld z1, ld t1, ld z2, ld t2) {
    ld s = hypotl(r, h);
    ld a = s * (1.L - z1/h), b = s * (1.L - z2/h);
    ld ang = (r/s) * fabsl(remainderl(t2 - t1, 2.L * PI));
    return hypotl(a - b, 2.L * sqrtl(a*b) * sinl(ang/2.L));
}
```

接口说明

接口	功能与返回值	复杂度
`BoxSurface3(L,W,H)`	构造轴对齐长方体，三个尺寸都必须大于零	$O(1)$
`box.shortest(p,q)`	求闭表面任意两点最短路；无效尺寸或点不在表面返回 `nullopt`	$O(1)$
`Result::dist`	最短**表面路径长度**，`ld`	$O(1)$
`Result::faces`	对应的一条合法最短路径经过的面编号序列	最多 6 项
`box_opposite_vertices_dist(L,W,H)`	两个空间对角顶点的表面最短距离	$O(1)$
`cylinder_side_dist(r,z1,t1,z2,t2)`	半径 `r` 的**无限圆柱侧面**两点最短路；`z` 为高度，`t` 为弧度	$O(1)$
`cone_side_dist(r,h,z1,t1,z2,t2)`	底面半径 `r`、高度 `h` 的**有限正圆锥侧面**两点最短路；圆锥顶点在 `z=h`，底面在 `z=0`	$O(1)$

`Result::faces` 描述**经过哪些面**，并非空间最短路的折点坐标；不要把不同面的展开直线当作三维空间中的直线段。

公式速查

长方体空间对角顶点： 设三条棱长为 L, W, H，有

$$D = \min\left\{\sqrt{(L+W)^2+H^2},\ \sqrt{(L+H)^2+W^2},\ \sqrt{(W+H)^2+L^2}\right\}.$$

圆柱侧面： 将侧面展开为矩形，周向距离取较短绕行方向。记 $\Delta\theta = \mathrm{remainder}(t_2 - t_1, 2\pi) \in [-\pi, \pi]$，则

$$D = \sqrt{(z_2 - z_1)^2 + (r\Delta\theta)^2}.$$

正圆锥侧面： 母线长度 $s=\sqrt{r^2+h^2}$；点的顶点沿母线距离 $a=s(1-z_1/h)$、$b=s(1-z_2/h)$。把圆锥展开为半径 s、圆心角 $2\pi r/s$ 的扇形，记

$$\phi=\frac{r}{s}|\mathrm{remainder}(t_2-t_1,2\pi)|\in[0,\pi],\qquad D=\sqrt{(a-b)^2+4ab\sin^2(\phi/2)}.$$

该式含圆锥顶点：某点位于顶点时，对应母线距离为零，答案为另一点的母线距离。

注意事项

- **输入对象约定：** `BoxSurface3` 只支持**轴对齐**的非退化长方体，坐标范围为 `[0,L]×[0,W]×[0,H]`。若长方体只是平移，先将两点减去同一个起点；若有旋转，先转至局部坐标系。
- **边与顶点：** 一点可能同时属于 2 或 3 个面，代码会枚举全部起点面、终点面。若存在一个同时包含两点的面，表面最短距离直接等于三维欧氏距离。
- **不可省略裁剪：** 某个面序列的展开直线可能穿出面片、绕过不应穿越的棱。必须验证它按该序列依次留在所有矩形内，不能仅取各展开图中的直线距离最小值。
- **不支持穿过物体内部：** 长方体、圆柱、圆锥三个表面距离都不是空间中的最短欧氏距离。圆柱/圆锥额外限定**只走侧面**；若允许走上下底面或圆锥底面，需要另行考虑其它路径。
- **浮点判定：** 长方体以各轴尺寸为尺度做表面容差判定，并用面裁剪容差处理棱上交点。几何规模极度悬殊、坐标接近机器精度极限时，浮点算法不保证严格正确。`shortest()` 的 `nullopt` 首先表示输入不满足板子约定；极端数值退化时也可能无法找到有效展开。
- **方位角：** `t1`、`t2` 单位是**弧度**，允许超出 `[0,2π)`；`remainderl` 自动选取等价的短周向差。圆柱要求 `r>0`，圆锥要求 `r>0,h>0,0≤z1,z2≤h`。
- **圆柱侧面边界：** 两点可以在有限圆柱的上下边缘；因为侧面展开域是矩形，连接两点的展开直线仍在侧面内。但是**经过圆柱端盖**的路线不包含在本接口中。

常见用法

长方体任意表面点

```
BoxSurface3 box(2, 3, 5);
Point3 p{0, 1, 2}, q{2, 2, 4};
auto res = box.shortest(p, q);
if (res) {
    cout << fixed << setprecision(12) << res->dist << '\n';
    // res->faces 是一条最短路径依次经过的面编号
}
```

单位正方体两个空间对角顶点

```
BoxSurface3 box(1, 1, 1);
auto res = box.shortest({0, 0, 0}, {1, 1, 1});
// res->dist == sqrtl(5.L)
ld d = box_opposite_vertices_dist(1, 1, 1); // sqrtl(5.L)
```

圆柱、圆锥的侧面最短路

```
ld a = cylinder_side_dist(2, 0, 0, 3, PI);  // sqrtl(9 + 4*PI*PI)
ld b = cone_side_dist(1, 1, 1, 0, 0, PI);  // sqrtl(2)，顶点到任意底圆点
```

复杂度

- `BoxSurface3::shortest`：最多枚举 6 个面的**不重复路径**，6 是常量，因此单次查询 $O(1)$ 时间、$O(1)$ 额外空间；如果把面数作为变量，一般情形的枚举是指数级，**不能当作任意多面体表面最短路算法**。
- 对角顶点、圆柱侧面、圆锥侧面距离函数：均为 $O(1)$ 时间和空间。

17. 经典模型、专题与结论表

17.1 构造、搜索与经典模型

N 皇后一组解

解决问题： 在 $n\times n$ 棋盘上放置 n 个皇后，使任意两个皇后不同列且不在同一条对角线上，只要求构造任意一组解
思路： 普通 N 皇后在 $n=1$ 或 $n\geq 4$ 时都有解，可按 $n \bmod 6$ 分类直接排列列号，无需 DFS；返回每行皇后的 0base 列号

```
optional<vector<int>> queens(int n) {
    if (n == 0) return vector<int>{};
    if (n == 2 || n == 3) return nullopt;
```

```
    if (n == 1) return vector<int>{0};
    vector<int> p;
    p.reserve(n);
    if (n % 6 != 2 && n % 6 != 3) {
        for (int x = 2; x <= n; x += 2) p.push_back(x);
        for (int x = 1; x <= n; x += 2) p.push_back(x);
    } else if (n % 6 == 2) {
        for (int x = 2; x <= n; x += 2) p.push_back(x);
        p.push_back(3);
        p.push_back(1);
        for (int x = 7; x <= n; x += 2) p.push_back(x);
        p.push_back(5);
    } else {
        for (int x = 4; x <= n; x += 2) p.push_back(x);
        p.push_back(2);
        for (int x = 5; x <= n; x += 2) p.push_back(x);
        p.push_back(1);
        p.push_back(3);
    }
    for (int &x : p) -- x;
    return p;
}
```

接口说明

- `queens(n)`：返回每行皇后的 0base 列号；`n=2,3` 返回 `nullopt`

注意事项

- 只解决普通 N 皇后的任意一组解；若要求方案数、全部方案、带障碍或额外约束，应改用位压 DFS / DLX

常见用法

```
auto p = queens(n);
if (!p) {
    // 无解
}
```

复杂度

- 时间 $O(n)$，额外空间 $O(n)$

幻方

解决问题： 构造 n 阶正常幻方，使 $1..n^2$ 各出现一次，且每行、每列、两条主对角线的元素和相同
思路： 奇数阶使用 Siamese 法；$4k$ 阶使用互补翻转；$4k+2$ 阶先构造半阶奇数幻方，再按四块偏移并交换指定列。$n=2$ 无解

```
vector<vector<int>> odd_magic(int n) {
    vector a(n, vector<int>(n));
    int r = 0, c = n / 2;
    for (int x = 1; x <= n * n; ++ x) {
        a[r][c] = x;
        int nr = (r - 1 + n) % n, nc = (c + 1) % n;
        if (a[nr][nc]) r = (r + 1) % n;
        else r = nr, c = nc;
    }
    return a;
}
optional<vector<vector<int>>> magic(int n) {
    if (n <= 0 || n == 2) return nullopt;
    if (n & 1) return odd_magic(n);
    vector a(n, vector<int>(n));
    if (n % 4 == 0) {
        for (int i = 0; i < n; ++ i) for (int j = 0; j < n; ++ j) {
            int x = i * n + j + 1;
            if (i % 4 == j % 4 || i % 4 + j % 4 == 3) x = n * n + 1 - x;
            a[i][j] = x;
        }
        return a;
    }
```

```
    int m = n / 2, k = (m - 1) / 2, d = m * m;
    auto b = odd_magic(m);
    for (int i = 0; i < m; ++ i) for (int j = 0; j < m; ++ j) {
        a[i][j] = b[i][j];
        a[i][j + m] = b[i][j] + 2 * d;
        a[i + m][j] = b[i][j] + 3 * d;
        a[i + m][j + m] = b[i][j] + d;
    }
    for (int i = 0; i < m; ++ i) {
        for (int j = 0; j < k; ++ j) if (i != k || j != 0) {
            swap(a[i][j], a[i + m][j]);
        }
        for (int j = n - k + 1; j < n; ++ j) swap(a[i][j], a[i + m][j]);
    }
    swap(a[k][k], a[k + m][k]);
    return a;
}
```

接口说明

- `magic(n)`：构造 n 阶正常幻方；`n=2` 或 `n≤0` 返回 `nullopt`
- `odd_magic(n)`：内部奇数阶构造，通常不单独调用

注意事项

- 普通正常幻方与 pandiagonal / most-perfect 等特殊幻方不是同一模型

常见用法

```
auto a = magic(n);
if (a) {
    cout << (*a)[i][j] << '\n';
}
```

复杂度

- 时间 $O(n^2)$，空间 $O(n^2)$，与输出规模同阶

$h \times w$ 宫 Sudoku 终盘

解决问题： 构造一个完整合法的广义 Sudoku 终盘。每个宫为 $h \times w$，整盘边长为 $n = hw$，每行、每列、每宫都恰含 $1..n$
思路： 让第 i 行相对首行循环平移 $(i \bmod h)w + \lfloor i/h \rfloor$，可同时保证行、列和宫内均为排列

```
vector<vector<int>> sudoku(int h, int w) {
    int n = h * w;
    vector a(n, vector<int>(n));
    for (int i = 0; i < n; ++ i) for (int j = 0; j < n; ++ j) {
        a[i][j] = ((i % h) * w + i / h + j) % n + 1;
    }
    return a;
}
```

接口说明

- `sudoku(h,w)`：返回边长为 `h*w` 的完整合法终盘，数字范围为 `1..h*w`

注意事项

- 只构造完整终盘；带空格求解、唯一解判定、字典序最小补全应使用搜索 / DLX

常见用法

```
auto a = sudoku(h, w);
cout << a[i][j] << '\n';
```

复杂度

- 设 $n = hw$，时间 $O(n^2)$，空间 $O(n^2)$

Hadamard 矩阵

解决问题： 当 $n=2^k$ 时构造元素为 ±1 的 Hadamard 矩阵，使任意不同两行或两列的点积均为 0
思路： 从 $H_1=[1]$ 出发反复使用 Sylvester 构造 $H_{2n}=\begin{bmatrix}H_n & H_n\\ H_n & -H_n\end{bmatrix}$

```
vector<vector<int>> hadamard(int n) {
    if (n <= 0 || (n & (n - 1))) return {};
    vector a(n, vector<int>(n));
    a[0][0] = 1;
    for (int len = 1; len < n; len <<= 1) {
        for (int i = 0; i < len; ++ i) for (int j = 0; j < len; ++ j) {
            a[i][j + len] = a[i + len][j] = a[i][j];
            a[i + len][j + len] = -a[i][j];
        }
    }
    return a;
}
```

接口说明

- `hadamard(n)`：构造 2^k 阶 Hadamard 矩阵；`n` 不是正二次幂时返回空矩阵

注意事项

- Hadamard 猜想并未给出所有 $4k$ 阶的统一构造；本板子只保证 2^k 阶

常见用法

```
auto h = hadamard(n);
if (h.empty()) {
    // 不支持该阶数
}
```

复杂度

- 时间 $O(n^2)$，空间 $O(n^2)$

Hanoi

解决问题： 将 n 个大小不同的圆盘从起点柱移动到目标柱，每次只能移动一个圆盘，且大盘不能放在小盘上
思路： 先把前 $n-1$ 个盘移到辅助柱，再移动最大盘，最后把前 $n-1$ 个盘移到目标柱；通过回调 `move(s,t)` 处理每次操作

```
template<class F>
void hanoi(int n, int s, int t, int h, const F &move) {
    if (!n) return;
    hanoi(n - 1, s, h, t, move);
    move(s, t);
    hanoi(n - 1, h, t, s, move);
}
```

接口说明

- `hanoi(n,s,t,h,move)`：把 `n` 个圆盘从 `s` 移到 `t`，`h` 为辅助柱；每一步调用 `move(u,v)`

注意事项

- 三柱 Hanoi 恰有 2^n-1 次移动；四柱及更多柱不能直接套本递归

常见用法

```
hanoi(n, 0, 2, 1, [&](int u, int v) {
    cout << u << ' ' << v << '\n';
});
```

复杂度

- 时间 $O(2^n)$，递归栈 $O(n)$；输出本身有 2^n-1 步

滑块拼图可达性

解决问题： 判断两个 $h \times w$ 滑块拼图状态能否通过移动空格互相到达，只判可达性，不求最短步数
思路： 忽略空格计算逆序奇偶性；宽度为奇数时奇偶性不变，宽度为偶数时还要结合空格距底部的行号形成不变量

```
int inv_parity(const vector<int> &a, int blank = 0) {
    int p = 0;
    for (int i = 0; i < (int)a.size(); ++ i) if (a[i] != blank) {
        for (int j = i + 1; j < (int)a.size(); ++ j) {
            if (a[j] != blank && a[i] > a[j]) p ^= 1;
        }
    }
    return p;
}
bool puzzle_ok(const vector<int> &a, const vector<int> &b, int w) {
    if (w & 1) return inv_parity(a) == inv_parity(b);
    auto f = [&](const vector<int> &s) {
        int z = find(s.begin(), s.end(), 0) - s.begin();
        int r = int(s.size()) / w - z / w;
        return inv_parity(s) ^ (r & 1);
    };
    return f(a) == f(b);
}
```

接口说明

- `inv_parity(a,blank)`：返回忽略空格后的逆序奇偶性
- `puzzle_ok(a,b,w)`：判断宽度为 `w` 的两个状态是否可达

注意事项

- 两个状态必须尺寸相同、宽度合法、都恰有一个空格 `0`，且其余编号集合相同；函数本身不检查这些条件
- 这里只判可达性；八数码最短路常用反向 BFS，十五数码常用 IDA*

常见用法

```
if (puzzle_ok(a, b, w)) {
    // 可达
}
```

复杂度

- 设格子总数为 N，当前逆序统计为 $O(N^2)$，额外空间 $O(1)$

稳定婚姻 Gale–Shapley

解决问题： 两侧各有 n 人，每人给出对另一侧的严格完整偏好序，求不存在阻塞对的稳定完美匹配
思路： A 方未匹配者按偏好顺序不断求婚；B 方只保留自己更喜欢的候选。每对组合最多被尝试一次，A 方主动时得到 A-optimal 稳定匹配

```
vector<int> stable_match(const vector<vector<int>> &pa, const vector<vector<int>> &pb) {
    int n = pa.size();
    vector rk(n, vector<int>(n));
    for (int b = 0; b < n; ++ b) for (int i = 0; i < n; ++ i) {
        rk[b][pb[b][i]] = i;
    }
    vector<int> ma(n, -1), mb(n, -1), nxt(n);
    queue<int> q;
    for (int a = 0; a < n; ++ a) q.push(a);
    while (!q.empty()) {
        int a = q.front();
        q.pop();
        int b = pa[a][nxt[a] ++];
        if (mb[b] == -1) {
            ma[a] = b;
            mb[b] = a;
        } else if (rk[b][a] < rk[b][mb[b]]) {
            int old = mb[b];
            ma[old] = -1;
            q.push(old);
            ma[a] = b;
            mb[b] = a;
```

```
        } else {
            q.push(a);
        }
    }
    return ma;
}
```

接口说明

- `stable_match(pa,pb)`：`pa[a]`、`pb[b]` 分别为两侧严格完整偏好排列，返回 `ma[a]=b`

注意事项

- A 方主动求婚，因此得到 A-optimal 稳定匹配
- 不完整偏好、不可接受对象、容量大于 1 或并列偏好需要改成对应模型

常见用法

```
auto ma = stable_match(pa, pb);
for (int a = 0; a < n; ++ a) {
    int b = ma[a];
}
```

复杂度

- 时间 $O(n^2)$，排名表与匹配数组共 $O(n^2)$ 空间

最优二叉搜索树

解决问题： 已知有序关键字的成功搜索权 `p[0..n)` 和相邻空隙的失败搜索权 `q[0..n]`，构造期望搜索代价最小的二叉搜索树
思路： 区间 DP 枚举根，区间总权值作为本层新增代价；利用最优根的 Knuth 单调性，把根的枚举范围缩到相邻区间最优根之间

```
pair<ld, vector<vector<int>>> opt_bst(const vector<ld> &p, const vector<ld> &q) {
    int n = p.size();
    vector dp(n + 1, vector<ld>(n + 1));
    vector sum(n + 1, vector<ld>(n + 1));
    vector rt(n + 1, vector<int>(n + 1));
    for (int i = 0; i <= n; ++ i) {
        dp[i][i] = sum[i][i] = q[i];
        rt[i][i] = i;
    }
    for (int len = 1; len <= n; ++ len) for (int l = 0, r = len; r <= n; ++ l, ++ r) {
        sum[l][r] = sum[l][r - 1] + p[r - 1] + q[r];
        int lo = len == 1 ? l : rt[l][r - 1];
        int hi = len == 1 ? r - 1 : min(r - 1, rt[l + 1][r]);
        dp[l][r] = numeric_limits<ld>::infinity();
        for (int k = lo; k <= hi; ++ k) {
            ld cur = dp[l][k] + dp[k + 1][r] + sum[l][r];
            if (cur < dp[l][r]) dp[l][r] = cur, rt[l][r] = k;
        }
    }
    return {dp[0][n], rt};
}
```

接口说明

- `opt_bst(p,q)`：要求 `q.size()==p.size()+1`，返回 `{最小期望代价, 最优根表}`；`rt[l][r]` 表示区间 `[l,r)` 的根

注意事项

- 权值必须非负；Knuth 单调性依赖该经典最优 BST 代价结构，其他区间 DP 不能直接套根区间优化

常见用法

```
auto [cost, rt] = opt_bst(p, q);
int root = rt[0][p.size()];
```

复杂度

- 时间 $O(n^2)$，空间 $O(n^2)$

最大 1 矩形

解决问题： 在 0/1 字符矩阵中求全部由指定字符组成的最大轴对齐子矩形，并返回面积与半开边界 `[t,b) * [l,r)`
思路： 逐行维护每列连续高度 `h`，同时维护当前高度能延伸到的左右边界，把每一行视为柱状图并在线更新最大矩形

```
struct Rect {
    i64 area{0};
    int t{0}, l{0}, b{0}, r{0};
};
Rect max_rect(const vector<string> &a, char good = '1') {
    int n = a.size(), m = n ? (int)a[0].size() : 0;
    vector<int> h(m), l(m), r(m, m);
    Rect ans;
    for (int i = 0; i < n; ++ i) {
        int bd = 0;
        for (int j = 0; j < m; ++ j) {
            if (a[i][j] == good) {
                ++ h[j];
                l[j] = max(l[j], bd);
            } else {
                h[j] = 0;
                l[j] = 0;
                r[j] = m;
                bd = j + 1;
            }
        }
        bd = m;
        for (int j = m - 1; j >= 0; -- j) {
            if (a[i][j] == good) r[j] = min(r[j], bd);
            else bd = j;
            i64 s = (i64)h[j] * (r[j] - l[j]);
            if (s > ans.area) ans = {s, i - h[j] + 1, l[j], i + 1, r[j]};
        }
    }
    return ans;
}
```

接口说明

- `max_rect(a,good)`：默认把字符 `'1'` 视为可用格，返回 `{area,t,l,b,r}`，边界为半开区间 `[t,b) * [l,r)`

注意事项

- 若题目给的是平面障碍点、障碍矩形或允许旋转的矩形，与本 0/1 网格模型不同

常见用法

```
Rect ans = max_rect(a);
cout << ans.area << '\n';
```

复杂度

- 对 $n \times m$ 矩阵，时间 $O(nm)$，额外空间 $O(m)$

两机流水作业 Johnson 排序

解决问题： 每个任务都必须依次经过机器 1、机器 2，且两台机器一次只能处理一个任务，求使全部任务最早完成的一组任务顺序
思路： 若任务在机器 1 上更快，则放在前半并按机器 1 时间升序；否则放在后半并按机器 2 时间降序，这就是 Johnson 规则

```
vector<int> johnson(const vector<pair<i64, i64>> &a) {
    vector<int> p(a.size());
    iota(p.begin(), p.end(), 0);
    sort(p.begin(), p.end(), [&](int i, int j) {
        bool x = a[i].first <= a[i].second;
        bool y = a[j].first <= a[j].second;
        if (x != y) return x > y;
        return x ? a[i].first < a[j].first : a[i].second > a[j].second;
```

```
    });
    return p;
}
```

接口说明

- `johnson(a)`：`a[i]={t1,t2}` 为任务在机器 1、2 上的处理时间，返回一组最优任务编号顺序

注意事项

- 只适用于所有任务都按“机器 1 → 机器 2”处理的经典两机 flow-shop；机器顺序不同、可并行、存在释放时间等变体不能直接套用

常见用法

```
auto ord = johnson(a);
for (int id : ord) {
    // 按该顺序加工
}
```

复杂度

- 排序时间 $O(n \log n)$，额外空间 $O(n)$

第 k 个元素

解决问题： 求无序序列按升序排列后的第 `k` 个元素，`k` 为 0base，但不需要完整排序
思路： 使用 `nth_element` 做选择，使目标位置左侧都不大于它、右侧都不小于它，避免完整排序

```
template<class T>
T kth(vector<T> a, int k) {
    nth_element(a.begin(), a.begin() + k, a.end());
    return a[k];
}
```

接口说明

- `kth(a,k)`：返回升序后的 0base 第 `k` 个元素，要求 `0 ≤ k < a.size()`；参数按值传入，不修改原数组

注意事项

- `nth_element` 只保证第 `k` 个位置正确及两侧偏序，不会把整个数组排好序

常见用法

```
int x = kth(a, k);
```

复杂度

- 标准库通常为平均 $O(n)$；本封装额外复制数组需要 $O(n)$ 时间与空间

有界布尔母函数

解决问题： 第 i 类物品重量为 `w[i]`、最多使用 `c[i]` 个，判断 `0..lim` 中哪些总和可达
思路： 把每类数量按二进制拆成若干个 0/1 物品，再用按位位移 OR 更新可达集合；每个 64 位整数同时处理 64 个状态

```
void shift_or(vector<u64> &f, int d, int lim) {
    if (d ≤ 0 || d > lim) return;
    int q = d >> 6, r = d & 63;
    for (int i = (int)f.size() - 1; i ≥ q; -- i) {
        u64 x = f[i - q] << r;
        if (r && i > q) x |= f[i - q - 1] >> (64 - r);
        f[i] |= x;
    }
    int k = (lim + 1) & 63;
    if (k) f.back() &= (1ULL << k) - 1;
}
vector<u64> bounded_sum(const vector<int> &w, const vector<int> &c, int lim) {
    vector<u64> f((lim + 64) / 64);
```

```
    f[0] = 1;
    for (int i = 0; i < (int)w.size(); ++ i) {
        int rem = c[i];
        for (i64 b = 1; rem > 0; b <<= 1) {
            int take = min<i64>(b, rem);
            i64 d = (i64)w[i] * take;
            if (d ⩽ lim) shift_or(f, (int)d, lim);
            rem -= take;
        }
    }
    return f;
}
```

接口说明

- `bounded_sum(w,c,lim)`：第 `i` 类物品重量为 `w[i]`、最多选 `c[i]` 个；返回位集，第 `s` 位为 1 表示总和 `s` 可达
- `shift_or(f,d,lim)`：内部 0/1 位集转移，通常不单独调用

注意事项

- 要求 `lim⩾0`、`w[i]⩾0`、`c[i]⩾0` 且 `w.size()==c.size()`；`w[i]=0` 不改变可达集合

常见用法

```
auto f = bounded_sum(w, c, lim);
bool ok = f[x >> 6] >> (x & 63) & 1ULL;
```

复杂度

- 设上界为 L，时间 $O\left(\frac{L}{64}\sum_i \log(c_i+1)\right)$，空间 $O(L/64)$

17.2 图论、偏序与变换结论

König、Gallai 与边覆盖

解决问题： 已知匹配、点覆盖、独立集或边覆盖中的一种量时，快速转化为另一种量，尤其用于二分图匹配后的答案换算
思路： König 把二分图最大匹配与最小点覆盖连接起来；Gallai 把任意图的最大独立集与最小点覆盖互补；无孤立点图的最小边覆盖可由最大匹配补齐

- König：二分图最小点覆盖大小 $\tau=\nu$，其中 ν 为最大匹配大小
- Gallai：任意图中最大独立集大小 $\alpha+\tau=n$
- 无孤立点图：最小边覆盖大小 $\rho=n-\nu$

接口说明

- 二分图最大匹配、最小点覆盖使用 6.1 二分图匹配与 KM
- 若只需要大小，求出最大匹配数 `mat` 后直接代公式；若要恢复具体覆盖/独立集，使用匹配板子的恢复接口

注意事项

- König 的 $\tau=\nu$ 只对二分图成立；Gallai 的 $\alpha+\tau=n$ 对任意图成立
- 图存在孤立点时，不存在覆盖全部顶点的普通边覆盖，不能直接使用 $\rho=n-\nu$

常见用法

```
int mvc = mat;
int mis = n - mat; // 二分图中也可由 König + Gallai 得到
int mec = n - mat;      // 仅当无孤立点
```

复杂度

- 公式换算为 $O(1)$；总复杂度由最大匹配算法决定，Hopcroft–Karp 为 $O(E\sqrt{V})$

Hall 定理与缺额

解决问题： 判断二分图是否存在覆盖左部所有点的匹配，或判断至少会有多少个左部点无法匹配
思路： 对任意左部子集 S，必须有足够多的右部邻点容纳它。定义缺额 $\mathrm{def}=\max_{S\subseteq L}(|S|-|N(S)|)$，则最大匹配能覆盖的左部点数为 $|L|-\mathrm{def}$。

$$\text{存在覆盖 } L \text{ 的匹配} \iff \forall S \subseteq L,\ |N(S)| \geq |S|$$

接口说明

- 赛场上通常不枚举所有 S，而是直接求最大匹配：若 `mat == |L|`，Hall 条件成立
- 带权最优匹配不由 Hall 解决，应使用 KM 或费用流

注意事项

- Hall 是存在性定理，不提供带权最优方案
- 完美匹配还要求两侧规模相同；“覆盖左部”本身不要求右部全部被匹配

常见用法

```
bool ok = mat == nl;
int def = nl - mat;
```

复杂度

- 直接检查全部子集是指数级；实际使用最大匹配判断，复杂度由匹配算法决定

Dilworth 与 Mirsky

解决问题： 对偏序集求最少链覆盖、最大反链、最大链或最少反链分层
思路： Dilworth 给出“最少链数 = 最大反链大小”；Mirsky 给出对偶的“最少反链层数 = 最大链长度”。偏序链覆盖可把元素复制成左右两部，对所有可比关系 $x < y$ 连边，再做最大匹配

- Dilworth：最少链覆盖数 $= n - \nu =$ 最大反链大小
- Mirsky：最少反链分层数 $=$ 最大链长度

接口说明

- 偏序链覆盖：对“可比关系”建二分图后调用 6.1 二分图匹配与 KM
- 若输入本身是 DAG 且题目求 DAG 顶点不相交最小路径覆盖，使用 DAG 路径覆盖模型，不必额外做传递闭包

注意事项

- **偏序链覆盖与 DAG 路径覆盖不是同一建图。** 偏序需要连所有 $x < y$ 的可比关系；输入 DAG 的原始边未必已经是传递闭包
- 只有确认题意对象是偏序时，才能直接用 Dilworth 的“最大反链”解释

常见用法

```
int chains = n - mat;
int anti = chains;
```

复杂度

- 传递闭包若用 bitset / Floyd 等另计；匹配复杂度取决于所用二分图匹配算法

Menger 与点边不相交路径

解决问题： 求 s 到 t 最多能同时走多少条互不共享边或内部点的路径，也可反向解释为最小割边数或最少删除点数
思路： Menger 把“不相交路径最大条数”与“最小分离集大小”对偶起来。边不相交直接把边容量设为 1；点不相交把中间点拆成 `in → out`，容量设为 1

- 边版：边不相交的 $s \to t$ 路径最大条数 = 最小 $s - t$ 边割大小
- 点版：对不相邻的 s, t，内部点不相交路径最大条数 = 删除后能断开 s, t 的最少内部点数

接口说明

- 使用最大流板子；边不相交令原边容量为 1
- 点不相交令每个中间点拆点，`vin → vout` 容量为 1，源汇容量设为足够大

注意事项

- 若某个点允许被多条路径经过，把拆点容量改成对应上限
- 点版通常不限制源点和汇点本身的使用次数
- 有向图与无向图都可建模，但无向边转容量边时要按题意处理方向

常见用法

```
// 点不相交：v 拆成 vin, vout
flow.add_edge(vin, vout, v == s || v == t ? INF : 1);
```

复杂度

- 建图为 $O(V+E)$ 级；求解复杂度由所用最大流算法决定

Erdős–Gallai 度序列判定

解决问题： 给定非负整数序列，判断它能否作为某个简单无向图的度数序列
思路： 先按非增排序并检查度数和为偶数，再验证所有前缀约束：

$$\sum_{i=1}^{k} d_i \le k(k-1) + \sum_{i=k+1}^{n} \min(d_i, k)$$

后半项可用前缀和加二分找到第一个小于 k 的位置，从而整体做到 $O(n \log n)$；若还要构造图，可用 Havel–Hakimi

接口说明

- 本节只保留判定结论；需要实现时建议写 `bool graphical(vector<int> d)`
- 只判存在性用 Erdős–Gallai；需要直接构造一张图时更适合 Havel–Hakimi

注意事项

- 目标是**简单无向图**：不允许自环和重边
- 必须先排序；度数应满足 `0 ≤ d[i] < n`

常见用法

```
sort(d.rbegin(), d.rend());
// 前缀和 + 对每个 k 检查 Erdős–Gallai 不等式
```

复杂度

- 排序、前缀和和二分实现为 $O(n \log n)$；朴素逐项计算为 $O(n^2)$

Landau 竞赛图度序列

解决问题： 判断一个整数序列能否作为竞赛图中各点的出度序列
思路： 将出度非降排序。任意前 k 个点之间至少产生 $\binom{k}{2}$ 场内部胜利，全部点总胜场数恰为 $\binom{n}{2}$

$$\sum_{i=1}^{k} s_i \ge \binom{k}{2} \quad (k < n), \qquad \sum_{i=1}^{n} s_i = \binom{n}{2}$$

接口说明

- 判定时先非降排序，再扫前缀和
- 若题目还要求构造竞赛图，需要额外使用构造算法，Landau 本身只给存在性

注意事项

- 每个出度都必须位于 `0..n-1`
- 这里是竞赛图：任意两点之间恰有一条定向边

常见用法

```
sort(s.begin(), s.end());
i64 pre = 0;
for (int k = 1; k ≤ n; ++ k) {
    pre += s[k - 1];
    // k < n 时检查 pre ≥ k * (k - 1) / 2
}
```

复杂度

- 排序后 $O(n)$ 检查，总复杂度 $O(n \log n)$

Ramsey 小表

解决问题： 遇到很小的完全图染边 / 团与独立集保证存在问题时，直接查常见 Ramsey 数，避免临场误记
思路： $R(a,b)$ 是最小的 n，使任意 K_n 的红蓝边染色必含红色 K_a 或蓝色 K_b

- $R(3,3)=6$
- $R(3,4)=9$
- $R(3,5)=14$
- $R(4,4)=18$

接口说明

- 本节仅作常数表，没有代码接口

注意事项

- 大量 Ramsey 数的精确值未知，不能根据小表外推
- 注意区分 Ramsey 数与 Turán 极值数

常见用法

- 看到"任意 6 人中必有 3 人两两认识或 3 人两两不认识"即使用 $R(3,3)=6$

平面图与极值边数

解决问题： 快速判断某些图规模是否必不可能平面，或由面长、二分性推出边数和低度点上界
思路： 对平面嵌入使用 Euler 公式，再由所有面的边长和为 $2m$ 推出边数上界。设点数、边数、面数、连通块数分别为 n,m,f,c：

$$n-m+f=1+c$$

连通时为 $n-m+f=2$。进一步有：

- 简单平面图且 $n\ge 3$：$m\le 3n-6$
- 简单二分平面图且 $n\ge 3$：$m\le 2n-4$
- 连通简单平面图每个面长度至少为 $g\ge 3$：$m\le \dfrac{g}{g-2}(n-2)$
- 简单平面图存在度数不超过 5 的点；简单二分平面图存在度数不超过 3 的点
- K_5 与 $K_{3,3}$ 非平面

接口说明

- 这些式子主要用于必要条件、计数和反证，不代替真正的平面性判定

注意事项

- $m\le 3n-6$ 不是平面性的充分条件
- 有自环、重边时不能直接套简单图边数上界
- Kuratowski 定理给出完整刻画，但竞赛中常先用 Euler 上界快速排除

常见用法

```
if (n ≥ 3 && m > 3LL * n - 6) {
    // 简单图一定不是平面图
}
```

Matrix-Tree 生成树计数

解决问题： 计算无向图生成树数量，或在统一 Laplacian 约定后计算有向根生成树数量。
思路： 构造 Laplacian 矩阵。无向图中 $L=D-A$，删除任意一行一列后的余子式行列式即生成树数

接口说明

- 使用 9.4 Prüfer、Matrix-Tree 与 De Bruijn 的 `spanning_tree_count`
- 行列式消元依赖 11.1 矩阵与线性方程组

注意事项

- 多重边在邻接矩阵中累加；自环不参与生成树

- 有向入树 / 出树的 Laplacian 定义和删行列方向必须与现有板子保持一致，不要凭记忆混用
- 模意义计数时，消元中的除法必须保证对应元素可逆；合数模通常不能直接套普通模高斯消元

常见用法

```
auto ans = spanning_tree_count(g);
```

复杂度

- 构造矩阵 $O(n^2+m)$；朴素 $(n-1)\times(n-1)$ 行列式消元为 $O(n^3)$

BEST Euler 回路计数

解决问题： 计算强连通 Euler 有向图的 Euler 回路数量
思路： BEST 定理把 Euler 回路计数拆成“根向生成树数 × 各点出边排列的阶乘项”。核心结构为

$$\text{Euler tours} \propto t_r(G)\prod_v(\text{out}(v)-1)!$$

具体系数取决于是否固定起点、首边、把循环旋转视为同一个方案、平行边是否有编号

接口说明

- 根向生成树数使用 Matrix–Tree；阶乘项使用组合数学预处理
- 先按题目定义确定计数口径，再写最终系数

注意事项

- BEST 最容易错在计数口径，不能只背一个裸公式
- 必须先满足 Euler 有向图的连通与入出度条件
- 建议用只有一个小环或两个平行环的手算样例核对系数

常见用法

```
// 伪代码：先明确题目是否固定起点/首边
ans = tree_cnt * fac_prod;
```

复杂度

- 主要瓶颈是 Matrix–Tree 的 $O(n^3)$ 行列式；阶乘项为 $O(n)$

Euler、度数、树与割

解决问题： 快速处理 Euler 路存在性、度数奇偶、森林边数与桥/环之间的基本关系
思路： 这些结论都来自握手定理、树的无环连通性和割的度数计数

- 无向图：$\sum_v \deg(v)=2m$，奇度点个数为偶数
- 无向 Euler 回路：所有非零度点连通且全为偶度
- 无向 Euler 路：所有非零度点连通且奇度点恰为 0 或 2
- 有向 Euler 回路：忽略方向后非零度点连通，且每点入度 = 出度
- 有向 Euler 路：除起点 `out=in+1`、终点 `in=out+1` 外，其余点入度 = 出度；忽略方向后非零度点连通
- 树有 $n-1$ 条边；森林有 $m=n-c$
- 对任意顶点集 S，$\sum_{v\in S}\deg(v)$ 与割边数 $|\delta(S)|$ 同奇偶
- 无向图中一条边属于某个简单环，当且仅当它不是桥

接口说明

- Euler 路构造使用图论章节的 Hierholzer
- 桥使用边双连通 / Tarjan 板子

注意事项

- Euler 判定中的“连通”只要求考虑非零度点
- 有向 Euler 路的连通条件不要误写成所有点强连通；标准判定可用忽略方向后的连通性配合度数条件

常见用法

```
int odd = 0;
for (int d : deg) odd += d & 1;
```

```
bool ok = odd == 0 || odd == 2;
```

复杂度

- 仅检查度数为 $O(V)$；若还需检查连通性则为 $O(V+E)$

Manhattan / Chebyshev 距离变换

解决问题： 把二维 Manhattan 距离转为 Chebyshev 距离，便于处理最远点、正方形范围、扫描和某些几何查询
思路： 令 $u=x+y$, $v=x-y$，则

$$|x_1-x_2|+|y_1-y_2|=\max(|u_1-u_2|,|v_1-v_2|)$$

反变换为 $x=(u+v)/2$, $y=(u-v)/2$，因此只有 `u,v` 同奇偶时才对应整数点

```
pair<i128, i128> man2cheb(i64 x, i64 y) {
    return {(i128)x + y, (i128)x - y};
}
optional<pair<i64, i64>> cheb2man(i128 u, i128 v) {
    if ((u - v) % 2 != 0) return nullopt;
    i128 x = (u + v) / 2, y = (u - v) / 2;
    if (x < numeric_limits<i64>::min() || x > numeric_limits<i64>::max()) return nullopt;
    if (y < numeric_limits<i64>::min() || y > numeric_limits<i64>::max()) return nullopt;
    return pair{(i64)x, (i64)y};
}
```

接口说明

- `man2cheb(x,y)`：返回变换坐标 `(u,v)`，使用 `i128` 防止 `x+y` 溢出
- `cheb2man(u,v)`：若能还原为 `i64` 整数点则返回坐标，否则返回 `nullopt`

注意事项

- `cheb2man()` 默认 `u+v`、`u-v` 不超过 `i128`；由 `i64` 坐标经 `man2cheb()` 得到的值一定满足
- 二维 Manhattan 最远点也可直接枚举四种 `±x±y`；d 维推广为枚举 2^d 个符号掩码

常见用法

```
auto [u1, v1] = man2cheb(x1, y1);
auto [u2, v2] = man2cheb(x2, y2);
i128 du = u1 > u2 ? u1 - u2 : u2 - u1;
i128 dv = v1 > v2 ? v1 - v2 : v2 - v1;
i128 dis = max(du, dv);
```

复杂度

- 正变换、反变换和单次距离计算均为 $O(1)$

最小乘积与参数化凸壳

解决问题： 可行解同时产生两个非负代价 X,T，目标最小化乘积 XT；典型例子是每条边带两个权值的最小乘积生成树
思路： 把每个可行解映射为平面点 (X,T)。第一象限中最优乘积只可能出现在下凸壳；用参数 λ 最小化 $\lambda X+T$ 得到支撑点，再递归寻找相邻支撑点之间是否存在新的凸壳点

接口说明

- 最小乘积生成树每次把边权改为 `lambda * x + y` 后调用 MST
- 递归停止条件应比较端点与新点的叉积 / 是否落在同一支撑线，而不是对 `lambda` 做普通三分

注意事项

- 前提是两个目标量处在适合该几何解释的非负区域
- 若 X 或 T 可为负，乘积等高线方向变化，不能直接套第一象限下凸壳结论
- 最终比较答案仍使用真实乘积 `X*T`，参数化线性权只负责找候选点

常见用法

```
// solve(lambda): 按 lambda * x + y 求一个最优对象，并返回其 (X, Y)
// 用两个端点递归寻找新的支撑点，再比较 X * Y
```

复杂度

- 取决于产生的凸壳候选数和单次基础优化复杂度；最小乘积生成树中每个候选通常需要一次 MST

最大 / 最小统计量求和

解决问题： 求所有大小为 k 的子集中最大值之和、最小值之和，或进一步处理元素作为第若干大 / 小时的贡献
思路： 排序后固定某个元素作为最大值或最小值，剩余元素只需从它一侧选取，因此贡献次数直接是组合数。若 $a_0 \le \cdots \le a_{n-1}$：

$$\sum_{|S|=k} \max S = \sum_{i=k-1}^{n-1} a_i \binom{i}{k-1},$$

$$\sum_{|S|=k} \min S = \sum_{i=0}^{n-k} a_i \binom{n-i-1}{k-1}.$$

接口说明

- 先排序，再预处理组合数；按公式线性累加
- 更高阶第 `t` 大 / 小同样按“固定当前元素后，两侧分别选多少个”计组合数

注意事项

- 先确认题目求的是“所有子集统计量之和”，还是随机子集的期望 / 分布；后者还要除以总方案数或保留计数分布
- 有模数时组合数和乘法都按模处理；精确整数计数注意溢出

常见用法

```
sort(a.begin(), a.end());
Z ans{};
for (int i = k - 1; i < n; ++ i) ans += a[i] * C(i, k - 1);
```

复杂度

- 排序 $O(n \log n)$；组合数已预处理时统计为 $O(n)$

Sherman–Morrison、Woodbury 与行列式引理

解决问题： 已知 A^{-1} 时，矩阵只发生低秩更新，希望避免重新做一次 $O(n^3)$ 求逆或行列式。
思路： 把更新写成秩一或低秩乘积，在较低维的中间矩阵上求逆：

- Sherman–Morrison：

$$(A + uv^T)^{-1} = A^{-1} - \frac{A^{-1}uv^TA^{-1}}{1 + v^TA^{-1}u}$$

- Woodbury：

$$(A + UCV^T)^{-1} = A^{-1} - A^{-1}U(C^{-1} + V^TA^{-1}U)^{-1}V^TA^{-1}$$

- Matrix determinant lemma：

$$\det(A + UV^T) = \det(I + V^TA^{-1}U)\det(A)$$

接口说明

- 本节只保留公式；矩阵乘法、求逆和行列式使用 11.1 矩阵与线性方程组
- 秩一更新优先使用 Sherman–Morrison；秩 k 更新使用 Woodbury，把主要新求逆降到 $k \times k$

注意事项

- 所有公式都要求相应逆存在；Sherman–Morrison 需 `1 + v^T A^{-1} u != 0`
- 上述 Woodbury 形式还要求 `C` 与中间矩阵可逆
- 有限域中“非零”和“可逆”都按模意义判断

常见用法

```
// 已知 invA，更新 A ← A + u v^T
// 先算 den = 1 + v^T invA u，再按 Sherman--Morrison 更新
```

复杂度

- 秩一更新在已有 A^{-1} 时通常为 $O(n^2)$
- 秩 k Woodbury 主要为矩阵乘法加 $k \times k$ 求逆，常见量级约 $O(n^2k + k^3)$

17.3 微积分、恒等式与常数表

常见导数与不定积分

解决问题： 临场快速查询常见函数的导数、原函数和基础微积分变换，避免重新推导
思路： 先匹配函数类型，再用链式法则、换元或分部积分把目标化到表中形式

$f(x)$	$f'(x)$	$\int f(x)\,dx$
x^a	ax^{a-1}	$x^{a+1}/(a+1),\ a \neq -1$
$1/x$	$-1/x^2$	$\ln\|x\|$
e^x	e^x	e^x
a^x	$a^x \ln a$	$a^x/\ln a$
$\ln x$	$1/x$	$x\ln x - x$
$\sin x$	$\cos x$	$-\cos x$
$\cos x$	$-\sin x$	$\sin x$
$\tan x$	$\sec^2 x$	$-\ln\|\cos x\|$
$\arcsin x$	$1/\sqrt{1-x^2}$	$x\arcsin x + \sqrt{1-x^2}$
$\arctan x$	$1/(1+x^2)$	$x\arctan x - \ln(1+x^2)/2$

分部积分：

$$\int u\,dv = uv - \int v\,du$$

接口说明

- 本节是公式表，没有代码接口；优先直接查表，再决定是否需要数值积分或符号变换
- 换元积分时同时替换被积函数、微分和积分限

注意事项

- 不定积分统一省略积分常数 `+C`
- a^x 的公式默认 $a > 0,\ a \neq 1$；涉及根号、对数、反三角函数时先检查定义域
- 浮点数值计算仍需考虑精度，不能因为公式精确就直接使用裸 `==`

常见用法

- 出现 $f'(g(x))g'(x)$ 型结构时优先换元
- 出现多项式乘指数 / 三角 / 对数时优先考虑分部积分

复杂度

- 查表与公式代换为 $O(1)$

常用极限与 Taylor 展开

解决问题： 处理极限、局部近似、误差估计和需要把复杂函数展开成低次多项式的题目
思路： 在 $x \to 0$ 附近使用标准极限和 Taylor 展开；只保留足以决定答案的最低非零阶
常用极限：

$$\lim_{x\to 0}\frac{\sin x}{x} = 1,\quad \lim_{x\to 0}\frac{e^x - 1}{x} = 1,\quad \lim_{x\to 0}\frac{\ln(1+x)}{x} = 1$$

$$\lim_{x\to 0}\frac{1-\cos x}{x^2} = \frac{1}{2},\quad \lim_{x\to 0}(1+x)^{1/x} = e$$

常用展开：

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + O(x^4)$$

$$\ln(1+x) = x - \frac{x^2}{2} + \frac{x^3}{3} - \frac{x^4}{4} + O(x^5)$$

$$\sin x = x - \frac{x^3}{3!} + \frac{x^5}{5!} + O(x^7)$$

$$\cos x = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} + O(x^6)$$

$$\frac{1}{1-x} = 1 + x + x^2 + x^3 + O(x^4)$$

接口说明

- $O(x^k)$ 表示被省略项至少为 k 阶；需要更高精度时继续展开
- 通常先把自变量平移 / 缩放到趋近 0 的形式，再套展开

注意事项

- Taylor 展开是局部工具；不要把 $x \to 0$ 的近似直接用于远离展开点的区间
- 比较两个表达式时要继续展开到首次出现不同的非零阶

常见用法

- $1 - \cos x$ 与 $x^2/2$ 同阶；$\sin x$ 与 x 同阶
- $\ln(1+x) - x$ 的首项为 $-x^2/2$，不能只用一阶近似判断其符号

复杂度

- 固定阶展开与比较为 $O(1)$

三角恒等式与降幂

解决问题： 化简三角表达式、降幂、积化和差或和差化积，常用于几何、积分和周期函数计算
思路： 先统一角度与函数类型，再按倍角、降幂或和差公式消去乘积和高次项

- $\sin^2 x + \cos^2 x = 1$，$1 + \tan^2 x = \sec^2 x$
- $\sin(a \pm b) = \sin a \cos b \pm \cos a \sin b$
- $\cos(a \pm b) = \cos a \cos b \mp \sin a \sin b$
- $\sin 2x = 2 \sin x \cos x$
- $\cos 2x = \cos^2 x - \sin^2 x = 1 - 2\sin^2 x = 2\cos^2 x - 1$
- $\sin^2 x = (1 - \cos 2x)/2$，$\cos^2 x = (1 + \cos 2x)/2$
- $\sin a + \sin b = 2 \sin \frac{a+b}{2} \cos \frac{a-b}{2}$
- $\cos a + \cos b = 2 \cos \frac{a+b}{2} \cos \frac{a-b}{2}$
- $2 \sin a \cos b = \sin(a+b) + \sin(a-b)$
- $2 \cos a \cos b = \cos(a+b) + \cos(a-b)$
- $2 \sin a \sin b = \cos(a-b) - \cos(a+b)$

接口说明

- 本节是公式表，无代码接口

注意事项

- C++ `sin/cos/tan` 默认弧度制，不是角度制
- 几何题中若只比较符号或大小，优先避免不必要的 `atan/acos`

常见用法

- $\sin^2 x, \cos^2 x$ 积分时优先降幂
- 三角函数乘积积分时优先积化和差

复杂度

- 查表与代换为 $O(1)$

等差、等比与常用幂和

解决问题： 快速计算基础有限和、无限等比和与低次幂和
思路： 等差直接用首尾项，等比利用乘 r 后相减，低次幂和直接查闭式

和式	结果
$\sum_{i=1}^{n} i$	$n(n+1)/2$
$\sum_{i=1}^{n} i^2$	$n(n+1)(2n+1)/6$
$\sum_{i=1}^{n} i^3$	$[n(n+1)/2]^2$
$\sum_{i=0}^{n-1} ar^i$	$a(1-r^n)/(1-r),\ r \neq 1$
$\sum_{i=0}^{\infty} ar^i$	$a/(1-r),\ \lvert r\rvert < 1$
$\sum_{i=1}^{n} ir^{i-1}$	$[1-(n+1)r^n+nr^{n+1}]/(1-r)^2$

接口说明

- 整数模意义下的等比和若分母不可逆，应使用递归 / 分治求和，而不是直接除以 `1-r`

注意事项

- 公式中的除法在整数或模环中不一定合法
- 计算 $n(n+1)(2n+1)$ 时要先扩大整数类型

常见用法

```
i128 s1 = (i128)n * (n + 1) / 2;
i128 s2 = (i128)n * (n + 1) * (2 * n + 1) / 6;
```

复杂度

- 闭式计算为 $O(1)$

数论函数求和

解决问题： 查询 φ, μ, τ 等常见数论函数的约数和恒等式，并识别可用整除分块处理的求和
思路： 利用狄利克雷卷积关系和“$\lfloor n/i \rfloor$ 只有 $O(\sqrt{n})$ 种取值”进行化简

和式	结果
$\sum_{d\mid n} \varphi(d)$	n
$\sum_{d\mid n} \mu(d)$	$[n=1]$
$\sum_{d\mid n} \tau(d)$	$\prod_{p^a \parallel n} \binom{a+2}{2}$
$\sum_{i=1}^{n} \lfloor n/i \rfloor$	$\sum_{d=1}^{n} \tau(d)$

接口说明

- $\sum_{i=1}^{n} \lfloor n/i \rfloor$ 可用整除分块：相同商值的一段一次处理
- 更复杂的 φ/μ 前缀和使用对应数论章节的筛法或杜教筛

注意事项

- 约定 $\varphi(1)=1,\ \mu(1)=1$
- $p^a \parallel n$ 表示 $p^a \mid n$ 且 $p^{a+1} \nmid n$

常见用法

```
i64 ans = 0;
for (i64 l = 1, r; l <= n; l = r + 1) {
    i64 q = n / l;
    r = n / q;
    ans += q * (r - l + 1);
}
```

复杂度

- 单个约数和恒等式查表为 $O(1)$；整除分块为 $O(\sqrt{n})$

组合恒等式

解决问题： 化简组合数求和、嵌套选取、交替和以及“固定某一步再计数”的组合表达式
思路： 优先把式子解释成“同一件事的两种计数方式”，常见入口是 Pascal、Vandermonde、Hockey-stick 和二项式定理

- Pascal：$\binom{n}{k}=\binom{n-1}{k}+\binom{n-1}{k-1}$
- 对称：$\binom{n}{k}=\binom{n}{n-k}$
- Hockey-stick：$\sum_{i=r}^{n}\binom{i}{r}=\binom{n+1}{r+1}$
- Vandermonde：$\sum_k\binom{r}{k}\binom{s}{n-k}=\binom{r+s}{n}$
- $\sum_k\binom{n}{k}=2^n$，$\sum_k(-1)^k\binom{n}{k}=[n=0]$
- $\sum_k\binom{n}{k}^2=\binom{2n}{n}$
- $\sum_k k\binom{n}{k}=n2^{n-1}$
- $\sum_k k^2\binom{n}{k}=n(n+1)2^{n-2}$
- $\sum_k(-1)^k\binom{n}{k}k^m=0\ (m<n)$；$m=n$ 时为 $(-1)^n n!$
- $\sum_{k\ge 0}\binom{n-k}{k}=F_{n+1}$
- 李善兰恒等式：$\sum_{k=0}^{n}\binom{n}{k}\binom{n+k}{k}=\sum_{k=0}^{n}\binom{n}{k}^2 2^k$
- 嵌套选集：$\binom{n}{m}\binom{m}{k}=\binom{n}{k}\binom{n-k}{m-k}$
- 多重集选取：从 n 类中可重复选 k 个共有 $\binom{n+k-1}{k}$ 种
- 容斥：$|\bigcup_i A_i|=\sum_{\varnothing\neq S}(-1)^{|S|+1}\left|\bigcap_{i\in S}A_i\right|$

接口说明

- 默认约定非法组合数 $\binom{n}{k}=0$，例如 `k<0` 或 `k>n`
- 实际计算组合数使用组合数学章节对应预处理 / Lucas / 任意模版本

注意事项

- 恒等式成立不代表当前模数下能直接做除法；公式实现时仍要看模数条件
- 出现期望时先判断是“总贡献”还是还需除以总方案数

常见用法

- 形如 $\sum_k\binom{a}{k}\binom{b}{m-k}$ 优先尝试 Vandermonde
- 形如 $\sum_{i=r}^{n}\binom{i}{r}$ 优先尝试 Hockey-stick

复杂度

- 公式化简为 $O(1)$；实际求值复杂度由组合数接口决定

Fibonacci / Lucas

解决问题： 快速查询 Fibonacci / Lucas 的加法、倍增、整除、最大公约数和求和恒等式
思路： Fibonacci 是二阶线性递推，很多恒等式来自矩阵乘法或加法公式；需要大下标时优先使用快速倍增

- $F_0=0,F_1=1$；$L_0=2,L_1=1$，且 $L_n=F_{n-1}+F_{n+1}$
- $F_{m+n}=F_{m-1}F_n+F_mF_{n+1}$
- $F_{2n}=F_n(2F_{n+1}-F_n)$
- $F_{2n+1}=F_n^2+F_{n+1}^2$
- $\gcd(F_m,F_n)=F_{\gcd(m,n)}$
- 若 $n\mid m$，则 $F_n\mid F_m$
- 当 $n\ge 3$ 时，$F_n\mid F_m$ 当且仅当 $n\mid m$；$F_1=F_2=1$ 是退化例外
- Cassini：$F_{n+1}F_{n-1}-F_n^2=(-1)^n$
- $\sum_{i=0}^{n}F_i=F_{n+2}-1$
- $\sum_{i=0}^{n}F_i^2=F_nF_{n+1}$

接口说明

- 大下标计算使用 8.6 floor_sum、Pell、Farey 与 Fibonacci 的快速倍增
- 本节公式适合化简和手算，不重复实现 Fibonacci 板子

注意事项

- `F1=F2=1` 会破坏“由整除反推下标整除”的一般直觉，因此反向结论要排除 `n=1,2`
- 模意义下只保留数值，不应从模后的相等关系反推整数整除性质

常见用法

- 需要 `F(2n)`、`F(2n+1)` 时直接用倍增式
- 出现 `gcd(F[a],F[b])` 时先把下标取 `gcd`

复杂度

- 查恒等式为 $O(1)$；快速倍增计算 F_n 为 $O(\log n)$

Catalan、Motzkin 与 Bell 小表

解决问题： 对很小的经典计数值做手算、样例校验或快速识别数列。、
思路： 只保存前若干项作为校验表；超过范围应使用递推、组合公式或生成函数
下标均从 0 开始：

数列	$n=0,\dots,10$
Catalan	1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, 16796
Motzkin	1, 1, 2, 4, 9, 21, 51, 127, 323, 835, 2188
Bell	1, 1, 2, 5, 15, 52, 203, 877, 4140, 21147, 115975

接口说明

- 经典数列的递推和计算见 9.2 经典数列与分拆

注意事项

- 这些数增长很快，表外不要默认 `i64` 足够
- 小表主要用于识别与自测，不替代正式算法

常见用法

- 输出前几项与表格比对，检查递推初值和下标偏移

复杂度

- 查表 $O(1)$

第二类 Stirling 数小表

解决问题： 校验“把 n 个有标号元素划分为 k 个非空无标号集合”的计数
思路： 使用递推 $S(n,k)=S(n-1,k-1)+kS(n-1,k)$；本节只保留小值表

```
n=0: 1
n=1: 0 1
n=2: 0 1 1
n=3: 0 1 3 1
n=4: 0 1 7 6 1
n=5: 0 1 15 25 10 1
n=6: 0 1 31 90 65 15 1
n=7: 0 1 63 301 350 140 21 1
```

接口说明

- 第 `n` 行第 `k` 个数为 $S(n,k)$，下标从 0 开始

注意事项

- 不要与第一类 Stirling 数混淆；两者递推和组合意义不同

常见用法

- 用 `S(4,2)=7`、`S(5,3)=25` 检查 DP 初值和转移

复杂度

- 查表 $O(1)$；直接 DP 计算到 n 通常为 $O(n^2)$

Bernoulli 数小表

解决问题： 查询 Faulhaber 幂和、Euler–Maclaurin 等公式中常见的 Bernoulli 数
思路： 固定采用生成函数 $\frac{x}{e^x-1}=\sum_{n\ge 0}B_n x^n/n!$，因此 $B_1=-1/2$

```
B0=1, B1=-1/2, B2=1/6, B3=0, B4=-1/30, B5=0,
B6=1/42, B7=0, B8=-1/30, B9=0, B10=5/66,
B12=-691/2730, B14=7/6, B16=-3617/510,
B18=43867/798, B20=-174611/330
```

奇数 B_n 在 $n>1$ 时全为 0

接口说明

- 本表采用 `B1=-1/2` 约定；使用其他公式前先核对其 Bernoulli 定义

注意事项

- 部分文献采用 `B1=+1/2`，这是最常见的约定冲突
- 分数分子分母增长很快，小表之外不要硬编码

常见用法

- 套 Faulhaber / Euler–Maclaurin 前先检查公式使用哪一种 `B1` 约定

复杂度

- 查表 $O(1)$

常用数学常数

解决问题： 手算、误差估计和样例校验时快速查询常见实数常数。
思路： 仅保存足够人工校验的十进制近似；代码中能由库函数稳定得到的常数优先现算

常数	近似值
π	3.14159265358979323846
e	2.71828182845904523536
$\sqrt{2}$	1.41421356237309504880
$\varphi=(1+\sqrt{5})/2$	1.61803398874989484820
Euler–Mascheroni γ	0.57721566490153286061

接口说明

- C++ 中 π 可写 `acosl(-1)`，$\sqrt{2}$ 可写 `sqrtl(2)`

注意事项

- 不要为了避免一次库函数调用而手抄过多小数；硬编码精度必须与实际类型匹配
- `double` 只有约 15–17 位有效十进制数字，表中更多位并不会提升其有效精度

常见用法

```
const ld PI = acosl(-1);
const ld PHI = (1 + sqrtl(5)) / 2;
```

复杂度

- 查表或初始化均为 $O(1)$

常用质数、原根与 NTT 模数

解决问题： 为 NTT 选择支持足够二次幂长度的质数模数和原根，并快速判断目标卷积长度是否可用
思路： 长度为 2^k 的 NTT 要求 $2^k \mid (\mathrm{mod}-1)$；表中记录常用友好模数、原根和最大二次幂长度

模数	$\mathrm{mod}-1$ 分解	原根	最大二次幂长度
998244353	$119\cdot 2^{23}$	3	2^{23}
1004535809	$479\cdot 2^{21}$	3	2^{21}
469762049	$7\cdot 2^{26}$	3	2^{26}

模数	$\mathrm{mod}-1$ 分解	原根	最大二次幂长度
167772161	$5\cdot 2^{25}$	3	2^{25}
754974721	$45\cdot 2^{24}$	11	2^{24}

常用大质数 $1\,000\,000\,007$ 与 $1\,000\,000\,009$ 适合普通取模，但 $\mathrm{mod}-1$ 的二次幂因子很小，不适合长 NTT

接口说明

- 两个多项式长度分别为 `a`、`b` 时，线性卷积至少需要 `a+b-1` 个系数，再向上取到二次幂 `len`
- 只有 `len | (mod-1)` 时，当前模数才支持该长度的标准二次幂 NTT

注意事项

- “模数是质数”远远不够；必须同时检查根阶和目标长度
- 原根正确也不代表 NTT 实现一定正确，还要检查正逆根方向、位逆序和逆变换归一化
- 需要的 `len` 可能大于表中最大长度，此时应换友好模数、CRT 或其他卷积

常见用法

```
int need = int(a.size() + b.size() - 1);
int len = 1;
while (len < need) len <<= 1;
if ((mod - 1) % len != 0) {
    // 当前模数不支持该 NTT 长度
}
```

复杂度

- 长度检查为 $O(1)$；实际 NTT 长度为 N 时复杂度为 $O(N\log N)$

17.4 大模拟、牌型与赛事排名

大模拟通用拆解

解决问题： 处理规则多、分支多、平局细则多的模拟题，避免把输入解析、状态修改、比较规则和输出逻辑混在一起
思路： 优先拆成“解析 → 标准状态 → 单步更新 → 评分 / 排序键 → 输出”。能转成字典序比较元组的规则尽量一次编码，状态修改尽量在检查合法后一次提交

题型	稳定核心	优先复用	题目特化部分
扑克牌 / 牌型	编码为可字典序比较的评分元组	`PokerVal`	短牌、大小王、自定义牌型
Blackjack	A 先按 1，再尽量提升一个 A 为 11	`bj_val`	保险、分牌、double 等流程
保龄球	10 局 + strike / spare 奖励球	`bowl_score`	字符串输入格式
足球 / 篮球联赛	逐场累计，再按规则依次比较	`League`	积分制、相互战绩、公平竞赛分
分组晋级	组内排序后抽取名次，再跨组排序	`select_qual`	是否扣除对末名成绩
循环赛赛程	固定一人，其余循环旋转	`round_robin`	主客场平衡、双循环
事件时间线	`(时间,优先级,输入序号)` 排序	扫描线 / 事件模拟	同刻事件顺序
循环座位 / 淘汰	活跃集合 + 方向 + 取模	约瑟夫 / 下标链表	跳过、反转、额外回合
日志 / 协议	分词 + FSM + 原子更新	字符串与 IO	非法命令处理

注意事项

- 开写前先固定编号方式、闭开区间、同刻事件顺序、平局规则和非法输入行为
- 状态修改前先检查全部前置条件，避免失败分支留下半次修改
- 赛事名称不能替代题面规则；同一运动不同年份、不同赛事的破平规则可能不同

常见用法

- 牌型：先转为 `(类别, 破平关键字)`
- 排名：把每条比较规则写成独立字段，严格按题面顺序排序
- 时间线：所有事件先标准化成统一结构，再排序执行

德州扑克：五张牌评分与七选五

解决问题： 比较标准德州扑克牌力；支持恰好五张牌评分，以及从 5～7 张牌中选择最强五张

思路： 五张牌先统计点数频次、同花和顺子，再编码为 `(cat,key)`；`cat` 越大牌型越强，`key` 按破平顺序存储，因此可直接字典序比较。七选五最多枚举 $\binom{7}{5}=21$ 组

```
struct Card {
    int r{0}, s{0};
    friend bool operator==(const Card &a, const Card &b) {
        return tie(a.r, a.s) == tie(b.r, b.s);
    }
};
optional<Card> parse_card(string_view x) {
    if (x.size() != 2 && x.size() != 3) return nullopt;
    static constexpr string_view rs = "23456789TJQKA";
    static constexpr string_view ss = "SHDC";
    int r;
    if (x.size() == 3) {
        if (x[0] != '1' || x[1] != '0') return nullopt;
        r = 10;
    } else {
        size_t p = rs.find(x[0]);
        if (p == string_view::npos) return nullopt;
        r = int(p) + 2;
    }
    size_t s = ss.find(x.back());
    if (s == string_view::npos) return nullopt;
    return Card{r, (int)s};
}
struct PokerVal {
    int cat{-1};
    array<int, 5> key{};
    friend bool operator==(const PokerVal &a, const PokerVal &b) {
        return tie(a.cat, a.key) == tie(b.cat, b.key);
    }
    friend bool operator<(const PokerVal &a, const PokerVal &b) {
        return tie(a.cat, a.key) < tie(b.cat, b.key);
    }
};
PokerVal eval5(const array<Card, 5> &a) {
    array<int, 15> cnt{};
    bool flush = true;
    for (Card c : a) {
        ++ cnt[c.r];
        flush &= c.s == a[0].s;
    }
    array<int, 5> rs{};
    array<pair<int, int>, 5> gp{};
    int nr = 0;
    for (int r = 14; r >= 2; -- r) if (cnt[r]) {
        rs[nr] = r;
        gp[nr ++] = {cnt[r], r};
    }
    sort(gp.begin(), gp.end(), greater<pair<int, int>>());
    int hi = 0;
    if (nr == 5) {
        if (rs[0] - rs[4] == 4) hi = rs[0];
        else if (rs[0] == 14 && rs[1] == 5 && rs[4] == 2) hi = 5;
    }
    PokerVal v;
    if (flush && hi) v.cat = 8, v.key[0] = hi;
    else if (gp[0].first == 4) v.cat = 7, v.key = {gp[0].second, gp[1].second};
    else if (gp[0].first == 3 && gp[1].first == 2) v.cat = 6, v.key = {gp[0].second, gp[1].second};
    else if (flush) v.cat = 5, v.key = rs;
    else if (hi) v.cat = 4, v.key[0] = hi;
    else if (gp[0].first == 3) {
        v.cat = 3;
        v.key[0] = gp[0].second;
        int p = 1;
        for (int i = 0; i < nr; ++ i) if (rs[i] != gp[0].second) v.key[p ++] = rs[i];
    } else if (gp[0].first == 2 && gp[1].first == 2) {
        v.cat = 2;
```

```
        v.key = {gp[0].second, gp[1].second, gp[2].second};
    } else if (gp[0].first == 2) {
        v.cat = 1;
        v.key[0] = gp[0].second;
        int p = 1;
        for (int i = 0; i < nr; ++ i) if (rs[i] != gp[0].second) v.key[p ++] = rs[i];
    } else {
        v.cat = 0;
        v.key = rs;
    }
    return v;
}
struct BestHand {
    PokerVal val;
    array<int, 5> id{};
};
optional<BestHand> best_hand(const vector<Card> &a) {
    if (a.size() < 5 || a.size() > 7) return nullopt;
    BestHand ans;
    for (int i = 0; i < a.size(); ++ i)
        for (int j = i + 1; j < a.size(); ++ j)
            for (int k = j + 1; k < a.size(); ++ k)
                for (int l = k + 1; l < a.size(); ++ l)
                    for (int t = l + 1; t < a.size(); ++ t) {
                        array<int, 5> id{i, j, k, l, t};
                        array<Card, 5> h{a[i], a[j], a[k], a[l], a[t]};
                        PokerVal v = eval5(h);
                        if (ans.val < v) ans = {v, id};
                    }
    return ans;
}
int cmp_poker(const PokerVal &a, const PokerVal &b) {
    return (b < a) - (a < b);
}
```

cat	牌型	key 比较顺序
8	同花顺	顺子最高张，A2345 记为 5
7	四条	四条点数、单张
6	葫芦	三条点数、对子点数
5	同花	五张点数从大到小
4	顺子	顺子最高张
3	三条	三条点数、两张散牌从大到小
2	两对	大对子、小对子、单张
1	一对	对子点数、三张散牌从大到小
0	高牌	五张点数从大到小

接口说明

- `parse_card(x)`：解析 `2S..AS` 或 `10S` 形式，点数为 `2..14`，花色编号只用于判同花
- `eval5(a)`：恰好五张牌评分，返回可直接比较的 `PokerVal`
- `best_hand(a)`：接受 5～7 张牌，返回最优五张下标与牌力；其他张数返回 `nullopt`
- `cmp_poker(a,b)`：A 强返回 `1`，相同返回 `0`，B 强返回 `-1`

注意事项

- A2345 是五高顺子，低于 23456；标准德州扑克花色无大小
- 皇家同花顺只是 A 高同花顺，不需要额外类别
- 必须保证输入没有重复实体牌；短牌、大小王、五条等规则需要修改评分逻辑

常见用法

```
vector<Card> a;
for (string s : in) a.push_back(*parse_card(s));
auto best = best_hand(a);
if (best) cout << best->val.cat << '\n';
```

复杂度

- `eval5()` 为 $O(1)$；`best_hand()` 最多检查 21 个五张子集，仍为 $O(1)$

Blackjack 点数

解决问题： 给定一手 Blackjack 牌，计算不超过 21 时尽可能大的标准点数，并判断当前是否为 soft hand
思路： 所有 A 先按 1 计，其余 J/Q/K 按 10；若至少有一个 A 且总点数加 10 不超过 21，就把一个 A 提升为 11

```
pair<int, bool> bj_val(const vector<Card> &a) {
    int sum = 0, ace = 0;
    for (Card c : a) {
        if (c.r == 14) ++ ace, ++ sum;
        else sum += min(c.r, 10);
    }
    bool soft = ace && sum + 10 <= 21;
    if (soft) sum += 10;
    return {sum, soft};
}
```

接口说明

- `bj_val(a)`：返回 `{最佳点数, soft}`；点数大于 21 表示 bust

注意事项

- 只实现标准点数，不处理 blackjack 天然牌奖金、保险、分牌、double、庄家流程等规则
- 地方规则若含“五小龙”等特殊胜负条件，应在点数之外单独判断

常见用法

```
auto [sum, soft] = bj_val(a);
if (sum > 21) {
    // bust
}
```

复杂度

- 设手牌数为 k，时间 $O(k)$，额外空间 $O(1)$

十瓶保龄球计分

解决问题： 对一场合法完整的十瓶保龄球投球序列计算总分，包括 strike 和 spare 的奖励球
思路： 逐局维护当前投球下标；strike 取后两球奖励，spare 取后一球奖励，普通局只加本局两球

```
int bowl_score(const vector<int> &a) {
    int ans = 0, p = 0;
    for (int f = 0; f < 10; ++ f) {
        if (a[p] == 10) {
            ans += 10 + a[p + 1] + a[p + 2];
            ++ p;
        } else {
            int s = a[p] + a[p + 1];
            ans += s;
            if (s == 10) ans += a[p + 2];
            p += 2;
        }
    }
    return ans;
}
```

接口说明

- `bowl_score(a)`：`a` 按实际投球顺序存击倒瓶数，并已包含第 10 局需要的奖励球

注意事项

- 函数假设输入合法，不检查单局击倒数、第 10 局奖励球数量等约束
- 若题目输入为 `X` / `-` 字符串，应先解析成投球数组
- 全 strike 的投球序列长度为 12

常见用法

```
int ans = bowl_score(a);
```

复杂度

- 固定 10 局，时间和额外空间均为 $O(1)$

联赛积分、相互战绩与排序

解决问题： 处理足球、篮球等联赛积分榜，支持总积分、净胜分、得分、胜场、失分、外部附加值，以及并列后按相互战绩小表重新排序
思路： 先逐场累计统一统计量；比较规则用 `Crit` 按题面顺序描述。需要相互战绩时，只重算并列队伍之间的比赛；若部分队伍仍并列，可按规则选择是否对子组再次重算小表

```
struct Match {
    int a, b;
    i64 x, y;
};
struct PointRule {
    i64 win{3}, draw{1}, loss{0};
};
struct TeamStat {
    int id{0}, game{0}, win{0}, draw{0}, loss{0};
    i64 pt{0}, sf{0}, sa{0}, extra{0};
    i64 diff() const { return sf - sa; }
};
vector<TeamStat> calc_stat(int n, const vector<Match> &e,
        PointRule pr = {}, const vector<int> &sub = {}) {
    vector<TeamStat> st(n);
    vector<char> use(n, sub.empty());
    for (int i = 0; i < n; ++ i) st[i].id = i;
    for (int u : sub) use[u] = true;
    for (const auto &g : e) if (use[g.a] && use[g.b]) {
        auto &a = st[g.a];
        auto &b = st[g.b];
        ++ a.game;
        ++ b.game;
        a.sf += g.x;
        a.sa += g.y;
        b.sf += g.y;
        b.sa += g.x;
        if (g.x > g.y) {
            ++ a.win;
            ++ b.loss;
            a.pt += pr.win;
            b.pt += pr.loss;
        } else if (g.x < g.y) {
            ++ b.win;
            ++ a.loss;
            b.pt += pr.win;
            a.pt += pr.loss;
        } else {
            ++ a.draw;
            ++ b.draw;
            a.pt += pr.draw;
            b.pt += pr.draw;
        }
    }
    return st;
}
enum class Field { PT, DIFF, SF, WIN, SA, EXTRA };
struct Crit {
    Field f;
    bool hi{true};
};
i64 stat_val(const TeamStat &s, Field f) {
    if (f == Field::PT) return s.pt;
    if (f == Field::DIFF) return s.diff();
    if (f == Field::SF) return s.sf;
```

```
    if (f == Field::WIN) return s.win;
    if (f == Field::SA) return s.sa;
    return s.extra;
}
int cmp_stat(const TeamStat &a, const TeamStat &b, const vector<Crit> &c) {
    for (auto q : c) {
        i64 x = stat_val(a, q.f), y = stat_val(b, q.f);
        if (x == y) continue;
        return (q.hi ? x > y : x < y) ? -1 : 1;
    }
    return 0;
}
vector<int> sort_rank(const vector<TeamStat> &st, vector<int> p,
        const vector<Crit> &c) {
    sort(p.begin(), p.end(), [&](int a, int b) {
        int d = cmp_stat(st[a], st[b], c);
        return d ? d < 0 : a < b;
    });
    return p;
}
vector<vector<int>> split_tie(const vector<TeamStat> &st,
        const vector<int> &p, const vector<Crit> &c) {
    vector<vector<int>> g;
    for (int u : p) {
        if (g.empty() || cmp_stat(st[g.back().back()], st[u], c)) g.push_back({});
        g.back().push_back(u);
    }
    return g;
}
struct League {
    int n;
    vector<Match> e;
    PointRule pr;
    vector<TeamStat> all;
    League(int n_, vector<Match> e_, PointRule pr_ = {})
        : n(n_), e(move(e_)), pr(pr_), all(calc_stat(n, e, pr)) {}
    vector<int> rank(const vector<Crit> &c) const {
        vector<int> p(n);
        iota(p.begin(), p.end(), 0);
        return sort_rank(all, move(p), c);
    }
    vector<int> rank_group(vector<int> g, const vector<Crit> &head,
            const vector<Crit> &fall, bool reap) const {
        if (g.size() <= 1) return g;
        auto st = calc_stat(n, e, pr, g);
        auto p = sort_rank(st, g, head);
        auto tie = split_tie(st, p, head);
        if (tie.size() == 1) return sort_rank(all, move(g), fall);
        vector<int> ans;
        for (auto &q : tie) {
            vector<int> cur = q.size() <= 1 ? q
                : reap ? rank_group(q, head, fall, true)
                       : sort_rank(all, q, fall);
            ans.insert(ans.end(), cur.begin(), cur.end());
        }
        return ans;
    }
    vector<int> rank_h2h(const vector<Crit> &pre, const vector<Crit> &head,
            const vector<Crit> &fall, bool reap = true) const {
        vector<int> p = rank(pre), ans;
        for (auto &g : split_tie(all, p, pre)) {
            auto q = rank_group(g, head, fall, reap);
            ans.insert(ans.end(), q.begin(), q.end());
        }
        return ans;
    }
};
```

接口说明

- `Match{a,b,x,y}`：队伍 `a,b` 的一场比赛，比分 / 得分为 `x:y`

- `calc_stat(n,e,pr,sub)`：统计全部队伍，`sub` 非空时只计算子集内部相互战绩
- `Crit{field,hi}`：`hi=true` 表示字段越大越优
- `League::rank(c)`：只按全局统计排序
- `League::rank_h2h(pre,head,fall,reap)`：先按 `pre` 划分并列组，再按相互战绩 `head` 排序，仍并列时用 `fall`；`reap=true` 对剩余并列子组重新计算相互战绩

注意事项

- 体育赛事没有统一破平规则，必须严格按题面填写 `Crit` 顺序
- `Field::EXTRA` 不由 `calc_stat()` 自动计算；纪律分、抽签值等需外部写入
- 若相互战绩也要比较某种 `EXTRA`，必须明确它能否从子组比赛重算
- `reap=false / true / rank()` 分别对应“不对子组重算 / 对子组重算 / 不使用相互战绩”

常见用法

```
League tb(n, e, {3, 1, 0});
vector<Crit> pre{{Field::PT, true}};
vector<Crit> head{
    {Field::PT, true},
    {Field::DIFF, true},
    {Field::SF, true}
};
vector<Crit> fall{
    {Field::DIFF, true},
    {Field::SF, true}
};
auto p = tb.rank_h2h(pre, head, fall, true);
```

复杂度

- `calc_stat()` 为 $O(n+m)$，其中 m 为比赛数
- `rank()` 为 $O(n+m+n\log n)$
- 相互战绩递归会重复扫描比赛，最坏可到 $O(nm+n^2\log n)$；常见小组规模下通常很小

分组晋级

解决问题： 多个小组已完成组内排名后，先取每组前若干名直接晋级，再从各组下一名中选择若干最佳候选
思路： 直接晋级者按组截取；每组第一支未直通队伍进入跨组候选池，再用题目给定比较器排序

```
struct Qual {
    vector<int> direct, wild;
};
template<class F>
Qual select_qual(const vector<vector<int>> &g, int d, int w, F better) {
    Qual ans;
    vector<int> c;
    d = max(d, 0);
    w = max(w, 0);
    for (const auto &p : g) {
        int k = min(d, (int)p.size());
        ans.direct.insert(ans.direct.end(), p.begin(), p.begin() + k);
        if (k < (int)p.size()) c.push_back(p[k]);
    }
    sort(c.begin(), c.end(), [&](int a, int b) {
        if (better(a, b)) return true;
        if (better(b, a)) return false;
        return a < b;
    });
    c.resize(min(w, (int)c.size()));
    ans.wild = move(c);
    return ans;
}
```

接口说明

- `select_qual(g,d,w,better)`：每个 `g[i]` 必须已经按组内规则排好；取前 `d` 名直通，再从第 `d` 名位置的候选中选 `w` 支
- `better(a,b)`：跨组比较器，`a` 更优时返回 `true`

注意事项

- 最佳第三名等规则若要求扣除对小组末名球队的成绩，应先重新计算跨组统计，不能直接把原积分表交给 `better`
- 不同组队数、退赛、取消比赛等情况都可能改变跨组比较口径

常见用法

```
auto q = select_qual(g, 2, 4, [&](int a, int b) {
    return key[a] > key[b];
});
```

复杂度

- 设共有 g 个小组、跨组候选数为 c，时间 $O(g + c \log c)$，额外空间 $O(c)$

并列显示名次

解决问题： 已有从优到劣的排序结果，需要输出竞赛排名 `1,2,2,4` 或稠密排名 `1,2,2,3`
思路： 相邻元素等价时沿用前一名次；否则竞赛排名使用当前位置 `i+1`，稠密排名只加 1

```
template<class Eq>
vector<int> show_rank(const vector<int> &p, Eq eq, bool dense = false) {
    vector<int> r(p.size());
    for (int i = 0; i < (int)p.size(); ++ i) {
        if (i == 0) r[i] = 1;
        else if (eq(p[i - 1], p[i])) r[i] = r[i - 1];
        else r[i] = dense ? r[i - 1] + 1 : i + 1;
    }
    return r;
}
```

接口说明

- `show_rank(p,eq,false)`：竞赛排名，如 `1,2,2,4`
- `show_rank(p,eq,true)`：稠密排名，如 `1,2,2,3`

注意事项

- `p` 必须已经按最终规则排好序
- `eq(a,b)` 必须表示“显示名次意义下完全并列”，不能只比较部分指标

常见用法

```
auto r = show_rank(p, [&](int a, int b) {
    return score[a] == score[b];
});
```

复杂度

- 时间 $O(n)$，额外空间 $O(n)$

单循环赛程：圆圈法

解决问题： 为 n 个选手生成单循环赛程，使每一对选手恰好相遇一次，并尽量让每轮每人只赛一场
思路： 偶数人数时固定一个位置，其余编号每轮旋转；奇数人数先补一个虚拟轮空选手再使用同一构造

```
vector<vector<pair<int, int>>> round_robin(int n) {
    if (n <= 1) return {};
    int m = n + (n & 1);
    vector<int> a(m);
    iota(a.begin(), a.end(), 0);
    if (m != n) a.back() = -1;
    vector<vector<pair<int, int>>> ans;
    for (int rd = 0; rd < m - 1; ++ rd) {
        vector<pair<int, int>> cur;
        for (int i = 0; i < m / 2; ++ i) {
            int x = a[i], y = a[m - 1 - i];
            if (x != -1 && y != -1) cur.push_back({x, y});
        }
```

```
        ans.push_back(move(cur));
        int x = a.back();
        for (int i = m - 1; i >= 2; --i) a[i] = a[i - 1];
        a[1] = x;
    }
    return ans;
}
```

接口说明

- `round_robin(n)`：返回各轮对阵；偶数 `n` 有 `n-1` 轮，奇数 `n>1` 有 `n` 轮
- 返回 pair 只表示一场对阵，不保证主客场方向平衡

注意事项

- 双循环可复制全部对阵并交换主客场
- 若要求主客场连续场次最少、场地冲突、休息天数等额外目标，则已经是更复杂的排程问题

常见用法

```
auto rd = round_robin(n);
for (auto &g : rd) {
    for (auto [a, b] : g) {
        // a vs b
    }
}
```

复杂度

- 输出比赛总数为 $\binom{n}{2}$，时间和输出空间均为 $\Theta(n^2)$

两回合淘汰赛

解决问题： 两队进行主客场两回合比赛，按总比分、可选客场进球规则和外部决胜结果判断晋级者
思路： 先比较两回合总比分；若相同且题面启用客场进球，再比较各队客场得分；仍相同则交给加时、点球等外部规则

```
struct TwoLeg {
    i64 a1, b1;
    i64 a2, b2;
};
optional<int> two_leg(const TwoLeg &g, bool away,
        optional<int> last = nullopt) {
    i64 a = g.a1 + g.a2, b = g.b1 + g.b2;
    if (a != b) return a > b ? 0 : 1;
    if (away && g.a2 != g.b1) return g.a2 > g.b1 ? 0 : 1;
    return last;
}
```

接口说明

- `a1,b1`：首回合 A 主场时 A/B 得分
- `a2,b2`：次回合仍按 A/B 记录，因此 A 客场得分是 `a2`，B 客场得分是 `b1`
- `two_leg(g,away,last)`：返回 `0/1` 表示 A/B 晋级；仍无法决定且 `last` 为空时返回 `nullopt`

注意事项

- 客场进球规则只有题面明确存在时才能启用，不要根据赛事名称猜测
- 加时阶段客场进球是否生效、点球是否计入总比分等都必须按题面单独处理

常见用法

```
auto w = two_leg(g, use_away, shootout_winner);
if (!w) {
    // 仍未决
}
```

复杂度

- 时间和额外空间均为 $O(1)$

系列赛胜者判定

解决问题： 在 BO3、BO5、BO7 等“先赢 `need` 场”系列赛中，根据当前双方胜场判断是否已经产生胜者
思路： 一方达到目标胜场且另一方尚未达到时即可确定胜者

```
optional<int> series_win(int a, int b, int need) {
    if (a ≥ need && b < need) return 0;
    if (b ≥ need && a < need) return 1;
    return nullopt;
}
```

接口说明

- `series_win(a,b,need)`：返回 `0/1` 表示 A/B 已胜出，否则返回 `nullopt`

注意事项

- 函数只判断当前胜场，不验证总比赛数是否合法
- 若系列赛存在让分、平局或重赛规则，应额外处理

常见用法

```
auto w = series_win(wa, wb, 4); // BO7
```

复杂度

- 时间和额外空间均为 $O(1)$